DEVOPS ENGINEERING MASTERY

NETWORKING & INTERNET INFRASTRUCTURE

নেটওয়ার্কিং, Internet Protocols ও Production Network Engineering

VOLUME 02 • FIRST EDITION

```
nahid@ubuntu-server:~$ ip -br addr
eth0   UP   192.168.56.10/24
nahid@ubuntu-server:~$ dig +short example.com
93.184.216.34
nahid@ubuntu-server:~$ curl -I https://example.com
```

Ubuntu Server 24.04 LTS (Noble Numbat) — Primary hands-on environment

First Edition • Published 8 November 2026 • Dhaka, Bangladesh

24 Chapters • 24 Guided Labs • 3 Major Projects • 120 MCQs + 40 Subnetting Questions

Networking & Internet Infrastructure

DEVOPS ENGINEERING MASTERY — Volume 02
নেটওয়ার্কিং, Internet Protocols ও Production Network Engineering

First Edition — Published 8 November 2026.

Hands-on environment: Ubuntu Server 24.04 LTS (Noble Numbat, released April 2024, supported until April 2029). All commands in this book were written for Ubuntu 24.04 LTS unless a section explicitly says otherwise. General networking principles are distinguished from Linux-specific commands throughout. Version-specific instructions are labelled as such. Volume 01 (Computer Systems & Linux Engineering) is the prerequisite for the Linux command-line work in this book.

How to use this book. Read Parts I–II in order — they build the mental model (layers, addressing, transport) everything else rests on. Part III follows one packet at a time through routing, DNS, DHCP, HTTP and TLS. Part IV is production work: firewalls, VPNs, CDNs, cloud networks and troubleshooting. Work in an isolated lab network — a VirtualBox VM or two VMs on a host-only network. Never scan, probe or interfere with a network you do not own or have no permission to test. Destructive or privileged commands are flagged with a WARNING box. Sample terminal outputs are illustrative — your output will differ in addresses, PIDs and timings, and that is expected.

A note on honesty. Reading a book alone does not make anyone a network engineer. This book teaches the foundations and the practical judgment to progress toward professional engineering — but the judgment itself comes from doing the labs, breaking a test network, reading the packets, and fixing it. There is no shortcut for that practice.

Conventions. Bangla is the explanatory language; protocol names, technical terms, command names and configuration keys stay in English. `monospace` marks commands, paths and code. **CONCEPT** explains an idea, **NETWORK JOURNEY** follows a packet, **REAL WORLD** shows a production scenario, **WARNING** marks risk, **PRO TIP** gives professional advice, and **LAB** marks hands-on work.

© 2026 DevOps Engineering Mastery series. All rights reserved. This book is for personal learning. Practise network changes in a virtual machine or a disposable test network, never directly on a production network. The author and publisher accept no liability for outages or data loss caused by running commands without a backup or without authorisation.

Who this book is for

A Bangladeshi learner with basic computer knowledge who has studied Linux and command-line fundamentals (Volume 01), has some programming and web development experience, and has beginner-level networking knowledge. Someone who wants to become a professional DevOps and infrastructure engineer and can give about three hours a day to focused study.

Learning outcomes

By the end of this book you will be able to: explain how computer networks communicate; use the OSI and TCP/IP models; explain Ethernet, MAC addresses and switching; perform IPv4 binary conversion, subnetting and CIDR calculations; understand IPv6 addressing; explain TCP, UDP, ports, sockets and the connection lifecycle; understand routing, gateways, ARP, ICMP and NAT; explain DNS resolution step by step; understand HTTP, HTTPS and TLS; configure and troubleshoot DNS, firewall and connectivity in labs; configure Nginx as a reverse proxy and load balancer; understand VPN, CDN and network segmentation; analyse packets with Wireshark and tcpdump; troubleshoot realistic production networking problems; and design a secure network architecture for a web application.

সূচিপত্র — Contents

PART I — Networking Fundamentals

Chapter 01 — Introduction to Computer Networking 7

Chapter 02 — Network Types, Topologies and Architecture 13

Chapter 03 — OSI Model: Understanding Network Communication 19

Chapter 04 — TCP/IP Model and Encapsulation 25

Chapter 05 — Network Devices and Their Responsibilities 31

PART II — Addressing and Transport

Chapter 06 — MAC Addresses, Ethernet and Switching 38

Chapter 07 — IPv4 Addressing and Binary Mathematics 44

Chapter 08 — Subnetting, CIDR and Network Planning 50

Chapter 09 — IPv6 Fundamentals and Addressing 73

Chapter 10 — TCP, UDP, Ports and Sockets 80

Chapter 11 — TCP Connection Lifecycle and Troubleshooting 87

PART III — Internet Protocols

Chapter 12 — Routing, Gateways and NAT 95

Chapter 13 — DNS: How Domain Name Resolution Works 101

Chapter 14 — DHCP, ARP, ICMP and Essential Protocols 107

Chapter 15 — HTTP and HTTPS: How the Web Works 113

Chapter 16 — TLS, SSL Certificates and Secure Communication 119

Chapter 17 — Proxies, Reverse Proxies and Load Balancing 125

PART IV — Production Networking

Chapter 18 — Firewalls, Network Security and Segmentation 133

Chapter 19 — VPN, Tunnels and Secure Remote Connectivity 138

Chapter 20 — CDN, Edge Networks and Global Traffic Delivery 144

Chapter 21 — Linux Network Configuration and Diagnostics 149

Chapter 22 — Network Troubleshooting and Packet Analysis 155

Chapter 23 — Cloud Networking Fundamentals 160

Chapter 24 — Designing and Operating Production Network Infrastructure 166

Appendices

Appendix A — Guided Labs (24 Labs) 172

Appendix B — Independent Assignments (15) 187

Appendix C — Major Projects (01, 02, 03 — TechNova) 189

Appendix D — Final Examination & Assessment Rubric 192

Appendix E — Glossary (160 Terms) 194

Appendix F — References 200

Chapter-end quizzes (120 MCQs) and the 30 subnetting practice problems in Chapter 8 are inside each chapter and Appendix D. Their answers — with explanations, not just option letters, and a fully worked subnetting answer key — are in the separate companion volume, *Quiz, Examination & Answer Guide*. The hands-on material is also collected in the separate *Practical Lab & Assignment Workbook*.

PART I • DEVOPS ENGINEERING MASTERY

Networking Fundamentals

নেটওয়ার্কিংয়ের ভিত্তি — প্রথম নীতি থেকে: নেটওয়ার্ক কী, স্তর, মডেল ও যন্ত্রপাতি।

অধ্যায় ১ • Part I — Networking Fundamentals

Chapter 01: Introduction to Computer Networking — কম্পিউটার নেটওয়ার্কিংয়ের পরিচিতি

শুরুতে: একটি বাস্তব দৃশ্য

ঢাকার একটি বিশ্ববিদ্যালয়ের ছাত্রী তানিয়া তার ল্যাপটপে ব্রাউজারে একটি অনলাইন দোকানের ঠিকানা লিখল। এক সেকেন্ডেরও কম সময়ে পর্দায় ছবি, দাম আর "Add to Cart" বোতাম ভেসে উঠল। সে ভাবল, ওয়েবসাইটটি যেন তার ল্যাপটপের ভেতরেই আছে। আসলে তা নয় — ওয়েবসাইটটি চলছে সিঙ্গাপুরের একটি data center-এর সার্ভারে, হাজার হাজার কিলোমিটার দূরে। তানিয়ার ল্যাপটপ থেকে সেই সার্ভার পর্যন্ত তথ্য গেছে তার বাড়ির Wi-Fi router হয়ে, ইন্টারনেট সেবাদাতার (ISP) নেটওয়ার্ক দিয়ে, সমুদ্রের তলদেশের ফাইবার কেবল পার হয়ে, আরও অনেক যন্ত্রের হাত ঘুরে। এই পুরো যাত্রাটাই networking। এই অধ্যায়ে আমরা বুঝব: network আসলে কী, কেন তা দরকার, আর একটি ক্লিক কীভাবে দূরের সার্ভারে পৌঁছায়।

Learning Objectives — শেখার উদ্দেশ্য

- Network কী এবং কেন তা দরকার — নিজের ভাষায় ব্যাখ্যা করতে পারবেন।
- Client ও server-এর ভূমিকা আলাদা করে বলতে পারবেন।
- LAN, WAN, PAN ও MAN-এর মধ্যে পার্থক্য করতে পারবেন।
- Internet ও private network-এর সম্পর্ক ব্যাখ্যা করতে পারবেন।
- Bandwidth, throughput ও latency-র সংজ্ঞা ও পার্থক্য বলতে পারবেন।
- একটি web application-এর অনুরোধের যাত্রা ধাপে ধাপে বর্ণনা করতে পারবেন।

Prerequisites — পূর্বজ্ঞান

Volume 01 থেকে Linux terminal-এর মৌলিক ধারণা (কমান্ড চালানো, ফাইল দেখা), কম্পিউটার কীভাবে প্রোগ্রাম চালায় তার সাধারণ ধারণা, আর ব্রাউজার ব্যবহারের অভিজ্ঞতা থাকলেই চলবে। Networking-এর পূর্বজ্ঞান দরকার নেই।

১.১ Network আসলে কী?

সহজ কথায়, **network** হলো দুই বা ততোধিক যন্ত্রের মধ্যে তথ্য আদান-প্রদানের ব্যবস্থা। যন্ত্রগুলো হতে পারে কম্পিউটার, ফোন, সার্ভার, প্রিন্টার, এমনকি সিসিটিভি ক্যামেরা। তাদের মধ্যে সংযোগ হতে পারে তার দিয়ে (Ethernet কেবল, ফাইবার) বা তার ছাড়া (Wi-Fi, মোবাইল নেটওয়ার্ক)। কিন্তু শুধু তার জোড়া লাগালেই network হয় না — যন্ত্রগুলোকে একই ভাষায় কথা বলতে হয়। সেই ভাষার নিয়মগুলোই **protocol**। যেমন দুজন মানুষ কথা বলতে হলে একই ভাষা জানতে হয়, তেমনি দুটি কম্পিউটার তথ্য বিনিময় করতে হলে একই protocol মানতে হয়।

একটি উপমা ভাবুন: ডাকঘর। আপনি চিঠি লেখেন, খামে ঠিকানা লেখেন, ডাকবাক্সে ফেলেন। ডাকঘর খামটি বাছাই করে, ট্রেনে বা বিমানে পাঠায়, গন্তব্যের ডাকঘর প্রাপকের কাছে পৌঁছে দেয়। কম্পিউটার network-ও একইভাবে কাজ করে: তথ্য ছোট ছোট টুকরো (**packet**) করে, প্রতিটি packet-এ ঠিকানা লাগিয়ে, অনেক যন্ত্রের হাত ঘুরে গন্তব্যে পৌঁছে দেওয়া হয়। এই বইয়ের পুরো যাত্রা আসলে এই ডাকঘরের ভেতরের নিয়মগুলো বোঝা।

CONCEPT: Network-এর তিনটি মৌলিক উপাদান — (১) যন্ত্র (device): কম্পিউটার, সার্ভার, router; (২) মাধ্যম (medium): কেবল বা বেতার তরঙ্গ; (৩) নিয়ম (protocol): তথ্য কীভাবে ভাগ হবে, ঠিকানা কীভাবে লেখা হবে, ভুল হলে কী হবে। তিনটির যেকোনো একটি না থাকলে যোগাযোগ হয় না।

১.২ কেন Network দরকার?

প্রথম কম্পিউটারগুলো একা একা কাজ করত। কিন্তু মানুষ দ্রুত বুঝল, তথ্য ভাগ করে নেওয়ার মূল্য অনেক বেশি। Network দরকার চারটি কারণে। প্রথম, **সম্পদ ভাগাভাগি (resource sharing)**: একই প্রিন্টার, একই ফাইল, একই ডেটাবেস অনেক ব্যবহারকারী ব্যবহার করতে পারে। দ্বিতীয়, **যোগাযোগ (communication)**: ইমেইল, মেসেজ, ভিডিও কল — সবই network। তৃতীয়, **কেন্দ্রীয় ব্যবস্থাপনা**: একটি কোম্পানির সব তথ্য একটি সার্ভারে রেখে ব্যাকআপ ও নিরাপত্তা এক জায়গায় করা যায়। চতুর্থ, **পরিষেবা (services)**: ওয়েবসাইট, অনলাইন ব্যাংকিং, ক্লাউড — এগুলো network ছাড়া অস্তিত্বই রাখে না। DevOps engineer-এর কাজের প্রায় পুরোটাই network-এর ওপর দাঁড়িয়ে: সার্ভার, ডেটাবেস, load balancer, CDN — সবই network দিয়ে যুক্ত।

১.৩ Client ও Server

Network-এর সবচেয়ে পরিচিত সম্পর্ক হলো **client–server**। Client হলো যে সেবা চায় — আপনার ব্রাউজার, মোবাইল অ্যাপ। Server হলো যে সেবা দেয় — ওয়েবসাইটের কম্পিউটার, ডেটাবেস সার্ভার। তানিয়া যখন দোকানের সাইট খুলল, তার ব্রাউজার (client) সার্ভারকে বলল, "আমাকে হোমপেজ দাও।" সার্ভার পাতাটি পাঠাল। এই অনুরোধ–উত্তরের জোড়ার নাম **request–response**। মনে রাখবেন, client ও server ভূমিকা, যন্ত্রের নাম নয়। একই ল্যাপটপ এক সময় client (ব্রাউজার চালিয়ে), আরেক সময় server (নিজের কম্পিউটারে একটি test web server চালিয়ে) হতে পারে। Volume 01-এ আপনি Nginx চালিয়েছেন — তখন আপনার VM-ই ছিল server।

চিত্র ১.১ — Client–Server অনুরোধের যাত্রা
তানিয়ার ল্যাপটপ (Client / Browser) → বাড়ির Wi-Fi Router → ISP নেটওয়ার্ক → সমুদ্রের তলদেশের ফাইবার → Data Center → Web Server → উত্তর একই পথে ফিরে আসে
প্রতিটি তীরের মাঝে packet হাত বদল হয়; প্রতিটি যন্ত্র শুধু পরের গন্তব্যটি জানে।

১.৪ LAN, WAN, PAN ও MAN

Network-কে আকার অনুযায়ী ভাগ করা হয়। **PAN (Personal Area Network)** সবচেয়ে ছোট — আপনার ফোন ও Bluetooth হেডফোন, কয়েক মিটার। **LAN (Local Area Network)** একটি বাড়ি, অফিস বা ক্যাম্পাসের ভেতরে — সাধারণত একটি সংস্থার নিয়ন্ত্রণে, দ্রুতগতির। আপনার বাড়ির Wi-Fi একটি LAN। **MAN (Metropolitan Area Network)** একটি শহরজুড়ে — যেমন ঢাকার বিভিন্ন শাখা অফিসের সংযোগ। **WAN (Wide Area Network)** দেশ বা মহাদেশজুড়ে — Internet-ই সবচেয়ে বড় WAN। পার্থক্যটি শুধু আকারে নয়: LAN-এ আপনি নিজে যন্ত্রপাতি নিয়ন্ত্রণ করেন, WAN-এ অনেক সংস্থার যন্ত্র একসঙ্গে কাজ করে।

ধরন	আকার	মালিকানা	উদাহরণ
PAN	কয়েক মিটার	ব্যক্তি	ফোন–হেডফোন Bluetooth
LAN	একটি ভবন/ক্যাম্পাস	একটি সংস্থা	অফিসের Wi-Fi, বিশ্ববিদ্যালয়ের ল্যাব

MAN	একটি শহর	একাধিক সংস্থা	শহরের শাখা অফিসের সংযোগ
WAN	দেশ/বিশ্ব	অনেক ISP	Internet

১.৫ Internet বনাম Private Network

Internet হলো অনেক network-এর network — হাজার হাজার ISP ও সংস্থার network একে অপরের সঙ্গে যুক্ত হয়ে একটি বিশ্বব্যাপী ব্যবস্থা তৈরি করেছে। এর মালিক কেউ একা নয়; সবাই নিজের অংশ চালায়, আর সাধারণ protocol (TCP/IP) সবাইকে যুক্ত রাখে। অন্যদিকে **private network** (যেমন একটি কোম্পানির internal network বা intranet) শুধু অনুমোদিত ব্যবহারকারীদের জন্য। একটি ব্যাংকের কর্মীরা internal network-এ গ্রাহকের তথ্য দেখেন, কিন্তু সেই network Internet থেকে আলাদা ও সুরক্ষিত থাকে। DevOps-এ এই ভাগটাই সবচেয়ে গুরুত্বপূর্ণ: web server Internet-এ খোলা থাকে, কিন্তু database server শুধু private network-এ থাকে — বাইরে থেকে কেউ সরাসরি ডেটাবেসে পৌঁছাতে পারে না। এই নীতির নাম **segmentation**, যা Chapter 02 ও 18-এ বিস্তারিত আসবে।

চিত্র ১.২ — Internet বনাম Private Network
[Internet — সবার জন্য খোলা] ⇄ [Firewall] ⇄ [Private Network: Web Server | App Server | Database]
Database শুধু private network-এর ভেতর থেকে পৌঁছানো যায়; Internet থেকে সরাসরি নয়।

১.৬ Network Performance: Bandwidth, Throughput ও Latency

তিনটি শব্দ প্রায়ই গুলিয়ে যায়। **Bandwidth** হলো রাস্তার প্রস্থ — এক সেকেন্ডে সর্বোচ্চ কত তথ্য যেতে পারে, যেমন 100 Mbps। **Throughput** হলো আসলে কত তথ্য গেল — জ্যাম থাকলে রাস্তা চওড়া হলেও গাড়ি ধীরে যায়; তেমনি নেটওয়ার্ক ব্যস্ত থাকলে throughput bandwidth-এর চেয়ে কম হয়। **Latency** হলো সময় — একটি packet গন্তব্যে পৌঁছাতে কতক্ষণ লাগে, মিলিসেকেন্ডে মাপা হয়। ঢাকা থেকে সিঙ্গাপুরে latency সাধারণত ৩০–৬০ মিলিসেকেন্ড। একটি ভিডিও কলে bandwidth কম হলেও চলতে পারে, কিন্তু latency বেশি হলে কথা কেটে কেটে আসে। আর **jitter** হলো latency-র ওঠানামা — ভিডিও কলের শত্রু। মনে রাখুন: bandwidth বাড়ালেই latency কমে না। ঢাকা–সিঙ্গাপুর দূরত্ব আলোর গতির সীমায় বাঁধা; রাস্তা যত চওড়াই হোক, দূরত্ব কমে না।

১.৭ বাস্তব উদাহরণ: একটি ওয়েব অনুরোধের যাত্রা

তানিয়া যখন ঠিকানা লিখে Enter চাপে, যা ঘটে: (১) ব্রাউজার ডোমেইন নামটি IP ঠিকানায় বদলায় — DNS (Chapter 13)। (২) ব্রাউজার সার্ভারের সঙ্গে TCP সংযোগ তৈরি করে — handshake (Chapter 11)। (৩) HTTP অনুরোধ পাঠায় — "GET /" (Chapter 15)। (৪) সার্ভার পাতাটি ফেরত পাঠায়, প্রয়োজনে ডেটাবেস থেকে তথ্য এনে। (৫) ব্রাউজার পাতাটি দেখায়। পুরো যাত্রায় packet-গুলো router থেকে router-এ যায়, প্রতিটি router শুধু জানে পরের ধাপটি কোথায়। এই বইয়ের প্রতিটি অধ্যায় এই যাত্রার একটি করে ধাপ খুলে দেখাবে।

REAL WORLD: একটি বাংলাদেশি ই-কমার্স সাইটে ঈদের সময় হঠাৎ ট্রাফিক দশ গুণ বেড়ে গেল। সাইট ধীর হয়ে গেল — কারণ bandwidth নয়, সার্ভারের সামনে থাকা একটিমাত্র সার্ভার সব অনুরোধ সামলাতে পারছিল না। সমাধান ছিল load balancer দিয়ে ট্রাফিক কয়েকটি সার্ভারে ভাগ করা (Chapter 17)। সমস্যাটি network-এর ছিল না, architecture-এর ছিল — আর সেটা বুঝতে হলে এই অধ্যায়ের ধারণাগুলোই দরকার।

১.৮ হাতে-কলমে অনুশীলন: নিজের সংযোগ দেখা

Ubuntu VM বা নিজের Linux মেশিনে নিচের কমান্ডগুলো চালান। প্রতিটি কমান্ডের কাজ ব্যাখ্যাসহ দেওয়া হলো।

```
$ ping -c 4 8.8.8.8
# ping: একটি ছোট packet পাঠিয়ে উত্তর আসে কি না ও কতক্ষণ লাগে, তা মাপে।
# 8.8.8.8 হলো Google-এর public DNS সার্ভার — শুধু সংযোগ পরীক্ষার জন্য।

$ ip addr show
# ip addr: আপনার মেশিনের network interface ও IP ঠিকানা দেখায়।

$ ip route show
# ip route: আপনার মেশিনের routing table — packet কোন পথে যাবে, তা দেখায়।

$ curl -I https://example.com
# curl -I: একটি HTTP অনুরোধ পাঠিয়ে সার্ভারের উত্তরের শিরোনাম (header) দেখায়।
```

উদাহরণ আউটপুট (illustrative):

```
PING 8.8.8.8: 64 bytes from 8.8.8.8: icmp_seq=1 ttl=118 time=32.4 ms
--- 8.8.8.8 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3003ms
rtt min/avg/max/mdev = 31.8/32.6/33.9/0.8 ms
```

এখানে `time=32.4 ms` হলো latency-র একটি নমুনা, `0% packet loss` মানে সব packet পৌঁছেছে। আপনার সংখ্যা ভিন্ন হবে — সেটাই স্বাভাবিক।

> **LAB:** (১) `ping -c 4 8.8.8.8` চালিয়ে latency লিখুন। (২) `ip addr show` দিয়ে নিজের IP ঠিকানা খুঁজুন। (৩) `curl -I https://example.com` চালিয়ে HTTP status code (200) দেখুন। যাচাই: তিনটি কমান্ডই সফল হতে হবে; ping ব্যর্থ হলে প্রথমে Wi-Fi/কেবল সংযোগ ও `ip link` দিয়ে interface up আছে কি না দেখুন।

১.৯ Common Mistakes — সাধারণ ভুল

- **Bandwidth আর গতিকে এক মনে করা:** 100 Mbps সংযোগ মানেই পাতা দ্রুত খুলবে — তা নয়; latency ও সার্ভারের সাড়া দেওয়ার সময়ও গুরুত্বপূর্ণ।
- **"Internet নেই" বলা, যখন আসলে DNS কাজ করছে না:** `ping 8.8.8.8` কাজ করলে কিন্তু ওয়েবসাইট না খুললে সমস্যা DNS-এ, সংযোগে নয়।
- **Wi-Fi-কে Internet মনে করা:** Wi-Fi শুধু আপনার LAN-এর সংযোগ; router থেকে ISP পর্যন্ত লাইন বন্ধ থাকলে Wi-Fi থাকলেও Internet থাকবে না।

১.১০ Troubleshooting — সমস্যা নির্ণয়

কোনো সাইট না খুললে ধাপে ধাপে পরীক্ষা করুন: (১) `ip link` — interface up আছে কি? (২) `ping 8.8.8.8` — Internet সংযোগ আছে কি? (৩) `ping example.com` — DNS নাম মিলছে কি? (৪) `curl -I https://example.com` — HTTP স্তরে সমস্যা কি? যে ধাপে প্রথম ব্যর্থ হয়, সমস্যা সেই স্তরে। এই পদ্ধতির নাম layer-by-layer diagnosis — Chapter 03 ও 22-এ এর পূর্ণ রূপ আসবে।

১.১১ Security Considerations

Network-এ যাওয়া তথ্য পথে যে কেউ পড়তে পারে, যদি তা encrypt করা না থাকে। তাই HTTP-এর বদলে HTTPS ব্যবহার করা হয় (Chapter 16)। আর নিজের Wi-Fi-তে শক্ত পাসওয়ার্ড ব্যবহার করুন; খোলা Wi-Fi-তে ব্যাংকিং বা গুরুত্বপূর্ণ লগইন এড়িয়ে চলুন। মনে রাখবেন: network-এর নিরাপত্তা শুরু হয় এই সচেতনতা থেকে যে তথ্য অনেক যন্ত্রের হাত ঘুরে যায়।

১.১২ Best Practices

- সমস্যা দেখলে অনুমান নয়, ধাপে ধাপে পরীক্ষা করুন — নিচের স্তর থেকে ওপরে।
- Bandwidth, throughput ও latency-কে আলাদা শব্দ হিসেবে ব্যবহার করুন।
- Production সার্ভারের database কখনও সরাসরি Internet-এ খুলবেন না।
- নেটওয়ার্কের প্রতিটি পরিবর্তন লিখে রাখুন — কী বদলালেন, কেন, কখন।

Knowledge Check — যাচাই করুন

1. Network বলতে কী বোঝায়? A) একটি বড় কম্পিউটার B) দুই বা ততোধিক যন্ত্রের মধ্যে তথ্য আদান-প্রদানের ব্যবস্থা C) একটি ওয়েবসাইট D) একটি অপারেটিং সিস্টেম
2. Client–server মডেলে client-এর ভূমিকা কী? A) সেবা দেওয়া B) সেবা চাওয়া C) packet সংরক্ষণ করা D) কেবল তৈরি করা
3. একটি বাড়ির Wi-Fi নেটওয়ার্ক কোন ধরনের? A) WAN B) MAN C) LAN D) PAN
4. Bandwidth ও latency-র মধ্যে পার্থক্য কী? A) দুটি একই জিনিস B) Bandwidth সর্বোচ্চ ধারণক্ষমতা, latency পৌঁছাতে লাগা সময় C) Latency সর্বোচ্চ ধারণক্ষমতা, bandwidth সময় D) কোনোটিই মাপা যায় না
5. ping 8.8.8.8 সফল হলেও ওয়েবসাইট না খুললে সবচেয়ে সম্ভাব্য কারণ কী? A) কেবল ছিঁড়ে গেছে B) DNS সমস্যা C) মনিটর নষ্ট D) RAM শেষ

Practical Assignment

নিজের বাড়ি বা হোস্টেলের নেটওয়ার্কের একটি চিত্র আঁকুন: আপনার ল্যাপটপ/ফোন, router, ISP, আর একটি ওয়েবসাইটের সার্ভার। প্রতিটি অংশের নাম লিখুন, LAN কোথায় শেষ হয়ে WAN শুরু হয় তা চিহ্নিত করুন, আর `ping` দিয়ে তিনটি ভিন্ন গন্তব্যের latency মেপে একটি ছোট টেবিলে লিখুন।

Chapter Summary

Network হলো যন্ত্র, মাধ্যম ও protocol-এর সমন্বয়ে তথ্য আদান-প্রদানের ব্যবস্থা। Client সেবা চায়, server সেবা দেয়। আকার অনুযায়ী PAN, LAN, MAN, WAN ভাগ করা হয়; Internet হলো অনেক network-এর network। Bandwidth ধারণক্ষমতা, throughput আসল প্রবাহ, latency সময় — তিনটি আলাদা মাপ। একটি web অনুরোধের যাত্রা DNS, TCP, HTTP ধাপে ধাপে এগোয় — এই বইয়ের বাকি অধ্যায়গুলো সেই ধাপগুলো খুলে দেখাবে।

Revision Checklist

- ☐ Network-এর তিনটি উপাদান বলতে পারি
- ☐ Client ও server-এর ভূমিকা আলাদা করতে পারি

- ☐ LAN, WAN, PAN, MAN-এর পার্থক্য বলতে পারি
- ☐ Bandwidth, throughput, latency আলাদা করতে পারি
- ☐ ping, ip addr, ip route ও curl চালিয়ে ফল ব্যাখ্যা করতে পারি

অধ্যায় ২ • Part I — Networking Fundamentals

Chapter 02: Network Types, Topologies and Architecture — নেটওয়ার্কের ধরন, টপোলজি ও স্থাপত্য

শুরুতে: একটি বাস্তব দৃশ্য

চট্টগ্রামের একটি ছোট সফটওয়্যার কোম্পানিতে দশজন কর্মী একই অফিসে বসে কাজ করেন। একদিন অফিসের মূল switch-টি নষ্ট হয়ে গেল — আর পুরো অফিসের নেটওয়ার্ক থেমে গেল। কেউ ফাইল সার্ভারে ঢুকতে পারল না, কেউ প্রিন্ট করতে পারল না। অথচ পাশের ভবনের আরেকটি কোম্পানিতে একই ঘটনা ঘটলেও তাদের কাজ থামেনি — কারণ তাদের নেটওয়ার্ক এমনভাবে সাজানো ছিল যে একটি যন্ত্র নষ্ট হলেও বিকল্প পথ ছিল। পার্থক্যটা টপোলজিতে — যন্ত্রগুলো কীভাবে সাজানো ও যুক্ত। এই অধ্যায়ে আমরা শিখব নেটওয়ার্ক সাজানোর ধরনগুলো, কোনটি কখন ভালো, আর ভুল সাজানো কেন ব্যবসার ঝুঁকি।

Learning Objectives — শেখার উদ্দেশ্য

- Star, bus, ring ও mesh টপোলজি ব্যাখ্যা ও তুলনা করতে পারবেন।
- Physical ও logical topology-র পার্থক্য বলতে পারবেন।
- Peer-to-peer ও client–server স্থাপত্যের মধ্যে পার্থক্য করতে পারবেন।
- Centralized ও distributed architecture-এর সুবিধা-অসুবিধা বলতে পারবেন।
- Network segmentation কেন দরকার, তা ব্যাখ্যা করতে পারবেন।
- একটি ছোট enterprise network-এর চিত্র পড়তে ও আঁকতে পারবেন।

Prerequisites — পূর্বজ্ঞান

Chapter 01-এর ধারণা: network, client–server, LAN/WAN, packet। নতুন কোনো কমান্ড দরকার নেই; এই অধ্যায় ধারণাগত।

২.১ Topology কী?

Topology হলো নেটওয়ার্কের আকৃতি — যন্ত্রগুলো কীভাবে একে অপরের সঙ্গে যুক্ত। দুটি দিক আছে। **Physical topology** হলো আসল তারের সাজানো: কেবল কোথা দিয়ে গেছে, কোন যন্ত্র কোথায় বসেছে। **Logical topology** হলো তথ্য কীভাবে প্রবাহিত হয়: তারের সাজানো যেমনই হোক, তথ্য কোন পথে যায়। যেমন, একটি অফিসে সব কম্পিউটার একটি switch-এর সঙ্গে তার দিয়ে যুক্ত (physical star), কিন্তু তথ্য প্রয়োজনে যেকোনো কম্পিউটার থেকে যেকোনো কম্পিউটারে যেতে পারে (logical-ভাবে সবাই সবার সঙ্গে যুক্ত)। দুটি গুলিয়ে ফেললে troubleshooting-এ ভুল হয়: তার ঠিক থাকলেও logical পথ ভুল হতে পারে, যেমন ভুল VLAN সাজানো।

> **CONCEPT:** Topology বেছে নেওয়ার সময় তিনটি প্রশ্ন করুন — (১) একটি যন্ত্র নষ্ট হলে কী হবে? (২) নতুন যন্ত্র যোগ করা কত সহজ? (৩) খরচ কত? কোনো টপোলজিই সব ক্ষেত্রে সেরা নয়; প্রসঙ্গই সিদ্ধান্ত নেয়।

২.২ Star, Bus, Ring ও Mesh

Star: সব যন্ত্র একটি কেন্দ্রীয় যন্ত্রের (switch বা hub) সঙ্গে যুক্ত — তারার মতো। আজকের প্রায় সব অফিস ও বাড়ির নেটওয়ার্ক star। সুবিধা: একটি কম্পিউটারের কেবল নষ্ট হলে বাকিরা চলে; নতুন যন্ত্র যোগ করা সহজ। অসুবিধা: কেন্দ্রীয় যন্ত্রটি নষ্ট হলে পুরো নেটওয়ার্ক থামে — একে বলে **single point of failure**। সমাধান: গুরুত্বপূর্ণ জায়গায় দুটি switch রাখা।

Bus: সব যন্ত্র একটি একক কেবলের সঙ্গে যুক্ত — পুরনো দিনের নেটওয়ার্ক। সস্তা, কিন্তু কেবলের এক জায়গায় ছিঁড়ে গেলে পুরো নেটওয়ার্ক ভেঙে যায়, আর একসঙ্গে একটি যন্ত্রই কথা বলতে পারে। আজ প্রায় ব্যবহার হয় না, তবে ধারণাটি বোঝা দরকার কারণ পুরনো ডকুমেন্টে পাবেন।

Ring: প্রতিটি যন্ত্র দুই প্রতিবেশীর সঙ্গে যুক্ত, একটি বৃত্ত তৈরি হয়। তথ্য এক দিকে ঘুরে ঘুরে গন্তব্যে যায়। একটি সংযোগ ছিঁড়ে গেলে তথ্য উল্টো দিকে ঘুরে যেতে পারে — কিছু পুরনো টেলিকম নেটওয়ার্কে এই সুবিধা ছিল। তবে একটি যন্ত্র নষ্ট হলে বৃত্ত ভেঙে যায়, যদি না dual ring থাকে।

Mesh: প্রতিটি যন্ত্র একাধিক বা সব যন্ত্রের সঙ্গে যুক্ত। সবচেয়ে নির্ভরযোগ্য — অনেক বিকল্প পথ থাকে। কিন্তু খরচ ও জটিলতা সবচেয়ে বেশি। তাই পুরো mesh ছোট নেটওয়ার্কে বা গুরুত্বপূর্ণ সংযোগে (যেমন data center-এর core router-গুলোর মধ্যে) ব্যবহার হয়; বড় নেটওয়ার্কে **partial mesh** — শুধু গুরুত্বপূর্ণ যন্ত্রগুলোর মধ্যে একাধিক সংযোগ — ব্যবহার হয়। Internet-এর core আসলে partial mesh-এর মতো।

চিত্র ২.১ — চারটি টপোলজির তুলনা

Star: কেন্দ্রে Switch, চারপাশে PC — কেন্দ্র নষ্ট হলে সব থামে। | Bus: একটিমাত্র কেবল, সব PC তাতে যুক্ত — কেবল ছিঁড়লে সব থামে।

Ring: PC-গুলো বৃত্তাকারে যুক্ত — তথ্য বৃত্ত ধরে ঘোরে। | Mesh: প্রতিটি PC একাধিক PC-র সঙ্গে যুক্ত — অনেক বিকল্প পথ।

টপোলজি	একটি ব্যর্থতায় কী হয়	খরচ	কোথায় দেখা যায়
Star	একটি PC বিচ্ছিন্ন; কেন্দ্র নষ্ট হলে সব থামে	মাঝারি	অফিস, বাড়ি, ক্যাম্পাস
Bus	কেবল ছিঁড়লে পুরো নেটওয়ার্ক থামে	কম	পুরনো নেটওয়ার্ক
Ring	একটি সংযোগ ছিঁড়লে বিকল্প দিকে যেতে পারে	মাঝারি	পুরনো টেলিকম, কিছু শিল্প নেটওয়ার্ক
Mesh	বিকল্প পথে তথ্য চলে যায়	বেশি	Data center core, Internet backbone

২.৩ Peer-to-Peer বনাম Client–Server

Peer-to-peer (P2P)-তে সব যন্ত্র সমান — প্রতিটি যন্ত্র একই সঙ্গে সেবা দিতে ও নিতে পারে। ছোট দলে ফাইল ভাগাভাগি বা কিছু distributed system-এ এটি দেখা যায়। সুবিধা: কেন্দ্রীয় সার্ভার দরকার নেই, সস্তা। অসুবিধা: নিরাপত্তা ও ব্যাকআপ নিয়ন্ত্রণ করা কঠিন; কে কোন ফাইলে ঢুকতে পারবে, তা প্রতিটি যন্ত্রে আলাদা করে ঠিক করতে হয়।

Client–server-তে সেবা কেন্দ্রীয় সার্ভারে থাকে। সুবিধা: নিরাপত্তা, ব্যাকআপ ও আপডেট এক জায়গায় নিয়ন্ত্রণ করা যায়। অসুবিধা: সার্ভার নষ্ট হলে সেবা থামে — তাই production-এ সার্ভারের বিকল্প (redundancy) রাখা হয়। ওয়েব অ্যাপ্লিকেশনের জগৎ প্রায় পুরোটাই client–server; P2P দেখা যায় ফাইল শেয়ারিং ও কিছু blockchain নেটওয়ার্কে। একটি ভুল ধারণা: "P2P মানেই অনিরাপদ" — তা নয়; নিরাপত্তা নির্ভর করে নকশা ও নিয়ন্ত্রণের ওপর।

২.৪ Centralized বনাম Distributed Architecture

Centralized স্থাপত্যে সিদ্ধান্ত ও তথ্য এক জায়গায় থাকে — যেমন একটি কোম্পানির সব কর্মীর তথ্য একটি সার্ভারে। নিয়ন্ত্রণ সহজ, কিন্তু এক জায়গায় চাপ ও ঝুঁকি বেশি। **Distributed** স্থাপত্যে কাজ অনেক যন্ত্রে ভাগ করা থাকে — যেমন একটি ওয়েবসাইটের তথ্য কয়েকটি data center-এ ছড়িয়ে থাকে। সুবিধা: একটি জায়গা নষ্ট হলেও সেবা চলে; ব্যবহারকারীর কাছাকাছি সার্ভার থেকে দ্রুত সাড়া পাওয়া যায়। অসুবিধা: তথ্যের সামঞ্জস্য (consistency) রাখা কঠিন — একটি data center-এ তথ্য বদলালে অন্যটিতেও বদলাতে হয়। আধুনিক production system সাধারণত দুটির মিশ্রণ: নিয়ন্ত্রণ কেন্দ্রীয়, কিন্তু সেবা দেওয়ার কাজ distributed। CDN (Chapter 20) এর সেরা উদাহরণ: একটিমাত্র origin server, কিন্তু বিষয়বস্তু বিশ্বজুড়ে edge server-এ ছড়িয়ে থাকে।

চিত্র ২.২ — Centralized বনাম Distributed
Centralized: সব Client → একটি Server (একটি ব্যর্থতা = পুরো সেবা বন্ধ)।
Distributed: Client → Load Balancer → Server A / Server B / Server C (একটি নষ্ট হলেও বাকিরা সেবা দেয়)।

২.৫ Network Segmentation — ভাগ করে সুরক্ষা

Segmentation মানে নেটওয়ার্ককে ছোট ছোট ভাগে ভাগ করা, যাতে একটি ভাগের সমস্যা অন্য ভাগে ছড়াতে না পারে। বাস্তব উদাহরণ: একটি হাসপাতালে অতিথিদের Wi-Fi, ডাক্তারদের কম্পিউটার ও চিকিৎসা যন্ত্রের নেটওয়ার্ক আলাদা থাকে — অতিথি নেটওয়ার্কে ঢুকে কেউ যন্ত্রের নেটওয়ার্কে যেতে পারে না। অফিসে HR বিভাগের ফাইল সার্ভার শুধু HR-এর VLAN-এ থাকে। Segmentation-এর উপায়: **VLAN** (Chapter 06) দিয়ে একই switch-এ যৌক্তিকভাবে আলাদা নেটওয়ার্ক তৈরি করা, আর **firewall** (Chapter 18) দিয়ে ভাগগুলোর মধ্যে কোন তথ্য যেতে পারবে, তা নিয়ন্ত্রণ করা। একটি সাধারণ ভুল: সব যন্ত্র একই flat network-এ রাখা — তখন একটি কম্পিউটারে ভাইরাস ঢুকলে পুরো অফিস আক্রান্ত হয়।

২.৬ Enterprise Network Overview

একটি মাঝারি কোম্পানির নেটওয়ার্ক সাধারণত তিন স্তরে সাজানো হয়। **Access layer:** কর্মীদের কম্পিউটার, ফোন, প্রিন্টার switch-এর সঙ্গে যুক্ত হয়। **Distribution layer:** বিভিন্ন বিভাগের switch-গুলো একত্র হয়, VLAN ও নীতি এখানে প্রয়োগ হয়। **Core layer:** দ্রুতগতির backbone — সব distribution switch এখানে যুক্ত হয়, আর এখান থেকেই Internet ও data center-এর সংযোগ যায়। এই তিন স্তরের নকশার নাম **three-tier architecture**। ছোট অফিসে তিন স্তর একটিমাত্র switch-এ মিশে থাকতে পারে; বড় প্রতিষ্ঠানে প্রতিটি স্তরে আলাদা যন্ত্র থাকে। DevOps engineer-কে এই চিত্রটি পড়তে জানতে হবে — সার্ভার কোথায় বসবে, কোন স্তরে firewall থাকবে, তা বোঝার জন্য।

> **REAL WORLD:** একটি ঢাকার স্টার্টআপের অফিসে কর্মীদের ল্যাপটপ, অতিথিদের Wi-Fi ও সার্ভার — সব একই নেটওয়ার্কে ছিল। একদিন এক অতিথির ফোনে থাকা malware অফিসের ফাইল সার্ভারে ছড়িয়ে পড়ল। পরে তারা তিনটি VLAN করল: staff, guest ও servers — আর firewall দিয়ে ঠিক করল guest VLAN থেকে server VLAN-এ কোনো সংযোগ যাবে না। খরচ প্রায় শূন্য, কারণ switch-টিতেই VLAN সুবিধা ছিল — শুধু নকশাটা বদলাতে হয়েছিল।

২.৭ হাতে-কলমে অনুশীলন: নিজের নেটওয়ার্কের মানচিত্র

```
$ ip link show
# ip link: আপনার মেশিনের সব network interface (eth0, wlan0, lo) ও তাদের অবস্থা দেখায়।
```

```
$ ip addr show
# প্রতিটি interface-এর IP ঠিকানা ও subnet দেখায় — আপনার মেশিন কোন LAN-এ আছে, তা বোঝা যায়।

$ ss -tlnp
# ss: আপনার মেশিনে কোন port-এ কোন সার্ভিস শুনছে (listening), তা দেখায়।
# -t: TCP, -l: listening, -n: নামের বদলে সংখ্যা, -p: process দেখায়। sudo ছাড়া কিছু process নাম নাও দেখাতে পারে।
```

উদাহরণ আউটপুট (illustrative):

```
$ ip link show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500
$ ss -tlnp
State  Recv-Q  Local Address:Port  Process
LISTEN 0       0.0.0.0:22          sshd
LISTEN 0       127.0.0.1:53        systemd-resolved
```

এখানে `eth0 UP` মানে interface চালু, `0.0.0.0:22` মানে SSH সব interface-এ শুনছে, আর `127.0.0.1:53` মানে DNS resolver শুধু নিজের মেশিনে শুনছে — বাইরে থেকে পৌঁছানো যায় না। এটাই segmentation-এর ছোট উদাহরণ: সেবাটি শুধু যেখানে দরকার, সেখানেই খোলা।

> **LAB:** (১) `ip link show` ও `ip addr show` চালিয়ে নিজের interface-গুলোর নাম ও IP লিখুন। (২) `ss -tlnp` চালিয়ে কোন কোন port খোলা আছে, তা লিখুন। (৩) কাগজে নিজের LAN-এর star চিত্র আঁকুন: router কেন্দ্রে, আপনার যন্ত্রগুলো চারপাশে। যাচাই: আপনার আঁকা চিত্রের প্রতিটি যন্ত্রের IP `ip addr`-এর সঙ্গে মিলতে হবে।

২.৮ Common Mistakes — সাধারণ ভুল

- **সবকিছু একই flat network-এ রাখা:** অতিথি, কর্মী ও সার্ভার একসঙ্গে — একটি আক্রান্ত যন্ত্র থেকে সব যন্ত্রে ছড়িয়ে পড়ে।
- **Single point of failure ভুলে যাওয়া:** একটি switch-এর ওপর পুরো অফিস নির্ভরশীল, অথচ তার কোনো বিকল্প নেই।
- **Physical ও logical topology গুলিয়ে ফেলা:** তার ঠিক থাকলেই তথ্য যাবে — তা নয়; VLAN বা firewall-এর ভুল সাজানোতেও সংযোগ বন্ধ হয়।
- **Mesh মানেই সবসময় ভালো মনে করা:** ছোট অফিসে full mesh অপ্রয়োজনীয় খরচ ও জটিলতা।

২.৯ Troubleshooting — সমস্যা নির্ণয়

একটি কম্পিউটার নেটওয়ার্কে ঢুকতে পারছে না, বাকিরা পারছে: সমস্যা সম্ভবত ওই কম্পিউটারের কেবল, port বা তার নিজের সেটিং-এ — star topology-তে এটাই প্রত্যাশিত। সবাই একসঙ্গে বিচ্ছিন্ন: কেন্দ্রীয় switch, router বা ISP লাইন দেখুন। একটি বিভাগের সবাই বিচ্ছিন্ন, বাকিরা ঠিক: সেই বিভাগের distribution switch বা VLAN সেটিং দেখুন। এইভাবে "কে আক্রান্ত, কে নয়" — এই প্রশ্নটিই topology-ভিত্তিক troubleshooting-এর শুরু।

২.১০ Security Considerations

Segmentation নিরাপত্তার ভিত্তি। Guest নেটওয়ার্ক থেকে internal সার্ভারে যাওয়া বন্ধ করুন। Switch-এর অপরিচিত port বন্ধ রাখুন — কেউ এসে নিজের ল্যাপটপ লাগিয়ে internal network-এ ঢুকতে না পারে। Wi-Fi-তে আলাদা SSID-তে guest ও staff ভাগ করুন। মনে রাখবেন: network-এর ভেতরে ঢুকে পড়া আক্রমণকারীর জন্য flat network হলো খোলা মাঠ।

২.১১ Best Practices

- গুরুত্বপূর্ণ সংযোগে একটিমাত্র পথ রাখবেন না — core ও distribution স্তরে redundancy রাখুন।
- নেটওয়ার্কের চিত্র (diagram) সবসময় হালনাগাদ রাখুন; চিত্র ছাড়া troubleshooting অন্ধকারে হাতড়ানো।
- Segmentation শুরুতেই পরিকল্পনা করুন; পরে ভাগ করা অনেক কঠিন।
- নতুন যন্ত্র যোগ করার আগে জানুন সেটি কোন VLAN/segment-এ যাবে।

Knowledge Check — যাচাই করুন

1. Star topology-তে কেন্দ্রীয় switch নষ্ট হলে কী হয়? A) শুধু একটি PC বিচ্ছিন্ন হয় B) পুরো নেটওয়ার্ক থামে C) কিছুই হয় না D) শুধু Internet বন্ধ হয়
2. Physical ও logical topology-র পার্থক্য কী? A) দুটি একই B) Physical তারের সাজানো, logical তথ্যের প্রবাহের পথ C) Logical তারের সাজানো, physical তথ্যের প্রবাহ D) কোনোটিই বাস্তবে নেই
3. Mesh topology-র প্রধান সুবিধা কী? A) সবচেয়ে সস্তা B) অনেক বিকল্প পথ থাকায় নির্ভরযোগ্যতা বেশি C) সাজানো সবচেয়ে সহজ D) কম বিদ্যুৎ খায়
4. Network segmentation-এর প্রধান উদ্দেশ্য কী? A) গতি কমানো B) একটি ভাগের সমস্যা অন্য ভাগে ছড়াতে না দেওয়া C) খরচ বাড়ানো D) যন্ত্র কমানো
5. Enterprise network-এর core layer-এর কাজ কী? A) কর্মীদের কম্পিউটার যুক্ত করা B) দ্রুতগতির backbone হিসেবে সব distribution switch যুক্ত করা C) প্রিন্টার চালানো D) Wi-Fi পাসওয়ার্ড দেওয়া

Practical Assignment

২০ জন কর্মীর একটি অফিসের নেটওয়ার্ক নকশা করুন: দুটি বিভাগ (Engineering ও Accounts), একটি ফাইল সার্ভার, একটি প্রিন্টার ও অতিথিদের Wi-Fi। Star topology-তে চিত্র আঁকুন, কোন যন্ত্র কোন VLAN-এ থাকবে তা লিখুন, আর একটি switch নষ্ট হলে কী হবে ও তার বিকল্প কী — তা ব্যাখ্যা করুন।

Chapter Summary

Topology হলো নেটওয়ার্কের সাজানো: star সহজ ও প্রচলিত, bus পুরনো, ring বৃত্তাকার, mesh সবচেয়ে নির্ভরযোগ্য কিন্তু ব্যয়বহুল। Physical topology তারের সাজানো, logical topology তথ্যের প্রবাহ। Client–server স্থাপত্যে সেবা কেন্দ্রীয়; distributed স্থাপত্যে কাজ ছড়িয়ে থাকে। Segmentation নেটওয়ার্ককে ভাগ করে সমস্যা ও আক্রমণের ছড়ানো আটকায়। Enterprise network access, distribution ও core — এই তিন স্তরে সাজানো হয়।

Revision Checklist

- ☐ চারটি টপোলজি ব্যাখ্যা ও তুলনা করতে পারি
- ☐ Physical ও logical topology আলাদা করতে পারি

- ☐ P2P ও client–server-এর পার্থক্য বলতে পারি
- ☐ Segmentation কেন দরকার, তা বলতে পারি
- ☐ তিন স্তরের enterprise network-এর চিত্র পড়তে পারি

অধ্যায় ৩ • Part I — Networking Fundamentals

Chapter 03: OSI Model: Understanding Network Communication — OSI মডেল: নেটওয়ার্ক যোগাযোগ বোঝা

শুরুতে: একটি বাস্তব দৃশ্য

একটি production সার্ভারে হঠাৎ ওয়েবসাইট খুলছে না। জুনিয়র engineer প্রথমে ভাবল সার্ভার নষ্ট। কিন্তু পরীক্ষা করে দেখা গেল সার্ভার চলছে, কেবল ঠিক আছে, IP ঠিকানাও ঠিক — সমস্যা ছিল DNS-এ। সে যদি এলোমেলোভাবে সবকিছু পরীক্ষা করত, ঘণ্টা চলে যেত। কিন্তু সে একটি মানচিত্র অনুসরণ করেছিল: নিচের স্তর থেকে ওপরের স্তরে, এক স্তর করে পরীক্ষা। সেই মানচিত্রের নাম **OSI model**। এই অধ্যায়ে আমরা শিখব সাতটি স্তরের সেই মানচিত্র — যা ছাড়া professional troubleshooting অসম্ভব।

Learning Objectives — শেখার উদ্দেশ্য

- Layered networking কেন দরকার, তা ব্যাখ্যা করতে পারবেন।
- OSI মডেলের সাতটি স্তরের নাম, ক্রম ও দায়িত্ব বলতে পারবেন।
- প্রতিটি স্তরে কোন protocol ও যন্ত্র কাজ করে, তা চিহ্নিত করতে পারবেন।
- Encapsulation ও decapsulation ধাপে ধাপে ব্যাখ্যা করতে পারবেন।
- একটি web অনুরোধের যাত্রা সাতটি স্তর ধরে বর্ণনা করতে পারবেন।
- OSI মডেল দিয়ে একটি নেটওয়ার্ক সমস্যার স্তর শনাক্ত করতে পারবেন।

Prerequisites — পূর্বজ্ঞান

Chapter 01–02: network, packet, client–server, topology। Protocol শব্দটির সঙ্গে পরিচয় থাকলেই চলবে।

৩.১ কেন Layered Model দরকার?

একটি উপমা ভাবুন: একটি কোম্পানিতে চিঠি পাঠানো। একজন চিঠি লেখে, আরেকজন খামে ভরে ঠিকানা লেখে, আরেকজন ডাকঘরে পাঠায়, আরেকজন ট্রাকে তোলে। প্রত্যেকে শুধু নিজের কাজটি জানে; কেউ পুরো ব্যবস্থা জানতে বাধ্য নয়। Networking-ও তেমনি: পুরো যোগাযোগকে স্তরে স্তরে ভাগ করা হয়েছে, যাতে প্রতিটি স্তর শুধু নিজের দায়িত্ব পালন করে। একটি স্তর বদলালে (যেমন Wi-Fi-এর বদলে কেবল ব্যবহার করলে) ওপরের স্তরগুলো টেরই পায় না। এই ভাগাভাগির সুবিধা তিনটি: (১) জটিলতা কমে — প্রতিটি স্তর আলাদাভাবে বোঝা ও ঠিক করা যায়; (২) আন্তঃকার্যকারিতা (interoperability) — ভিন্ন কোম্পানির যন্ত্র একসঙ্গে কাজ করে, কারণ সবাই একই স্তরের নিয়ম মানে; (৩) troubleshooting সহজ হয় — সমস্যা কোন স্তরে, তা খুঁজে বের করা যায়। OSI model হলো এই স্তরগুলোর একটি আদর্শ মানচিত্র — এটি নিজে কোনো সফটওয়্যার নয়, একটি ধারণাগত কাঠামো (reference model)।

> **CONCEPT:** প্রতিটি স্তর নিচের স্তরকে সেবা দেয় আর ওপরের স্তর থেকে সেবা নেয়। একটি স্তর শুধু তার ঠিক নিচের ও ঠিক ওপরের স্তরের সঙ্গে কথা বলে — অন্য স্তরের ভেতরের কাজ তার জানার দরকার নেই। এই নীতির নাম **abstraction**।

৩.২ সাতটি স্তর — নিচ থেকে ওপরে

স্তরগুলো মনে রাখার একটি পরিচিত বাক্য (নিচ থেকে ওপরে): **"Please Do Not Throw Sausage Pizza Away"** — Physical, Data Link, Network, Transport, Session, Presentation, Application। নিচের চারটি স্তর (Physical থেকে Transport) তথ্য পৌঁছে দেওয়ার দায়িত্বে; ওপরের তিনটি স্তর (Session, Presentation, Application) তথ্যের অর্থ ও ব্যবহার নিয়ে কাজ করে।

চিত্র ৩.১ — OSI মডেলের সাতটি স্তর (নিচ থেকে ওপরে)

7. Application — ব্রাউজার, ইমেইল: ব্যবহারকারীর সেবা
6. Presentation — তথ্যের রূপ: encryption, compression, অক্ষর-সংকেত
5. Session — সংলাপ পরিচালনা: সংযোগ শুরু, চালানো, শেষ করা
4. Transport — নির্ভরযোগ্য পৌঁছে দেওয়া: TCP / UDP
3. Network — ঠিকানা ও পথ: IP, routing
2. Data Link — একই নেটওয়ার্কে পৌঁছে দেওয়া: Ethernet, MAC, switch
1. Physical — বাস্তব সংকেত: কেবল, বেতার তরঙ্গ, বিদ্যুৎ/আলো

৩.৩ প্রতিটি স্তরের দায়িত্ব

Layer 1 — Physical: তথ্যকে বাস্তব সংকেতে বদলানো — কেবলে বিদ্যুতের ঢেউ, ফাইবারে আলোর ঝলক, বাতাসে বেতার তরঙ্গ। এখানে bit (০/১) ছাড়া আর কিছু নেই। যন্ত্র: কেবল, repeater, hub। সমস্যা এখানে হলে: কেবল ছিঁড়ে যাওয়া, Wi-Fi সংকেত দুর্বল।

Layer 2 — Data Link: একই নেটওয়ার্কের (LAN) ভেতরে এক যন্ত্র থেকে আরেক যন্ত্রে frame পৌঁছে দেওয়া। এখানে MAC address ব্যবহার হয় (Chapter 06)। যন্ত্র: switch, network interface card (NIC)। সমস্যা এখানে হলে: ভুল VLAN, MAC address না মেলা।

Layer 3 — Network: ভিন্ন ভিন্ন নেটওয়ার্কের মধ্যে পথ খোঁজা — routing। এখানে IP address ব্যবহার হয় (Chapter 07)। যন্ত্র: router। সমস্যা এখানে হলে: ভুল gateway, routing table-এ পথ না থাকা।

Layer 4 — Transport: তথ্য সম্পূর্ণ ও ঠিক ক্রমে পৌঁছানো নিশ্চিত করা। TCP এখানে কাজ করে: হারানো packet আবার পাঠায়, ক্রম ঠিক রাখে (Chapter 10–11)। UDP-ও এই স্তরের, তবে নির্ভরযোগ্যতার প্রতিশ্রুতি ছাড়া। সমস্যা এখানে হলে: port বন্ধ, সংযোগ প্রত্যাখ্যাত (connection refused)।

Layer 5 — Session: দুটি application-এর মধ্যে সংলাপ (session) শুরু, চালানো ও শেষ করা। যেমন লগইন করার পর আপনার session যতক্ষণ চালু থাকে, তা এই স্তরের ধারণা। বাস্তবে এই কাজের অনেকটাই application নিজে করে।

Layer 6 — Presentation: তথ্যের রূপ বদলানো — encryption (HTTPS-এ TLS), compression, অক্ষর-সংকেতের অনুবাদ। প্রেরক ও গ্রাহকের তথ্যের রূপ এক না হলে এখানে মেলানো হয়।

Layer 7 — Application: ব্যবহারকারী সরাসরি যা ব্যবহার করে — HTTP (ওয়েব), DNS (নাম খোঁজা), SMTP (ইমেইল), SSH। সমস্যা এখানে হলে: ভুল URL, ভুল পাসওয়ার্ড, সার্ভারের ভেতরের ত্রুটি।

স্তর	নাম	মূল কাজ	Protocol / যন্ত্র
7	Application	ব্যবহারকারীর সেবা	HTTP, DNS, SSH
6	Presentation	তথ্যের রূপ, encryption	TLS, compression

5	Session	সংলাপ পরিচালনা	Session পরিচালনা
4	Transport	নির্ভরযোগ্য পৌঁছে দেওয়া	TCP, UDP
3	Network	ঠিকানা ও পথ	IP, router
2	Data Link	একই LAN-এ পৌঁছে দেওয়া	Ethernet, MAC, switch
1	Physical	বাস্তব সংকেত	কেবল, Wi-Fi

৩.৪ Encapsulation ও Decapsulation

তথ্য পাঠানোর সময় প্রতিটি স্তর ওপরের স্তরের তথ্যের সঙ্গে নিজের একটি **header** (শিরোনাম) জুড়ে দেয় — যেমন খামের ওপর খাম। Application স্তরে তৈরি হয় HTTP অনুরোধ। Transport স্তর তাতে TCP header জুড়ে **segment** বানায়। Network স্তর IP header জুড়ে **packet** বানায়। Data Link স্তর Ethernet header জুড়ে **frame** বানায়। Physical স্তর frame-কে bit-এ বদলে কেবলে পাঠায়। গন্তব্যে পৌঁছে উল্টো ঘটে: প্রতিটি স্তর নিজের header খুলে ফেলে (decapsulation) আর ভেতরের তথ্য ওপরের স্তরে দেয়। Header-এ যা থাকে: প্রেরক ও গ্রাহকের ঠিকানা, ক্রমিক নম্বর, ভুল ধরা পড়ার তথ্য। এই পদ্ধতির জন্যই প্রতিটি স্তর শুধু নিজের header পড়ে সিদ্ধান্ত নিতে পারে — ভেতরের তথ্য খুলে দেখার দরকার নেই।

চিত্র ৩.২ — Encapsulation: খামের ওপর খাম

HTTP অনুরোধ (Data) → + TCP Header = Segment → + IP Header = Packet → + Ethernet Header = Frame → Bit (কেবলে)

গন্তব্যে উল্টো ক্রমে header খোলা হয়: Frame → Packet → Segment → HTTP অনুরোধ।

৩.৫ বাস্তব যাত্রা: একটি web অনুরোধ সাতটি স্তরে

তানিয়া ব্রাউজারে ঠিকানা লিখল: (৭) ব্রাউজার HTTP অনুরোধ তৈরি করে। (৬) TLS তথ্য encrypt করে। (৫) ব্রাউজার ও সার্ভারের মধ্যে session তৈরি হয়। (৪) TCP অনুরোধকে segment-এ ভাগ করে, port নম্বর জোড়ে। (৩) IP প্রতিটি packet-এ সার্ভারের ঠিকানা লেখে, router পথ খোঁজে। (২) Ethernet frame-এ MAC address লেখে, switch LAN-এর ভেতরে পৌঁছে দেয়। (১) bit কেবল/বেতার তরঙ্গে রূপান্তরিত হয়ে ছুটে যায়। সার্ভারে উল্টো ক্রমে সব খুলে HTTP অনুরোধ বেরিয়ে আসে। উত্তর ফেরার সময় একই পথ উল্টোদিকে। মনে রাখুন: আসল যন্ত্রপাতিতে Session ও Presentation স্তরের কাজ প্রায়ই application বা TLS-এর ভেতরে মিশে থাকে — OSI একটি মানচিত্র, বাস্তব বাস্তবায়নের আক্ষরিক ছবি নয়। বাস্তব Internet চলে TCP/IP মডেলে (Chapter 04), যেখানে স্তর চারটি।

REAL WORLD: একটি কোম্পানির অফিসে হঠাৎ কিছু কম্পিউটার Internet-এ ঢুকতে পারছিল না। Layer 1 (কেবল) ঠিক, Layer 2 (switch) ঠিক, Layer 3-এ গিয়ে দেখা গেল তাদের gateway ঠিকানা ভুল — DHCP থেকে ভুল gateway দেওয়া হয়েছিল। OSI ক্রমে পরীক্ষা করায় সমস্যাটি ১০ মিনিটে ধরা পড়ল। এলোমেলোভাবে পরীক্ষা করলে প্রথমে ব্রাউজার reinstall করা হতো — যা কোনো কাজে আসত না।

৩.৬ OSI দিয়ে Troubleshooting

পদ্ধতিটি সহজ: নিচ থেকে ওপরে পরীক্ষা করুন। Layer 1: কেবল লাগানো আছে? Wi-Fi সংযুক্ত? interface up? (`ip link`)। Layer 2: একই LAN-এর যন্ত্রে `ping` যায়? (`arp -n` দিয়ে প্রতিবেশী দেখুন)। Layer 3: `ping 8.8.8.8` যায়?

`ip route` ঠিক আছে? Layer 4: `ss -tlnp` দিয়ে port খোলা আছে? `curl` দিয়ে সংযোগ হয়? Layer 7: DNS নাম মিলছে? HTTP status কী? যে স্তরে প্রথম ব্যর্থ হয়, সেখানেই সমস্যা — তার ওপরের স্তর পরীক্ষা করে সময় নষ্ট করবেন না। একটি সতর্কতা: স্তরগুলো মানসিক মানচিত্র; বাস্তব সমস্যা কখনও কখনও দুটি স্তরের সীমানায় থাকে (যেমন ভুল MTU — Layer 2/3-এর সীমানা)। তখন দুটি স্তরই পরীক্ষা করুন।

৩.৭ হাতে-কলমে অনুশীলন: স্তর ধরে পরীক্ষা

```
$ ip link show
# Layer 1/2: interface UP আছে কি? eth0/wlan0-র অবস্থা দেখুন।

$ ip addr show
# Layer 3: IP ঠিকানা পেয়েছেন কি?

$ ping -c 3 8.8.8.8
# Layer 3: Internet পর্যন্ত পথ আছে কি?

$ ping -c 3 example.com
# Layer 7 (DNS): নাম IP-তে বদলাচ্ছে কি?

$ curl -I https://example.com
# Layer 7 (HTTP) ও Layer 4 (TCP/TLS): সার্ভার সাড়া দিচ্ছে কি?
```

উদাহরণ আউটপুট (illustrative):

```
$ ping -c 3 example.com
PING example.com (93.184.216.34) 56(84) bytes of data.
64 bytes from 93.184.216.34: icmp_seq=1 ttl=56 time=41.2 ms
--- example.com ping statistics ---
3 packets transmitted, 3 received, 0% packet loss
```

এখানে `93.184.216.34` হলো DNS-এর দেওয়া IP — অর্থাৎ Layer 7-এর DNS কাজ করছে, আর Layer 3-এর পথও খোলা।

> **LAB:** (১) ওপরের পাঁচটি কমান্ড ক্রমে চালান ও প্রতিটির ফল লিখুন। (২) Wi-Fi বন্ধ করে আবার `ping 8.8.8.8` চালান — কোন স্তরে ব্যর্থ হয়, তা লিখুন। (৩) Wi-Fi চালু করে আবার পরীক্ষা করুন। যাচাই: আপনি প্রতিটি কমান্ডকে সঠিক OSI স্তরের সঙ্গে মেলাতে পারছেন কি না।

৩.৮ Common Mistakes — সাধারণ ভুল

- **OSI-কে বাস্তব software মনে করা:** OSI একটি reference model; Internet-এর বাস্তব protocol-গুলো TCP/IP মডেলে সাজানো। দুটির স্তর একে একে মেলাতে গিয়ে বিভ্রান্ত হবেন না।
- **সমস্যা দেখলেই ওপরের স্তরে ঝাঁপ দেওয়া:** ব্রাউজার reinstall করার আগে কেবল ও IP পরীক্ষা করুন।
- **Layer 5 ও 6-কে আলাদা protocol খোঁজা:** বাস্তবে এদের কাজ প্রায়ই application-এর ভেতরে থাকে; আলাদা যন্ত্র বা protocol খুঁজে পাবেন না।
- **Header-কে তথ্য মনে করা:** Header হলো খামের ঠিকানা; আসল চিঠি (payload) আলাদা।

৩.৯ Troubleshooting — সমস্যা নির্ণয়

`ping` কাজ করে কিন্তু ওয়েবসাইট খোলে না: Layer 3 ঠিক, সমস্যা Layer 4 (port বন্ধ) বা Layer 7 (DNS/HTTP)-এ। একই LAN-এর কম্পিউটারে `ping` যায় না: Layer 1/2 — কেবল, switch port, firewall-এর host নিয়ম দেখুন। `curl` "Could not resolve host" বলে: DNS — Layer 7, তবে তার আগে Layer 3 ঠিক আছে কি না নিশ্চিত হোন। প্রতিটি বার্তা কোন স্তরের, তা জিজ্ঞেস করুন — উত্তরেই সমস্যার অবস্থান লুকিয়ে থাকে।

৩.১০ Security Considerations

প্রতিটি স্তরে আলাদা আক্রমণ হয়: Layer 1-এ কেবলে আড়ি পাতা; Layer 2-এ ভুয়া MAC address; Layer 3-এ ভুয়া routing; Layer 7-এ ভুয়া ওয়েবসাইট (phishing)। তাই নিরাপত্তাও স্তরে স্তরে সাজাতে হয় — একটিমাত্র firewall যথেষ্ট নয়। বিশেষ করে Layer 6-এর encryption (TLS) ছাড়া ওপরের স্তরের তথ্য পথে পড়া যায় — এজন্য HTTPS বাধ্যতামূলক।

৩.১১ Best Practices

- প্রতিটি সমস্যাকে প্রথমে OSI স্তরে বসান, তারপর সমাধান খুঁজুন।
- সাতটি স্তরের নাম ও ক্রম মুখস্থ নয় — প্রতিটি স্তরের দায়িত্ব বুঝে মনে রাখুন।
- নেটওয়ার্কের চিত্র আঁকার সময় কোন যন্ত্র কোন স্তরে কাজ করে, তা লিখুন।
- নতুন protocol শিখলে জিজ্ঞেস করুন: এটি কোন স্তরের?

Knowledge Check — যাচাই করুন

1. OSI মডেলে মোট কয়টি স্তর আছে? A) 4টি B) 5টি C) 7টি D) 9টি
2. IP address কোন স্তরে ব্যবহার হয়? A) Physical B) Data Link C) Network D) Application
3. Encapsulation বলতে কী বোঝায়? A) তথ্য মুছে ফেলা B) প্রতিটি স্তর নিজের header জুড়ে দেওয়া C) তথ্য encrypt করা D) কেবল বদলানো
4. TCP কোন স্তরের protocol? A) Network B) Transport C) Session D) Physical
5. ping কাজ করে কিন্তু ওয়েবসাইট খোলে না — সমস্যা সম্ভবত কোন স্তরে? A) শুধু Layer 1 B) Layer 4 বা Layer 7 C) শুধু Layer 2 D) কোনো স্তরেই নয়

Practical Assignment

একটি সাইট না খোলার তিনটি ভিন্ন পরিস্থিতি লিখুন: (ক) কেবল খোলা, (খ) DNS ভুল, (গ) সার্ভারের port বন্ধ। প্রতিটির ক্ষেত্রে কোন কোন কমান্ডে কী ফল হবে, আর কোন OSI স্তরে সমস্যাটি — তা টেবিলে লিখুন।

Chapter Summary

OSI model যোগাযোগকে সাতটি স্তরে ভাগ করে: Physical, Data Link, Network, Transport, Session, Presentation, Application। প্রতিটি স্তর নিজের header জুড়ে encapsulation করে, গন্তব্যে উল্টোভাবে decapsulation হয়। এই মানচিত্র troubleshooting-এর ভিত্তি: নিচ থেকে ওপরে পরীক্ষা করলে সমস্যার স্তর দ্রুত ধরা পড়ে। বাস্তব Internet চলে TCP/IP মডেলে — পরের অধ্যায়ে সেই মডেলটি দেখব।

Revision Checklist

- ☐ সাতটি স্তরের নাম ক্রমসহ বলতে পারি
- ☐ প্রতিটি স্তরের দায়িত্ব ও একটি করে protocol বলতে পারি
- ☐ Encapsulation ও decapsulation ব্যাখ্যা করতে পারি
- ☐ একটি web অনুরোধের যাত্রা স্তর ধরে বলতে পারি
- ☐ সমস্যাকে OSI স্তরে বসিয়ে পরীক্ষা করতে পারি

অধ্যায় ৪ • Part I — Networking Fundamentals

Chapter 04: TCP/IP Model and Encapsulation — TCP/IP মডেল ও Encapsulation

শুরুতে: একটি বাস্তব দৃশ্য

১৯৭০-এর দশকে মার্কিন প্রতিরক্ষা গবেষণায় একটি সমস্যা ছিল: যুদ্ধ বা দুর্যোগে নেটওয়ার্কের একটি অংশ নষ্ট হলেও যোগাযোগ চালু রাখা যায় কীভাবে? উত্তর ছিল এমন একটি নকশা, যেখানে কোনো কেন্দ্রীয় নিয়ন্ত্রক নেই, তথ্য ছোট টুকরো হয়ে অনেক পথে যেতে পারে, আর গন্তব্য নিজে তথ্য জোড়া লাগিয়ে নেয়। সেই নকশাই TCP/IP — আজকের Internet-এর ভিত্তি। OSI যেখানে আদর্শ মানচিত্র, TCP/IP সেখানে বাস্তবে চলা ব্যবস্থা। এই অধ্যায়ে আমরা দেখব চারটি স্তরের এই মডেল কীভাবে কাজ করে, আর একটি packet কীভাবে তৈরি হয়।

Learning Objectives — শেখার উদ্দেশ্য

- TCP/IP মডেলের চারটি স্তরের নাম ও দায়িত্ব বলতে পারবেন।
- OSI মডেলের সঙ্গে TCP/IP মডেলের মিল-অমিল ব্যাখ্যা করতে পারবেন।
- PDU (segment, packet, frame)-এর পার্থক্য বলতে পারবেন।
- Header ও payload-এর ভূমিকা ব্যাখ্যা করতে পারবেন।
- একটি packet-এর encapsulation ধাপে ধাপে বর্ণনা করতে পারবেন।
- একটি web অনুরোধের end-to-end যাত্রা ব্যাখ্যা করতে পারবেন।

Prerequisites — পূর্বজ্ঞান

Chapter 03 (OSI model, encapsulation)। IP ও TCP শব্দ দুটির সঙ্গে প্রাথমিক পরিচয় থাকলেই চলবে।

৪.১ TCP/IP Architecture — চারটি স্তর

TCP/IP মডেলের চারটি স্তর: **Application, Transport, Internet, Link** (কখনও Network Access বলা হয়)। লক্ষ্য করুন, OSI-র নিচের দুটি স্তর (Physical ও Data Link) এখানে একটি স্তরে (Link) মিশে গেছে, আর ওপরের তিনটি স্তর (Session, Presentation, Application) একটি স্তরে (Application) মিশে গেছে। এটা কোনো ভুল নয় — TCP/IP বাস্তব protocol-গুলোকে গুছিয়ে দেখায়, আর OSI ধারণাগতভাবে ভাগ করে দেখায়। দুটি মডেল প্রতিদ্বন্দ্বী নয়; একই বাস্তবতাকে দুটি ভিন্নভাবে দেখার উপায়।

চিত্র ৪.১ — OSI থেকে TCP/IP-এ মিল

OSI 7 Application ┐
OSI 6 Presentation ┤ → TCP/IP Application (HTTP, DNS, SSH, TLS)
OSI 5 Session ┘
OSI 4 Transport → TCP/IP Transport (TCP, UDP)
OSI 3 Network → TCP/IP Internet (IP, ICMP, routing)
OSI 2 Data Link ┐
OSI 1 Physical ┘ → TCP/IP Link (Ethernet, Wi-Fi, MAC)

TCP/IP স্তর	দায়িত্ব	প্রধান Protocol	OSI-তে মিল
Application	ব্যবহারকারীর সেবা, তথ্যের রূপ	HTTP, DNS, SSH, TLS	স্তর 5–7
Transport	end-to-end পৌঁছে দেওয়া	TCP, UDP	স্তর 4
Internet	ঠিকানা ও routing	IP, ICMP	স্তর 3
Link	একই নেটওয়ার্কে পৌঁছে দেওয়া	Ethernet, Wi-Fi, ARP	স্তর 1–2

৪.২ Protocol Data Unit (PDU) — প্রতিটি স্তরের তথ্যের নাম

প্রতিটি স্তরে তথ্যের টুকরোর আলাদা নাম আছে — এগুলো গুলিয়ে ফেললে packet analysis-এ বিভ্রান্তি হয়। Application স্তরে তথ্যের নাম **data** বা message। Transport স্তরে TCP-র টুকরোর নাম **segment**, UDP-র টুকরোর নাম **datagram**। Internet স্তরে টুকরোর নাম **packet**। Link স্তরে টুকরোর নাম **frame**। আর Physical স্তরে শুধু **bit**। সাধারণ কথায় সবাই "packet" বলে, কিন্তু পরীক্ষায় বা Wireshark-এ সঠিক নামটি জানা দরকার: Wireshark-এ আপনি frame দেখেন, তার ভেতরে packet, তার ভেতরে segment, তার ভেতরে HTTP data।

> **CONCEPT:** PDU মনে রাখার উপায় — স্তর যত নিচে নামে, তথ্যের টুকরো তত "মোড়ক" পায়: data → segment → packet → frame → bit। প্রতিটি মোড়কেই নতুন ঠিকানা ও নিয়ন্ত্রণ তথ্য যোগ হয়।

৪.৩ Header ও Payload

প্রতিটি স্তরের header হলো সেই স্তরের "খামের ওপর লেখা তথ্য"। **TCP header**-এ থাকে: source port, destination port, sequence number, acknowledgment number, flags (SYN, ACK, FIN)। **IP header**-এ থাকে: source IP, destination IP, TTL (time to live), protocol নম্বর। **Ethernet header**-এ থাকে: source MAC, destination MAC, EtherType। আর **payload** হলো ভেতরের আসল তথ্য — ওপরের স্তরের পুরো টুকরো। যেমন IP packet-এর payload হলো TCP segment; TCP segment-এর payload হলো HTTP অনুরোধ। Header-এর আকার ছোট, কিন্তু সিদ্ধান্ত নেওয়ার সব তথ্য সেখানেই থাকে — router শুধু IP header পড়ে পথ ঠিক করে, ভেতরের HTTP তথ্য সে পড়েই না।

৪.৪ Packet Encapsulation — ধাপে ধাপে

ধরা যাক ব্রাউজার "GET /" অনুরোধ পাঠাচ্ছে। ধাপগুলো: **(১) Application:** HTTP অনুরোধ তৈরি হয় — "GET / HTTP/1.1" ও header-গুলো। **(২) Transport:** TCP এই তথ্যকে segment-এ ভাগ করে, প্রতিটিতে TCP header জোড়ে

— source port (যেমন 49152, এলোমেলো) ও destination port (443, HTTPS)। **(৩) Internet:** IP প্রতিটি segment-এর সঙ্গে IP header জোড়ে — source IP (আপনার ঠিকানা) ও destination IP (সার্ভারের ঠিকানা), TTL বসায়। এখন এটি packet। **(৪) Link:** Ethernet frame তৈরি হয় — destination MAC (আপনার gateway router-এর MAC) ও source MAC (আপনার NIC-এর MAC) জোড়ে। **(৫) Physical:** frame bit-এ বদলে কেবল বা বেতার তরঙ্গে যায়। প্রতিটি router-এ শুধু Link ও Internet স্তর খোলা হয়: router frame খুলে IP header পড়ে, নতুন frame বানিয়ে পরের router-এ পাঠায় — TCP ও HTTP তথ্য সে ছোঁয়ই না। একে বলে **end-to-end principle**: মাঝের যন্ত্রগুলো শুধু পৌঁছে দেয়, আসল কাজ শেষ প্রান্তে হয়।

চিত্র ৪.২ — Encapsulation-এর স্তর ও PDU

Application: HTTP Data

↓ + TCP Header (port, sequence)

Transport: TCP Segment

↓ + IP Header (source/destination IP, TTL)

Internet: IP Packet

↓ + Ethernet Header (source/destination MAC)

Link: Ethernet Frame → Bit → কেবল/Wi-Fi

Router শুধু Frame খুলে IP পড়ে, নতুন Frame বানায়; TCP/HTTP সে ছোঁয় না।

৪.৫ End-to-End যোগাযোগের উদাহরণ

তানিয়ার ল্যাপটপ থেকে সিঙ্গাপুরের সার্ভার পর্যন্ত যাত্রা: ল্যাপটপ packet বানিয়ে Wi-Fi router-কে দেয়। Router IP header পড়ে দেখে গন্তব্য নিজের LAN-এ নেই — তাই ISP-র router-এ পাঠায়। ISP-র router আবার পথ খুঁজে পরের router-এ দেয়। এভাবে ১০–১৫টি router পার হয়ে packet সার্ভারে পৌঁছায়। সার্ভার decapsulation করে: Ethernet header খুলে, IP header খুলে, TCP header খুলে, HTTP অনুরোধ পড়ে। উত্তর ফেরার সময় একই প্রক্রিয়া উল্টোদিকে — তবে ফেরার পথ যাওয়ার পথের মতো নাও হতে পারে; Internet-এ প্রতিটি packet নিজের পথ নিজে খুঁজে নেয়। এটাই TCP/IP-এর শক্তি: কোনো একক পথের ওপর নির্ভরতা নেই।

REAL WORLD: একটি বাংলাদেশি SaaS কোম্পানির ব্যবহারকারীরা অভিযোগ করল সাইট মাঝে মাঝে ধীর। তদন্তে দেখা গেল সমস্যা সার্ভারে নয় — একটি মাঝের router-এ packet হারাচ্ছিল (packet loss)। TCP হারানো packet আবার পাঠাচ্ছিল বলে সাইট চলছিল, কিন্তু ধীরে। `mtr` দিয়ে প্রতিটি hop-এর loss মেপে সমস্যার router শনাক্ত করা হলো, আর ISP-কে জানানো হলো। Encapsulation বুঝলে বোঝা যায় কেন সমস্যাটি TCP স্তরে ধরা পড়ল, অথচ কারণ ছিল Internet স্তরের পথে।

৪.৫ক MTU ও Fragmentation — বড় Packet ভাঙা

প্রতিটি Link-এর একটি সর্বোচ্চ আকার থাকে, যাকে বলে **MTU (Maximum Transmission Unit)**। Ethernet-এর সাধারণ MTU 1500 byte — এর চেয়ে বড় frame একবারে পাঠানো যায় না। কিন্তু একটি বড় তথ্য, যেমন একটি বড় ছবি, 1500 byte-এর চেয়ে অনেক বড় হতে পারে। তখন কী হয়? Transport স্তরে TCP তথ্যকে ছোট ছোট segment-এ ভাগ করে, যাতে প্রতিটি segment MTU-র ভেতরে ধরে। এটাই স্বাভাবিক পথ। কিন্তু কখনও কখনও Internet স্তরে একটি packet MTU-র চেয়ে বড় হয়ে যায় — তখন IP তাকে **fragment** করে, অর্থাৎ কয়েকটি ছোট packet-এ ভেঙে পাঠায়, আর গন্তব্য সেগুলো জোড়া লাগায়। Fragmentation কাজ করে, কিন্তু ধীর ও ঝুঁকিপূর্ণ: একটি fragment হারিয়ে গেলে পুরো packet আবার পাঠাতে হয়। তাই আধুনিক system **Path MTU Discovery** ব্যবহার করে — প্রথমে ছোট packet পাঠিয়ে পথের

সবচেয়ে ছোট MTU জেনে নেয়, তারপর সেই আকারে তথ্য ভাগ করে। একটি পরিচিত সমস্যা: ভুল MTU সেটিং থাকলে ছোট ওয়েবপেজ খোলে, কিন্তু বড় ফাইল ডাউনলোড হয় না — কারণ বড় packet-গুলো পথে আটকে যায়। তখন `ping -M do -s 1472 8.8.8.8` দিয়ে পরীক্ষা করা যায় কোন আকার পর্যন্ত packet যাচ্ছে।

> **PRO TIP:** "কিছু সাইট খোলে, কিছু খোলে না" — এই লক্ষণ দেখলে MTU সন্দেহ করুন। বিশেষ করে VPN বা PPPoE সংযোগে MTU ছোট হয় (সাধারণত 1400-এর কাছাকাছি), আর ভুল সেটিং থাকলে বড় packet হারিয়ে যায়।

৪.৬ হাতে-কলমে অনুশীলন: নিজের packet দেখা

```
$ ip addr show
# নিজের IP ঠিকানা লিখুন — এটি IP header-এর source IP হবে।

$ ip route show
# default gateway কোনটি? আপনার packet প্রথমে সেখানেই যায়।

$ sudo tcpdump -i any -c 5 -n icmp
# tcpdump: চলমান packet ধরে দেখায়। -i any: সব interface, -c 5: ৫টি packet,
# -n: নামের বদলে সংখ্যা, icmp: শুধু ping-এর packet। sudo দরকার, কারণ packet ধরা
# একটি privileged কাজ। অন্য terminal-এ ping চালিয়ে দেখুন।
# নিরাপত্তা: শুধু নিজের মেশিনের ও নিজের নেটওয়ার্কের packet দেখুন।

$ ping -c 2 8.8.8.8
# ওপরের tcpdump চালু থাকা অবস্থায় এটি চালান — ICMP packet ধরা পড়বে।
```

উদাহরণ আউটপুট (illustrative):

```
12:00:01.123456 IP 192.168.1.10 > 8.8.8.8: ICMP echo request, id 1, seq 1, length 64
12:00:01.155789 IP 8.8.8.8 > 192.168.1.10: ICMP echo reply, id 1, seq 1, length 64
```

এখানে `192.168.1.10 > 8.8.8.8` হলো IP header-এর source ও destination — tcpdump আপনাকে packet-এর ভেতরের ঠিকানা দেখাচ্ছে। আপনার IP ভিন্ন হবে।

> **LAB:** (১) `ip route show` দিয়ে default gateway লিখুন। (২) একটি terminal-এ `sudo tcpdump -i any -c 4 -n icmp` চালান, আরেকটিতে `ping -c 2 8.8.8.8` চালান। (৩) tcpdump-এর আউটপুটে source ও destination IP চিহ্নিত করুন। যাচাই: request ও reply — দুটি দিকের packet-ই দেখা যাচ্ছে কি না। Cleanup: tcpdump নিজে থামে (-c 4 দেওয়ায়); আর কিছু পরিষ্কার করার নেই।

৪.৭ Common Mistakes — সাধারণ ভুল

- **OSI ও TCP/IP-কে একই ভাবা:** OSI-র ৭ স্তর আর TCP/IP-র ৪ স্তর এক নয়; মিলের চিত্রটি বুঝে ব্যবহার করুন, আক্ষরিকভাবে নয়।
- **Segment, packet ও frame গুলিয়ে ফেলা:** প্রতিটি স্তরের টুকরোর নাম আলাদা — Wireshark-এ এগুলো আলাদা করে দেখা যায়।
- **Router-কে সব স্তর খোলে মনে করা:** Router সাধারণত শুধু Link ও Internet স্তর পর্যন্ত খোলে; TCP/HTTP সে পড়ে না।

- **TTL-কে সময় মনে করা:** TTL (time to live) আসলে hop সংখ্যা — প্রতিটি router-এ ১ করে কমে; শূন্য হলে packet ফেলে দেওয়া হয়।

৪.৮ Troubleshooting — সমস্যা নির্ণয়

`traceroute` বা `mtr` দিয়ে দেখুন packet কোন hop পর্যন্ত যাচ্ছে — যেখানে থামে, সমস্যা সেখানে বা তার পরে। `tcpdump` দিয়ে দেখুন packet বের হচ্ছে কি না: বের হচ্ছে কিন্তু উত্তর আসছে না — সমস্যা গন্তব্যে বা ফেরার পথে; বেরই হচ্ছে না — সমস্যা নিজের মেশিনে (gateway, firewall)। TTL exceeded বার্তা মানে routing loop — packet ঘুরপাক খাচ্ছে।

৪.৯ Security Considerations

Header-এর তথ্য পথে পড়া যায় — তাই IP ঠিকানা, port নম্বর গোপন নয়। আসল সুরক্ষা payload-এর encryption-এ (TLS)। আর tcpdump-এর মতো tool শুধু নিজের নেটওয়ার্কে ব্যবহার করুন; অন্যের নেটওয়ার্কের packet ধরা অনধিকার চর্চা ও অনেক ক্ষেত্রে অবৈধ। পরীক্ষার জন্য নিজের VM ও নিজের traffic-ই যথেষ্ট।

৪.১০ Best Practices

- নতুন protocol শিখলে জিজ্ঞেস করুন: এটি কোন TCP/IP স্তরের, আর তার PDU-র নাম কী?
- Packet analysis-এর সময় সবসময় স্তর ধরে পড়ুন: frame → packet → segment → data।
- TTL ও MTU-র মতো সীমাগুলো মনে রাখুন — এগুলোই অনেক "রহস্যময়" সমস্যার কারণ।
- tcpdump চালানোর আগে জানুন কী ধরতে চান; অকারণে সব packet ধরা নিরাপত্তা ঝুঁকি।

Knowledge Check — যাচাই করুন

1. TCP/IP মডেলে মোট কয়টি স্তর আছে? A) 7টি B) 5টি C) 4টি D) 3টি
2. Transport স্তরে TCP-র তথ্যের টুকরোর নাম কী? A) Frame B) Packet C) Segment D) Bit
3. IP header-এ কোন তথ্যটি থাকে? A) MAC address B) Source ও destination IP address C) HTTP method D) পাসওয়ার্ড
4. Router সাধারণত packet-এর কোন স্তর পর্যন্ত খোলে? A) Application পর্যন্ত B) শুধু Link ও Internet স্তর C) শুধু Physical স্তর D) কোনো স্তরই নয়
5. TTL-এর কাজ কী? A) তথ্য encrypt করা B) প্রতিটি router-এ ১ করে কমে packet-এর ঘোরাঘুরি থামানো C) গতি বাড়ানো D) port নম্বর দেওয়া

Practical Assignment

একটি HTTP অনুরোধের encapsulation চিত্র নিজে আঁকুন: প্রতিটি স্তরে কী header যোগ হয়, PDU-র নাম কী, আর কোন যন্ত্র কোন স্তর পর্যন্ত খোলে — তা লিখুন। তারপর `tcpdump` দিয়ে নিজের একটি ping-এর packet ধরে source ও destination IP চিহ্নিত করুন।

Chapter Summary

TCP/IP মডেলের চারটি স্তর: Application, Transport, Internet, Link। প্রতিটি স্তরে তথ্যের টুকরোর নাম আলাদা — data, segment/datagram, packet, frame, bit। Header-এ ঠিকানা ও নিয়ন্ত্রণ তথ্য থাকে, payload-এ আসল তথ্য। Encapsulation-এ প্রতিটি স্তর header জুড়ে দেয়; router শুধু Link ও Internet স্তর খোলে। End-to-end principle-এর কারণে মাঝের যন্ত্রগুলো শুধু পৌঁছে দেয় — আসল কাজ হয় শেষ প্রান্তে।

Revision Checklist

- ☐ TCP/IP-র চারটি স্তরের নাম ও দায়িত্ব বলতে পারি
- ☐ OSI-র সঙ্গে TCP/IP-র মিল ব্যাখ্যা করতে পারি
- ☐ Segment, packet, frame ও bit আলাদা করতে পারি
- ☐ Header ও payload-এর ভূমিকা বলতে পারি
- ☐ Encapsulation ধাপে ধাপে বর্ণনা করতে পারি

অধ্যায় ৫ • Part I — Networking Fundamentals

Chapter 05: Network Devices and Their Responsibilities — নেটওয়ার্ক যন্ত্রপাতি ও তাদের দায়িত্ব

শুরুতে: একটি বাস্তব দৃশ্য

একটি নতুন অফিস সাজানো হচ্ছে। কেনাকাটার তালিকায় আছে: router, switch, access point, firewall, modem। দোকানদার প্রতিটির দাম বলছেন, কিন্তু কেউ বলতে পারছে না কোনটি আসলে কী করে — আর কোনটি না কিনলেও চলবে। শেষে দেখা গেল তারা দুটি router কিনেছে, অথচ switch কিনতেই ভুলে গেছে — ফলে অফিসের কম্পিউটারগুলো একে অপরের সঙ্গে যুক্তই হতে পারছে না। এই অধ্যায়ের লক্ষ্য ঠিক এটাই: প্রতিটি যন্ত্র কী করে, কোন স্তরে কাজ করে, আর packet এক যন্ত্র থেকে আরেক যন্ত্রে কীভাবে যায় — তা পরিষ্কার করে বোঝা।

Learning Objectives — শেখার উদ্দেশ্য

- NIC, repeater, hub, switch, router, access point, firewall, modem ও gateway-এর কাজ বলতে পারবেন।
- Layer 2 ও Layer 3 যন্ত্রের পার্থক্য ব্যাখ্যা করতে পারবেন।
- একটি packet LAN-এর ভেতরে ও LAN-এর বাইরে কীভাবে যায়, তা বর্ণনা করতে পারবেন।
- বাড়ি ও ছোট অফিসের জন্য কোন যন্ত্র দরকার, তা নির্ধারণ করতে পারবেন।
- যন্ত্রের ভুল সাজানো শনাক্ত করতে পারবেন।

Prerequisites — পূর্বজ্ঞান

Chapter 01–04: OSI/TCP/IP স্তর, MAC ও IP address-এর প্রাথমিক ধারণা, frame ও packet। MAC ও IP-র বিস্তারিত Chapter 06–07-এ আসবে; এখানে শুধু ভূমিকাটুকু দরকার।

৫.১ NIC — যন্ত্রের নেটওয়ার্কের দরজা

Network Interface Card (NIC) হলো প্রতিটি যন্ত্রের নেটওয়ার্কের সঙ্গে যুক্ত হওয়ার দরজা। ল্যাপটপের Ethernet port, ফোনের Wi-Fi চিপ — এগুলোই NIC। প্রতিটি NIC-এর একটি স্থায়ী পরিচয় থাকে: **MAC address** (Chapter 06)। NIC দুটি কাজ করে: (১) কম্পিউটারের তথ্যকে frame-এ বদলে কেবল/বেতারে পাঠানো; (২) আসা frame গ্রহণ করে কম্পিউটারকে দেওয়া — তবে শুধু নিজের ঠিকানায় আসা frame (আর broadcast frame) গ্রহণ করে, অন্যের frame ফেলে দেয়। আধুনিক NIC বেশিরভাগ কাজ hardware-এই করে, তাই CPU-র চাপ কমে। একটি যন্ত্রে একাধিক NIC থাকতে পারে — যেমন সার্ভারে দুটি Ethernet port: একটি LAN-এর জন্য, একটি backup-এর জন্য।

৫.২ Repeater ও Hub — পুরনো দিনের যন্ত্র

Repeater শুধু সংকেতকে শক্তিশালী করে আবার পাঠায় — কেবল লম্বা হলে সংকেত দুর্বল হয়ে যায়, repeater তা ঠিক করে। এটি Layer 1-এর যন্ত্র; তথ্যের অর্থ সে বোঝে না। **Hub** হলো অনেক port-ওয়ালা repeater: এক port-এ আসা তথ্য সে সব port-এ পাঠিয়ে দেয়। ফলে সব যন্ত্র সব তথ্য পায় — নিরাপত্তা ও দক্ষতা দুটোই খারাপ। আজ hub প্রায় ব্যবহার হয় না; switch তাকে পুরোপুরি প্রতিস্থাপন করেছে। তবে ধারণাটি জানা দরকার: পুরনো নেটওয়ার্কের চিত্রে hub দেখলে বুঝবেন সেখানে তথ্য সবদিকে ছড়িয়ে যেত।

৫.৩ Switch — LAN-এর হৃদয়

Switch হলো Layer 2-এর যন্ত্র — আধুনিক LAN-এর কেন্দ্র। Hub-এর সঙ্গে পার্থক্য: switch জানে কোন MAC address কোন port-এ আছে (এই তালিকার নাম **MAC address table**, Chapter 06)। তাই তথ্য আসলে switch শুধু গন্তব্যের port-এই পাঠায়, সব port-এ নয়। ফলে দ্রুত, নিরাপদ ও দক্ষ। Switch-এর কাজ ধাপে ধাপে: (১) frame পায়; (২) source MAC দেখে শেখে — এই MAC এই port-এ আছে; (৩) destination MAC তালিকায় খোঁজে; (৪) পেলে শুধু সেই port-এ পাঠায়, না পেলে (অচেনা ঠিকানা) সব port-এ পাঠায় (flooding) — একবার উত্তর এলে তালিকা শিখে নেয়। বাড়ির router-এর ভেতরে আসলে একটি ছোট switch-ই থাকে — তার LAN port-গুলো সেই switch-এর port।

> **CONCEPT:** Switch সিদ্ধান্ত নেয় MAC address দিয়ে, router সিদ্ধান্ত নেয় IP address দিয়ে। Switch ভাবে "এই যন্ত্রটি আমার কোন port-এ?", router ভাবে "এই নেটওয়ার্কে যেতে কোন পথে যাব?"। এই এক বাক্যে দুটি যন্ত্রের পার্থক্য ধরা পড়ে।

৫.৪ Router — নেটওয়ার্কের মধ্যে পথপ্রদর্শক

Router হলো Layer 3-এর যন্ত্র — ভিন্ন ভিন্ন নেটওয়ার্কের মধ্যে packet পৌঁছে দেয়। তার কাজ: (১) packet পায়; (২) IP header-এর destination IP পড়ে; (৩) **routing table**-এ দেখে এই গন্তব্যের নেটওয়ার্কে যেতে কোন পথে (next hop) যেতে হবে; (৪) packet-কে নতুন frame-এ ভরে পরের router বা গন্তব্যের দিকে পাঠায়। লক্ষ্য করুন: router packet-এর ভেতরের TCP/HTTP তথ্য পড়ে না — শুধু IP header দেখে। আপনার বাড়ির router-টি আসলে একসঙ্গে কয়েকটি যন্ত্র: router + switch + Wi-Fi access point + firewall — এক বাক্সে বানানো। বড় নেটওয়ার্কে এগুলো আলাদা আলাদা যন্ত্র থাকে। Router ছাড়া LAN-এর বাইরে যাওয়া যায় না — তাই প্রতিটি যন্ত্রের **default gateway** হলো তার LAN-এর router-এর ঠিকানা (Chapter 12)।

চিত্র ৫.১ — Switch বনাম Router: কে কী দেখে

Switch: Frame-এর destination MAC পড়ে → একই LAN-এর সঠিক port-এ পাঠায়।

Router: Packet-এর destination IP পড়ে → routing table দেখে → অন্য নেটওয়ার্কের দিকে পাঠায়।

একটি packet: PC → Switch (MAC দিয়ে) → Router (IP দিয়ে) → Internet → দূরের Router → দূরের Switch → Server।

৫.৫ Wireless Access Point, Modem ও Gateway

Wireless Access Point (AP) তারযুক্ত নেটওয়ার্ককে বেতার করে — আপনার ফোন Wi-Fi-তে যুক্ত হয় AP-র মাধ্যমে, AP আবার switch/router-এর সঙ্গে তার দিয়ে যুক্ত থাকে। বাড়ির router-এর Wi-Fi অংশটিই একটি AP। **Modem** (modulator-demodulator) এক ধরনের সংকেতকে আরেক ধরনে বদলায় — যেমন টেলিফোন লাইনের সংকেতকে ডিজিটাল তথ্যে, বা ফাইবারের আলোকে Ethernet-এ। আজকাল অনেক বাড়িতে modem ও router এক বাক্সে আসে, তবে কাজ দুটি আলাদা। **Gateway** একটি সাধারণ নাম: যে যন্ত্র একটি নেটওয়ার্ক থেকে আরেক নেটওয়ার্কে যাওয়ার দরজা — সাধারণত router-ই gateway হিসেবে কাজ করে। আর **firewall** হলো পাহারাদার: কোন তথ্য যেতে পারবে, কোনটি নয় — তা নিয়ম অনুযায়ী ঠিক করে (Chapter 18)। Firewall আলাদা যন্ত্র হতে পারে, আবার router-এর ভেতরের সফটওয়্যারও হতে পারে।

৫.৬ Layer 2 বনাম Layer 3 যন্ত্র — এক নজরে

যন্ত্র	স্তর	সিদ্ধান্ত নেয় কী দিয়ে	কাজের এলাকা
NIC	Layer 1–2	নিজের MAC	একটি যন্ত্র
Repeater / Hub	Layer 1	কিছুই না — সবদিকে পাঠায়	সংকেত বাড়ানো
Switch	Layer 2	MAC address	একটি LAN
Access Point	Layer 2	MAC address	বেতার LAN
Router	Layer 3	IP address	নেটওয়ার্কের মধ্যে
Firewall	Layer 3–7	নিয়ম (IP, port, তথ্য)	নেটওয়ার্কের সীমানা
Modem	Layer 1	সংকেতের রূপান্তর	ISP সংযোগ

৫.৭ Packet-এর যাত্রা: এক যন্ত্র থেকে আরেক যন্ত্রে

একটি পূর্ণ যাত্রা ধাপে ধাপে। তানিয়ার ল্যাপটপ (IP 192.168.1.10) সার্ভারে (203.0.113.50) অনুরোধ পাঠাচ্ছে: (১) ল্যাপটপ দেখে গন্তব্য নিজের LAN-এ নেই — তাই default gateway (192.168.1.1, বাড়ির router)-কে পাঠাতে হবে। (২) ল্যাপটপ ARP দিয়ে gateway-র MAC address জানে (Chapter 14)। (৩) Frame বানায়: destination MAC = gateway-র MAC, ভেতরে IP packet: destination IP = 203.0.113.50। (৪) Switch frame-টি পড়ে শুধু router-এর port-এ পাঠায়। (৫) Router frame খুলে IP পড়ে, routing table দেখে ISP-র দিকে পাঠায় — নতুন frame বানিয়ে, যেখানে destination MAC = পরের router-এর MAC। লক্ষ্য করুন: যাত্রাপথে IP ঠিকানা একই থাকে, কিন্তু MAC ঠিকানা প্রতিটি ধাপে বদলায়। এই পার্থক্যটিই MAC ও IP-র ভূমিকা বোঝায় — IP হলো চূড়ান্ত ঠিকানা, MAC হলো পরের ধাপের ঠিকানা। (৬) সার্ভারের LAN-এ পৌঁছে শেষ router frame-টি সার্ভারের MAC-এ পাঠায়; সার্ভার decapsulation করে অনুরোধ পড়ে।

চিত্র ৫.২ — Packet-এর যাত্রা: IP একই থাকে, MAC বদলায়

ল্যাপটপ [IP: 192.168.1.10 → 203.0.113.50 | MAC: ল্যাপটপ → Gateway] → Switch → Router [নতুন Frame: MAC: Router → ISP Router | IP একই] → ... → Server

প্রতিটি hop-এ নতুন frame, কিন্তু IP packet অপরিবর্তিত।

৫.৮ বাস্তব উদাহরণ: ছোট অফিসের যন্ত্রপাতি

দশজন কর্মীর অফিসের জন্য দরকার: (১) একটি Internet সংযোগ (ISP থেকে modem/ONT); (২) একটি router (LAN থেকে Internet-এ যাওয়ার দরজা, firewall-সহ); (৩) একটি switch (সব কম্পিউটার ও প্রিন্টার যুক্ত করার জন্য); (৪) একটি access point (Wi-Fi-র জন্য)। ভুল যা প্রায়ই হয়: router-এর LAN port মাত্র ৪টি — দশটি কম্পিউটারের জন্য switch ছাড়া চলবে না। আরেকটি ভুল: switch-এর বদলে আরেকটি router কিনে LAN-কে দুই ভাগ করে ফেলা — তখন দুই ভাগের কম্পিউটার একে অপরকে দেখতে পায় না। মনে রাখুন: LAN বাড়াতে switch কিনুন, নেটওয়ার্ক ভাগ করতে বা Internet-এ যেতে router কিনুন।

REAL WORLD: একটি ঢাকার কো-ওয়ার্কিং স্পেসে শতাধিক ব্যবহারকারী একই Wi-Fi-তে কাজ করতেন। একদিন একজন ব্যবহারকারীর ল্যাপটপে ভাইরাস ঢুকে পুরো নেটওয়ার্কে ছড়িয়ে পড়ল — কারণ সবাই একটিমাত্র flat LAN-এ ছিল, কোনো segmentation ছিল না, আর switch-টি ছিল unmanaged (VLAN সুবিধা নেই)। সমাধান: managed switch দিয়ে VLAN-এ ভাগ করা ও client isolation চালু করা — যাতে এক ব্যবহারকারীর যন্ত্র আরেক ব্যবহারকারীর যন্ত্রকে সরাসরি দেখতে না পায়। যন্ত্র কেনার সময় "managed" বনাম "unmanaged" পার্থক্যটি তাই গুরুত্বপূর্ণ।

৫.৯ হাতে-কলমে অনুশীলন: নিজের পথের যন্ত্র চেনা

```
$ ip route show
# default via 192.168.1.1 dev eth0 — এটিই আপনার gateway (router)।

$ arp -n
# arp: আপনার LAN-এর পরিচিত যন্ত্রগুলোর IP ও MAC address-এর তালিকা দেখায়।
# এখানে gateway-র MAC address-ও দেখা যাবে।

$ traceroute 8.8.8.8
# traceroute: আপনার packet কোন কোন router (hop) পার হয়ে যাচ্ছে, তা দেখায়।
# প্রথম hop-টি আপনার gateway; তারপর ISP-র router-গুলো।
# traceroute না থাকলে: sudo apt update && sudo apt install -y traceroute

$ ip neigh show
# আধুনিক বিকল্প: প্রতিবেশী যন্ত্রের IP–MAC তালিকা (arp-এর মতো)।
```

উদাহরণ আউটপুট (illustrative):

```
$ traceroute 8.8.8.8
 1  192.168.1.1  2.1 ms
 2  103.86.112.1  8.4 ms
 3  8.8.8.8  32.6 ms
```

প্রথম hop (192.168.1.1) আপনার বাড়ির router, দ্বিতীয়টি ISP-র router, তৃতীয়টি গন্তব্য। Hop সংখ্যা ও সময় থেকে বোঝা যায় পথটি কেমন।

LAB: (১) `ip route show` দিয়ে নিজের gateway লিখুন। (২) `arp -n` বা `ip neigh show` দিয়ে gateway-র MAC address লিখুন। (৩) `traceroute 8.8.8.8` চালিয়ে hop-গুলো লিখুন। যাচাই: প্রথম hop-টি gateway-র IP-র সঙ্গে মিলছে কি না। সতর্কতা: traceroute শুধু নিজের সংযোগ পরীক্ষায় ব্যবহার করুন; অন্যের নেটওয়ার্ক স্ক্যান করবেন না।

৫.১০ Common Mistakes — সাধারণ ভুল

- **Switch আর router-কে এক মনে করা:** Switch LAN-এর ভেতরে MAC দিয়ে পাঠায়; router নেটওয়ার্কের মধ্যে IP দিয়ে পাঠায়।
- **Router-এর LAN port দিয়ে অফিস বাড়ানো:** ৪টি port শেষ হলে switch কিনুন, আরেকটি router নয়।
- **Modem আর router গুলিয়ে ফেলা:** Modem ISP-র সংকেত বদলায়; router LAN ও Internet-এর মধ্যে পথ ঠিক করে।

- **Hub-এর যুগের ধারণা ধরে রাখা:** আধুনিক নেটওয়ার্কে hub নেই; তথ্য সব port-এ যায় না, শুধু গন্তব্যে যায়।

৫.১১ Troubleshooting — সমস্যা নির্ণয়

একটি PC Internet-এ যেতে পারছে না, বাকিরা পারছে: তার gateway সেটিং (`ip route`) ও কেবল/switch port দেখুন। সবাই পারছে না: router ও modem-এর বাতি, ISP লাইন দেখুন। একই LAN-এর PC-গুলো একে অপরকে দেখতে পাচ্ছে না: switch, VLAN সেটিং, host firewall দেখুন — router-এর দোষ নয়, কারণ LAN-এর ভেতরের যোগাযোগে router লাগে না। `traceroute` -এ প্রথম hop-এই থেমে গেলে সমস্যা gateway-তে; মাঝখানে থেমে গেলে ISP-র পথে।

৫.১২ Security Considerations

Switch-এর অপরিচিত port বন্ধ রাখুন — কেউ এসে কেবল লাগিয়ে LAN-এ ঢুকতে না পারে। Router-এর admin প্যানেলের ডিফল্ট পাসওয়ার্ড বদলান (এই বইয়ে কোনো আসল পাসওয়ার্ড লেখা হয়নি — নিজের শক্ত পাসওয়ার্ড নিজে তৈরি করুন)। Wi-Fi-তে WPA2/WPA3 ব্যবহার করুন; খোলা Wi-Fi কখনও অফিসের internal network-এর সঙ্গে যুক্ত করবেন না। Firewall-এর নিয়ম শুরুতে কঠোর রাখুন, প্রয়োজনমতো খুলুন — উল্টোটা নয়।

৫.১৩ Best Practices

- প্রতিটি যন্ত্রের ভূমিকা ও IP ঠিকানা একটি চিত্রে লিখে রাখুন।
- LAN বাড়াতে switch, নেটওয়ার্কের মধ্যে যেতে router — এই নিয়ম মনে রাখুন।
- গুরুত্বপূর্ণ switch ও router-এর জন্য বিকল্প (redundancy) পরিকল্পনা করুন।
- যন্ত্র কেনার আগে জানুন সেটি managed না unmanaged, আর কোন স্তরে কাজ করে।

Knowledge Check — যাচাই করুন

1. Switch সিদ্ধান্ত নেয় কী দিয়ে? A) IP address B) MAC address C) Port নম্বর D) পাসওয়ার্ড
2. Router-এর প্রধান কাজ কী? A) একই LAN-এর যন্ত্র যুক্ত করা B) ভিন্ন নেটওয়ার্কের মধ্যে IP address দিয়ে packet পৌঁছে দেওয়া C) Wi-Fi সংকেত দেওয়া D) ফাইল সংরক্ষণ করা
3. Packet-এর যাত্রায় কোনটি অপরিবর্তিত থাকে? A) MAC address B) IP address (source ও destination) C) Frame-এর আকার D) TTL
4. Hub ও switch-এর প্রধান পার্থক্য কী? A) Hub দ্রুত B) Hub সব port-এ পাঠায়, switch শুধু গন্তব্যের port-এ পাঠায় C) Hub নতুন প্রযুক্তি D) কোনো পার্থক্য নেই
5. একটি PC Internet-এ যেতে না পারলেও একই LAN-এর PC-তে ping যায় — সমস্যা সম্ভবত কোথায়? A) Switch-এ B) Gateway/router বা তার বাইরের পথে C) NIC-এ D) মনিটরে

Practical Assignment

আপনার বাড়ি বা হোস্টেলের নেটওয়ার্কের যন্ত্রগুলোর তালিকা করুন: modem/ONT, router, switch (যদি থাকে), access point। প্রতিটি কোন স্তরে কাজ করে, কী দিয়ে সিদ্ধান্ত নেয়, আর `traceroute` -এর প্রথম দুটি hop কোন যন্ত্র — তা লিখুন। তারপর ১৫ জনের একটি অফিসের জন্য কেনাকাটার তালিকা বানান (যন্ত্রের নাম, সংখ্যা, কেন দরকার)।

Chapter Summary

NIC যন্ত্রকে নেটওয়ার্কে যুক্ত করে; repeater/hub পুরনো Layer 1 যন্ত্র; switch Layer 2-এ MAC দিয়ে LAN-এর ভেতরে frame পৌঁছে দেয়; router Layer 3-এ IP দিয়ে নেটওয়ার্কের মধ্যে packet পৌঁছে দেয়; access point বেতার সংযোগ দেয়; modem সংকেত বদলায়; firewall নিয়ম অনুযায়ী তথ্য আটকায় বা ছাড়ে। Packet-এর যাত্রায় IP ঠিকানা একই থাকে, MAC ঠিকানা প্রতিটি hop-এ বদলায়। সঠিক যন্ত্র সঠিক জায়গায় বসানোই ভালো নেটওয়ার্ক নকশার শুরু।

Revision Checklist

- ☐ প্রতিটি যন্ত্রের কাজ ও স্তর বলতে পারি
- ☐ Switch ও router-এর পার্থক্য ব্যাখ্যা করতে পারি
- ☐ Packet-এর যাত্রায় IP ও MAC-এর ভূমিকা বলতে পারি
- ☐ traceroute, arp ও ip route চালিয়ে ফল ব্যাখ্যা করতে পারি
- ☐ ছোট অফিসের জন্য যন্ত্রের তালিকা বানাতে পারি

PART II • DEVOPS ENGINEERING MASTERY

Addressing and Transport

অ্যাড্রেসিং ও ট্রান্সপোর্ট — MAC, IPv4, সাবনেটিং, IPv6, TCP ও UDP।

Chapter 6: MAC Addresses, Ethernet and Switching — ম্যাক ঠিকানা, ইথারনেট ও সুইচিং

অধ্যায় ৬ | DevOps Engineering Mastery Vol 02 — Networking and Internet Infrastructure | Ubuntu Server 24.04 LTS | প্রকাশ: ৮ নভেম্বর ২০২৬

শুরুতে: একই ঘরে দুটি কম্পিউটার, তবু কথা হয় কীভাবে

ঢাকার একটি ছোট অফিসে দশটি কম্পিউটার, একটি প্রিন্টার আর একটি রাউটার একটি ছোট বাক্সের সঙ্গে তার দিয়ে যুক্ত, যার নাম সুইচ। একজন কর্মী প্রিন্টারে একটি ফাইল পাঠাতে চান। প্রিন্টার কীভাবে বুঝবে, এই ফাইলটি তার জন্য, পাশের কম্পিউটারের জন্য নয়? এই অধ্যায়ের প্রশ্ন এটাই: একটি স্থানীয় নেটওয়ার্কের ভেতরে একটি যন্ত্র আরেকটি যন্ত্রকে কীভাবে চিনে, আর তথ্য কীভাবে সঠিক গন্তব্যে পৌঁছায়? একটি চিঠির উদাহরণ ভাবুন। খামের ওপর বাড়ির ঠিকানা লেখা থাকে, আর ভবনের ভেতরে দরকার হয় ফ্ল্যাট নম্বর। আইপি ঠিকানা অনেকটা বাড়ির ঠিকানার মতো, আর ম্যাক ঠিকানা অনেকটা ফ্ল্যাটের দরজায় লাগানো নম্বর প্লেটের মতো। দুটোই দরকার, তবে কাজ আলাদা। এই অধ্যায়ে আমরা শিখব ম্যাক ঠিকানার গঠন, ইথারনেট ফ্রেমের ভেতরে কী থাকে, আর সুইচ কীভাবে প্রতিটি ফ্রেমকে শুধু সঠিক পোর্টে পাঠায়।

শেখার উদ্দেশ্য — Learning Objectives

এই অধ্যায় শেষে আপনি পারবেন:

- ম্যাক ঠিকানা কী, কেন দরকার, এবং এটি আইপি ঠিকানা থেকে কীভাবে আলাদা, তা ব্যাখ্যা করতে।
- ৪৮ বিটের ম্যাক ঠিকানার গঠন পড়তে এবং unicast, multicast ও broadcast চিনতে।
- ইথারনেট ফ্রেমের প্রধান ক্ষেত্রগুলো চিহ্নিত করতে এবং ফ্রেম কীভাবে তৈরি হয়, তা বলতে।
- সুইচের ম্যাক টেবিল কীভাবে শেখে, ফরওয়ার্ড করে ও ফ্লাড করে, তা ব্যাখ্যা করতে।
- উবুন্টু ২৪.০৪-এ ip link, ip -br link, ip addr ও ip neigh চালিয়ে ম্যাক ও প্রতিবেশীর তথ্য পড়তে।
- ল্যানে যন্ত্র না পাওয়া গেলে ম্যাক, তার, পোর্ট ও এআরপি স্তরে সমস্যা খুঁজতে।

পূর্বশর্ত — Prerequisites

ভলিউম ০১-এর লিনাক্স জ্ঞান দরকার: টার্মিনালে লগইন, সাধারণ কমান্ড, sudo ব্যবহার এবং ip কমান্ডের মৌলিক ধারণা। আইপি সম্পর্কে হালকা ধারণা থাকলে ভালো, তবে না থাকলেও চলবে, কারণ পরের অধ্যায়ে আইপি নিয়ে বিস্তারিত আলোচনা আছে। একটি উবুন্টু সার্ভার ২৪.০৪ ল্যাব যন্ত্র থাকলে অনুশীলন সরাসরি করা যাবে।

প্রথম নীতি: পরিচয় ছাড়া যোগাযোগ হয় না

যোগাযোগের প্রথম শর্ত পরিচয়। একটি নেটওয়ার্কে অনেক যন্ত্র একই মাধ্যম ব্যবহার করে, তাই প্রতিটি তথ্যের গায়ে লিখে দিতে হয়, এটি কার জন্য। সেই লেখাটাই ঠিকানা। ঠিকানাটি প্রতিটি যন্ত্রের জন্য আলাদা হতে হবে, যন্ত্রের সঙ্গেই থাকতে হবে, আর এমনভাবে সাজাতে হবে যাতে যন্ত্র দ্রুত বুঝতে পারে, এই তথ্য আমার জন্য কি না। ম্যাক ঠিকানা এই তিনটি শর্ত পূরণের জন্য তৈরি। ম্যাক শব্দের পুরো অর্থ Media Access Control। এটি নেটওয়ার্ক ইন্টারফেস কার্ডের সঙ্গে যুক্ত পরিচয়। একটি কম্পিউটারে একাধিক ইন্টারফেস থাকতে পারে, যেমন তারের ইন্টারফেস ও ওয়াইফাই ইন্টারফেস। প্রতিটি ইন্টারফেসের আলাদা ম্যাক ঠিকানা থাকে। অর্থাৎ ম্যাক ঠিকানা কম্পিউটারের নয়, ইন্টারফেসের পরিচয়।

ম্যাক ঠিকানা ৪৮ বিট লম্বা। মানুষের পড়ার সুবিধার জন্য এই ৪৮ বিটকে ছয়টি ভাগে ভাগ করা হয়, প্রতিটি ভাগে আট বিট। প্রতিটি আট বিটকে হেক্সাডেসিমাল সংখ্যায় লেখা হয়, দুটি অঙ্ক দিয়ে। ছয়টি ভাগকে কোলন বা হাইফেন দিয়ে আলাদা করা হয়। উদাহরণ: 52:54:00:12:34:56। প্রথম তিনটি বাইট, অর্থাৎ প্রথম ২৪ বিট, প্রস্তুতকারক প্রতিষ্ঠানকে চেনায়। একে বলা হয় OUI, অর্থাৎ Organizationally Unique Identifier। পরের তিনটি বাইট, অর্থাৎ শেষ ২৪ বিট, সেই প্রতিষ্ঠানের তৈরি যন্ত্রগুলোর মধ্যে একটি নির্দিষ্ট যন্ত্রকে চেনায়। বাস্তবে প্রস্তুতকারক ভুলে একই ঠিকানা দুবার দিতে পারে, আর ভার্চুয়াল মেশিনে ম্যাক ঠিকানা বদলানোও যায়। তাই ম্যাক ঠিকানাকে একেবারে অপরিবর্তনীয় সত্য মনে করা ভুল।

> মূল ধারণা: ম্যাক ঠিকানা হলো স্থানীয় নেটওয়ার্কে একটি ইন্টারফেসের ভৌত পরিচয়। এটি ৪৮ বিটের, প্রথম ২৪ বিট প্রস্তুতকারককে চেনায়, শেষ ২৪ বিট যন্ত্রটিকে আলাদা করে। আইপি ঠিকানা নেটওয়ার্ক বদলালে বদলাতে পারে, কিন্তু ম্যাক ঠিকানা ইন্টারফেসের সঙ্গেই থাকে।

ইথারনেট: একটি নিয়মের বই

ইথারনেট হলো স্থানীয় নেটওয়ার্কে তথ্য পাঠানোর নিয়মের সংগ্রহ। তারের ভেতর দিয়ে বিদ্যুৎ সংকেত যায়, কিন্তু সেই সংকেতের অর্থ কী, কোথায় শুরু, কোথায় শেষ, কে পাঠিয়েছে, কে পাবে — এই সবকিছু ঠিক করে ইথারনেট। ইথারনেটের সবচেয়ে গুরুত্বপূর্ণ ধারণা হলো ফ্রেম। ফ্রেম হলো তথ্যের একটি খাম। একটি ফ্রেমের শুরুতে থাকে গন্তব্য ম্যাক ঠিকানা, তারপর উৎস ম্যাক ঠিকানা, তারপর একটি ক্ষেত্র যা বলে, ভেতরে কোন ধরনের তথ্য আছে, যেমন আইপি তথ্য। তারপর আসল তথ্য, আর শেষে একটি পরীক্ষার মান, যা দিয়ে গ্রহণকারী যন্ত্র বুঝতে পারে, পথে তথ্য নষ্ট হয়েছে কি না। সুইচ শুধু ফ্রেমের শুরুর অংশটি পড়েই সিদ্ধান্ত নেয়। সে ভেতরের তথ্য খুলে দেখে না। সে শুধু দেখে, এই ফ্রেমটি কোন ম্যাক ঠিকানার জন্য, আর সেই ম্যাকের যন্ত্র কোন পোর্টে আছে।

পুরোনো ইথারনেটে সব যন্ত্র একই তার ভাগ করে নিত, আর কেউ কথা বলার সময় অন্যরা চুপ থাকত। আধুনিক সুইচযুক্ত নেটওয়ার্কে সেই সমস্যা অনেকটাই কমে গেছে, কারণ সুইচ প্রতিটি পোর্টকে আলাদা পথ দেয়। গতি বাড়লেও ফ্রেমের গঠন ও ম্যাক ঠিকানার কাজ বদলায়নি। পুরোনো ইথারনেট ১০ মেগাবিট প্রতি সেকেন্ডে চলত, এখন ১ গিগাবিট বা তার বেশি গতি সাধারণ। ছোট অফিস, বিশ্ববিদ্যালয়ের ল্যাব ও ডেটা সেন্টার — সব জায়গায় একই ম্যাক নিয়ম কাজ করে।

সুইচ কীভাবে কাজ করে: ভেতরের হাঁটাচলা

সুইচকে একটি ডাকঘরের বাছাই ঘরের সঙ্গে তুলনা করা যায়। ডাকঘরে চিঠি এলে কর্মী খামের ওপরের ঠিকানা পড়ে সঠিক থলিতে ফেলেন। সুইচও ঠিক তাই করে, তবে সে ঠিকানা মনে রাখার জন্য একটি টেবিল ব্যবহার করে। এই টেবিলের নাম ম্যাক ঠিকানা টেবিল, বা CAM টেবিল। সুইচ যখন প্রথম চালু হয়, তখন তার টেবিল খালি থাকে। কোনো ফ্রেম যখন একটি পোর্ট দিয়ে ঢোকে, সুইচ সেই ফ্রেমের উৎস ম্যাক ঠিকানাটি পড়ে, আর মনে রাখে, এই ম্যাকের যন্ত্র এই পোর্টে আছে। তারপর সে গন্তব্য ম্যাকটি দেখে। গন্তব্য টেবিলে থাকলে সে ফ্রেমটি শুধু সেই পোর্টে পাঠায়। একে বলা হয় ফরওয়ার্ডিং। গন্তব্য টেবিলে না থাকলে সে ফ্রেমটি সব পোর্টে পাঠিয়ে দেয়, যাতে সঠিক যন্ত্রটি নিজে থেকে সাড়া দেয়। একে বলা হয় ফ্লাডিং। ফ্লাডিং শুনতে অপচয় মনে হলেও এটি শেখার উপায়। সঠিক যন্ত্রটি যখন উত্তর পাঠায়, তখন সুইচ তার উৎস ম্যাক থেকে শিখে নেয়, সে কোন পোর্টে আছে। পরের বার আর ফ্লাড করতে হয় না।

উদাহরণ: ক পোর্ট একে, খ পোর্ট দুইয়ে, গ পোর্ট তিনে, ঘ পোর্ট চারে। প্রথমে ক খ-কে ফ্রেম পাঠায়। সুইচ ফ্রেমটি পোর্ট একে পেয়ে শেখে, ক আছে পোর্ট একে। গন্তব্য খ টেবিলে নেই, তাই সে ফ্রেমটি পোর্ট দুই, তিন ও চারে পাঠায়। গ ও ঘ দেখে, গন্তব্য তাদের নয়, তাই চুপ থাকে। খ উত্তর পাঠায়। উত্তরের ফ্রেমটি পোর্ট দুই দিয়ে ঢোকে, সুইচ শেখে, খ আছে পোর্ট দুইয়ে। এবার গন্তব্য ক টেবিলে আছে, তাই উত্তরটি শুধু পোর্ট একে যায়। এভাবেই কয়েকটি ফ্রেমের পর টেবিলটি পূর্ণ হয়।

চিত্র ১: সুইচের ম্যাক টেবিল শেখা

পোর্ট ১ — কম্পিউটার ক — ম্যাক ক | পোর্ট ২ — কম্পিউটার খ — ম্যাক খ | পোর্ট ৩ — কম্পিউটার গ — ম্যাক গ | পোর্ট ৪ — কম্পিউটার ঘ — ম্যাক ঘ

ধাপ ১: ক থেকে ফ্রেম আসে, সুইচ শেখে ক আছে পোর্ট ১-এ। ধাপ ২: খ টেবিলে নেই, তাই ফ্রেম সব পোর্টে যায়। ধাপ ৩: খ উত্তর দেয়, সুইচ শেখে খ আছে পোর্ট ২-এ। ধাপ ৪: এরপর ক ও খ-এর মধ্যে ফ্রেম শুধু পোর্ট ১ ও ২-এর মধ্যে চলে।

পরিভাষার তালিকা

পরিভাষা	ইংরেজি	অর্থ
ম্যাক ঠিকানা	MAC address	৪৮ বিটের ভৌত পরিচয়, প্রতিটি নেটওয়ার্ক ইন্টারফেসের জন্য আলাদা।
ইথারনেট	Ethernet	স্থানীয় নেটওয়ার্কে ফ্রেম পাঠানোর নিয়মের সংগ্রহ।
ফ্রেম	Frame	ম্যাক ঠিকানাসহ তথ্যের খাম, যা তারে চলে।
সুইচ	Switch	ম্যাক ঠিকানা দেখে ফ্রেম সঠিক পোর্টে পাঠানো যন্ত্র।
ম্যাক টেবিল	MAC address table	কোন ম্যাক কোন পোর্টে আছে, তার তালিকা।
ব্রডকাস্ট	Broadcast	নেটওয়ার্কের সব যন্ত্রের জন্য পাঠানো ফ্রেম, গন্তব্য ff:ff:ff:ff:ff:ff।
ইউনিকাস্ট	Unicast	একটি নির্দিষ্ট যন্ত্রের জন্য পাঠানো ফ্রেম।
মাল্টিকাস্ট	Multicast	একটি নির্দিষ্ট দলের যন্ত্রের জন্য পাঠানো ফ্রেম।
ফ্লাডিং	Flooding	গন্তব্য অজানা হলে সব পোর্টে ফ্রেম পাঠানো।
ওইউআই	OUI	ম্যাক ঠিকানার প্রথম ২৪ বিট, প্রস্তুতকারক প্রতিষ্ঠানের পরিচয়।
ইন্টারফেস	Interface	কম্পিউটারের নেটওয়ার্ক সংযোগের মুখ, যেমন eth0 বা enp0s3।
এআরপি	ARP	আইপি ঠিকানা থেকে ম্যাক ঠিকানা খুঁজে বের করার নিয়ম।

চিত্র ২: একটি ইথারনেট ফ্রেমের ভেতরের সাজানো অংশ

গন্তব্য ম্যাক ঠিকানা — ৬ বাইট | উৎস ম্যাক ঠিকানা — ৬ বাইট | ধরন — ২ বাইট | তথ্য — ৪৬ থেকে ১৫০০ বাইট | পরীক্ষার মান — ৪ বাইট

পাঠের নিয়ম: সুইচ প্রথমে গন্তব্য ম্যাক পড়ে। সে তথ্যের ভেতর ঢোকে না। গন্তব্য জানা থাকলে শুধু সেই পোর্টে পাঠায়, অজানা থাকলে সব পোর্টে পাঠায়।

বাস্তব পরিস্থিতি

প্রথম পরিস্থিতি: একটি বিশ্ববিদ্যালয়ের ল্যাবে ত্রিশটি কম্পিউটার একটি সুইচে যুক্ত। ম্যাক টেবিল না থাকলে প্রতিটি ফ্রেম সব কম্পিউটারে যেত, নেটওয়ার্ক ধীর হয়ে যেত, আর সবাই সবার তথ্য দেখতে পেত। ম্যাক টেবিলের কারণে প্রতিটি ফ্রেম শুধু

সঠিক কম্পিউটারে যায়। দ্বিতীয় পরিস্থিতি: একটি ইন্টারনেট সেবাদাতা প্রতিষ্ঠান নতুন গ্রাহকের বাড়িতে রাউটার বসায়। প্রথম কয়েকটি ফ্রেমে সুইচ শিখে নেয়, গ্রাহকের কম্পিউটার কোন পোর্টে আছে। তৃতীয় পরিস্থিতি: একজন কর্মী তারের সংযোগ খুলে ওয়াইফাইয়ে যুক্ত হন। তারের ম্যাক আর ওয়াইফাইয়ের ম্যাক আলাদা, তাই নেটওয়ার্ক তাঁকে দুটি আলাদা পরিচয়ে দেখে। আইপি একই থাকতে পারে, কিন্তু ম্যাক বদলে যায়। তাই সমস্যা খুঁজতে গেলে শুধু আইপি দেখলেই হয় না, ম্যাকও দেখতে হয়।

কাজের উদাহরণ

উদাহরণ এক: ম্যাক ঠিকানা 52:54:00:12:34:56-এ প্রথম তিনটি বাইট 52:54:00 হলো প্রস্তুতকারকের অংশ, আর শেষ তিনটি বাইট 12:34:56 হলো যন্ত্রের নিজস্ব অংশ। উদাহরণ দুই: গন্তব্য ম্যাক ff:ff:ff:ff:ff:ff দেখলে বুঝতে হবে, এটি ব্রডকাস্ট। এই ফ্রেমটি নেটওয়ার্কের সব যন্ত্রের জন্য। সুইচ এই ফ্রেমকে সব পোর্টে পাঠায়। এআরপি অনুরোধে এই ঠিকানা ব্যবহার হয়। উদাহরণ তিন: সুইচের টেবিলে লেখা আছে, ম্যাক ক আছে পোর্ট ১-এ, ম্যাক খ আছে পোর্ট ২-এ। এখন ক থেকে খ-এর জন্য ফ্রেম এলে সুইচ শুধু পোর্ট ২-এ পাঠাবে। পোর্ট ৩ ও ৪-এর যন্ত্র কিছুই দেখবে না। উদাহরণ চার: একটি যন্ত্রের ম্যাক বদলে গেছে, কারণ সে ভার্চুয়াল মেশিনে নতুন ইন্টারফেস পেয়েছে। পুরোনো ম্যাক টেবিলে পুরোনো ঠিকানা কিছুক্ষণ থেকে যেতে পারে। তখন ফ্রেম ভুল পোর্টে যেতে পারে। টেবিলের পুরোনো লেখা মুছে গেলে বা নতুন ফ্রেম এলে টেবিল আবার শিখে নেয়।

উবুন্টুতে কমান্ড: নিজের ম্যাক ঠিকানা দেখা

উবুন্টু সার্ভার ২৪.০৪-এ নেটওয়ার্ক দেখার প্রধান কমান্ড ip। আগের ifconfig কমান্ড এখন আর সাধারণভাবে ব্যবহৃত হয় না। নিচের কমান্ডগুলো নিজের সার্ভারে চালান। প্রথম কমান্ডে সব ইন্টারফেসের সংক্ষিপ্ত তালিকা, দ্বিতীয় কমান্ডে বিস্তারিত তথ্য, তৃতীয় কমান্ডে আইপি ঠিকানা, আর চতুর্থ কমান্ডে প্রতিবেশী যন্ত্রের ম্যাক ঠিকানা পাবেন।

```
ip -br link
ip link
ip addr
ip neigh
arp -n
```

ip -br link চালালে প্রতিটি ইন্টারফেস এক লাইনে দেখা যায়। সেখানে ইন্টারফেসের নাম, অবস্থা ও ম্যাক ঠিকানা থাকে। ip link চালালে আরও বিস্তারিত তথ্য পাওয়া যায়, যেমন ইন্টারফেসটি চালু আছে কি না। ip addr চালালে একই ইন্টারফেসের আইপি ঠিকানাও দেখা যায়। ফলে ম্যাক ও আইপি একসঙ্গে মিলিয়ে দেখা যায়। ip neigh চালালে প্রতিবেশী যন্ত্রের তালিকা পাওয়া যায়, যেখানে আইপি ঠিকানার পাশে ম্যাক ঠিকানা লেখা থাকে। পুরোনো arp -n কমান্ডও একই তালিকা দেখায়, তবে আধুনিক উবুন্টুতে ip neigh-ই প্রধান উপায়। এই অধ্যায়ের কমান্ডগুলো শুধু দেখার জন্য, বদলানোর জন্য নয়।

নির্দেশিত অনুশীলন

> ল্যাব: নিজের সার্ভারের ম্যাক ঠিকানা ও প্রতিবেশী তালিকা তৈরি করুন। ধাপ ১: টার্মিনালে ip -br link চালিয়ে সব ইন্টারফেসের নাম লিখে নিন। ধাপ ২: প্রধান ইন্টারফেসের ম্যাক ঠিকানাটি আলাদা করে লিখুন। ধাপ ৩: ip addr চালিয়ে সেই ইন্টারফেসের আইপি ঠিকানাটি পাশে লিখুন। ধাপ ৪: ip neigh চালিয়ে প্রতিবেশী যন্ত্রের আইপি ও ম্যাকের জোড়া লিখুন। ধাপ ৫: একটি প্রতিবেশী আইপিতে ping চালিয়ে দেখুন, ip neigh তালিকায় নতুন কিছু যুক্ত হয় কি না।

উদাহরণ আউটপুট (illustrative)

```
lo               UNKNOWN        00:00:00:00:00:00
enp0s3           UP             52:54:00:12:34:56
192.168.1.10 dev enp0s3 lladdr 52:54:00:aa:bb:cc REACHABLE
```

উপরের উদাহরণ আউটপুটে enp0s3 ইন্টারফেসটি চালু আছে, তার ম্যাক 52:54:00:12:34:56, আর প্রতিবেশী 192.168.1.10-এর ম্যাক 52:54:00:aa:bb:cc। আপনার আসল আউটপুটে নাম, সংখ্যা ও ম্যাক আলাদা হবে। গুরুত্বপূর্ণ বিষয় হলো, কোন কলামে কী আছে, তা চিনতে পারা। lo হলো লুপব্যাক ইন্টারফেস, যা নিজের ভেতরে কথা বলার জন্য ব্যবহৃত হয়। REACHABLE শব্দের অর্থ, প্রতিবেশী যন্ত্রটি সম্প্রতি সাড়া দিয়েছে। STALE বা FAILED লেখা থাকলে তার অর্থ আলাদা, যা সমস্যা সমাধানের অংশে ব্যাখ্যা করা আছে।

সাধারণ ভুল

প্রথম ভুল: ম্যাক ঠিকানাকে কম্পিউটারের ঠিকানা মনে করা। আসলে এটি ইন্টারফেসের ঠিকানা। দ্বিতীয় ভুল: ম্যাক ঠিকানাকে চিরস্থায়ী মনে করা। ভার্চুয়াল মেশিন বা নতুন কার্ডে ম্যাক বদলানো যায়। তৃতীয় ভুল: সুইচ ও হাবকে এক মনে করা। হাব সব ফ্রেম সব পোর্টে পাঠায়, সুইচ শুধু সঠিক পোর্টে পাঠায়। চতুর্থ ভুল: ব্রডকাস্ট ঠিকানাকে সাধারণ ঠিকানা মনে করা। ff:ff:ff:ff:ff:ff মানে সবাই, একটি নির্দিষ্ট যন্ত্র নয়। পঞ্চম ভুল: ip neigh তালিকা খালি দেখে ভাবা, নেটওয়ার্ক নষ্ট। আসলে তালিকাটি তখনই তৈরি হয়, যখন কোনো যন্ত্রের সঙ্গে কথা হয়। ষষ্ঠ ভুল: ম্যাক ঠিকানা দেখে আইপি ঠিকানা অনুমান করা। দুটি ঠিকানার মধ্যে কোনো গাণিতিক সম্পর্ক নেই।

সমস্যা সমাধান — Troubleshooting

সমস্যা: ip -br link-এ ইন্টারফেস DOWN দেখাচ্ছে। পরীক্ষা: তার ঠিকমতো লাগানো আছে কি না, সুইচের পোর্টে আলো জ্বলছে কি না, আর ভার্চুয়াল মেশিনে নেটওয়ার্ক যুক্ত আছে কি না, দেখুন। সমস্যা: ম্যাক ঠিকানা আছে, কিন্তু ip neigh খালি। পরীক্ষা: গেটওয়ে বা প্রতিবেশী আইপিতে ping চালান, তারপর আবার ip neigh দেখুন। সমস্যা: প্রতিবেশী FAILED দেখাচ্ছে। পরীক্ষা: সেই যন্ত্রটি চালু আছে কি না, একই নেটওয়ার্কে আছে কি না, আর তার ফায়ারওয়াল ping বন্ধ করেছে কি না, দেখুন। সমস্যা: একই ম্যাক দুটি যন্ত্রে দেখা যাচ্ছে। পরীক্ষা: ভার্চুয়াল মেশিনে ম্যাক ঠিকানা হাতে বসানো হয়েছে কি না, দেখুন। একই ম্যাক দুবার থাকলে সুইচের টেবিল বারবার বদলায়, আর সংযোগ অস্থির হয়।

নিরাপত্তা বিবেচনা

ম্যাক ঠিকানা কোনো গোপন তথ্য নয়, কারণ স্থানীয় নেটওয়ার্কে সব যন্ত্র এটি দেখতে পায়। তবু এটি পরিচয় যাচাইয়ের একমাত্র উপায় নয়। কেউ ইচ্ছা করলে নিজের ম্যাক বদলে অন্য যন্ত্রের ম্যাক ব্যবহার করতে পারে। একে বলা হয় ম্যাক স্পুফিং। এর ফলে সুইচ ভুল পোর্টে ফ্রেম পাঠাতে পারে। সুইচে পোর্ট নিরাপত্তা নামে একটি ব্যবস্থা থাকে, যা একটি পোর্টে শুধু নির্দিষ্ট ম্যাককে অনুমতি দেয়। কিন্তু সেটিও সম্পূর্ণ নিরাপত্তা দেয় না, কারণ ম্যাক নকল করা যায়। আরেকটি ঝুঁকি হলো, কেউ অনেক ভুয়া ম্যাক পাঠিয়ে সুইচের টেবিল ভরে ফেলতে পারে। তখন সুইচ আর নতুন ম্যাক শিখতে পারে না, আর সব ফ্রেম ফ্লাড করতে শুরু করে। একে বলা হয় ম্যাক ফ্লাডিং।

সেরা অনুশীলন — Best Practices

প্রথমত, নেটওয়ার্কের নথিতে প্রতিটি গুরুত্বপূর্ণ যন্ত্রের ইন্টারফেসের নাম, ম্যাক ঠিকানা ও আইপি ঠিকানা একসঙ্গে লিখে রাখুন। দ্বিতীয়ত, ভার্চুয়াল মেশিনে ম্যাক ঠিকানা হাতে বসাবেন না, যদি না বিশেষ দরকার থাকে। তৃতীয়ত, সমস্যা খুঁজতে গেলে

ধাপে ধাপে যান: প্রথমে তার ও পোর্ট, তারপর ip link-এ ইন্টারফেসের অবস্থা, তারপর ip neigh-এ প্রতিবেশী। চতুর্থত, সুইচের পোর্টে অপরিচিত যন্ত্র যুক্ত হতে দেবেন না। পঞ্চমত, ব্রডকাস্ট বেশি হলে তার কারণ খুঁজুন।

জ্ঞান যাচাই — Knowledge Check

1. ম্যাক ঠিকানা কত বিট লম্বা? A) ৩২ বিট B) ৪৮ বিট C) ৬৪ বিট D) ১২৮ বিট
2. ম্যাক ঠিকানার প্রথম ২৪ বিট কী চেনায়? A) যন্ত্রের আইপি ঠিকানা B) প্রস্তুতকারক প্রতিষ্ঠান C) নেটওয়ার্কের নাম D) ব্যবহারকারীর নাম
3. ff:ff:ff:ff:ff:ff ঠিকানাটি কী ধরনের? A) ইউনিকাস্ট B) মাল্টিকাস্ট C) ব্রডকাস্ট D) লুপব্যাক
4. সুইচ গন্তব্য ম্যাক টেবিলে না পেলে কী করে? A) ফ্রেমটি ফেলে দেয় B) ফ্রেমটি সব পোর্টে পাঠায় C) ফ্রেমটি শুধু প্রথম পোর্টে পাঠায় D) ফ্রেমটি রাউটারে পাঠায়
5. উবুন্টুতে প্রতিবেশী যন্ত্রের ম্যাক ঠিকানা দেখার আধুনিক কমান্ড কোনটি? A) ifconfig B) ip neigh C) ping D) traceroute

স্বাধীন অ্যাসাইনমেন্ট

আপনার ল্যাব সার্ভারে ip -br link, ip addr ও ip neigh চালিয়ে একটি ছোট নথি তৈরি করুন। নথিতে প্রতিটি ইন্টারফেসের নাম, অবস্থা, ম্যাক ঠিকানা ও আইপি ঠিকানা লিখুন। তারপর একটি প্রতিবেশী আইপিতে ping চালিয়ে ip neigh আবার দেখুন, আর কী বদলাল, তা লিখুন। শেষে তিনটি বাক্যে লিখুন, সুইচ কীভাবে আপনার ফ্রেমটিকে সঠিক যন্ত্রে পৌঁছে দিয়েছে বলে আপনি মনে করেন। এই নথিটি পরের অধ্যায়ের আইপি অনুশীলনের সঙ্গে মিলিয়ে রাখুন।

সারসংক্ষেপ — Summary

এই অধ্যায়ে আমরা শিখেছি, স্থানীয় নেটওয়ার্কে প্রতিটি ইন্টারফেসের একটি ৪৮ বিটের ম্যাক ঠিকানা থাকে। প্রথম ২৪ বিট প্রস্তুতকারককে চেনায়, শেষ ২৪ বিট যন্ত্রটিকে আলাদা করে। ইথারনেট ফ্রেমের গায়ে গন্তব্য ও উৎস ম্যাক লেখা থাকে, আর সুইচ সেই গন্তব্য পড়ে সিদ্ধান্ত নেয়। সুইচ প্রথমে উৎস ম্যাক থেকে শেখে, তারপর জানা গন্তব্যে ফরওয়ার্ড করে, আর অজানা গন্তব্যে ফ্লাড করে। উবুন্টুতে ip link, ip addr ও ip neigh দিয়ে এই তথ্য দেখা যায়। ম্যাক পরিচয় দেয়, কিন্তু নিরাপত্তার একমাত্র ভরসা নয়।

পুনরালোচনার তালিকা — Revision Checklist

- ম্যাক ঠিকানা ৪৮ বিটের, ছয়টি বাইটে লেখা হয়, তা মনে আছে কি?
- OUI ও যন্ত্রের অংশ আলাদা করে বলতে পারেন কি?
- ইউনিকাস্ট, মাল্টিকাস্ট ও ব্রডকাস্টের পার্থক্য বলতে পারেন কি?
- সুইচের শেখা, ফরওয়ার্ড ও ফ্লাড ধাপগুলো বলতে পারেন কি?
- ip -br link, ip link, ip addr ও ip neigh কমান্ডের কাজ জানেন কি?
- ম্যাক স্পুফিং ও ম্যাক ফ্লাডিংয়ের ঝুঁকি ব্যাখ্যা করতে পারেন কি?

Chapter 7: IPv4 Addressing and Binary Mathematics — আইপিভি৪ ঠিকানা ও বাইনারি গণিত

অধ্যায় ৭ | DevOps Engineering Mastery Vol 02 — Networking and Internet Infrastructure | Ubuntu Server 24.04 LTS | প্রকাশ: ৮ নভেম্বর ২০২৬

শুরুতে: একটি বাড়ির ঠিকানা কীভাবে লিখতে হয়

আগের অধ্যায়ে আমরা শিখেছি, স্থানীয় নেটওয়ার্কে যন্ত্র চেনা যায় ম্যাক ঠিকানা দিয়ে। কিন্তু একটি শহরে হাজার হাজার বাড়ি থাকলে শুধু ফ্ল্যাট নম্বর দিয়ে কি চিঠি পৌঁছানো যায়? না, দরকার হয় রাস্তার নাম, এলাকার নাম ও শহরের নাম। ইন্টারনেটেও একই সমস্যা। একটি যন্ত্র আরেকটি যন্ত্রকে খুঁজে পেতে হলে দরকার এমন একটি ঠিকানা, যা দিয়ে বোঝা যায়, যন্ত্রটি কোন নেটওয়ার্কে আছে, আর সেই নেটওয়ার্কের ভেতরে সে কোন যন্ত্র। সেই ঠিকানার নাম আইপি ঠিকানা। এই অধ্যায়ে আমরা আইপিভি৪ ঠিকানা শিখব, আর তার জন্য দরকারি বাইনারি গণিত হাতে-কলমে করব। দশমিক গণিতে আমরা দশটি অঙ্ক ব্যবহার করি, বাইনারিতে ব্যবহার করি দুটি অঙ্ক: শূন্য ও এক। কম্পিউটার শুধু এই দুটি অবস্থা বোঝে: চালু বা বন্ধ। আমরা মানুষ দশমিক সংখ্যায় লিখি, যেমন 192.168.1.10, কিন্তু কম্পিউটার ভেতরে সেটিকে বাইনারিতে বোঝে। এই অধ্যায়ে আমরা দুটি লেখার মধ্যে অনুবাদ শিখব, ধাপে ধাপে, কোনো ধাপ বাদ না দিয়ে।

শেখার উদ্দেশ্য — Learning Objectives

এই অধ্যায় শেষে আপনি পারবেন:

- 128, 64, 32, 16, 8, 4, 2, 1 স্থানীয় মান ব্যবহার করে যেকোনো দশমিক সংখ্যাকে বাইনারিতে ও বাইনারিকে দশমিকে রূপান্তর করতে।
- আইপিভি৪ ঠিকানার চারটি অক্টেট, নেটওয়ার্ক অংশ ও হোস্ট অংশ আলাদা করে চিনতে।
- সাবনেট মাস্ক পড়তে এবং একটি ঠিকানা কোন নেটওয়ার্কে আছে, তা বের করতে।
- প্রাইভেট, পাবলিক, লুপব্যাক ও লিংক-লোকাল ঠিকানার পরিসর মুখস্থ না করে যুক্তি দিয়ে চিনতে।
- উবুন্টু ২৪.০৪-এ ip addr, ip -br addr ও ip route চালিয়ে আইপি, মাস্ক ও গেটওয়ে পড়তে।
- ভুল মাস্ক, ভুল গেটওয়ে ও ভুল নেটওয়ার্কের কারণে সংযোগ না পাওয়ার সমস্যা খুঁজতে।

পূর্বশর্ত — Prerequisites

আগের অধ্যায়ের ম্যাক ঠিকানা ও সুইচের ধারণা জানা থাকতে হবে। দশমিক গণিতে যোগ ও বিয়োগ করতে পারলেই চলবে। ভলিউম ০১-এর টার্মিনাল ও ip কমান্ডের অভিজ্ঞতা কাজে লাগবে। একটি উবুন্টু সার্ভার ২৪.০৪ ল্যাব থাকলে অনুশীলন সরাসরি করা যাবে। কাগজ ও কলম সঙ্গে রাখুন, কারণ হাতে বাইনারি লেখাই সবচেয়ে ভালো উপায়।

প্রথম নীতি: কম্পিউটার শুধু দুটি অঙ্ক জানে

মানুষ দশটি আঙুল দিয়ে গোনে বলে দশমিক পদ্ধতি ব্যবহার করে। কম্পিউটারের আঙুল নেই, তার আছে বিদ্যুৎ: হয় বিদ্যুৎ আছে, নয় নেই। এই দুটি অবস্থাকে আমরা লিখি ১ ও ০ দিয়ে। এটাই বাইনারি পদ্ধতি। দশমিক পদ্ধতিতে প্রতিটি স্থানের মান দশ গুণ করে বাড়ে: একক, দশক, শতক। বাইনারিতে প্রতিটি স্থানের মান দুই গুণ করে বাড়ে: ১, ২, ৪, ৮, ১৬, ৩২, ৬৪, ১২৮। একটি বাইনারি সংখ্যা পড়ার সময় ডান দিক থেকে বাম দিকে যেতে হয়। সবচেয়ে ডানের স্থানের মান ১, তার বামেরটি ২, তারপর ৪, তারপর ৮, তারপর ১৬, তারপর ৩২, তারপর ৬৪, তারপর ১২৮। আটটি স্থান মিলে একটি বাইট হয়। আইপি

ঠিকানায় প্রতিটি অংশ একটি বাইট, তাই প্রতিটি অংশের মান ০ থেকে ২৫৫-এর মধ্যে থাকে। ২৫৫-এর বেশি কোনো অক্টেট হতে পারে না। আটটি এক, অর্থাৎ 11111111, যোগ করলে হয় ১২৮ যোগ ৬৪ যোগ ৩২ যোগ ১৬ যোগ ৮ যোগ ৪ যোগ ২ যোগ ১, মোট ২৫৫।

> মূল ধারণা: বাইনারিতে প্রতিটি স্থানের মান আগেরটির দ্বিগুণ। আটটি স্থানের মান ডান থেকে বামে 1, 2, 4, 8, 16, 32, 64, 128। কোনো স্থানে 1 থাকলে সেই মান যোগ হয়, 0 থাকলে যোগ হয় না।

দশমিক থেকে বাইনারি: প্রতিটি ধাপ দেখে

দশমিক থেকে বাইনারিতে যাওয়ার নিয়মটি সহজ। বড় স্থান থেকে শুরু করে দেখুন, সংখ্যাটি সেই মানের সমান বা বড় কি না। বড় হলে সেই স্থানে ১ লিখুন ও মানটি বিয়োগ করুন। ছোট হলে সেই স্থানে ০ লিখুন ও বিয়োগ করবেন না। তারপর পরের ছোট স্থানে যান। এই নিয়মে কোনো অনুমান লাগে না। উদাহরণ এক: 192-কে বাইনারিতে নিন। স্থানীয় মানগুলো: 128, 64, 32, 16, 8, 4, 2, 1। 192 কি 128-এর সমান বা বড়? হ্যাঁ, তাই 128-এর ঘরে ১ লিখুন। বাকি থাকে 192 বিয়োগ 128, অর্থাৎ 64। এবার 64 কি 64-এর সমান বা বড়? হ্যাঁ, তাই 64-এর ঘরে ১ লিখুন। বাকি থাকে 64 বিয়োগ 64, অর্থাৎ 0। এবার 32-এর ঘরে: 0 কি 32-এর সমান বা বড়? না, তাই ০ লিখুন। 16-এর ঘরে: 0 কি 16-এর সমান বা বড়? না, তাই ০। 8-এর ঘরে: না, তাই ০। 4-এর ঘরে: না, তাই ০। 2-এর ঘরে: না, তাই ০। 1-এর ঘরে: না, তাই ০। ফলে 192-এর বাইনারি হয় 11000000। প্রতিটি ঘর পরীক্ষা করা হয়েছে, কোনো ঘর বাদ যায়নি।

উদাহরণ দুই: 168-কে বাইনারিতে নিন। 168 কি 128-এর সমান বা বড়? হ্যাঁ, ১ লিখুন, বাকি 40। 40 কি 64-এর সমান বা বড়? না, ০ লিখুন, বাকি 40। 40 কি 32-এর সমান বা বড়? হ্যাঁ, ১ লিখুন, বাকি 8। 8 কি 16-এর সমান বা বড়? না, ০ লিখুন, বাকি 8। 8 কি 8-এর সমান বা বড়? হ্যাঁ, ১ লিখুন, বাকি 0। 0 কি 4-এর সমান বা বড়? না, ০। 0 কি 2-এর সমান বা বড়? না, ০। 0 কি 1-এর সমান বা বড়? না, ০। ফলে 168-এর বাইনারি হয় 10101000। উদাহরণ তিন: 10-কে বাইনারিতে নিন। 10 কি 128-এর সমান বা বড়? না, ০। 64-এর ঘরে: না, ০। 32-এর ঘরে: না, ০। 16-এর ঘরে: না, ০। 8-এর ঘরে: হ্যাঁ, ১, বাকি 2। 4-এর ঘরে: না, ০, বাকি 2। 2-এর ঘরে: হ্যাঁ, ১, বাকি 0। 1-এর ঘরে: না, ০। ফলে 10-এর বাইনারি হয় 00001010। এখানে বামের শূন্যগুলোও লিখতে হয়, কারণ আটটি ঘর পূর্ণ করতে হয়। শূন্য বাদ দিলে বোঝা যাবে না, এটি আট বিটের সংখ্যা।

বাইনারি থেকে দশমিক: যোগ করে ফেরা

বাইনারি থেকে দশমিকে যাওয়া আরও সহজ। যেসব ঘরে ১ আছে, শুধু সেই ঘরগুলোর মান যোগ করুন। উদাহরণ এক: 11000000। এখানে 128-এর ঘরে ১, 64-এর ঘরে ১, বাকি সব ঘরে ০। যোগ করুন: 128 যোগ 64, মোট 192। উদাহরণ দুই: 10101000। এখানে 128-এর ঘরে ১, 32-এর ঘরে ১, 8-এর ঘরে ১, বাকি ঘরে ০। যোগ করুন: 128 যোগ 32 যোগ 8, মোট 168। উদাহরণ তিন: 00001010। এখানে 8-এর ঘরে ১, 2-এর ঘরে ১, বাকি ঘরে ০। যোগ করুন: 8 যোগ 2, মোট 10। উদাহরণ চার: 11111111। সব ঘরে ১। যোগ করুন: 128 যোগ 64 যোগ 32 যোগ 16 যোগ 8 যোগ 4 যোগ 2 যোগ 1, মোট 255। শুধু মনে রাখুন, ১ থাকলে যোগ, ০ থাকলে যোগ নয়।

চিত্র ১: আটটি স্থানীয় মান

128 | 64 | 32 | 16 | 8 | 4 | 2 | 1

192 = 1 1 0 0 0 0 0 0 | 168 = 1 0 1 0 1 0 0 0 | 10 = 0 0 0 0 1 0 1 0

পড়ার নিয়ম: ডান দিক থেকে বাম দিকে মান বাড়ে। প্রতিটি ঘরে 1 থাকলে সেই মান যোগ হয়, 0 থাকলে হয় না।

আইপিভি৪ ঠিকানার গঠন: নেটওয়ার্ক ও হোস্ট

আইপিভি৪ ঠিকানা ৩২ বিট লম্বা। এই ৩২ বিটকে চারটি ভাগে ভাগ করা হয়, প্রতিটি ভাগে আট বিট। প্রতিটি ভাগকে বলা হয় অক্টেট। প্রতিটি অক্টেটকে দশমিক সংখ্যায় লেখা হয়, আর চারটি অক্টেটকে বিন্দু দিয়ে আলাদা করা হয়। উদাহরণ: 192.168.1.10। বাইনারিতে পুরো ঠিকানাটি হয় 11000000.10101000.00000001.00001010। আইপি ঠিকানার দুটি অংশ থাকে: নেটওয়ার্ক অংশ ও হোস্ট অংশ। নেটওয়ার্ক অংশ বলে, যন্ত্রটি কোন নেটওয়ার্কে আছে। হোস্ট অংশ বলে, সেই নেটওয়ার্কের ভেতরে যন্ত্রটি কোনটি। রাস্তার নাম হলো নেটওয়ার্ক অংশ, বাড়ির নম্বর হলো হোস্ট অংশ। একই নেটওয়ার্কের সব যন্ত্রের নেটওয়ার্ক অংশ একই, হোস্ট অংশ আলাদা।

নেটওয়ার্ক অংশ কতটুকু, আর হোস্ট অংশ কতটুকু, তা ঠিক করে সাবনেট মাস্ক। মাস্কও ৩২ বিটের একটি সংখ্যা। মাস্কের যেসব বিট ১, সেসব বিট নেটওয়ার্ক অংশ। যেসব বিট ০, সেসব বিট হোস্ট অংশ। সবচেয়ে সাধারণ মাস্ক হলো 255.255.255.0। বাইনারিতে এটি 11111111.11111111.11111111.00000000। এখানে প্রথম ২৪টি বিট ১, শেষ আটটি বিট ০। তাই প্রথম তিনটি অক্টেট নেটওয়ার্ক অংশ, শেষ অক্টেট হোস্ট অংশ। একে সংক্ষেপে লেখা হয় /24। উদাহরণ: 192.168.1.10/24 ঠিকানায় নেটওয়ার্ক অংশ 192.168.1, হোস্ট অংশ 10। আরেকটি যন্ত্র হতে পারে 192.168.1.20/24। আর /16 মাস্কে, অর্থাৎ 255.255.0.0, প্রথম দুটি অক্টেট নেটওয়ার্ক, শেষ দুটি হোস্ট। আর /8 মাস্কে, অর্থাৎ 255.0.0.0, প্রথম অক্টেট নেটওয়ার্ক, শেষ তিনটি হোস্ট।

চিত্র ২: 192.168.1.10/24 ঠিকানার ভাগ

192 . 168 . 1 . 10 | নেটওয়ার্ক অংশ: 192.168.1 | হোস্ট অংশ: 10 | মাস্ক: 255.255.255.0 | সংক্ষেপ: /24

নিয়ম: মাস্কে 1 থাকলে সেই বিট নেটওয়ার্কের, 0 থাকলে হোস্টের। /24 মানে প্রথম 24 বিট নেটওয়ার্ক, শেষ 8 বিট হোস্ট।

পরিভাষার তালিকা

পরিভাষা	ইংরেজি	অর্থ
বাইনারি	Binary	শুধু 0 ও 1 দিয়ে লেখা সংখ্যা পদ্ধতি।
বিট	Bit	একটি 0 বা 1, তথ্যের ক্ষুদ্রতম একক।
বাইট	Byte	আটটি বিটের দল, মান 0 থেকে 255।
অক্টেট	Octet	আইপি ঠিকানার একটি আট বিটের অংশ।
আইপিভি৪	IPv4	৩২ বিটের ইন্টারনেট ঠিকানা পদ্ধতি।
নেটওয়ার্ক অংশ	Network portion	ঠিকানার যে অংশ নেটওয়ার্ককে চেনায়।
হোস্ট অংশ	Host portion	ঠিকানার যে অংশ নেটওয়ার্কের ভেতরে যন্ত্রকে চেনায়।
সাবনেট মাস্ক	Subnet mask	নেটওয়ার্ক ও হোস্ট অংশ আলাদা করার মান।
প্রাইভেট ঠিকানা	Private address	শুধু নিজের নেটওয়ার্কে ব্যবহারের ঠিকানা।
পাবলিক ঠিকানা	Public address	ইন্টারনেটে পৌঁছানো যায় এমন ঠিকানা।
লুপব্যাক	Loopback	নিজের সঙ্গে কথা বলার ঠিকানা।

গেটওয়ে	Gateway	বাইরে যাওয়ার দরজা।

বাস্তব পরিস্থিতি

প্রথম পরিস্থিতি: ঢাকার একটি অফিসে নতুন সার্ভার বসানো হলো। তার ঠিকানা 192.168.1.50/24। অফিসের অন্য কম্পিউটারগুলো 192.168.1.10 থেকে 192.168.1.40-এর মধ্যে আছে। সবার নেটওয়ার্ক অংশ একই, 192.168.1, তাই তারা একে অপরকে সরাসরি খুঁজে পায়। দ্বিতীয় পরিস্থিতি: একটি বাড়িতে রাউটার থেকে ঠিকানা আসে 192.168.0.0/24 নেটওয়ার্কে। বাড়ির ফোন, ল্যাপটপ ও টেলিভিশন সবাই একই নেটওয়ার্কে থাকে, আর রাউটার তাদের ইন্টারনেটে যাওয়ার দরজা। তৃতীয় পরিস্থিতি: একজন ডেভেলপার নিজের ল্যাপটপে একটি সার্ভার চালান। তিনি ব্রাউজারে 127.0.0.1 লেখেন। এই ঠিকানা নিজের যন্ত্রকেই দেখায়। নেটওয়ার্কে তার না থাকলেও এই ঠিকানা কাজ করে, কারণ এটি লুপব্যাক। এই তিনটি পরিস্থিতিতেই একই নিয়ম: নেটওয়ার্ক অংশ মিললে যন্ত্রগুলো একই নেটওয়ার্কে, না মিললে গেটওয়ে দরকার।

কাজের উদাহরণ: প্রাইভেট, পাবলিক, লুপব্যাক ও লিংক-লোকাল

সব আইপি ঠিকানা ইন্টারনেটে ব্যবহার করা যায় না। কিছু ঠিকানা শুধু নিজের নেটওয়ার্কের জন্য রাখা আছে। এগুলোকে বলা হয় প্রাইভেট ঠিকানা। তিনটি প্রাইভেট পরিসর মনে রাখুন। প্রথমটি 10.0.0.0/8, অর্থাৎ 10 দিয়ে শুরু হওয়া সব ঠিকানা, 10.0.0.0 থেকে 10.255.255.255 পর্যন্ত। দ্বিতীয়টি 172.16.0.0/12, অর্থাৎ 172.16.0.0 থেকে 172.31.255.255 পর্যন্ত। এখানে দ্বিতীয় অক্টেট 16 থেকে 31-এর মধ্যে থাকে। তৃতীয়টি 192.168.0.0/16, অর্থাৎ 192.168.0.0 থেকে 192.168.255.255 পর্যন্ত। বাংলাদেশের বাড়ি ও অফিসে সবচেয়ে বেশি দেখা যায় এই তৃতীয় পরিসরটি। এই তিনটি পরিসরের বাইরের ঠিকানা সাধারণত পাবলিক ঠিকানা। একটি বাড়ির রাউটারের বাইরের দিকে একটি পাবলিক ঠিকানা থাকে, ভেতরের দিকে একটি প্রাইভেট ঠিকানা থাকে।

দুটি বিশেষ ঠিকানা আলাদা করে মনে রাখুন। 127.0.0.1 হলো লুপব্যাক ঠিকানা। এটি সবসময় নিজের যন্ত্রকে দেখায়। 127.0.0.1-এ ping চালালে উত্তর আসে নিজের যন্ত্র থেকেই। আর 169.254.0.0/16 পরিসরটি হলো লিংক-লোকাল। কোনো যন্ত্র যখন কোনো সার্ভার থেকে আইপি ঠিকানা পায় না, তখন সে নিজে থেকে এই পরিসরে একটি ঠিকানা বেছে নেয়, যেমন 169.254.10.20। এই ঠিকানা দেখলে বুঝতে হবে, যন্ত্রটি ঠিকানা পায়নি। উদাহরণ: 192.168.1.10/24 হলে সেটি প্রাইভেট। 8.8.8.8 হলে সেটি পাবলিক। 127.0.0.1 হলে সেটি লুপব্যাক। 169.254.5.5 হলে সেটি লিংক-লোকাল।

উবুন্টুতে কমান্ড: নিজের আইপি ঠিকানা দেখা

উবুন্টু সার্ভার ২৪.০৪-এ আইপি ঠিকানা দেখার প্রধান কমান্ড ip addr। নিচের কমান্ডগুলো নিজের সার্ভারে চালান।

```
ip addr
ip -br addr
ip route
ip addr show enp0s3
```

ip addr চালালে প্রতিটি ইন্টারফেসের নিচে inet লেখা লাইনে আইপি ঠিকানা ও মাস্ক দেখা যায়, যেমন inet 192.168.1.10/24। এখানে /24 হলো মাস্কের সংক্ষিপ্ত রূপ। ip -br addr চালালে একই তথ্য এক লাইনে পাওয়া যায়। ip route চালালে default লেখা লাইনে গেটওয়ের ঠিকানা দেখা যায়, যেমন default via 192.168.1.1। এই গেটওয়ে হলো নিজের নেটওয়ার্ক থেকে বাইরে যাওয়ার দরজা। ip addr show enp0s3 চালালে শুধু একটি ইন্টারফেসের তথ্য দেখা যায়। এখানে enp0s3 হলো উদাহরণ নাম, আপনার যন্ত্রে নামটি আলাদা হতে পারে।

নির্দেশিত অনুশীলন

ল্যাব: নিজের ঠিকানা ভেঙে লিখুন। ধাপ ১: ip -br addr চালিয়ে নিজের আইপি ঠিকানা ও মাস্ক লিখে নিন। ধাপ ২: প্রতিটি অক্টেটকে হাতে বাইনারিতে রূপান্তর করুন, 128, 64, 32, 16, 8, 4, 2, 1 মান ব্যবহার করে। ধাপ ৩: মাস্ক থেকে নেটওয়ার্ক অংশ ও হোস্ট অংশ আলাদা করুন। ধাপ ৪: ip route চালিয়ে গেটওয়ের ঠিকানা লিখুন। ধাপ ৫: ঠিকানাটি প্রাইভেট, পাবলিক, লুপব্যাক না লিংক-লোকাল, তা লিখুন।

উদাহরণ আউটপুট (illustrative)

```
enp0s3           UP             192.168.1.10/24
default via 192.168.1.1 dev enp0s3
192.168.1.10 = 11000000.10101000.00000001.00001010
```

উপরের উদাহরণ আউটপুটে ঠিকানা 192.168.1.10, মাস্ক /24, গেটওয়ে 192.168.1.1। নেটওয়ার্ক অংশ 192.168.1, হোস্ট অংশ 10। আপনার আসল আউটপুটে সংখ্যাগুলো আলাদা হবে। বাইনারি লাইনটি নিজে হাতে মিলিয়ে দেখুন: 192 মানে 11000000, 168 মানে 10101000, 1 মানে 00000001, 10 মানে 00001010।

সাধারণ ভুল

প্রথম ভুল: বামের শূন্য বাদ দেওয়া। 10-এর বাইনারি 1010 নয়, আট বিটে 00001010। দ্বিতীয় ভুল: স্থানীয় মানের ক্রম উল্টে ফেলা। ডান দিক থেকে মান বাড়ে। তৃতীয় ভুল: 255-এর বেশি অক্টেট লেখা। চতুর্থ ভুল: /24 মাস্ককে 255.255.255.255 ভাবা। /24 মানে 255.255.255.0। পঞ্চম ভুল: প্রাইভেট ঠিকানাকে ইন্টারনেটে সরাসরি ব্যবহার করা যাবে ভাবা। ষষ্ঠ ভুল: 172.16/12 পরিসরকে শুধু 172.16 ভাবা। এই পরিসরে 172.17 থেকে 172.31 পর্যন্তও আছে।

সমস্যা সমাধান — Troubleshooting

সমস্যা: ip addr-এ কোনো আইপি নেই। পরীক্ষা: ইন্টারফেস UP আছে কি না, তার লাগানো আছে কি না, আর ঠিকানা দেওয়ার সার্ভার কাজ করছে কি না, দেখুন। সমস্যা: আইপি আছে 169.254 দিয়ে শুরু। পরীক্ষা: যন্ত্রটি ঠিকানা পায়নি। রাউটার বা ঠিকানা দেওয়ার সার্ভার চালু আছে কি না, দেখুন। সমস্যা: একই নেটওয়ার্কের যন্ত্রে ping যায়, বাইরে যায় না। পরীক্ষা: ip route-এ default গেটওয়ে আছে কি না, দেখুন। সমস্যা: ভুল মাস্ক দেওয়া হয়েছে। পরীক্ষা: /24-এর জায়গায় /16 দিলে যন্ত্র ভাববে, অনেক বেশি যন্ত্র একই নেটওয়ার্কে আছে। মাস্কটি মিলিয়ে দেখুন।

নিরাপত্তা বিবেচনা

আইপি ঠিকানা দেখে যন্ত্রের অবস্থান সম্পর্কে ধারণা পাওয়া যায়, তবে এটি পরিচয়ের প্রমাণ নয়। কেউ নিজের আইপি ঠিকানা বদলে অন্য ঠিকানা ব্যবহার করতে পারে। একে বলা হয় আইপি স্পুফিং। তাই শুধু আইপি দেখে কাউকে বিশ্বাস করা উচিত নয়। প্রাইভেট ঠিকানা ইন্টারনেটে সরাসরি পৌঁছায় না, তবে এটি নিরাপত্তার একমাত্র ভরসা নয়। রাউটার প্রাইভেট ঠিকানাকে পাবলিক ঠিকানায় বদলে দেয়, তাই বাইরে থেকে ভেতরের যন্ত্র সরাসরি দেখা যায় না। তবু ভেতরের যন্ত্রে দুর্বল পাসওয়ার্ড বা পুরোনো সফটওয়্যার থাকলে ঝুঁকি থাকে। লুপব্যাক ঠিকানা 127.0.0.1 শুধু নিজের যন্ত্রে কাজ করে। কোনো সার্ভারকে শুধু লুপব্যাকে চালালে বাইরে থেকে কেউ তাতে ঢুকতে পারে না।

সেরা অনুশীলন — Best Practices

প্রথমত, নেটওয়ার্কের নথিতে প্রতিটি যন্ত্রের আইপি, মাস্ক, গেটওয়ে ও কাজ একসঙ্গে লিখে রাখুন। দ্বিতীয়ত, বাইনারি রূপান্তর প্রথমে হাতে করুন, তারপর প্রয়োজনে যাচাই করুন। তৃতীয়ত, প্রাইভেট পরিসরের বাইরে ঠিকানা বসাবেন না, যদি না বিশেষ দরকার থাকে। চতুর্থত, গুরুত্বপূর্ণ সার্ভারের ঠিকানা বারবার বদলাবেন না। পঞ্চমত, সমস্যা খুঁজতে গেলে ধাপে ধাপে যান: প্রথমে ip addr-এ ঠিকানা, তারপর ip route-এ গেটওয়ে, তারপর ping দিয়ে পরীক্ষা।

জ্ঞান যাচাই — Knowledge Check

1. 192-এর বাইনারি রূপ কোনটি? A) 10101000 B) 11000000 C) 11111111 D) 00001010
2. 10101000-এর দশমিক মান কত? A) 192 B) 168 C) 10 D) 255
3. 192.168.1.10/24 ঠিকানায় নেটওয়ার্ক অংশ কোনটি? A) 192.168.1 B) 192.168.1.10 C) 10 D) 192
4. নিচের কোনটি প্রাইভেট ঠিকানা? A) 8.8.8.8 B) 10.0.0.5 C) 127.0.0.1 D) 169.254.1.1
5. 127.0.0.1 ঠিকানাটি কী? A) পাবলিক ঠিকানা B) লুপব্যাক ঠিকানা C) লিংক-লোকাল ঠিকানা D) ব্রডকাস্ট ঠিকানা

স্বাধীন অ্যাসাইনমেন্ট

> পাঁচটি ঠিকানা নিন: 10.20.30.40, 172.16.5.5, 172.31.1.1, 192.168.0.100 ও 169.254.10.20। প্রতিটি অক্টেটকে হাতে বাইনারিতে রূপান্তর করুন, 128, 64, 32, 16, 8, 4, 2, 1 মান ব্যবহার করে। তারপর প্রতিটি ঠিকানা প্রাইভেট, পাবলিক, লুপব্যাক না লিংক-লোকাল, তা লিখুন। শেষে 192.168.1.10/24 ও 192.168.2.10/24 একই নেটওয়ার্কে আছে কি না, তা কারণসহ লিখুন।

সারসংক্ষেপ — Summary

এই অধ্যায়ে আমরা শিখেছি, বাইনারিতে প্রতিটি স্থানের মান আগেরটির দ্বিগুণ: 1, 2, 4, 8, 16, 32, 64, 128। দশমিক থেকে বাইনারিতে যেতে বড় মান থেকে বিয়োগ করে যেতে হয়, আর বাইনারি থেকে দশমিকে যেতে 1 থাকা ঘরগুলোর মান যোগ করতে হয়। আইপিভি৪ ঠিকানা ৩২ বিটের, চারটি অক্টেটে লেখা হয়। মাস্ক ঠিক করে, কোন অংশ নেটওয়ার্ক, কোন অংশ হোস্ট। 10/8, 172.16/12 ও 192.168/16 হলো প্রাইভেট পরিসর, 127.0.0.1 হলো লুপব্যাক, আর 169.254 হলো লিংক-লোকাল।

পুনরালোচনার তালিকা — Revision Checklist

- 128, 64, 32, 16, 8, 4, 2, 1 মানগুলো ক্রম অনুযায়ী বলতে পারেন কি?
- 192, 168 ও 10-এর বাইনারি রূপ হাতে বের করতে পারেন কি?
- 11000000 ও 10101000-এর দশমিক মান বের করতে পারেন কি?
- /24, /16 ও /8 মাস্কের অর্থ বলতে পারেন কি?
- 10/8, 172.16/12, 192.168/16, 127.0.0.1 ও 169.254 পরিসর চিনতে পারেন কি?
- ip addr, ip -br addr ও ip route কমান্ডের কাজ জানেন কি?

অধ্যায় ৮ • Part 2: Networking & Internet Infrastructure • Ubuntu Server 24.04 LTS • প্রকাশ: ৮ নভেম্বর ২০২৬

Chapter 8: Subnetting, CIDR and Network Planning — সাবনেটিং, CIDR এবং নেটওয়ার্ক পরিকল্পনা

ভূমিকা: একটি ঠিকানা কেন ভাগ করতে হয়

ধরুন, আপনার হাতে একটি বড় বাড়ি আছে। বাড়িতে একশোটি ঘর আছে, কিন্তু সেখানে থাকে মাত্র তিনটি পরিবার। প্রতিটি পরিবারের জন্য আলাদা দরজা, আলাদা রান্নাঘর, আলাদা নিয়ম না থাকলে কী হবে? সবাই একই করিডোর দিয়ে হাঁটবে, একজনের শব্দ অন্যজনের ঘরে যাবে, একজন দরজা বন্ধ করলে সবাই আটকে যাবে, আর কে কোন ঘরে থাকে তা খুঁজে পাওয়া কঠিন হবে। একটি বড় নেটওয়ার্ক ঠিক এই রকম। একটি বড় ঠিকানা-পরিসর, যেমন 192.168.1.0/24, একসঙ্গে ব্যবহার করলে সব ডিভাইস একই সম্প্রচার-ক্ষেত্রে থাকে। একটি ডিভাইস কিছু খুঁজতে সম্প্রচার করলে সেই বার্তা সবাই পায়। একটি বিভাগের সমস্যা অন্য বিভাগকে থামিয়ে দেয়। নিরাপত্তার নিয়ম লেখা কঠিন হয়, কারণ সবাই একই জায়গায়। আর ঠিকানা নষ্ট হয় — ছোট একটি দলের জন্য বড় একটি পরিসর বরাদ্দ দিলে বাকি ঠিকানা অব্যবহৃত পড়ে থাকে।

সাবনেটিং হলো এই বড় বাড়িকে পরিকল্পিতভাবে ছোট ছোট ফ্ল্যাটে ভাগ করা। প্রতিটি ফ্ল্যাটের নিজের দরজা আছে, নিজের সীমানা আছে, নিজের ঠিকানার পরিসর আছে। CIDR হলো সেই ভাগের লিখিত নিয়ম — কতটুকু নেটওয়ার্কের অংশ, কতটুকু হোস্টের অংশ, তা একটি সংখ্যা দিয়ে প্রকাশ করা। এই অধ্যায়ের প্রশ্ন তাই শুধু গণিত নয়। প্রশ্ন হলো: একটি প্রতিষ্ঠানের জন্য কতগুলো নেটওয়ার্ক দরকার, প্রতিটিতে কতগুলো ডিভাইস থাকবে, ভবিষ্যতে কতটুকু বাড়বে, কোন বিভাগকে কার থেকে আলাদা রাখতে হবে, আর রাউটার কীভাবে জানবে কোন প্যাকেট কোথায় যাবে? উত্তর না জেনে ঠিকানা বরাদ্দ দিলে পরে পুরো নেটওয়ার্ক ভেঙে নতুন করে সাজাতে হয়। উত্তর জেনে পরিকল্পনা করলে নেটওয়ার্ক ছোট থাকতেও গোছানো থাকে, বড় হলেও ভাঙে না।

একজন DevOps প্রকৌশলীর কাছে এই দক্ষতা প্রতিদিনের কাজ। তিনি যখন ক্লাউডে একটি VPC তৈরি করেন, Docker-এ একটি নেটওয়ার্ক বানান, Kubernetes-এ Pod-এর জন্য ঠিকানা-পরিসর বেছে নেন, অফিসের জন্য VLAN সাজান, অথবা VPN-এর দুই প্রান্তের ঠিকানা যেন না মেলে তা নিশ্চিত করেন — প্রতিটি জায়গায় একই প্রশ্ন ফিরে আসে: এই prefix-এ কতগুলো ঠিকানা আছে, network address কোনটি, broadcast address কোনটি, ব্যবহারযোগ্য হোস্ট কয়টি, আর এই পরিসরটি অন্য পরিসরের সঙ্গে মিলছে কি না? এই অধ্যায় শেষে আপনি সেই প্রশ্নের উত্তর কাগজে-কলমে, হাতে গণনা করে, কোনো ক্যালকুলেটরের উপর নির্ভর না করে দিতে পারবেন। ক্যালকুলেটর পরে ব্যবহার করবেন যাচাইয়ের জন্য, শেখার জন্য নয়।

> মূল ধারণা: IP ঠিকানা একা কিছু বলে না। ঠিকানার সঙ্গে prefix বা subnet mask না থাকলে বোঝা যায় না কোন অংশটি নেটওয়ার্কের পরিচয় আর কোন অংশটি সেই নেটওয়ার্কের ভেতরের ডিভাইসের পরিচয়। 192.168.1.10/24 আর 192.168.1.10/26 একই ঠিকানা হলেও সম্পূর্ণ আলাদা নেটওয়ার্কের সদস্য হতে পারে।

এই অধ্যায়ে আমরা প্রথমে কারণ বুঝব, তারপর নিয়ম শিখব, তারপর হাতে গণনা করব, তারপর Ubuntu Server-এ যাচাই করব। তাড়াহুড়ো করে সূত্র মুখস্থ করবেন না। প্রতিটি সূত্রের পেছনে একটি যুক্তি আছে: 2-এর ঘাত আসে বাইনারি থেকে, বিয়োগ 2 আসে network আর broadcast ঠিকানা সংরক্ষিত থাকার কারণে, আর block size আসে mask-এর শূন্য-বিটগুলো থেকে। যুক্তি বুঝলে সূত্র ভুলবেন না, আর ভুল করলেও নিজে ধরে ফেলতে পারবেন।

Learning Objectives — শেখার লক্ষ্য

এই অধ্যায় শেষে আপনি নিচের কাজগুলো নিজে, নির্ভুলভাবে করতে পারবেন। প্রতিটি লক্ষ্য পরিমাপযোগ্য — অর্থাৎ শেষে আপনি নিজেই যাচাই করতে পারবেন আপনি পেরেছেন কি না।

- যেকোনো private IPv4 ঠিকানা ও prefix (/8 থেকে /30) দেওয়া থাকলে ২ মিনিটের মধ্যে network address, broadcast address, প্রথম ও শেষ ব্যবহারযোগ্য হোস্ট, মোট ঠিকানা ও ব্যবহারযোগ্য হোস্টের সংখ্যা হাতে গণনা করে বের করতে পারবেন।
- CIDR notation (/24, /25, /26 ইত্যাদি) থেকে subnet mask (যেমন 255.255.255.192) এবং mask থেকে CIDR prefix — দুই দিকেই রূপান্তর করতে পারবেন।
- দশমিক ঠিকানাকে বাইনারিতে রূপান্তর করে AND অপারেশন দিয়ে network address বের করতে পারবেন এবং ব্যাখ্যা করতে পারবেন কেন এই পদ্ধতি কাজ করে।
- একটি /24 নেটওয়ার্ককে VLSM পদ্ধতিতে বিভিন্ন আকারের সাবনেটে ভাগ করতে পারবেন — বড় বিভাগকে বড় পরিসর, ছোট বিভাগকে ছোট পরিসর দিয়ে — এবং কোনো ঠিকানা যেন দুইবার বরাদ্দ না হয় তা যাচাই করতে পারবেন।
- একটি ছোট অফিস বা ল্যাবের জন্য সম্পূর্ণ IP পরিকল্পনা লিখতে পারবেন: বিভাগভিত্তিক সাবনেট, gateway ঠিকানা, ভবিষ্যৎ বৃদ্ধির জন্য সংরক্ষিত পরিসর, এবং নথিভুক্তির ছক।
- Ubuntu Server 24.04 LTS-এ ip addr ও ip route কমান্ডের আউটপুট পড়ে কোনো ইন্টারফেস কোন নেটওয়ার্কে আছে, তার broadcast কী, আর default route কোনটি তা বলতে পারবেন।
- সাধারণ ভুল — যেমন network ঠিকানাকে হোস্ট হিসেবে ব্যবহার, overlapping সাবনেট, ভুল broadcast ধরা, /31 ও /32-কে সাধারণ নিয়মে গণনা — চিনতে পারবেন এবং সংশোধন করতে পারবেন।

Prerequisites — পূর্বশর্ত

এই অধ্যায় শুরু করার আগে আপনার Vol 01-এর Linux ভিত্তি এবং আগের অধ্যায়গুলোর নেটওয়ার্ক ধারণা জানা থাকা দরকার। বিশেষ করে: IPv4 ঠিকানা চারটি octet-এ লেখা হয় এবং প্রতিটি octet 0 থেকে 255 হয়; বাইনারি সংখ্যার ধারণা — 2-এর ঘাত (1, 2, 4, 8, 16, 32, 64, 128, 256) চেনা; Ubuntu Server-এ টার্মিনালে কমান্ড চালানো ও আউটপুট পড়া; এবং gateway, DNS, interface ধারণার প্রাথমিক পরিচয়। বাইনারি রূপান্তরে দুর্বলতা থাকলে চিন্তা করবেন না — এই অধ্যায়ে প্রয়োজনীয় বাইনারি ধাপে ধাপে দেখানো হয়েছে। তবে দশমিক থেকে বাইনারি রূপান্তরের অভ্যাস না থাকলে প্রথম কয়েকটি উদাহরণ ধীরে পড়ুন, প্রতিটি ধাপ নিজে কাগজে লিখুন।

পরিবেশ: Ubuntu Server 24.04 LTS, একটি টার্মিনাল, কাগজ-কলম, আর ইচ্ছা করলে একটি subnet ক্যালকুলেটর শুধু উত্তর যাচাইয়ের জন্য। এই বইয়ের নিয়ম মনে রাখুন: কোনো সংস্করণ, RFC নম্বর বা কমান্ডের আউটপুট নিজে থেকে বানানো হবে না। এই অধ্যায়ের সব নমুনা আউটপুট স্পষ্টভাবে "উদাহরণ আউটপুট (illustrative)" বলে চিহ্নিত করা হয়েছে। বাস্তব সিস্টেমে আউটপুটের ইন্টারফেসের নাম, ঠিকানা ও সংখ্যা আপনার সিস্টেম অনুযায়ী আলাদা হবে।

প্রথম নীতি থেকে তত্ত্ব: সাবনেটিং কেন দরকার

একটি ঠিকানার দুটি অংশ

একটি IPv4 ঠিকানা 32 বিটের একটি সংখ্যা। মানুষের পড়ার সুবিধার জন্য তাকে চারটি octet-এ ভাগ করে দশমিকে লেখা হয়, যেমন 192.168.1.10। কিন্তু কম্পিউটারের কাছে এটি একটি 32 বিটের স্ট্রিং: 11000000.10101000.00000001.00001010। এই 32 বিটের মধ্যে প্রথম কিছু বিট বলে "তুমি কোন নেটওয়ার্কে আছো",

আর বাকি বিট বলে "সেই নেটওয়ার্কের ভেতরে তুমি কে"। প্রথম অংশকে network portion, দ্বিতীয় অংশকে host portion বলে। কোথায় ভাগ হবে, তা ঠিক করে subnet mask বা CIDR prefix।

একটি ডাক ঠিকানার সঙ্গে তুলনা করুন। "বাড়ি নং ১০, রোড নং ৫, ধানমন্ডি, ঢাকা" — এখানে "ধানমন্ডি, ঢাকা" হলো এলাকার পরিচয়, "বাড়ি নং ১০, রোড নং ৫" হলো সেই এলাকার ভেতরের নির্দিষ্ট বাড়ি। ডাকঘর প্রথমে এলাকা দেখে চিঠি ঢাকায় পাঠায়, তারপর এলাকার ভেতরে বাড়ি খোঁজে। রাউটারও একই কাজ করে। সে প্রথমে network portion দেখে প্যাকেট কোন নেটওয়ার্কে যাবে তা ঠিক করে, তারপর সেই নেটওয়ার্কের ভেতরে host portion দেখে নির্দিষ্ট ডিভাইসে পৌঁছে দেয়। Mask ছাড়া রাউটার জানতে পারে না ঠিকানার কোন অংশ এলাকা আর কোন অংশ বাড়ি। তাই IP ঠিকানা সবসময় mask বা prefix-সহ ভাবতে হয়।

কেন একটি বড় নেটওয়ার্ক ভাগ করতে হয়

প্রথম কারণ: broadcast। একই নেটওয়ার্কের একটি ডিভাইস যখন সবাইকে কিছু জানাতে চায়, সে broadcast ঠিকানায় পাঠায়, আর সেই নেটওয়ার্কের প্রতিটি ডিভাইস তা গ্রহণ করে ও প্রক্রিয়া করে। ডিভাইস দশটি হলে সমস্যা নেই। ডিভাইস এক হাজার হলে প্রতিটি broadcast হাজারটি ডিভাইসকে থামিয়ে দেয়। সাবনেট ভাগ করলে broadcast-এর পরিধি ছোট হয় — একটি সাবনেটের broadcast অন্য সাবনেটে যায় না, রাউটার তা আটকে দেয়।

দ্বিতীয় কারণ: নিরাপত্তা ও নিয়ন্ত্রণ। হিসাব বিভাগের কম্পিউটার আর অতিথিদের Wi-Fi একই নেটওয়ার্কে থাকলে তাদের আলাদা নিয়ম দেওয়া যায় না। সাবনেট আলাদা হলে রাউটার বা ফায়ারওয়ালে লেখা যায়: হিসাব বিভাগের সাবনেট থেকে সার্ভারের সাবনেটে যাওয়া যাবে, কিন্তু অতিথি সাবনেট থেকে যাওয়া যাবে না। তৃতীয় কারণ: ঠিকানার অপচয় রোধ। একটি দলে পাঁচজন থাকলে তাদের জন্য 254টি ঠিকানার পরিসর দেওয়া অপচয়। সাবনেটিং সেই পাঁচজনের জন্য 8টি ঠিকানার (/29) একটি ছোট পরিসর দিতে দেয়, বাকি ঠিকানা অন্য দলের কাজে লাগে। চতুর্থ কারণ: ব্যবস্থাপনা ও সমস্যা নির্ণয়। কোন সাবনেটে সমস্যা তা জানা থাকলে খুঁজতে সুবিধা হয় — "192.168.10.0/24-এ সমস্যা" বলা যায়, "কোথাও একটা সমস্যা" বলতে হয় না।

পঞ্চম কারণ, যা DevOps-এ সবচেয়ে গুরুত্বপূর্ণ: পরিকল্পনা ছাড়া বৃদ্ধি সম্ভব নয়। আজ 20টি সার্ভার আছে, কাল 200টি হবে। আজ একটি অফিস আছে, কাল তিনটি শহরে তিনটি অফিস হবে। শুরুতে যদি 10.0.0.0/8-এর ভেতর থেকে প্রতিটি অফিসকে একটি /16, প্রতিটি বিভাগকে একটি /24, প্রতিটি সার্ভিসকে একটি /26 দেওয়ার ছক থাকে, তাহলে বৃদ্ধি মানে শুধু পরের ফাঁকা পরিসরটি নেওয়া। ছক না থাকলে বৃদ্ধি মানে পুরোনো সব ঠিকানা বদলানো, আর ঠিকানা বদলানো মানে DNS, ফায়ারওয়াল, VPN, মনিটরিং — সব জায়গায় পরিবর্তন। তাই পরিকল্পনা প্রথমে, বরাদ্দ পরে।

> বাস্তব উদাহরণ: একটি ছোট সফটওয়্যার কোম্পানিতে ডেভেলপার 30 জন, QA 12 জন, সার্ভার 8টি, প্রিন্টার ও অতিথি ডিভাইস 10টি। সবাইকে 192.168.1.0/24-এ রাখলে 254টি ঠিকানা যথেষ্ট মনে হবে। কিন্তু QA-এর একটি ভুল broadcast টেস্ট ডেভেলপারদের কাজ থামিয়ে দেবে, অতিথি ডিভাইস সার্ভার দেখতে পাবে, আর নতুন বিভাগ এলে ঠিকানা ফুরিয়ে যাবে। সঠিক পরিকল্পনা হলো: ডেভেলপারদের /26 (62টি ব্যবহারযোগ্য), QA-কে /27 (30টি), সার্ভারদের /28 (14টি), অতিথিদের /28 (14টি) — মোট চারটি সাবনেট, প্রতিটি আলাদা, প্রতিটির gateway আলাদা, প্রতিটির নিয়ম আলাদা লেখা যায়।

Subnet Mask: ভাগের সীমানা

Subnet mask-ও 32 বিটের একটি সংখ্যা। তার নিয়ম খুব সরল: network portion-এর বিটগুলো 1, host portion-এর বিটগুলো 0। যেমন /24 মানে প্রথম 24 বিট 1, বাকি 8 বিট 0। দশমিকে তা 255.255.255.0। /26 মানে প্রথম 26 বিট 1, বাকি 6 বিট 0 — 255.255.255.192, কারণ শেষ octet-এ 11000000 = 128 + 64 = 192। /25 মানে 255.255.255.128, /27 মানে 255.255.255.224, /28 মানে 255.255.255.240, /29 মানে 255.255.255.248, /30 মানে 255.255.255.252। এই সংখ্যাগুলো মুখস্থ করার চেয়ে পদ্ধতিটি মনে রাখুন: mask-এর 1-বিটগুলো network, 0-বিটগুলো host।

দুটি ঠিকানা একই নেটওয়ার্কে আছে কি না, তা বোঝা যায় AND অপারেশন দিয়ে। ঠিকানার প্রতিটি বিটের সঙ্গে mask-এর সংশ্লিষ্ট বিটের AND নিন: 1 AND 1 = 1, বাকি সব 0। ফলে host portion মুছে যায়, শুধু network portion থাকে। সেটিই network address। যেমন 192.168.1.77-এর শেষ octet 77 = 01001101, /26 mask-এর শেষ octet 192 = 11000000, AND = 01000000 = 64। তাই network address 192.168.1.64। একই নেটওয়ার্কের যেকোনো ঠিকানার AND ফল একই হবে — 192.168.1.65 থেকে 192.168.1.126 পর্যন্ত সবার AND ফল 64। এটিই রাউটার ও অপারেটিং সিস্টেমের ভেতরের যুক্তি: ঠিকানা ও mask-এর AND-ই নেটওয়ার্কের পরিচয়।

CIDR Notation: ছোট লেখা, বড় অর্থ

আগে নেটওয়ার্ককে Class A, B, C-তে ভাগ করা হতো — /8, /16, /24-এ বাঁধা। তাতে ঠিকানা নষ্ট হতো: 300টি হোস্ট দরকার হলে /16 (65,534টি ব্যবহারযোগ্য) নিতে হতো, কারণ /24-এ মাত্র 254টি। CIDR (Classless Inter-Domain Routing) সেই বাঁধন ভেঙেছে। এখন যেকোনো prefix ব্যবহার করা যায় — /23, /22, /21, /20, /19, /18, /17, /25, /26, /27, /28, /29, /30 — প্রয়োজন অনুযায়ী। লেখার নিয়ম: ঠিকানা, তারপর স্ল্যাশ, তারপর network বিটের সংখ্যা। যেমন 192.168.1.0/24, 10.0.0.0/8, 172.16.0.0/12।

মনে রাখার সহজ ছক: host বিট = 32 − prefix। মোট ঠিকানা = 2^(host বিট)। /24-এ host বিট 8, মোট 2^8 = 256। /25-এ host বিট 7, মোট 128। /26-এ host বিট 6, মোট 64। /27-এ 32, /28-এ 16, /29-এ 8, /30-এ 4। অন্যদিকে /23-এ host বিট 9, মোট 512। /22-এ 1024, /21-এ 2048, /20-এ 4096, /19-এ 8192, /18-এ 16384, /17-এ 32768, /16-এ 65536, /12-এ 1,048,576, /8-এ 16,777,216। এই সারণিটি হাতে গণনার মেরুদণ্ড। প্রতিটি উদাহরণে প্রথমে host বিট বের করুন, তারপর 2-এর ঘাত, তারপর block size।

Network Address, Broadcast Address, ব্যবহারযোগ্য হোস্ট

প্রতিটি সাবনেটে তিন ধরনের ঠিকানা থাকে। Network address: host portion-এর সব বিট 0। এটি সাবনেটটির নাম, কোনো ডিভাইসকে দেওয়া যায় না। Broadcast address: host portion-এর সব বিট 1। এটি সাবনেটের সবাইকে পাঠানো বার্তার ঠিকানা, কোনো ডিভাইসকে দেওয়া যায় না। বাকি সব ঠিকানা ব্যবহারযোগ্য — প্রথম ব্যবহারযোগ্য = network address + 1, শেষ ব্যবহারযোগ্য = broadcast address − 1। তাই ব্যবহারযোগ্য হোস্ট = মোট ঠিকানা − 2। এই "বিয়োগ 2" ভুলে গেলে সবচেয়ে বেশি ভুল হয়।

উদাহরণ: 192.168.1.0/24-এ network 192.168.1.0, broadcast 192.168.1.255, ব্যবহারযোগ্য 192.168.1.1 থেকে 192.168.1.254, মোট 256, ব্যবহারযোগ্য 254। 192.168.1.64/26-এ network 192.168.1.64, broadcast 192.168.1.127, ব্যবহারযোগ্য 192.168.1.65 থেকে 192.168.1.126, মোট 64, ব্যবহারযোগ্য 62। লক্ষ করুন: broadcast সবসময় পরের network-এর ঠিক আগের ঠিকানা। /26-এ 64-এর পরের network 128, তাই broadcast 127। এই সম্পর্ক যাচাইয়ের দ্রুত উপায়।

দুটি ব্যতিক্রম মনে রাখুন। /31-এ মোট ঠিকানা 2, ব্যবহারযোগ্য সাধারণ নিয়মে 0 হয়, কিন্তু point-to-point লিংকে আধুনিক সিস্টেমে /31-এর দুটি ঠিকানাই হোস্ট হিসেবে ব্যবহার করা যায় — network ও broadcast আলাদা রাখা হয় না। /32 মানে একটি মাত্র ঠিকানা, কোনো network নয় — একটি নির্দিষ্ট হোস্টকে বোঝাতে, যেমন loopback বা একটি সার্ভারের ঠিকানা রাউটে লিখতে ব্যবহৃত হয়। এই অধ্যায়ের গণনায় /31 ও /32-কে ব্যতিক্রম হিসেবে চিহ্নিত করা হয়েছে; মূল সূত্রটি /30 পর্যন্ত নিয়মিত নেটওয়ার্কের জন্য।

Block Size ও সীমানা খোঁজার পদ্ধতি

হাতে গণনার সবচেয়ে কার্যকর পদ্ধতি হলো block size। যে octet-এ mask 255 নয়, সেই octet-এ block size = 256 − mask-এর ওই octet-এর মান। /26-এ শেষ octet-এর mask 192, block size 256 − 192 = 64। তাই network-গুলো 0, 64, 128, 192-এ শুরু হয়। /27-এ mask 224, block size 32 — network 0, 32, 64, 96, 128, 160, 192, 224। /28-এ

block 16, /29-এ block 8, /30-এ block 4, /25-এ block 128। কোনো ঠিকানা দেওয়া থাকলে দেখুন তা কোন block-এ পড়ে: যেমন 192.168.1.77/26-এ 77 পড়ে 64-এর block-এ (64 থেকে 127), তাই network 64, broadcast 127।

তৃতীয় octet-এ ভাগ হলে একই যুক্তি তৃতীয় octet-এ প্রয়োগ করুন। /23-এ তৃতীয় octet-এর mask 254, block size 256 − 254 = 2 — network তৃতীয় octet-এ 0, 2, 4, 6... ধাপে বাড়ে। /22-এ mask 252, block 4 — 0, 4, 8, 12...। /21-এ mask 248, block 8। /20-এ mask 240, block 16। /19-এ mask 224, block 32। /18-এ mask 192, block 64। /17-এ mask 128, block 128। /16-এ তৃতীয় octet-এর mask 0, block 256 — অর্থাৎ তৃতীয় octet পুরোটা host অংশ। এই ছকটি মুখস্থ না করে বুঝে নিন: block size সবসময় 2-এর ঘাত, আর network সবসময় block size-এর গুণিতক থেকে শুরু হয়। কোনো network address যদি block size-এর গুণিতক না হয়, তবে গণনা ভুল হয়েছে — এটিই দ্রুত যাচাই।

VLSM: একই নেটওয়ার্কে বিভিন্ন আকারের সাবনেট

Fixed Length Subnet Mask (FLSM)-এ একটি নেটওয়ার্ককে সমান আকারের টুকরোয় ভাগ করা হয়। যেমন 192.168.1.0/24-কে চারটি /26-এ ভাগ করলে প্রতিটি 62টি ব্যবহারযোগ্য হোস্ট দেয়। কিন্তু বাস্তবে বিভাগগুলো সমান হয় না — একটিতে 100 জন, আরেকটিতে 10 জন। সমান ভাগ করলে ছোট বিভাগে ঠিকানা নষ্ট হয়, বড় বিভাগে ঠিকানা কম পড়ে। Variable Length Subnet Mask (VLSM) এই সমস্যার সমাধান: একই মূল নেটওয়ার্কের ভেতরে বিভিন্ন prefix ব্যবহার করা। বড় বিভাগকে /25, মাঝারিকে /26, ছোটকে /27 বা /28, আর রাউটারের মধ্যে point-to-point লিংকে /30।

VLSM-এর নিয়ম তিনটি। প্রথম: বড় চাহিদা আগে। সবচেয়ে বেশি হোস্ট যার দরকার, তাকে প্রথমে বরাদ্দ দিন, তারপর পরের জনকে। ছোটকে আগে দিলে বড়টির জন্য টানা জায়গা নাও থাকতে পারে। দ্বিতীয়: প্রতিটি বরাদ্দের জন্য প্রয়োজনীয় host বিট বের করুন — হোস্ট সংখ্যা + 2 (network + broadcast) ঢোকে এমন সবচেয়ে ছোট 2-এর ঘাত নিন। যেমন 50 জন হোস্টের জন্য 52 দরকার, সবচেয়ে ছোট ঘাত 64, তাই /26। 20 জনের জন্য 22 দরকার, ঘাত 32, তাই /27। 10 জনের জন্য 12 দরকার, ঘাত 16, তাই /28। 2 জনের জন্য 4 দরকার, তাই /30। তৃতীয়: প্রতিটি বরাদ্দের পর পরের ফাঁকা ঠিকানা থেকে শুরু করুন, আর প্রতিটি network যেন block size-এর গুণিতক থেকে শুরু হয় তা যাচাই করুন।

VLSM-এর সবচেয়ে বড় ভুল হলো overlapping — দুটি সাবনেটের পরিসর একে অপরের ভেতরে ঢুকে যাওয়া। যেমন 192.168.1.0/25 (0–127) আর 192.168.1.64/26 (64–127) একসঙ্গে বরাদ্দ দিলে 64–127 পরিসরটি দুইবার দেওয়া হলো। রাউটার তখন জানে না 192.168.1.100-এর প্যাকেট কোন সাবনেটে যাবে। তাই প্রতিটি বরাদ্দের পর একটি সারণি লিখুন: network, broadcast, পরিসর, বরাদ্দপ্রাপ্ত বিভাগ। সারণি দেখেই overlap ধরা পড়ে। এই অধ্যায়ের উদাহরণ ৩০-এ একটি সম্পূর্ণ VLSM ভাগ ধাপে ধাপে দেখানো হয়েছে।

> সতর্কতা: VLSM-এ "প্রায় হবে" বলে কিছু নেই। 50 জন হোস্টের জন্য /27 (30টি ব্যবহারযোগ্য) দিলে 20 জন বাদ পড়বে — পরে আর ঠিকানা বাড়ানো যাবে না, কারণ পরের পরিসরটি অন্য বিভাগকে দেওয়া হয়ে গেছে। সবসময় network + broadcast-সহ গণনা করুন, আর ভবিষ্যৎ বৃদ্ধির জন্য কমপক্ষে 20–30% অতিরিক্ত জায়গা রাখুন।

একটি IP পরিকল্পনা কীভাবে সাজাবেন

পরিকল্পনা শুরু হয় প্রশ্ন দিয়ে, ঠিকানা দিয়ে নয়। প্রথমে লিখুন: কতগুলো সাইট (অফিস/শাখা) আছে, প্রতিটি সাইটে কতগুলো বিভাগ, প্রতিটি বিভাগে আজ কতগুলো ডিভাইস, এক বছরে কত হবে, কোন বিভাগকে আলাদা রাখতে হবে, কোন সার্ভিসগুলো (সার্ভার, প্রিন্টার, অতিথি, IoT, ব্যবস্থাপনা) আলাদা সাবনেট চায়। তারপর মূল পরিসর বেছে নিন। ছোট প্রতিষ্ঠানের জন্য 192.168.0.0/16 যথেষ্ট মনে হতে পারে, কিন্তু একাধিক সাইট থাকলে 10.0.0.0/8 ভালো — প্রতিটি সাইটকে একটি /16 দিলে 256টি সাইট পর্যন্ত জায়গা থাকে, প্রতিটি সাইটের ভেতরে বিভাগভিত্তিক /24 বা VLSM। 172.16.0.0/12 মাঝারি আকারের জন্য উপযুক্ত।

দ্বিতীয় ধাপ: সাইট ও বিভাগের জন্য পরিসর বরাদ্দ করুন এবং লিখে রাখুন। যেমন ঢাকা অফিস 10.1.0.0/16, চট্টগ্রাম অফিস 10.2.0.0/16, সিলেট অফিস 10.3.0.0/16। ঢাকার ভেতরে: ডেভেলপার 10.1.1.0/24, QA 10.1.2.0/24, সার্ভার 10.1.10.0/24, ব্যবস্থাপনা 10.1.99.0/24, অতিথি 10.1.200.0/24। এই ছকে প্রতিটি সাবনেটের নম্বর থেকেই বোঝা যায় সেটি কীসের — 10 নম্বর সার্ভার, 200 নম্বর অতিথি। তৃতীয় ধাপ: gateway ঠিকানা ঠিক করুন। প্রচলিত নিয়ম: সাবনেটের প্রথম ব্যবহারযোগ্য ঠিকানা (.1) gateway, অথবা শেষ ব্যবহারযোগ্য ঠিকানা (.254)। যেকোনো একটিকে নিয়ম বানান, সব সাবনেটে একই নিয়ম রাখুন। চতুর্থ ধাপ: সংরক্ষিত পরিসর রাখুন — ভবিষ্যৎ বিভাগের জন্য কিছু সাবনেট ফাঁকা রাখুন, প্রতিটি সাবনেটের ভেতরে কিছু ঠিকানা DHCP-র বাইরে static-এর জন্য রাখুন। পঞ্চম ধাপ: নথি লিখুন — কোন সাবনেট কার, gateway কী, DNS কী, কে বরাদ্দ দিয়েছে, কবে। নথি ছাড়া পরিকল্পনা এক মাসেই হারিয়ে যায়।

পরিকল্পনার আরও কয়েকটি নীতি। এক: একই সাবনেটে 254-এর বেশি ডিভাইস রাখবেন না, broadcast বড় হয়ে যাবে — বড় বিভাগকে /23 বা /22 দিন, তবু broadcast-এর সীমা মনে রাখুন। দুই: সার্ভার, ওয়ার্কস্টেশন, প্রিন্টার, অতিথি, ব্যবস্থাপনা (management) — এই ধরনের ডিভাইস আলাদা সাবনেটে রাখুন। তিন: VPN-এর দুই প্রান্তের পরিসর যেন না মেলে তা যাচাই করুন — দুই প্রান্তেই 192.168.1.0/24 থাকলে রাউটার গুলিয়ে ফেলবে। চার: DHCP-র পরিসর (pool) সাবনেটের ভেতরে সীমাবদ্ধ রাখুন, gateway ও static সার্ভারের ঠিকানা pool-এর বাইরে রাখুন। পাঁচ: প্রতিটি সাবনেটের জন্য একটি নাম ও দায়িত্বপ্রাপ্ত ব্যক্তি লিখুন।

কাজ করা উদাহরণ: ৩০টি সাবনেটিং গণনা

এই অংশে ৩০টি উদাহরণ ধাপে ধাপে করা হয়েছে। প্রতিটি উদাহরণে একই ছক ব্যবহার করা হয়েছে: (১) prefix থেকে host বিট ও মোট ঠিকানা, (২) mask ও block size, (৩) network address, (৪) broadcast address, (৫) প্রথম ও শেষ ব্যবহারযোগ্য হোস্ট, (৬) মোট ও ব্যবহারযোগ্য সংখ্যা। প্রথম কয়েকটি ধীরে পড়ুন, তারপর নিজে ঢেকে উত্তর বের করে মিলিয়ে নিন। সব ঠিকানা private পরিসর (10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16) থেকে নেওয়া হয়েছে।

কাজ করা উদাহরণ ১ — 192.168.1.0/24

ধাপ ১: host বিট = 32 − 24 = 8। মোট ঠিকানা = 2^8 = 256। ধাপ ২: mask 255.255.255.0, block size 256। ধাপ ৩: network 192.168.1.0। ধাপ ৪: broadcast = network + 256 − 1 = 192.168.1.255। ধাপ ৫: প্রথম ব্যবহারযোগ্য 192.168.1.1, শেষ ব্যবহারযোগ্য 192.168.1.254।

- Network: 192.168.1.0
- Broadcast: 192.168.1.255
- প্রথম ব্যবহারযোগ্য: 192.168.1.1; শেষ ব্যবহারযোগ্য: 192.168.1.254
- মোট ঠিকানা: 256; ব্যবহারযোগ্য হোস্ট: 254; CIDR: /24; Mask: 255.255.255.0

কাজ করা উদাহরণ ২ — 192.168.1.0/25

ধাপ ১: host বিট = 7, মোট = 2^7 = 128। ধাপ ২: mask 255.255.255.128, block size = 256 − 128 = 128। তাই /25 network-গুলো 0 ও 128-এ শুরু হয়। ধাপ ৩: 0-এর block-এ network 192.168.1.0। ধাপ ৪: broadcast 192.168.1.127। ধাপ ৫: ব্যবহারযোগ্য 192.168.1.1 থেকে 192.168.1.126।

- Network: 192.168.1.0
- Broadcast: 192.168.1.127
- প্রথম ব্যবহারযোগ্য: 192.168.1.1; শেষ ব্যবহারযোগ্য: 192.168.1.126
- মোট ঠিকানা: 128; ব্যবহারযোগ্য হোস্ট: 126; CIDR: /25; Mask: 255.255.255.128

কাজ করা উদাহরণ ৩ — 192.168.1.128/25

ধাপ ১: host বিট 7, মোট 128। ধাপ ২: mask 255.255.255.128, block 128। ধাপ ৩: 128-এর block-এ network 192.168.1.128। ধাপ ৪: broadcast 192.168.1.255। ধাপ ৫: ব্যবহারযোগ্য 192.168.1.129 থেকে 192.168.1.254। এটি উদাহরণ ২-এর দ্বিতীয় অর্ধেক — /24-কে দুটি /25-এ ভাগ করলে এই দুটি সাবনেট পাওয়া যায়।

- Network: 192.168.1.128
- Broadcast: 192.168.1.255
- প্রথম ব্যবহারযোগ্য: 192.168.1.129; শেষ ব্যবহারযোগ্য: 192.168.1.254
- মোট ঠিকানা: 128; ব্যবহারযোগ্য হোস্ট: 126; CIDR: /25; Mask: 255.255.255.128

কাজ করা উদাহরণ ৪ — 192.168.1.64/26

ধাপ ১: host বিট = 6, মোট = 2^6 = 64। ধাপ ২: mask 255.255.255.192, block size = 256 − 192 = 64। Network-গুলো 0, 64, 128, 192। ধাপ ৩: network 192.168.1.64। ধাপ ৪: broadcast = 64 + 64 − 1 = 192.168.1.127। ধাপ ৫: ব্যবহারযোগ্য 192.168.1.65 থেকে 192.168.1.126। বাইনারিতে: 64 = 01000000, mask-এর শেষ octet 11000000, AND করলে network-এর শেষ octet 01000000 = 64 মেলে।

- Network: 192.168.1.64
- Broadcast: 192.168.1.127
- প্রথম ব্যবহারযোগ্য: 192.168.1.65; শেষ ব্যবহারযোগ্য: 192.168.1.126
- মোট ঠিকানা: 64; ব্যবহারযোগ্য হোস্ট: 62; CIDR: /26; Mask: 255.255.255.192

কাজ করা উদাহরণ ৫ — 192.168.1.192/26

ধাপ ১: host বিট 6, মোট 64। ধাপ ২: mask 255.255.255.192, block 64। ধাপ ৩: network 192.168.1.192। ধাপ ৪: broadcast 192.168.1.255। ধাপ ৫: ব্যবহারযোগ্য 192.168.1.193 থেকে 192.168.1.254। এটি /24-এর চতুর্থ /26 — 0–63, 64–127, 128–191, 192–255 এই চারটি ভাগের শেষটি।

- Network: 192.168.1.192
- Broadcast: 192.168.1.255
- প্রথম ব্যবহারযোগ্য: 192.168.1.193; শেষ ব্যবহারযোগ্য: 192.168.1.254
- মোট ঠিকানা: 64; ব্যবহারযোগ্য হোস্ট: 62; CIDR: /26; Mask: 255.255.255.192

কাজ করা উদাহরণ ৬ — 192.168.1.32/27

ধাপ ১: host বিট = 5, মোট = 2^5 = 32। ধাপ ২: mask 255.255.255.224, block size = 256 − 224 = 32। Network-গুলো 0, 32, 64, 96, 128, 160, 192, 224। ধাপ ৩: network 192.168.1.32। ধাপ ৪: broadcast 192.168.1.63। ধাপ ৫: ব্যবহারযোগ্য 192.168.1.33 থেকে 192.168.1.62।

- Network: 192.168.1.32
- Broadcast: 192.168.1.63
- প্রথম ব্যবহারযোগ্য: 192.168.1.33; শেষ ব্যবহারযোগ্য: 192.168.1.62
- মোট ঠিকানা: 32; ব্যবহারযোগ্য হোস্ট: 30; CIDR: /27; Mask: 255.255.255.224

কাজ করা উদাহরণ ৭ — 192.168.1.96/27

ধাপ ১: host বিট 5, মোট 32। ধাপ ২: mask 255.255.255.224, block 32। ধাপ ৩: network 192.168.1.96। ধাপ ৪: broadcast = 96 + 32 − 1 = 192.168.1.127। ধাপ ৫: ব্যবহারযোগ্য 192.168.1.97 থেকে 192.168.1.126। লক্ষ করুন, 96 হলো 32-এর গুণিতক (32 × 3), তাই network হিসেবে বৈধ।

- Network: 192.168.1.96
- Broadcast: 192.168.1.127
- প্রথম ব্যবহারযোগ্য: 192.168.1.97; শেষ ব্যবহারযোগ্য: 192.168.1.126
- মোট ঠিকানা: 32; ব্যবহারযোগ্য হোস্ট: 30; CIDR: /27; Mask: 255.255.255.224

কাজ করা উদাহরণ ৮ — 192.168.2.16/28

ধাপ ১: host বিট = 4, মোট = 2^4 = 16। ধাপ ২: mask 255.255.255.240, block size = 256 − 240 = 16। Network-গুলো 0, 16, 32, 48...। ধাপ ৩: network 192.168.2.16। ধাপ ৪: broadcast 192.168.2.31। ধাপ ৫: ব্যবহারযোগ্য 192.168.2.17 থেকে 192.168.2.30।

- Network: 192.168.2.16
- Broadcast: 192.168.2.31
- প্রথম ব্যবহারযোগ্য: 192.168.2.17; শেষ ব্যবহারযোগ্য: 192.168.2.30
- মোট ঠিকানা: 16; ব্যবহারযোগ্য হোস্ট: 14; CIDR: /28; Mask: 255.255.255.240

কাজ করা উদাহরণ ৯ — 192.168.2.0/29

ধাপ ১: host বিট = 3, মোট = 2^3 = 8। ধাপ ২: mask 255.255.255.248, block size = 256 − 248 = 8। ধাপ ৩: network 192.168.2.0। ধাপ ৪: broadcast 192.168.2.7। ধাপ ৫: ব্যবহারযোগ্য 192.168.2.1 থেকে 192.168.2.6। ছোট দল বা প্রিন্টার-সার্ভারের মতো 6টি ডিভাইসের জন্য /29 উপযুক্ত।

- Network: 192.168.2.0
- Broadcast: 192.168.2.7
- প্রথম ব্যবহারযোগ্য: 192.168.2.1; শেষ ব্যবহারযোগ্য: 192.168.2.6
- মোট ঠিকানা: 8; ব্যবহারযোগ্য হোস্ট: 6; CIDR: /29; Mask: 255.255.255.248

কাজ করা উদাহরণ ১০ — 192.168.2.8/29

ধাপ ১: host বিট 3, মোট 8। ধাপ ২: mask 255.255.255.248, block 8। ধাপ ৩: network 192.168.2.8। ধাপ ৪: broadcast 192.168.2.15। ধাপ ৫: ব্যবহারযোগ্য 192.168.2.9 থেকে 192.168.2.14।

- Network: 192.168.2.8
- Broadcast: 192.168.2.15
- প্রথম ব্যবহারযোগ্য: 192.168.2.9; শেষ ব্যবহারযোগ্য: 192.168.2.14
- মোট ঠিকানা: 8; ব্যবহারযোগ্য হোস্ট: 6; CIDR: /29; Mask: 255.255.255.248

কাজ করা উদাহরণ ১১ — 192.168.2.4/30

ধাপ ১: host বিট = 2, মোট = 2^2 = 4। ধাপ ২: mask 255.255.255.252, block size = 256 − 252 = 4। Network-গুলো 0, 4, 8, 12...। ধাপ ৩: network 192.168.2.4। ধাপ ৪: broadcast 192.168.2.7। ধাপ ৫: ব্যবহারযোগ্য 192.168.2.5 ও 192.168.2.6 — মাত্র দুটি। /30 সাধারণত দুটি রাউটারের মধ্যে point-to-point লিংকে ব্যবহৃত হয়।

- Network: 192.168.2.4
- Broadcast: 192.168.2.7
- প্রথম ব্যবহারযোগ্য: 192.168.2.5; শেষ ব্যবহারযোগ্য: 192.168.2.6
- মোট ঠিকানা: 4; ব্যবহারযোগ্য হোস্ট: 2; CIDR: /30; Mask: 255.255.255.252

কাজ করা উদাহরণ ১২ — 10.0.0.0/8

ধাপ ১: host বিট = 24, মোট = 2^24 = 16,777,216। ধাপ ২: mask 255.0.0.0। ধাপ ৩: network 10.0.0.0। ধাপ ৪: broadcast 10.255.255.255 — host অংশের সব বিট 1। ধাপ ৫: ব্যবহারযোগ্য 10.0.0.1 থেকে 10.255.255.254। এটি সবচেয়ে বড় private পরিসর; একে কখনো একটি সাবনেট হিসেবে ব্যবহার করবেন না, বরং এর ভেতর থেকে ছোট ছোট সাবনেট কেটে নিন।

- Network: 10.0.0.0
- Broadcast: 10.255.255.255
- প্রথম ব্যবহারযোগ্য: 10.0.0.1; শেষ ব্যবহারযোগ্য: 10.255.255.254
- মোট ঠিকানা: 16,777,216; ব্যবহারযোগ্য হোস্ট: 16,777,214; CIDR: /8; Mask: 255.0.0.0

কাজ করা উদাহরণ ১৩ — 10.10.0.0/16

ধাপ ১: host বিট = 16, মোট = 2^16 = 65,536। ধাপ ২: mask 255.255.0.0। ধাপ ৩: network 10.10.0.0। ধাপ ৪: broadcast 10.10.255.255। ধাপ ৫: ব্যবহারযোগ্য 10.10.0.1 থেকে 10.10.255.254। একটি সাইটকে /16 দিলে তার ভেতরে 256টি /24 সাবনেট বানানো যায়।

- Network: 10.10.0.0
- Broadcast: 10.10.255.255
- প্রথম ব্যবহারযোগ্য: 10.10.0.1; শেষ ব্যবহারযোগ্য: 10.10.255.254
- মোট ঠিকানা: 65,536; ব্যবহারযোগ্য হোস্ট: 65,534; CIDR: /16; Mask: 255.255.0.0

কাজ করা উদাহরণ ১৪ — 10.20.30.0/23

ধাপ ১: host বিট = 9, মোট = 2^9 = 512। ধাপ ২: mask 255.255.254.0। তৃতীয় octet-এ block size = 256 − 254 = 2, তাই তৃতীয় octet জোড় সংখ্যা থেকে শুরু হয়: 30 জোড়, বৈধ। ধাপ ৩: network 10.20.30.0। ধাপ ৪: broadcast 10.20.31.255 — তৃতীয় octet 30 ও 31 দুটি মিলে 512টি ঠিকানা। ধাপ ৫: ব্যবহারযোগ্য 10.20.30.1 থেকে 10.20.31.254।

- Network: 10.20.30.0
- Broadcast: 10.20.31.255
- প্রথম ব্যবহারযোগ্য: 10.20.30.1; শেষ ব্যবহারযোগ্য: 10.20.31.254
- মোট ঠিকানা: 512; ব্যবহারযোগ্য হোস্ট: 510; CIDR: /23; Mask: 255.255.254.0

কাজ করা উদাহরণ ১৫ — 172.16.0.0/12

ধাপ ১: host বিট = 20, মোট = 2^20 = 1,048,576। ধাপ ২: mask 255.240.0.0। বাইনারিতে দ্বিতীয় octet 11110000 = 240। ধাপ ৩: network 172.16.0.0। ধাপ ৪: broadcast 172.31.255.255 — দ্বিতীয় octet 16 থেকে 31 পর্যন্ত 16টি মান, প্রতিটিতে 65,536টি ঠিকানা। ধাপ ৫: ব্যবহারযোগ্য 172.16.0.1 থেকে 172.31.255.254।

- Network: 172.16.0.0
- Broadcast: 172.31.255.255
- প্রথম ব্যবহারযোগ্য: 172.16.0.1; শেষ ব্যবহারযোগ্য: 172.31.255.254
- মোট ঠিকানা: 1,048,576; ব্যবহারযোগ্য হোস্ট: 1,048,574; CIDR: /12; Mask: 255.240.0.0

কাজ করা উদাহরণ ১৬ — 172.16.0.0/16

ধাপ ১: host বিট = 16, মোট = 65,536। ধাপ ২: mask 255.255.0.0। ধাপ ৩: network 172.16.0.0। ধাপ ৪: broadcast 172.16.255.255। ধাপ ৫: ব্যবহারযোগ্য 172.16.0.1 থেকে 172.16.255.254। 172.16.0.0/12-এর ভেতরে 16টি /16 আছে — 172.16.0.0/16, 172.17.0.0/16, এবং এভাবে 172.31.0.0/16 পর্যন্ত।

- Network: 172.16.0.0
- Broadcast: 172.16.255.255
- প্রথম ব্যবহারযোগ্য: 172.16.0.1; শেষ ব্যবহারযোগ্য: 172.16.255.254
- মোট ঠিকানা: 65,536; ব্যবহারযোগ্য হোস্ট: 65,534; CIDR: /16; Mask: 255.255.0.0

কাজ করা উদাহরণ ১৭ — 172.16.4.0/22

ধাপ ১: host বিট = 10, মোট = 2^10 = 1,024। ধাপ ২: mask 255.255.252.0। তৃতীয় octet-এ block size = 256 − 252 = 4, network তৃতীয় octet-এ 0, 4, 8, 12 ধাপে। ধাপ ৩: network 172.16.4.0। ধাপ ৪: broadcast 172.16.7.255 — তৃতীয় octet 4, 5, 6, 7 মিলে 1,024টি ঠিকানা। ধাপ ৫: ব্যবহারযোগ্য 172.16.4.1 থেকে 172.16.7.254।

- Network: 172.16.4.0
- Broadcast: 172.16.7.255
- প্রথম ব্যবহারযোগ্য: 172.16.4.1; শেষ ব্যবহারযোগ্য: 172.16.7.254
- মোট ঠিকানা: 1,024; ব্যবহারযোগ্য হোস্ট: 1,022; CIDR: /22; Mask: 255.255.252.0

কাজ করা উদাহরণ ১৮ — 172.16.8.0/21

ধাপ ১: host বিট = 11, মোট = 2^11 = 2,048। ধাপ ২: mask 255.255.248.0। তৃতীয় octet-এ block size = 256 − 248 = 8, network 0, 8, 16, 24 ধাপে। ধাপ ৩: network 172.16.8.0। ধাপ ৪: broadcast 172.16.15.255 — তৃতীয় octet 8 থেকে 15 পর্যন্ত 8টি মান। ধাপ ৫: ব্যবহারযোগ্য 172.16.8.1 থেকে 172.16.15.254।

- Network: 172.16.8.0
- Broadcast: 172.16.15.255
- প্রথম ব্যবহারযোগ্য: 172.16.8.1; শেষ ব্যবহারযোগ্য: 172.16.15.254
- মোট ঠিকানা: 2,048; ব্যবহারযোগ্য হোস্ট: 2,046; CIDR: /21; Mask: 255.255.248.0

কাজ করা উদাহরণ ১৯ — 192.168.0.0/20

ধাপ ১: host বিট = 12, মোট = 2^12 = 4,096। ধাপ ২: mask 255.255.240.0। তৃতীয় octet-এ block size = 256 − 240 = 16, network 0, 16, 32 ধাপে। ধাপ ৩: network 192.168.0.0। ধাপ ৪: broadcast 192.168.15.255 — তৃতীয় octet 0 থেকে 15। ধাপ ৫: ব্যবহারযোগ্য 192.168.0.1 থেকে 192.168.15.254। একটি /20-এর ভেতরে 16টি /24 আছে।

- Network: 192.168.0.0
- Broadcast: 192.168.15.255
- প্রথম ব্যবহারযোগ্য: 192.168.0.1; শেষ ব্যবহারযোগ্য: 192.168.15.254
- মোট ঠিকানা: 4,096; ব্যবহারযোগ্য হোস্ট: 4,094; CIDR: /20; Mask: 255.255.240.0

কাজ করা উদাহরণ ২০ — 192.168.16.0/20

ধাপ ১: host বিট 12, মোট 4,096। ধাপ ২: mask 255.255.240.0, block 16। ধাপ ৩: network 192.168.16.0 — 16 হলো 16-এর গুণিতক, বৈধ। ধাপ ৪: broadcast 192.168.31.255 — তৃতীয় octet 16 থেকে 31। ধাপ ৫: ব্যবহারযোগ্য 192.168.16.1 থেকে 192.168.31.254। এটি উদাহরণ ১৯-এর পরের /20।

- Network: 192.168.16.0
- Broadcast: 192.168.31.255
- প্রথম ব্যবহারযোগ্য: 192.168.16.1; শেষ ব্যবহারযোগ্য: 192.168.31.254
- মোট ঠিকানা: 4,096; ব্যবহারযোগ্য হোস্ট: 4,094; CIDR: /20; Mask: 255.255.240.0

কাজ করা উদাহরণ ২১ — 10.0.0.0/17

ধাপ ১: host বিট = 15, মোট = 2^15 = 32,768। ধাপ ২: mask 255.255.128.0। তৃতীয় octet-এ block size = 256 − 128 = 128, network তৃতীয় octet-এ 0 ও 128-এ। ধাপ ৩: network 10.0.0.0। ধাপ ৪: broadcast 10.0.127.255 — তৃতীয় octet 0 থেকে 127। ধাপ ৫: ব্যবহারযোগ্য 10.0.0.1 থেকে 10.0.127.254।

- Network: 10.0.0.0
- Broadcast: 10.0.127.255
- প্রথম ব্যবহারযোগ্য: 10.0.0.1; শেষ ব্যবহারযোগ্য: 10.0.127.254
- মোট ঠিকানা: 32,768; ব্যবহারযোগ্য হোস্ট: 32,766; CIDR: /17; Mask: 255.255.128.0

কাজ করা উদাহরণ ২২ — 10.0.128.0/17

ধাপ ১: host বিট 15, মোট 32,768। ধাপ ২: mask 255.255.128.0, block 128। ধাপ ৩: network 10.0.128.0। ধাপ ৪: broadcast 10.0.255.255 — তৃতীয় octet 128 থেকে 255। ধাপ ৫: ব্যবহারযোগ্য 10.0.128.1 থেকে 10.0.255.254। উদাহরণ ২১ ও ২২ মিলে 10.0.0.0/16-কে দুটি /17-এ ভাগ করে।

- Network: 10.0.128.0
- Broadcast: 10.0.255.255
- প্রথম ব্যবহারযোগ্য: 10.0.128.1; শেষ ব্যবহারযোগ্য: 10.0.255.254
- মোট ঠিকানা: 32,768; ব্যবহারযোগ্য হোস্ট: 32,766; CIDR: /17; Mask: 255.255.128.0

কাজ করা উদাহরণ ২৩ — 172.16.0.0/18

ধাপ ১: host বিট = 14, মোট = 2^14 = 16,384। ধাপ ২: mask 255.255.192.0। তৃতীয় octet-এ block size = 256 − 192 = 64, network 0, 64, 128, 192 ধাপে। ধাপ ৩: network 172.16.0.0। ধাপ ৪: broadcast 172.16.63.255 — তৃতীয় octet 0 থেকে 63। ধাপ ৫: ব্যবহারযোগ্য 172.16.0.1 থেকে 172.16.63.254।

- Network: 172.16.0.0
- Broadcast: 172.16.63.255
- প্রথম ব্যবহারযোগ্য: 172.16.0.1; শেষ ব্যবহারযোগ্য: 172.16.63.254
- মোট ঠিকানা: 16,384; ব্যবহারযোগ্য হোস্ট: 16,382; CIDR: /18; Mask: 255.255.192.0

কাজ করা উদাহরণ ২৪ — 172.16.64.0/18

ধাপ ১: host বিট 14, মোট 16,384। ধাপ ২: mask 255.255.192.0, block 64। ধাপ ৩: network 172.16.64.0। ধাপ ৪: broadcast 172.16.127.255 — তৃতীয় octet 64 থেকে 127। ধাপ ৫: ব্যবহারযোগ্য 172.16.64.1 থেকে 172.16.127.254।

- Network: 172.16.64.0
- Broadcast: 172.16.127.255
- প্রথম ব্যবহারযোগ্য: 172.16.64.1; শেষ ব্যবহারযোগ্য: 172.16.127.254
- মোট ঠিকানা: 16,384; ব্যবহারযোগ্য হোস্ট: 16,382; CIDR: /18; Mask: 255.255.192.0

কাজ করা উদাহরণ ২৫ — 192.168.5.100/19 (হোস্ট ঠিকানা থেকে network বের করা)

এখানে দেওয়া ঠিকানাটি network address নয়, একটি হোস্টের ঠিকানা। ধাপ ১: host বিট = 13, মোট = 2^13 = 8,192। ধাপ ২: mask 255.255.224.0। তৃতীয় octet-এ block size = 256 − 224 = 32, network তৃতীয় octet-এ 0, 32, 64 ধাপে। ধাপ ৩: বাইনারিতে তৃতীয় octet 5 = 00000101, mask 224 = 11100000, AND = 00000000 = 0। তাই network 192.168.0.0। ধাপ ৪: broadcast = 192.168.31.255 — তৃতীয় octet 0 থেকে 31। ধাপ ৫: ব্যবহারযোগ্য 192.168.0.1 থেকে 192.168.31.254। দেওয়া হোস্ট 192.168.5.100 এই পরিসরের ভেতরে আছে, তাই সঠিক।

- Network: 192.168.0.0
- Broadcast: 192.168.31.255
- প্রথম ব্যবহারযোগ্য: 192.168.0.1; শেষ ব্যবহারযোগ্য: 192.168.31.254
- মোট ঠিকানা: 8,192; ব্যবহারযোগ্য হোস্ট: 8,190; CIDR: /19; Mask: 255.255.224.0

কাজ করা উদাহরণ ২৬ — 192.168.10.77/26 (হোস্ট ঠিকানা থেকে network বের করা)

ধাপ ১: host বিট = 6, মোট = 64। ধাপ ২: mask 255.255.255.192, block 64। ধাপ ৩: 77 পড়ে 64-এর block-এ (64–127), তাই network 192.168.10.64। বাইনারিতে 77 = 01001101, mask 11000000, AND = 01000000 = 64। ধাপ ৪: broadcast 192.168.10.127। ধাপ ৫: ব্যবহারযোগ্য 192.168.10.65 থেকে 192.168.10.126। হোস্ট 77 এই পরিসরের ভেতরে আছে।

- Network: 192.168.10.64
- Broadcast: 192.168.10.127
- প্রথম ব্যবহারযোগ্য: 192.168.10.65; শেষ ব্যবহারযোগ্য: 192.168.10.126
- মোট ঠিকানা: 64; ব্যবহারযোগ্য হোস্ট: 62; CIDR: /26; Mask: 255.255.255.192

কাজ করা উদাহরণ ২৭ — 10.5.6.7/27 (হোস্ট ঠিকানা থেকে network বের করা)

ধাপ ১: host বিট = 5, মোট = 32। ধাপ ২: mask 255.255.255.224, block 32। ধাপ ৩: 7 পড়ে 0-এর block-এ (0–31), তাই network 10.5.6.0। ধাপ ৪: broadcast 10.5.6.31। ধাপ ৫: ব্যবহারযোগ্য 10.5.6.1 থেকে 10.5.6.30। হোস্ট 7 এই পরিসরের ভেতরে আছে।

- Network: 10.5.6.0
- Broadcast: 10.5.6.31
- প্রথম ব্যবহারযোগ্য: 10.5.6.1; শেষ ব্যবহারযোগ্য: 10.5.6.30
- মোট ঠিকানা: 32; ব্যবহারযোগ্য হোস্ট: 30; CIDR: /27; Mask: 255.255.255.224

কাজ করা উদাহরণ ২৮ — 172.16.10.200/28 (হোস্ট ঠিকানা থেকে network বের করা)

ধাপ ১: host বিট = 4, মোট = 16। ধাপ ২: mask 255.255.255.240, block 16। ধাপ ৩: 200 পড়ে 192-এর block-এ (192–207), তাই network 172.16.10.192। বাইনারিতে 200 = 11001000, mask 11110000, AND = 11000000 = 192। ধাপ ৪: broadcast 172.16.10.207। ধাপ ৫: ব্যবহারযোগ্য 172.16.10.193 থেকে 172.16.10.206।

- Network: 172.16.10.192
- Broadcast: 172.16.10.207
- প্রথম ব্যবহারযোগ্য: 172.16.10.193; শেষ ব্যবহারযোগ্য: 172.16.10.206
- মোট ঠিকানা: 16; ব্যবহারযোগ্য হোস্ট: 14; CIDR: /28; Mask: 255.255.255.240

কাজ করা উদাহরণ ২৯ — 192.168.50.130/25 (হোস্ট ঠিকানা থেকে network বের করা)

ধাপ ১: host বিট = 7, মোট = 128। ধাপ ২: mask 255.255.255.128, block 128। ধাপ ৩: 130 পড়ে 128-এর block-এ (128–255), তাই network 192.168.50.128। ধাপ ৪: broadcast 192.168.50.255। ধাপ ৫: ব্যবহারযোগ্য 192.168.50.129 থেকে 192.168.50.254।

- Network: 192.168.50.128
- Broadcast: 192.168.50.255
- প্রথম ব্যবহারযোগ্য: 192.168.50.129; শেষ ব্যবহারযোগ্য: 192.168.50.254
- মোট ঠিকানা: 128; ব্যবহারযোগ্য হোস্ট: 126; CIDR: /25; Mask: 255.255.255.128

কাজ করা উদাহরণ ৩০ — VLSM ভাগ: 192.168.100.0/24-কে পাঁচটি বিভাগে ভাগ করা

চাহিদা: বিভাগ A-তে 100টি হোস্ট, B-তে 50টি, C-তে 20টি, D-তে 10টি, E-তে 2টি (point-to-point লিংক)। নিয়ম অনুযায়ী বড় থেকে ছোট ক্রমে বরাদ্দ দিন। A: 100 + 2 = 102, সবচেয়ে ছোট ঘাত 128, তাই /25। B: 50 + 2 = 52, ঘাত 64, তাই /26। C: 20 + 2 = 22, ঘাত 32, তাই /27। D: 10 + 2 = 12, ঘাত 16, তাই /28। E: 2 + 2 = 4, তাই /30।

বরাদ্দ: A = 192.168.100.0/25, network 192.168.100.0, broadcast 192.168.100.127, ব্যবহারযোগ্য 192.168.100.1–192.168.100.126 (126টি)। B = 192.168.100.128/26, network 192.168.100.128, broadcast 192.168.100.191, ব্যবহারযোগ্য 192.168.100.129–192.168.100.190 (62টি)। C = 192.168.100.192/27, network 192.168.100.192, broadcast 192.168.100.223, ব্যবহারযোগ্য 192.168.100.193–192.168.100.222 (30টি)। D = 192.168.100.224/28, network 192.168.100.224, broadcast 192.168.100.239, ব্যবহারযোগ্য 192.168.100.225–192.168.100.238 (14টি)। E = 192.168.100.240/30, network 192.168.100.240, broadcast 192.168.100.243,

ব্যবহারযোগ্য 192.168.100.241–192.168.100.242 (2টি)। বাকি 192.168.100.244–192.168.100.255 (12টি ঠিকানা) ভবিষ্যতের জন্য সংরক্ষিত — কোনো overlap নেই, প্রতিটি network block size-এর গুণিতক থেকে শুরু হয়েছে।

- A: 192.168.100.0/25 — Network 192.168.100.0, Broadcast 192.168.100.127, ব্যবহারযোগ্য 126টি
- B: 192.168.100.128/26 — Network 192.168.100.128, Broadcast 192.168.100.191, ব্যবহারযোগ্য 62টি
- C: 192.168.100.192/27 — Network 192.168.100.192, Broadcast 192.168.100.223, ব্যবহারযোগ্য 30টি
- D: 192.168.100.224/28 — Network 192.168.100.224, Broadcast 192.168.100.239, ব্যবহারযোগ্য 14টি
- E: 192.168.100.240/30 — Network 192.168.100.240, Broadcast 192.168.100.243, ব্যবহারযোগ্য 2টি
- মূল নেটওয়ার্ক: 192.168.100.0/24; মোট 256টি ঠিকানা; সংরক্ষিত: 192.168.100.244–192.168.100.255

ভেতরের কার্যপ্রণালী: একটি প্যাকেট কীভাবে সাবনেট বোঝে

একটি কম্পিউটারে 192.168.1.10/24 বসানো আছে, gateway 192.168.1.1। সে 192.168.1.50-এ একটি প্যাকেট পাঠাতে চায়। অপারেটিং সিস্টেম প্রথমে নিজের ঠিকানা ও mask-এর AND নেয়: 192.168.1.0 — নিজের network। তারপর গন্তব্য ঠিকানা ও নিজের mask-এর AND নেয়: 192.168.1.50 AND 255.255.255.0 = 192.168.1.0। দুটি ফল একই, তাই গন্তব্য একই নেটওয়ার্কে আছে। সিস্টেম সরাসরি গন্তব্যের MAC ঠিকানা খুঁজে (ARP) প্যাকেট পাঠায়, gateway-এর দরকার হয় না। এটিই local delivery।

এবার একই কম্পিউটার 192.168.2.50-এ পাঠাতে চায়। গন্তব্যের AND ফল 192.168.2.0, নিজের network 192.168.1.0-এর সঙ্গে মেলে না। তাই গন্তব্য অন্য নেটওয়ার্কে। সিস্টেম রাউটিং টেবিল দেখে, gateway 192.168.1.1-এর কাছে প্যাকেট পাঠায়। রাউটার তার নিজের রাউটিং টেবিলে 192.168.2.0/24-এর route খুঁজে, সেই নেটওয়ার্কের দিকে প্যাকেট পাঠায়। এই পুরো সিদ্ধান্তের মূলে আছে AND অপারেশন ও prefix — কোনো জাদু নেই। Mask ভুল হলে এই সিদ্ধান্ত ভুল হয়: /24-এর জায়গায় /25 বসালে 192.168.1.200-কে সিস্টেম অন্য নেটওয়ার্ক ভাববে, gateway-এর কাছে পাঠাবে, gateway হয়তো ফিরিয়ে দেবে বা ঘুরপথে পাঠাবে। তাই mask-এর ভুল শুধু গণনার ভুল নয়, যোগাযোগের ভুল।

DHCP সার্ভারও একই যুক্তি ব্যবহার করে। তাকে একটি scope দেওয়া হয় — যেমন 192.168.1.0/24-এর ভেতরে 192.168.1.100 থেকে 192.168.1.200। সে ঠিকানা দেওয়ার সময় network ও broadcast বাদ দেয়, gateway ও DNS-এর ঠিকানা জানিয়ে দেয়, আর lease-এর মেয়াদ লিখে রাখে। আপনি যদি scope-এর ভেতরে gateway-এর ঠিকানা ঢুকিয়ে দেন, DHCP হয়তো সেই ঠিকানা অন্য ডিভাইসকে দিয়ে দেবে — তখন দুটি ডিভাইস একই ঠিকানা ব্যবহার করবে, দুটোই যোগাযোগ হারাবে। তাই পরিকল্পনায় static ঠিকানা (সার্ভার, gateway, প্রিন্টার) ও DHCP pool আলাদা রাখা হয়।

রাউটারের দৃষ্টিতে সাবনেট হলো একটি route। রাউটিং টেবিলে প্রতিটি লাইনে একটি network, একটি mask বা prefix, একটি next-hop, আর একটি ইন্টারফেস থাকে। সবচেয়ে নির্দিষ্ট (longest prefix) route জেতে। যেমন 10.0.0.0/8 ও 10.1.0.0/16 দুটি route থাকলে 10.1.5.10-এর প্যাকেট /16-এর route দিয়ে যাবে, কারণ /16 বেশি নির্দিষ্ট। VLSM-এর কারণে রাউটিং টেবিলে ছোট ছোট অনেক route থাকতে পারে, আবার সেগুলোকে একত্র করে (summarization) একটি বড় route-এ লেখা যায় — যেমন চারটি /24-কে একটি /22-এ। তবে summarization-এর সময় সতর্ক থাকুন: একত্র করা পরিসরগুলো যেন টানা হয়, ফাঁকা জায়গা যেন অন্য কারো না হয়।

পরিভাষা — Terminology

পরিভাষা	অর্থ	উদাহরণ
IPv4 Address	32 বিটের ঠিকানা, চারটি octet-এ দশমিকে লেখা হয়	192.168.1.10

Subnet Mask	32 বিটের সংখ্যা; 1 মানে network অংশ, 0 মানে host অংশ	255.255.255.192
CIDR / Prefix Length	স্ল্যাশের পর network বিটের সংখ্যা	/26
Network Address	host বিট সব 0; সাবনেটের পরিচয়, হোস্টকে দেওয়া যায় না	192.168.1.64
Broadcast Address	host বিট সব 1; সাবনেটের সবাইকে পাঠানোর ঠিকানা	192.168.1.127
Usable Host	network ও broadcast ছাড়া বাকি ঠিকানা	192.168.1.65–192.168.1.126
Block Size	একটি সাবনেটে মোট ঠিকানার সংখ্যা; network-গুলোর মধ্যবর্তী দূরত্ব	/26-এ 64
Subnetting	একটি বড় নেটওয়ার্ককে ছোট ছোট সাবনেটে ভাগ করা	/24 থেকে চারটি /26
VLSM	একই নেটওয়ার্কে বিভিন্ন আকারের সাবনেট ব্যবহার	/25, /26, /27 একসঙ্গে
FLSM	একই নেটওয়ার্কে সব সাবনেট সমান আকারের	চারটি /26
Private Address	ইন্টারনেটে সরাসরি রাউট হয় না; স্থানীয় নেটওয়ার্কে ব্যবহৃত হয়	10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16
Gateway	সাবনেট থেকে বাইরে যাওয়ার দরজা; সাধারণত প্রথম বা শেষ ব্যবহারযোগ্য ঠিকানা	192.168.1.1
Overlapping Subnet	দুটি সাবনেটের পরিসর একে অপরের সঙ্গে মিলে যাওয়া — গুরুতর ভুল	/25 ও তার ভেতরের /26 একসঙ্গে বরাদ্দ
Route Summarization	টানা কয়েকটি ছোট route-কে একটি বড় route-এ একত্র করা	চারটি /24 থেকে একটি /22

চিত্র: CIDR ও VLSM-এর গঠন

চিত্র ১ — Prefix, Mask, Block Size ও ব্যবহারযোগ্য হোস্টের সম্পর্ক

CIDR	Mask (শেষ octet)	Block Size	মোট ঠিকানা	ব্যবহারযোগ্য
/24	255.255.255.0	256	256	254
/25	255.255.255.128	128	128	126
/26	255.255.255.192	64	64	62
/27	255.255.255.224	32	32	30
/28	255.255.255.240	16	16	14
/29	255.255.255.248	8	8	6
/30	255.255.255.252	4	4	2
/23	255.255.254.0	512	512	510
/22	255.255.252.0	1,024	1,024	1,022
/21	255.255.248.0	2,048	2,048	2,046
/20	255.255.240.0	4,096	4,096	4,094

পড়ার নিয়ম: prefix এক কমলে ঠিকানা দ্বিগুণ হয়, prefix এক বাড়লে ঠিকানা অর্ধেক হয়। /24 থেকে /25-এ গেলে 256 থেকে 128, /25 থেকে /26-এ গেলে 128 থেকে 64।

চিত্র ২ — একটি /24-কে VLSM-এ ভাগ করার গাছ

192.168.100.0/24 (256টি ঠিকানা)

- ├── 192.168.100.0/25 — বিভাগ A, 100 জন (0–127)
- ├── 192.168.100.128/26 — বিভাগ B, 50 জন (128–191)
- ├── 192.168.100.192/27 — বিভাগ C, 20 জন (192–223)
- ├── 192.168.100.224/28 — বিভাগ D, 10 জন (224–239)
- ├── 192.168.100.240/30 — লিংক E, 2 জন (240–243)
- └── 192.168.100.244–255 — সংরক্ষিত, ভবিষ্যতের জন্য (244–255)

গাছটি উপর থেকে নিচে পড়ুন: বড় ডাল আগে, ছোট ডাল পরে। কোনো ডাল অন্য ডালের ভেতরে ঢোকেনি — এটিই VLSM-এর যাচাই।

বাস্তব পরিস্থিতি — Real-World Scenarios

পরিস্থিতি ১: ঢাকার একটি ছোট অফিস, 60 জন কর্মী

একটি স্টার্টআপে ডেভেলপার 35 জন, ডিজাইনার 10 জন, হিসাব 5 জন, সার্ভার 6টি, অতিথি Wi-Fi-তে একসঙ্গে 15 জন পর্যন্ত। সবাইকে একটি /24-এ রাখা হয়েছিল। একদিন ডেভেলপারদের একটি টেস্ট broadcast-এ পুরো অফিসের নেটওয়ার্ক ধীর হয়ে গেল, আর অতিথি ডিভাইস থেকে সার্ভারের ফাইল শেয়ার দেখা গেল। সমাধান: 192.168.10.0/24-কে VLSM-এ ভাগ করা হলো — ডেভেলপার 192.168.10.0/26 (62টি ব্যবহারযোগ্য, 35 জনের জন্য যথেষ্ট, বৃদ্ধির জায়গা আছে), ডিজাইনার 192.168.10.64/28 (14টি), হিসাব 192.168.10.80/28 (14টি), সার্ভার 192.168.10.96/28 (14টি), অতিথি 192.168.10.128/27 (30টি)। প্রতিটি সাবনেটের gateway আলাদা, ফায়ারওয়ালে নিয়ম লেখা হলো: অতিথি থেকে সার্ভারে যাওয়া যাবে না, হিসাব থেকে শুধু নির্দিষ্ট সার্ভারে যাওয়া যাবে। বাকি পরিসর ভবিষ্যতের জন্য সংরক্ষিত রইল।

পরিস্থিতি ২: তিন শহরে তিনটি শাখা, VPN দিয়ে যুক্ত

একটি কোম্পানির ঢাকা, চট্টগ্রাম ও সিলেটে অফিস। তিন জায়গাতেই আগে 192.168.1.0/24 ব্যবহার করা হতো। VPN চালু করার পর সমস্যা দেখা দিল: ঢাকার কম্পিউটার চট্টগ্রামের 192.168.1.50-এ পৌঁছাতে চাইলে সিস্টেম ভাবল গন্তব্য নিজের নেটওয়ার্কেই আছে, VPN-এ পাঠাল না। সমাধান: মূল পরিসর 10.0.0.0/8 থেকে ঢাকাকে 10.1.0.0/16, চট্টগ্রামকে 10.2.0.0/16, সিলেটকে 10.3.0.0/16 দেওয়া হলো। প্রতিটি শহরের ভেতরে বিভাগভিত্তিক /24। VPN-এর দুই প্রান্তের পরিসর আর মেলে না, রাউটিং পরিষ্কার, আর নতুন শাখা এলে পরের /16 (10.4.0.0/16) নিলেই হয়। শিক্ষা: একাধিক সাইট থাকলে শুরুতেই বড় private পরিসর (10.0.0.0/8) থেকে সাইটভিত্তিক ভাগ করুন; 192.168.1.0/24 সব জায়গায় ব্যবহার করবেন না।

পরিস্থিতি ৩: Docker ও Kubernetes-এর ঠিকানা-পরিসর

একজন DevOps প্রকৌশলী Docker-এ একটি নেটওয়ার্ক বানাতে গিয়ে 172.18.0.0/16 বেছে নিলেন, কিন্তু অফিসের VPN-ও 172.18.0.0/16 ব্যবহার করত। ফলে কন্টেইনার থেকে VPN-এর সার্ভারে পৌঁছানো গেল না — দুটি নেটওয়ার্কের পরিসর মিলে গেল। সমাধান: পরিকল্পনার নথিতে লেখা হলো কোন পরিসর কোন কাজে ব্যবহৃত হয় — অফিস 10.0.0.0/8, Docker 172.20.0.0/16, Kubernetes Pod 172.21.0.0/16, ল্যাব 192.168.0.0/16। নতুন কোনো নেটওয়ার্ক বানানোর আগে নথি দেখে নিশ্চিত করা হয় পরিসরটি ফাঁকা আছে কি না। শিক্ষা: কন্টেইনার ও ক্লাউডের নেটওয়ার্কও একই সাবনেটিং নিয়ম মানে; পরিসর বাছার আগে overlap যাচাই করুন।

> Pro Tip: পরিকল্পনার নথিতে প্রতিটি সাবনেটের পাশে লিখুন: নাম, network/CIDR, gateway, ব্যবহার (বিভাগ/সার্ভিস), দায়িত্বপ্রাপ্ত ব্যক্তি, আর বরাদ্দের তারিখ। ছয় মাস পর আপনি নিজেই ভুলে যাবেন 192.168.10.64/28 কার ছিল — নথি মনে করিয়ে দেবে।

কমান্ড: Ubuntu Server-এ যাচাই

গণনা কাগজে করুন, যাচাই সার্ভারে করুন। নিচের কমান্ডগুলো শুধু পড়ার জন্য — এগুলো কোনো সেটিং বদলায় না। ip addr দেখায় প্রতিটি ইন্টারফেসের ঠিকানা, prefix, broadcast। ip route দেখায় রাউটিং টেবিল — কোন network কোন ইন্টারফেস দিয়ে, default route কোনটি।

```
ip addr
ip route
```

```
ip addr show eth0
ip route show default
```

উদাহরণ আউটপুট (illustrative):

```
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500
    inet 192.168.1.10/24 brd 192.168.1.255 scope global eth0
    inet6 fe80::1ff:fe00:1/64 scope link
default via 192.168.1.1 dev eth0 proto static
192.168.1.0/24 dev eth0 proto kernel scope link src 192.168.1.10
```

এই উদাহরণ আউটপুট (illustrative) পড়ুন: eth0-তে 192.168.1.10/24 আছে, broadcast 192.168.1.255, default gateway 192.168.1.1, আর 192.168.1.0/24 নেটওয়ার্কটি সরাসরি eth0 দিয়ে পৌঁছানো যায়। আপনার সিস্টেমে ইন্টারফেসের নাম (eth0, ens33, enp0s3 ইত্যাদি), ঠিকানা ও gateway আলাদা হবে — নাম ও সংখ্যা মিলিয়ে নেওয়ার চেষ্টা করবেন না, কাঠামোটি মিলিয়ে নিন। একটি /26 ইন্টারফেসের আউটপুটে brd 192.168.1.127 দেখলে বুঝবেন সেটি 192.168.1.64/26-এর সদস্য।

যাচাইয়ের প্রশ্ন: আপনার সার্ভারে ip addr চালিয়ে দেখুন — ঠিকানার পাশের /24 বা /26 কী বলছে? Broadcast ঠিকানাটি আপনার হাতে গণনা করা broadcast-এর সঙ্গে মেলে কি? না মিললে হয় আপনার গণনা ভুল, নয়তো ইন্টারফেসে ভুল prefix বসানো হয়েছে। দুটোই সম্ভব — দুটোই যাচাই করুন।

নির্দেশিত অনুশীলন — Guided Practical Exercise

ল্যাব: নিজের ল্যাবের জন্য একটি IP পরিকল্পনা বানান

লক্ষ্য: 192.168.50.0/24-কে VLSM-এ ভাগ করে একটি ছোট ল্যাবের পরিকল্পনা লেখা। সময়: 30 মিনিট। সরঞ্জাম: কাগজ-কলম, Ubuntu Server টার্মিনাল।

ধাপ ১: চাহিদা লিখুন। ধরুন ল্যাবে: ওয়ার্কস্টেশন 40টি, সার্ভার 8টি, টেস্ট ডিভাইস 12টি, point-to-point লিংক 2টি। ধাপ ২: প্রতিটি চাহিদার জন্য prefix বের করুন। 40 জনের জন্য 42 দরকার, ঘাত 64, তাই /26। 12 জনের জন্য 14 দরকার, ঘাত 16, তাই /28। 8টি সার্ভারের জন্য 10 দরকার, ঘাত 16, তাই /28। 2টি লিংকের জন্য /30। ধাপ ৩: বড় থেকে ছোট ক্রমে বরাদ্দ দিন: ওয়ার্কস্টেশন 192.168.50.0/26, টেস্ট 192.168.50.64/28, সার্ভার 192.168.50.80/28, লিংক 192.168.50.96/30। ধাপ ৪: প্রতিটির network, broadcast, প্রথম ও শেষ ব্যবহারযোগ্য হোস্ট লিখুন এবং যাচাই করুন — কোনো overlap আছে কি, প্রতিটি network block size-এর গুণিতক থেকে শুরু হয়েছে কি, বাকি পরিসর কতটুকু ফাঁকা আছে। ধাপ ৫: প্রতিটি সাবনেটের gateway ঠিক করুন — নিয়ম হিসেবে প্রথম ব্যবহারযোগ্য ঠিকানা (.1, .65, .81, .97) gateway ধরুন। ধাপ ৬: Ubuntu Server-এ ip addr ও ip route চালিয়ে আপনার বর্তমান নেটওয়ার্কের prefix, broadcast ও gateway পড়ুন এবং আপনার পরিকল্পনার সঙ্গে তুলনা করুন।

প্রত্যাশিত ফল (illustrative): ওয়ার্কস্টেশন সাবনেটের network 192.168.50.0, broadcast 192.168.50.63, ব্যবহারযোগ্য 192.168.50.1–192.168.50.62, gateway 192.168.50.1। টেস্ট সাবনেটের network 192.168.50.64, broadcast 192.168.50.79, ব্যবহারযোগ্য 192.168.50.65–192.168.50.78। সার্ভার সাবনেটের network 192.168.50.80, broadcast 192.168.50.95, ব্যবহারযোগ্য 192.168.50.81–192.168.50.94। লিংকের network 192.168.50.96, broadcast 192.168.50.99, ব্যবহারযোগ্য 192.168.50.97–192.168.50.98। বাকি 192.168.50.100–192.168.50.255 সংরক্ষিত। আপনার গণনা এই ফলের সঙ্গে মিললে ল্যাবটি সফল।

স্বাধীন অনুশীলনী: ৩০টি সমস্যা

নিচের ৩০টি সমস্যার উত্তর এই অধ্যায়ে দেওয়া হয়নি — নিজে গণনা করুন। প্রতিটি সমস্যায় network address, broadcast address, প্রথম ও শেষ ব্যবহারযোগ্য হোস্ট, মোট ঠিকানা ও ব্যবহারযোগ্য হোস্টের সংখ্যা বের করুন। ২৭–৩০ নম্বর সমস্যায় VLSM বা নকশা করতে হবে। উত্তর যাচাইয়ের জন্য কাজ করা উদাহরণগুলোর পদ্ধতি ব্যবহার করুন, আর একটি সমস্যায় আটকে গেলে আগের অধ্যায়ের সারণিটি দেখুন।

1. 192.168.20.0/24 — network, broadcast ও ব্যবহারযোগ্য হোস্ট বের করুন।
2. 192.168.20.0/25 — network, broadcast ও ব্যবহারযোগ্য হোস্ট বের করুন।
3. 192.168.20.128/25 — network, broadcast ও ব্যবহারযোগ্য হোস্ট বের করুন।
4. 192.168.30.128/26 — network, broadcast ও ব্যবহারযোগ্য হোস্ট বের করুন।
5. 192.168.30.0/27 — network, broadcast ও ব্যবহারযোগ্য হোস্ট বের করুন।
6. 192.168.30.64/27 — network, broadcast ও ব্যবহারযোগ্য হোস্ট বের করুন।
7. 192.168.40.48/28 — network, broadcast ও ব্যবহারযোগ্য হোস্ট বের করুন।
8. 192.168.40.0/29 — network, broadcast ও ব্যবহারযোগ্য হোস্ট বের করুন।
9. 192.168.40.16/29 — network, broadcast ও ব্যবহারযোগ্য হোস্ট বের করুন।
10. 192.168.40.20/30 — network, broadcast ও ব্যবহারযোগ্য হোস্ট বের করুন।
11. 10.0.0.0/8 — network, broadcast ও ব্যবহারযোগ্য হোস্ট বের করুন।
12. 10.50.0.0/16 — network, broadcast ও ব্যবহারযোগ্য হোস্ট বের করুন।
13. 10.60.100.0/23 — network, broadcast ও ব্যবহারযোগ্য হোস্ট বের করুন।
14. 172.16.0.0/12 — network, broadcast ও ব্যবহারযোগ্য হোস্ট বের করুন।
15. 172.16.32.0/22 — network, broadcast ও ব্যবহারযোগ্য হোস্ট বের করুন।
16. 172.16.16.0/21 — network, broadcast ও ব্যবহারযোগ্য হোস্ট বের করুন।
17. 192.168.64.0/20 — network, broadcast ও ব্যবহারযোগ্য হোস্ট বের করুন।
18. 10.1.0.0/17 — network, broadcast ও ব্যবহারযোগ্য হোস্ট বের করুন।
19. 10.1.128.0/17 — network, broadcast ও ব্যবহারযোগ্য হোস্ট বের করুন।
20. 172.17.0.0/18 — network, broadcast ও ব্যবহারযোগ্য হোস্ট বের করুন।
21. 192.168.90.0/19 — দেওয়া ঠিকানা থেকে আসল network, broadcast ও ব্যবহারযোগ্য হোস্ট বের করুন।
22. 192.168.7.45/26 — দেওয়া হোস্ট ঠিকানা থেকে network, broadcast ও ব্যবহারযোগ্য হোস্ট বের করুন।
23. 10.8.9.100/27 — দেওয়া হোস্ট ঠিকানা থেকে network, broadcast ও ব্যবহারযোগ্য হোস্ট বের করুন।
24. 172.16.5.150/28 — দেওয়া হোস্ট ঠিকানা থেকে network, broadcast ও ব্যবহারযোগ্য হোস্ট বের করুন।
25. 192.168.99.200/25 — দেওয়া হোস্ট ঠিকানা থেকে network, broadcast ও ব্যবহারযোগ্য হোস্ট বের করুন।
26. 10.20.30.40/23 — দেওয়া হোস্ট ঠিকানা থেকে network, broadcast ও ব্যবহারযোগ্য হোস্ট বের করুন।
27. 192.168.200.0/24-কে VLSM-এ ভাগ করুন: 60 জন, 30 জন, 12 জন ও 6 জন হোস্টের চারটি বিভাগের জন্য প্রতিটি সাবনেটের network, broadcast ও ব্যবহারযোগ্য হোস্ট লিখুন।
28. 10.10.0.0/22-কে VLSM-এ ভাগ করুন: 200 জন, 100 জন, 50 জন ও 20 জন হোস্টের চারটি বিভাগের জন্য প্রতিটি সাবনেটের network, broadcast ও ব্যবহারযোগ্য হোস্ট লিখুন।

29. 172.16.10.0/24 থেকে একটি অফিসের জন্য পরিকল্পনা করুন: 120 জন, 45 জন ও 18 জন হোস্টের তিনটি বিভাগ এবং দুটি point-to-point লিংক — প্রতিটি সাবনেটের network, broadcast ও ব্যবহারযোগ্য হোস্ট লিখুন এবং সংরক্ষিত পরিসর উল্লেখ করুন।
30. 192.168.1.0/24-কে চারটি সমান /26 সাবনেটে ভাগ করুন এবং প্রতিটি সাবনেটের network, broadcast, প্রথম ও শেষ ব্যবহারযোগ্য হোস্ট লিখুন।

সাধারণ ভুল — Common Mistakes

প্রথম ভুল: network বা broadcast ঠিকানাকে হোস্ট হিসেবে ব্যবহার করা। 192.168.1.64/26-এ 192.168.1.64 হলো network, 192.168.1.127 হলো broadcast — কোনোটিই কম্পিউটারকে দেওয়া যায় না। এই ঠিকানা বসালে কম্পিউটার নেটওয়ার্কে উঠবে না, অথবা উঠলেও অন্য ডিভাইসের সঙ্গে যোগাযোগ করবে না। যাচাই: হোস্টের ঠিকানা সবসময় network-এর চেয়ে বড় ও broadcast-এর চেয়ে ছোট হতে হবে।

দ্বিতীয় ভুল: "বিয়োগ 2" ভুলে যাওয়া। /26-এ 64টি ঠিকানা আছে বলে 64 জন হোস্ট রাখা যায় ভাবা। আসলে রাখা যায় 62 জন। 64 জন বসাতে গেলে দুইজন বাদ পড়বে, আর তাদের ঠিকানা হবে network বা broadcast। সবসময় হোস্ট সংখ্যার সঙ্গে 2 যোগ করে তারপর 2-এর ঘাত নিন।

তৃতীয় ভুল: overlapping সাবনেট। 192.168.1.0/25 আর 192.168.1.64/26 একসঙ্গে বরাদ্দ দেওয়া — দ্বিতীয়টি প্রথমটির ভেতরে। রাউটার তখন 192.168.1.100-এর প্যাকেট কোথায় পাঠাবে বুঝতে পারে না। প্রতিটি বরাদ্দের পর সারণিতে পরিসর লিখে মিলিয়ে দেখুন।

চতুর্থ ভুল: network address block size-এর গুণিতক নয় এমন ঠিকানা থেকে শুরু করা। যেমন /26-এ network 192.168.1.100 লেখা — 100 হলো 64-এর গুণিতক নয়, তাই এটি network হতে পারে না; আসল network 192.168.1.64। এই ভুলটি ধরা সহজ: network-এর শেষ octet-কে block size দিয়ে ভাগ করুন, ভাগশেষ শূন্য হতে হবে।

পঞ্চম ভুল: ভুল octet-এ গণনা করা। /23-এর ভাগ তৃতীয় octet-এ হয়, চতুর্থ octet-এ নয়। 10.20.30.0/23-এর broadcast 10.20.30.255 ভাবলে ভুল হবে — আসল broadcast 10.20.31.255, কারণ /23-এ তৃতীয় octet-এ দুটি মান (30 ও 31) মিলে একটি সাবনেট। কোন octet-এ mask 255 নয়, সেই octet-এই ভাগ হচ্ছে তা প্রথমে চিহ্নিত করুন।

ষষ্ঠ ভুল: /31 ও /32-কে সাধারণ নিয়মে গণনা করা। /31-এ সাধারণ নিয়মে ব্যবহারযোগ্য 0 হয়, কিন্তু point-to-point লিংকে এটি বিশেষভাবে ব্যবহৃত হয়। /32 কোনো নেটওয়ার্ক নয়, একটি হোস্ট। এগুলোকে /30-এর মতো ভাববেন না। সপ্তম ভুল: সব জায়গায় একই পরিসর ব্যবহার করা — প্রতিটি সাইটে 192.168.1.0/24। VPN বা মার্জারের সময় এই ভুলের দাম দিতে হয়। অষ্টম ভুল: পরিকল্পনা নথিভুক্ত না করা। মুখে মনে রাখা পরিকল্পনা এক মাসে হারিয়ে যায়, আর পরের প্রকৌশলী আবার শূন্য থেকে শুরু করেন।

> সতর্কতা: ঠিকানা বদলানো সহজ কাজ নয়। একটি সার্ভারের ঠিকানা বদলালে DNS, ফায়ারওয়াল নিয়ম, মনিটরিং, ব্যাকআপ, VPN route — সব জায়গায় বদলাতে হয়। তাই প্রথমবারই সঠিক পরিকল্পনা করুন; "পরে ঠিক করে নেব" বলে ভুল পরিসর বসাবেন না।

সমস্যা নির্ণয় — Troubleshooting

সমস্যা: দুটি ডিভাইস একই সাবনেটে থাকা সত্ত্বেও ping যায় না। যাচাই করুন: (১) দুটির ঠিকানা ও mask ঠিক আছে কি — একটি /24, অন্যটি /25 হলে তারা একে অপরকে অন্য নেটওয়ার্ক ভাবতে পারে। (২) network বা broadcast ঠিকানা

কাউকে দেওয়া হয়নি তো? (৩) ip addr দিয়ে ইন্টারফেস UP আছে কি, ঠিকানাটি সত্যিই বসেছে কি দেখুন। (৪) ip route দিয়ে দেখুন গন্তব্য নেটওয়ার্কের route আছে কি, default route কোন gateway-এর দিকে।

সমস্যা: একটি ডিভাইস নিজের সাবনেটে পৌঁছায়, কিন্তু বাইরে (ইন্টারনেট বা অন্য সাবনেটে) পৌঁছায় না। সম্ভাব্য কারণ: gateway ভুল, অথবা gateway-এর ঠিকানা সাবনেটের বাইরে। যেমন 192.168.1.10/26-এর gateway 192.168.1.200 হলে সেটি 192.168.1.64/26-এর বাইরে — সিস্টেম gateway-কেই পৌঁছাতে পারবে না। Gateway সবসময় নিজের সাবনেটের ভেতরের ব্যবহারযোগ্য ঠিকানা হতে হবে। ip route-এ default লাইনটি দেখুন: default via-এর ঠিকানাটি নিজের সাবনেটের ভেতরে আছে কি?

সমস্যা: DHCP থেকে ঠিকানা পাওয়া যায়, কিন্তু সেটি ভুল সাবনেটের। কারণ: DHCP scope-এর mask বা network ভুল, অথবা একই নেটওয়ার্কে দুটি DHCP সার্ভার চলছে। Scope-এর network, mask, gateway ও DNS যাচাই করুন, আর ip addr-এর আউটপুটের সঙ্গে আপনার পরিকল্পনার সারণি মিলিয়ে দেখুন।

সমস্যা: নতুন সাবনেট যোগ করার পর পুরোনো সাবনেটের সঙ্গে যোগাযোগ বন্ধ। সম্ভাব্য কারণ: নতুন সাবনেট পুরোনোটির সঙ্গে overlap করছে, অথবা রাউটারে নতুন route যোগ করা হয়নি। প্রথমে overlap যাচাই করুন — দুটি সাবনেটের পরিসর মিলছে কি? তারপর রাউটিং টেবিলে নতুন network-এর route আছে কি দেখুন। Overlapping থাকলে route যোগ করে লাভ নেই — পরিসর বদলাতে হবে।

সমস্যা: ঠিকানা ফুরিয়ে গেছে — DHCP pool-এ আর ঠিকানা নেই। কারণ: সাবনেটটি চাহিদার চেয়ে ছোট, অথবা lease-এর মেয়াদ অনেক বড়, অথবা static ঠিকানা pool-এর ভেতরে ঢুকে গেছে। সমাধান: ব্যবহারের হিসাব নিন — কতগুলো ডিভাইস, কতগুলো lease, কতগুলো static। প্রয়োজনে সাবনেটটি বড় করুন (যেমন /26 থেকে /25), তবে মনে রাখবেন prefix বদলালে network ও broadcast-ও বদলায় — সব ডিভাইসের mask একসঙ্গে বদলাতে হবে। অর্ধেক ডিভাইসে নতুন mask আর অর্ধেকে পুরোনো mask থাকলে তারা একে অপরকে ভুল নেটওয়ার্কে ভাববে।

> সমস্যা নির্ণয়ের ক্রম: প্রথমে গণনা যাচাই করুন (network, broadcast, mask ঠিক আছে কি?), তারপর ip addr দিয়ে ঠিকানা যাচাই করুন, তারপর ip route দিয়ে route যাচাই করুন, তারপর gateway-কে ping করুন, তারপর গন্তব্যকে ping করুন। গণনা ভুল থাকলে কমান্ডের আউটপুট যতই ঠিক মনে হোক, যোগাযোগ ঠিক হবে না।

নিরাপত্তা — Security

সাবনেটিং নিজেই একটি নিরাপত্তা-নিয়ন্ত্রণ। সাবনেট আলাদা থাকলে ফায়ারওয়াল নিয়ম লেখা যায়: কোন সাবনেট থেকে কোন সাবনেটে যাওয়া যাবে, কোন পোর্ট খোলা থাকবে। কিন্তু সাবনেটিং নিরাপত্তার বিকল্প নয় — এটি শুধু সীমানা আঁকে, সীমানা পাহারা দেয় ফায়ারওয়াল। কয়েকটি নিয়ম মনে রাখুন। এক: ব্যবস্থাপনা (management) ইন্টারফেস — যেমন সার্ভারের SSH, রাউটারের অ্যাডমিন প্যানেল — আলাদা সাবনেটে রাখুন, সাধারণ ব্যবহারকারীর সাবনেট থেকে সেখানে যাওয়া বন্ধ করুন। দুই: অতিথি ও IoT ডিভাইসকে আলাদা সাবনেটে রাখুন, তাদের থেকে অভ্যন্তরীণ সার্ভারে যাওয়া বন্ধ করুন। তিন: সার্ভার সাবনেটে শুধু প্রয়োজনীয় পোর্ট খুলুন; একটি ওয়েব সার্ভারের সাবনেটে সব পোর্ট খোলা রাখবেন না।

চার: broadcast ঠিকানায় পাঠানো বার্তা (directed broadcast) বন্ধ রাখুন — বাইরে থেকে একটি সাবনেটের broadcast ঠিকানায় প্যাকেট পাঠিয়ে সেই সাবনেটের সব ডিভাইসকে ব্যস্ত করা যায়। আধুনিক রাউটারে এটি সাধারণত বন্ধ থাকে; যাচাই করে নিন। পাঁচ: DHCP-তে অজানা ডিভাইসকে ঠিকানা দেওয়ার আগে ভাবুন — অফিসের নেটওয়ার্কে কেউ নিজের ল্যাপটপ লাগিয়ে ঠিকানা পেয়ে গেলে সে অভ্যন্তরীণ নেটওয়ার্ক দেখতে পাবে। সংবেদনশীল সাবনেটে DHCP-র পরিবর্তে static ঠিকানা বা অনুমোদিত MAC-এর তালিকা ব্যবহার করুন। ছয়: IP পরিকল্পনার নথি সংবেদনশীল — এতে পুরো নেটওয়ার্কের গঠন লেখা থাকে। নথিটি সবার জন্য খোলা জায়গায় রাখবেন না। সাত: VPN-এর পরিসর ও অভ্যন্তরীণ পরিসর যেন না মেলে তা নিয়মিত যাচাই করুন; মিলে গেলে ট্র্যাফিক ভুল দিকে যেতে পারে এবং সংবেদনশীল তথ্য ভুল নেটওয়ার্কে পৌঁছাতে পারে।

সেরা অনুশীলন — Best Practices

- পরিকল্পনা প্রথমে, বরাদ্দ পরে। ঠিকানা বসানোর আগে সারণি লিখুন, overlap যাচাই করুন, gateway নিয়ম ঠিক করুন।
- বড় চাহিদা আগে, ছোট পরে — VLSM-এ সবসময় এই ক্রম মানুন।
- ভবিষ্যৎ বৃদ্ধির জন্য 20–30% জায়গা রাখুন; প্রতিটি সাবনেটে ও প্রতিটি সাইটে ফাঁকা পরিসর সংরক্ষণ করুন।
- নামকরণে নিয়ম রাখুন: সাইটের জন্য /16, বিভাগের জন্য /24 বা VLSM, সার্ভারের জন্য নির্দিষ্ট নম্বর (যেমন .10), অতিথির জন্য নির্দিষ্ট নম্বর (যেমন .200)।
- Gateway-এর নিয়ম একটাই রাখুন — সব সাবনেটে প্রথম ব্যবহারযোগ্য ঠিকানা, অথবা সব সাবনেটে শেষ ব্যবহারযোগ্য ঠিকানা। মিশ্র নিয়ম ভুলের কারণ হয়।
- Static ঠিকানা DHCP pool-এর বাইরে রাখুন; সার্ভার, প্রিন্টার, gateway-কে static দিন, ওয়ার্কস্টেশনকে DHCP দিন।
- প্রতিটি সাবনেট নথিভুক্ত করুন: নাম, CIDR, gateway, ব্যবহার, দায়িত্বপ্রাপ্ত ব্যক্তি, বরাদ্দের তারিখ।
- পরিবর্তনের আগে ব্যাকআপ নিন: রাউটিং টেবিল, DHCP scope, DNS রেকর্ড — পরিবর্তনের আগে বর্তমান অবস্থা লিখে রাখুন।
- নিয়মিত যাচাই করুন: ip addr ও ip route দিয়ে বাস্তব অবস্থা পরিকল্পনার সঙ্গে মেলে কি না দেখুন।
- একাধিক সাইট থাকলে 10.0.0.0/8 থেকে সাইটভিত্তিক ভাগ করুন; সব সাইটে 192.168.1.0/24 ব্যবহার করবেন না।

Knowledge Check — জ্ঞান যাচাই

নিচের পাঁচটি প্রশ্নের উত্তর নিজে বের করুন। উত্তর এই অধ্যায়ে দেওয়া হয়নি। প্রতিটি প্রশ্নে একটি সঠিক উত্তর আছে।

1. 192.168.1.0/26-এ ব্যবহারযোগ্য হোস্ট কয়টি? A) 62 B) 64 C) 126 D) 30
2. 192.168.1.0/25 সাবনেটের broadcast address কোনটি? A) 192.168.1.255 B) 192.168.1.127 C) 192.168.1.128 D) 192.168.1.0
3. নিচের কোনটি private IPv4 ঠিকানা নয়? A) 10.0.0.5 B) 172.16.0.5 C) 192.168.1.5 D) 8.8.8.8
4. VLSM ব্যবহারের মূল উদ্দেশ্য কী? A) সব সাবনেট সমান আকারের করা B) প্রতিটি বিভাগের চাহিদা অনুযায়ী বিভিন্ন আকারের সাবনেট দেওয়া C) broadcast ঠিকানা বাড়ানো D) gateway-এর সংখ্যা বাড়ানো
5. 192.168.2.4/30-এ ব্যবহারযোগ্য হোস্ট কয়টি? A) 4 B) 2 C) 6 D) 14

স্বাধীন অ্যাসাইনমেন্ট — Independent Assignment

একটি কাল্পনিক কোম্পানির জন্য সম্পূর্ণ IP পরিকল্পনা লিখুন। কোম্পানির ঢাকায় প্রধান অফিস ও চট্টগ্রামে একটি শাখা আছে। ঢাকায়: ডেভেলপার 80 জন, QA 25 জন, হিসাব 8 জন, সার্ভার 12টি, অতিথি Wi-Fi-তে একসঙ্গে 20 জন। চট্টগ্রামে: কর্মী 30 জন, সার্ভার 4টি। ঢাকা ও চট্টগ্রামের মধ্যে একটি point-to-point VPN লিংক আছে। মূল পরিসর হিসেবে 10.0.0.0/8 ব্যবহার করুন। আপনার জমা দিতে হবে: (১) প্রতিটি সাইটের /16 বরাদ্দ, (২) প্রতিটি বিভাগের সাবনেট (CIDR, network, broadcast, প্রথম ও শেষ ব্যবহারযোগ্য হোস্ট, gateway), (৩) VLSM যুক্তি — কেন এই prefix বেছে নিলেন, (৪) ভবিষ্যৎ বৃদ্ধির জন্য সংরক্ষিত পরিসর, (৫) ফায়ারওয়ালের জন্য দুটি নিয়ম (যেমন অতিথি থেকে সার্ভারে যাওয়া যাবে না), (৬) একটি সারণি যাতে সব সাবনেট একসঙ্গে দেখা যায়। গণনার প্রতিটি ধাপ দেখান, শুধু উত্তর লিখবেন না। পরিকল্পনাটি এমন হতে হবে যাতে নতুন একটি বিভাগ (15 জন) যোগ করা যায় পুরোনো কোনো সাবনেট না ভেঙে।

সারাংশ — Summary

এই অধ্যায়ে আমরা শিখেছি: IP ঠিকানার network ও host অংশকে আলাদা করে subnet mask বা CIDR prefix; কেন একটি বড় নেটওয়ার্ককে ছোট ছোট সাবনেটে ভাগ করতে হয় — broadcast কমানো, নিরাপত্তা, ঠিকানার অপচয় রোধ, ব্যবস্থাপনা ও ভবিষ্যৎ বৃদ্ধির জন্য; মোট ঠিকানা = 2^(32 − prefix) ও ব্যবহারযোগ্য হোস্ট = মোট − 2 এই সূত্র; block size দিয়ে দ্রুত network ও broadcast বের করার পদ্ধতি; AND অপারেশন দিয়ে network address বের করা; VLSM দিয়ে বিভিন্ন আকারের বিভাগের জন্য বিভিন্ন আকারের সাবনেট বানানো; এবং একটি সম্পূর্ণ IP পরিকল্পনা সাজানোর ধাপ — চাহিদা লেখা, মূল পরিসর বেছে নেওয়া, সাইট ও বিভাগভিত্তিক বরাদ্দ, gateway নিয়ম, সংরক্ষিত পরিসর, নথিভুক্তি।

মনে রাখার মূল সংখ্যাগুলো: /24 = 256টি ঠিকানা, 254টি ব্যবহারযোগ্য; /25 = 128টি, 126টি; /26 = 64টি, 62টি; /27 = 32টি, 30টি; /28 = 16টি, 14টি; /29 = 8টি, 6টি; /30 = 4টি, 2টি; /23 = 512টি, 510টি; /22 = 1,024টি, 1,022টি; /16 = 65,536টি, 65,534টি; /8 = 16,777,216টি, 16,777,214টি। এই সংখ্যাগুলো মুখস্থ রাখুন, কিন্তু তার চেয়েও বেশি মনে রাখুন পদ্ধতিটি: host বিট বের করুন, 2-এর ঘাত নিন, block size দিয়ে সীমানা খুঁজুন, network ও broadcast বাদ দিয়ে ব্যবহারযোগ্য পরিসর লিখুন। পদ্ধতি জানা থাকলে যেকোনো prefix-এর উত্তর আপনি নিজে বের করতে পারবেন। পরের অধ্যায়ে আমরা এই সাবনেটগুলোকে রাউটার দিয়ে যুক্ত করা, রাউটিং টেবিল ও gateway-এর কাজ আরও গভীরে দেখব।

Revision Checklist — পুনরালোচনার তালিকা

- আমি /24 থেকে /30 পর্যন্ত যেকোনো prefix-এর মোট ও ব্যবহারযোগ্য হোস্টের সংখ্যা মুখস্থ না করে গণনা করে বলতে পারি।
- আমি CIDR থেকে mask এবং mask থেকে CIDR — দুই দিকেই রূপান্তর করতে পারি।
- আমি একটি হোস্ট ঠিকানা (যেমন 192.168.10.77/26) থেকে network ও broadcast বের করতে পারি।
- আমি বাইনারি AND দিয়ে network address বের করে ব্যাখ্যা করতে পারি।
- আমি একটি /24-কে VLSM-এ ভাগ করতে পারি এবং overlap যাচাই করতে পারি।
- আমি একটি ছোট অফিসের জন্য IP পরিকল্পনার সারণি লিখতে পারি — সাবনেট, gateway, সংরক্ষিত পরিসরসহ।
- আমি ip addr ও ip route-এর আউটপুট পড়ে prefix, broadcast ও default gateway বলতে পারি।
- আমি network/broadcast-কে হোস্ট হিসেবে ব্যবহার, overlapping, ভুল block size — এই ভুলগুলো চিনতে পারি।
- আমি /31 ও /32-এর ব্যতিক্রমটি ব্যাখ্যা করতে পারি।
- আমি ৩০টি স্বাধীন সমস্যার সবগুলো নিজে গণনা করে শেষ করেছি।

Chapter 9: IPv6 Fundamentals and Addressing — IPv6 এর মৌলিক ধারণা ও অ্যাড্রেসিং

অধ্যায় ৯ • Networking & Internet Infrastructure • Ubuntu Server 24.04 LTS • আনুমানিক পড়ার সময়: ৫০ মিনিট • হাতে-কলমে অনুশীলনসহ

শুরুতে: একটি বাস্তব সমস্যা

ধরুন, আপনি ঢাকার একটি ছোট অফিসে কাজ করেন। অফিসে বিশটি কম্পিউটার, দশটি মোবাইল ফোন, পাঁচটি প্রিন্টার, কয়েকটি সিসিটিভি ক্যামেরা এবং একটি সার্ভার আছে। প্রতিটি যন্ত্রকে ইন্টারনেটে চেনার জন্য একটি করে ঠিকানা দরকার। IPv4-এ ঠিকানার সংখ্যা সীমিত, প্রায় ৪৩০ কোটি। পৃথিবীতে মানুষ আটশ কোটির বেশি, আর প্রত্যেকের হাতে একাধিক যন্ত্র। তাই IPv4 ঠিকানা প্রায় শেষ হয়ে গেছে। এই সমস্যার কারণেই ইন্টারনেট সেবাদাতা প্রতিষ্ঠানগুলো অনেক সময় একটি পাবলিক ঠিকানার পেছনে অনেক গ্রাহককে লুকিয়ে রাখে, যাকে NAT বলা হয়। NAT কাজ করলেও এটি সরাসরি সংযোগ, সার্ভার চালানো এবং সমস্যা নির্ণয়কে জটিল করে তোলে। IPv6 এই সীমাবদ্ধতা দূর করার জন্য তৈরি হয়েছে। IPv6-এ ঠিকানার সংখ্যা এত বেশি যে প্রতিটি যন্ত্র, প্রতিটি সেন্সর, প্রতিটি বাড়ি ও প্রতিটি প্রতিষ্ঠানের জন্য আলাদা পাবলিক ঠিকানা দেওয়া সম্ভব। এই অধ্যায়ে আপনি শিখবেন IPv6 কেন দরকার, এর ঠিকানা কীভাবে লেখা হয়, কীভাবে ছোট করা হয়, কোন ঠিকানা কোন কাজে ব্যবহৃত হয় এবং Ubuntu Server-এ IPv6 কীভাবে দেখতে ও পরীক্ষা করতে হয়।

উপমা: IPv4 একটি ছোট মহল্লার বাড়ির নম্বর, IPv6 এমন শহর যেখানে প্রতিটি বাড়ি ও বাতির আলাদা নম্বর আছে। এই অধ্যায়ের লক্ষ্য ঠিকানা মুখস্থ করা নয়, বরং গঠন বুঝে পড়তে, লিখতে ও পরীক্ষা করতে পারা।

Learning Objectives — শেখার লক্ষ্য

- এই অধ্যায় শেষে আপনি ব্যাখ্যা করতে পারবেন কেন IPv4 ঠিকানা শেষ হয়ে গেছে এবং IPv6 কেন প্রয়োজন।
- আপনি 128-bit IPv6 ঠিকানার গঠন বর্ণনা করতে পারবেন এবং 8 groups of 16-bit hexadecimal অংশ চিনতে পারবেন।
- আপনি :: shortening নিয়ম প্রয়োগ করে দীর্ঘ IPv6 ঠিকানাকে সঠিকভাবে ছোট করতে এবং ছোট ঠিকানাকে আবার বড় করতে পারবেন।
- আপনি loopback (::1), link-local (fe80::/10), global unicast (2000::/3) এবং multicast ঠিকানার মধ্যে পার্থক্য করতে পারবেন।
- আপনি prefix length, যেমন /64, ব্যাখ্যা করতে পারবেন এবং network অংশ ও host অংশ আলাদা করতে পারবেন।
- আপনি SLAAC ও dual-stack ধারণা ব্যাখ্যা করতে পারবেন এবং Ubuntu Server 24.04-এ ip -6 addr, ping ও অন্যান্য কমান্ড দিয়ে IPv6 অবস্থা যাচাই করতে পারবেন।

Prerequisites — পূর্বশর্ত

- Vol 01 থেকে Linux কমান্ড লাইন, sudo, এবং টেক্সট ফাইল পড়ার অভিজ্ঞতা থাকতে হবে।
- IPv4 ঠিকানা, subnet mask ও prefix ধারণার প্রাথমিক পরিচয় থাকতে হবে; না থাকলে Vol 02-এর আগের অধ্যায়গুলো আবার পড়ে নিন।
- Hexadecimal সংখ্যা পদ্ধতির প্রাথমিক ধারণা থাকতে হবে: 0-9 এবং a-f দিয়ে সংখ্যা লেখা হয়।
- Ubuntu Server 24.04 LTS চালু থাকা একটি পরীক্ষামূলক মেশিন বা virtual machine থাকতে হবে।

- ইন্টারনেট সংযোগ থাকলে ভালো, তবে এই অধ্যায়ের বেশিরভাগ অনুশীলন local ঠিকানা দিয়েই করা যায়।

প্রথম নীতি: ঠিকানা মানে কী

নেটওয়ার্কে ঠিকানা মানে পরিচয় ও অবস্থান, দুটোই। ডাক বিভাগের ঠিকানার মতো, IP ঠিকানা বলে দেয় একটি প্যাকেট কোথায় যাবে। IPv4-এ ঠিকানা 32-bit, অর্থাৎ 32টি binary অঙ্ক। 2-এর 32 ঘাত প্রায় ৪৩০ কোটি, তাই ঠিকানার সংখ্যা সীমিত। IPv6-এ ঠিকানা 128-bit, অর্থাৎ 2-এর 128 ঘাত। এই সংখ্যা এত বড় যে তাকে সাধারণ ভাষায় বোঝানো কঠিন। সহজভাবে বলা যায়, পৃথিবীর প্রতিটি বালুকণার জন্যও অনেক ঠিকানা বাকি থাকবে। কিন্তু বড় সংখ্যা একা যথেষ্ট নয়। ঠিকানার গঠন এমন হতে হয় যাতে রাউটার দ্রুত সিদ্ধান্ত নিতে পারে, ঠিকানাকে ভাগ করা যায় এবং মানুষ পড়তে পারে। IPv6 সেই গঠন দেয়।

IPv6 ঠিকানা hexadecimal-এ লেখা হয়। প্রতিটি hexadecimal অঙ্ক 4-bit প্রকাশ করে। 128-bit ঠিকানাকে 16-bit করে আটটি ভাগে ভাগ করা হয়, প্রতিটি ভাগে চারটি অঙ্ক থাকে, ভাগগুলো colon (:) দিয়ে আলাদা হয়। উদাহরণ: 2001:0db8:0000:0000:0000:ff00:0042:8329।

কেন IPv4 যথেষ্ট নয়

IPv4 যখন তৈরি হয়, তখন ইন্টারনেট ছিল গবেষণা ও বিশ্ববিদ্যালয়ের ছোট নেটওয়ার্ক। কেউ ভাবেনি প্রতিটি মানুষের পকেটে কম্পিউটার থাকবে, প্রতিটি বাড়িতে স্মার্ট যন্ত্র থাকবে, প্রতিটি কারখানায় সেন্সর থাকবে। ঠিকানা শেষ হয়ে যাওয়ার পর NAT একটি অস্থায়ী সমাধান হিসেবে এসেছে। NAT-এ একটি পাবলিক ঠিকানার পেছনে অনেক private ঠিকানা থাকে। এতে ঠিকানা বাঁচে, কিন্তু বাইরে থেকে ভেতরের যন্ত্রের সঙ্গে সরাসরি সংযোগ করা কঠিন হয়। একটি সার্ভার চালাতে port forwarding দরকার হয়, দুটি NAT-এর পেছনে থাকা যন্ত্রের মধ্যে সরাসরি সংযোগ জটিল হয় এবং প্রতিটি সংযোগের হিসাব রাখতে হয়। IPv6 এই জটিলতা কমায়, কারণ প্রতিটি যন্ত্রের নিজস্ব পাবলিক ঠিকানা থাকতে পারে। তবে IPv6 মানেই নিরাপত্তা নয়। পাবলিক ঠিকানা থাকলে firewall ও সঠিক কনফিগারেশন আরও গুরুত্বপূর্ণ হয়ে ওঠে, যা এই অধ্যায়ের শেষে আলোচনা করা হয়েছে।

128-bit ঠিকানার গঠন

IPv6 ঠিকানার প্রথম অংশ সাধারণত network বা prefix নির্দেশ করে, আর শেষ অংশ host বা interface identifier নির্দেশ করে। একটি সাধারণ বাড়ির নেটওয়ার্কে prefix হতে পারে /64। এর মানে প্রথম 64-bit network অংশ, আর বাকি 64-bit host অংশ। /64 এত বড় host অংশ দেয় যে একটি নেটওয়ার্কে অসংখ্য যন্ত্র থাকতে পারে। এই ভাগ বোঝা গুরুত্বপূর্ণ, কারণ prefix length বদলালে network-এর আকার বদলে যায়। /64-এর চেয়ে ছোট prefix, যেমন /48, আরও বড় network বোঝায়। /128 মানে একটি মাত্র ঠিকানা, কোনো host অংশ নেই। Loopback ঠিকানা ::1 হলো /128 ঠিকানা।

Hexadecimal-এ a মানে 10, f মানে 15। ঠিকানার শুরু দেখে ধরন বোঝা যায়: 2000::/3 global unicast, fe80::/10 link-local।

:: Shortening নিয়ম

IPv6 ঠিকানা লেখার সময় শূন্যের ভাগ অনেক থাকে। তাই দুটি নিয়মে ঠিকানা ছোট করা হয়। প্রথম নিয়ম: প্রতিটি ভাগের সামনের শূন্য বাদ দেওয়া যায়। উদাহরণস্বরূপ, 0db8 লেখা যায় db8, 0042 লেখা যায় 42, 0000 লেখা যায় 0। দ্বিতীয় নিয়ম: পরপর শূন্যের ভাগ থাকলে সেগুলোকে একবার :: দিয়ে বদলে দেওয়া যায়। উদাহরণস্বরূপ, 2001:0db8:0000:0000:0000:ff00:0042:8329 ছোট করে লেখা যায় 2001:db8::ff00:42:8329। এখানে তিনটি শূন্যের ভাগ :: দিয়ে বদলে গেছে। মনে রাখতে হবে, :: একটি ঠিকানায় মাত্র একবার ব্যবহার করা যায়। দুবার ব্যবহার করলে বোঝা যায় না কোন :: কতগুলো শূন্যের ভাগ বদলেছে। এটাই সবচেয়ে সাধারণ ভুল, যা নিচে আবার আলোচনা করা হয়েছে।

ছোট ঠিকানা বড় করতে ভাগ গুনুন: ::1-এ একটি ভাগ আছে, তাই :: সাতটি শূন্যের ভাগ নিয়েছে, পূর্ণ রূপ 0000:0000:0000:0000:0000:0000:0000:0001।

ঠিকানার ধরন

IPv6-এ প্রধান ধরন তিনটি: unicast, multicast ও anycast। Unicast একটি নির্দিষ্ট যন্ত্রের ঠিকানা। Global unicast 2000::/3 পরিসরে থাকে, link-local fe80::/10-এ, loopback ::1 নিজের মেশিনের ঠিকানা। Multicast ff00::/8 দিয়ে শুরু হয় এবং একসঙ্গে অনেক যন্ত্রের কাছে যায়। Anycast-এ একই ঠিকানা অনেক যন্ত্রে থাকতে পারে, প্যাকেট সবচেয়ে কাছের যন্ত্রের কাছে যায়।

প্রতিটি IPv6 interface-এ একটি link-local ঠিকানা থাকে, global ঠিকানা না থাকলেও। এটি একই local নেটওয়ার্কে কাজ করে, রাউটারের ওপারে যায় না। ব্যবহারের সময় interface লিখতে হয়, যেমন fe80::1%eth0, যেখানে %eth0 হলো zone index।

SLAAC ও Dual-Stack

SLAAC-এর পূর্ণ রূপ Stateless Address Autoconfiguration। এর অর্থ, যন্ত্র নিজে থেকে নিজের IPv6 ঠিকানা তৈরি করতে পারে, কোনো central server ছাড়াই। রাউটার network prefix ঘোষণা করে, আর যন্ত্র সেই prefix-এর সঙ্গে নিজের interface identifier যোগ করে ঠিকানা তৈরি করে। এই প্রক্রিয়ায় DHCP server দরকার হয় না। তবে SLAAC শুধু ঠিকানা দেয়, DNS server-এর তথ্য সবসময় দেয় না। তাই অনেক নেটওয়ার্কে SLAAC-এর সঙ্গে DHCPv6 বা router advertisement-এর মাধ্যমে DNS তথ্য দেওয়া হয়। এই পার্থক্য বোঝা গুরুত্বপূর্ণ, কারণ ঠিকানা থাকলেই নাম দিয়ে সার্ভারে যাওয়া যায় না, DNS দরকার হয়।

Dual-stack মানে একই যন্ত্রে IPv4 ও IPv6 দুটোই চালু থাকা। যন্ত্র প্রথমে IPv6 চেষ্টা করতে পারে, না পারলে IPv4-এ ফিরে যায়। Ubuntu Server-এ দুটো ঠিকানাই ip addr-এ দেখা যায়।

ভেতরের পথচলা: একটি IPv6 ঠিকানা কীভাবে তৈরি হয়

ধরুন, আপনার সার্ভার একটি নেটওয়ার্কে যুক্ত হলো। প্রথমে interface চালু হয় এবং একটি link-local ঠিকানা তৈরি হয়, যেমন fe80::a00:27ff:fe4e:5a1b। এই ঠিকানা interface-এর MAC ঠিকানা থেকে তৈরি হতে পারে, অথবা privacy-এর জন্য এলোমেলোভাবে তৈরি হতে পারে। এরপর সার্ভার router solicitation পাঠায়, অর্থাৎ রাউটারকে জিজ্ঞেস করে, এই নেটওয়ার্কের prefix কী। রাউটার router advertisement-এর মাধ্যমে prefix জানায়, যেমন 2001:db8:abcd:1::/64। সার্ভার সেই prefix-এর সঙ্গে নিজের interface identifier যোগ করে global ঠিকানা তৈরি করে, যেমন 2001:db8:abcd:1:a00:27ff:fe4e:5a1b। এরপর duplicate address detection চলে, অর্থাৎ সার্ভার পরীক্ষা করে এই ঠিকানা অন্য কেউ ব্যবহার করছে কি না। যদি না করে, তবে ঠিকানাটি ব্যবহারযোগ্য হয়। এই ধাপগুলো সাধারণত কয়েক সেকেন্ডের মধ্যে ঘটে এবং ব্যবহারকারী কিছু টের পায় না।

এরপর DNS AAAA record দিয়ে IPv6 ঠিকানা দেয়, A record দিয়ে IPv4 ঠিকানা দেয়। রাউটার prefix দেখে প্যাকেট পাঠায়, পুরো 128-bit মুখস্থ করে না।

পরিভাষা

পরিভাষা	অর্থ	উদাহরণ
IPv6	Internet Protocol version 6, 128-bit ঠিকানার ইন্টারনেট প্রোটোকল	2001:db8::1

Hexadecimal	16 ভিত্তিক সংখ্যা পদ্ধতি, 0-9 ও a-f	ff00, db8
Prefix / Prefix length	ঠিকানার network অংশের দৈর্ঘ্য, / দিয়ে লেখা হয়	/64, /48, /128
Global unicast	ইন্টারনেটে ব্যবহারযোগ্য একক যন্ত্রের ঠিকানা	2001:db8::1
Link-local	একই link-এর ভেতরে ব্যবহারের ঠিকানা, রাউটার পার হয় না	fe80::1
Loopback	নিজের মেশিনের ঠিকানা	::1
Multicast	একসঙ্গে অনেক যন্ত্রের কাছে পাঠানো ঠিকানা	ff02::1
SLAAC	Stateless Address Autoconfiguration, নিজে থেকে ঠিকানা তৈরি	Router advertisement থেকে prefix নেওয়া
Dual-stack	একই যন্ত্রে IPv4 ও IPv6 দুটোই চালু থাকা	IPv4 ও IPv6 একসঙ্গে
Interface identifier	ঠিকানার host অংশ, যা যন্ত্রকে চেনায়	2001:db8::/64-এর শেষ 64-bit

চিত্র ৯.১ — IPv6 ঠিকানার গঠন

2001 : 0db8 : 0000 : 0000 : 0000 : ff00 : 0042 : 8329

↑ 8টি ভাগ, প্রতিটি 16-bit, মোট 128-bit ↑

প্রথম 64-bit: network / prefix অংশ | শেষ 64-bit: interface identifier / host অংশ

ছোট রূপ: 2001:db8::ff00:42:8329

চিত্র ৯.২ — ঠিকানার ধরন চেনার ছক

::1 → loopback → নিজের মেশিন

fe80::/10 → link-local → একই local নেটওয়ার্ক

2000::/3 → global unicast → ইন্টারনেট

ff00::/8 → multicast → একসঙ্গে অনেক যন্ত্র

:: → unspecified → কোনো ঠিকানা নেই

বাস্তব পরিস্থিতি

প্রথম পরিস্থিতি: একটি বিশ্ববিদ্যালয়ের ল্যাবে একশটি কম্পিউটার আছে। IPv4-এ private ঠিকানা দিয়ে কাজ চলছে, কিন্তু ছাত্ররা নিজেদের সার্ভার ইন্টারনেটে দেখাতে পারে না। IPv6-এ প্রতিটি কম্পিউটার global ঠিকানা পেলে ছাত্ররা নিজের প্রজেক্ট সরাসরি দেখাতে পারে, তবে firewall দিয়ে শুধু প্রয়োজনীয় port খোলা রাখতে হয়। দ্বিতীয় পরিস্থিতি: একটি IoT কারখানায় হাজার হাজার সেন্সর আছে। প্রতিটি সেন্সরের আলাদা ঠিকানা দরকার। IPv4-এ NAT দিয়ে এটি জটিল, IPv6-এ সহজ। তৃতীয় পরিস্থিতি: একটি মোবাইল নেটওয়ার্কে লক্ষ লক্ষ ফোন আছে। IPv6 ছাড়া প্রতিটি ফোনকে আলাদা পাবলিক

ঠিকানা দেওয়া প্রায় অসম্ভব। তাই মোবাইল অপারেটররা IPv6 ব্যবহার করে। এই তিনটি উদাহরণ থেকে বোঝা যায় IPv6 শুধু তত্ত্ব নয়, বরং বড় নেটওয়ার্কের বাস্তব প্রয়োজন।

কাজের উদাহরণ

উদাহরণ ১: 2001:0db8:0000:0000:0000:ff00:0042:8329 ঠিকানাটি ছোট করুন। প্রথমে প্রতিটি ভাগের সামনের শূন্য বাদ দিন: 2001:db8:0:0:0:ff00:42:8329। তারপর পরপর তিনটি শূন্যের ভাগকে :: দিয়ে বদলান: 2001:db8::ff00:42:8329। এখানে :: একবারই ব্যবহৃত হয়েছে। উদাহরণ ২: ::1 ঠিকানাটি বড় করুন। এটি loopback ঠিকানা। পূর্ণ রূপ: 0000:0000:0000:0000:0000:0000:0000:0001। উদাহরণ ৩: 2001:db8:abcd:1::/64 ঠিকানায় network অংশ কী? প্রথম 64-bit হলো 2001:db8:abcd:1, আর host অংশ বাকি 64-bit। উদাহরণ ৪: fe80::a00:27ff:fe4e:5a1b ঠিকানাটি কোন ধরনের? এটি fe80 দিয়ে শুরু, তাই link-local। এটি ইন্টারনেটে রাউট করা যাবে না, শুধু local নেটওয়ার্কে ব্যবহার করা যাবে।

কমান্ড

নিচের কমান্ডগুলো Ubuntu Server 24.04-এ IPv6 দেখতে ও পরীক্ষা করতে ব্যবহৃত হয়। প্রতিটি কমান্ড চালানোর আগে তার কাজ বুঝে নিন।

```
ip -6 addr
ip -6 route
ping -6 ::1
ping -6 fe80::1%eth0
ss -tuln
cat /proc/net/if_inet6
```

ip -6 addr কমান্ড শুধু IPv6 ঠিকানা দেখায়। ip -6 route কমান্ড IPv6 রাউটিং টেবিল দেখায়। ping -6 ::1 কমান্ড নিজের মেশিনের loopback ঠিকানায় ping পাঠায়। ping -6 fe80::1%eth0 কমান্ড link-local ঠিকানায় ping পাঠায়, এখানে eth0 আপনার interface-এর নাম দিয়ে বদলাতে হবে। ss -tuln কমান্ড listening port দেখায়, যেখানে IPv6 port-ও দেখা যায়। cat /proc/net/if_inet6 কমান্ড kernel-এর IPv6 ঠিকানার তালিকা দেখায়।

নির্দেশিত অনুশীলন

> Lab 9.1 — নিজের IPv6 ঠিকানা খুঁজুন
>
> ১. টার্মিনালে ip -6 addr চালান। ২. inet6 লেখা লাইনগুলো দেখুন। ::1, fe80:: দিয়ে শুরু হওয়া ঠিকানা এবং global ঠিকানা আলাদা করে লিখুন। ৩. ping -6 ::1 চালিয়ে loopback পরীক্ষা করুন। ৪. ip -6 route চালিয়ে default route আছে কি না দেখুন। ৫. একটি ঠিকানা বেছে নিয়ে তার পূর্ণ রূপ লিখুন এবং :: নিয়ম ব্যাখ্যা করুন।

উদাহরণ আউটপুট (illustrative)

```
$ ip -6 addr
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500
    inet6 ::1/128 scope host
    inet6 fe80::a00:27ff:fe4e:5a1b/64 scope link
    inet6 2001:db8:abcd:1:a00:27ff:fe4e:5a1b/64 scope global
```

```
$ ping -6 ::1
64 bytes from ::1: icmp_seq=1 ttl=64 time=0.05 ms
```

উপরের আউটপুটে scope host মানে loopback, scope link মানে link-local, আর scope global মানে global ঠিকানা। আপনার মেশিনে ঠিকানা ও interface-এর নাম আলাদা হতে পারে, কিন্তু scope-এর অর্থ একই থাকবে।

সাধারণ ভুল

- :: একটি ঠিকানায় দুবার ব্যবহার করা। এটি ভুল, কারণ তখন শূন্যের ভাগ গোনা যায় না।
- IPv6 ঠিকানায় colon-এর জায়গায় dot ব্যবহার করা। IPv6-এ dot নয়, colon ব্যবহার হয়।
- Link-local ঠিকানাকে global ঠিকানা ভাবা। fe80:: দিয়ে শুরু হওয়া ঠিকানা ইন্টারনেটে যাবে না।
- Prefix length না দেখে ঠিকানা পড়া। /64 ও /128-এর অর্থ সম্পূর্ণ আলাদা।
- SLAAC থাকলেই DNS কাজ করবে ভাবা। SLAAC ঠিকানা দেয়, DNS তথ্য আলাদাভাবে দরকার হতে পারে।

সমস্যা নির্ণয়

যদি ip -6 addr-এ কোনো global ঠিকানা না থাকে, তবে প্রথমে দেখুন link-local ঠিকানা আছে কি না। Link-local থাকলে interface চালু আছে, কিন্তু রাউটার prefix দিচ্ছে না বা নেটওয়ার্কে IPv6 চালু নেই। যদি ::1-এ ping না যায়, তবে IPv6 kernel-এ বন্ধ থাকতে পারে বা loopback interface-এ সমস্যা আছে। যদি global ঠিকানা থাকে কিন্তু ইন্টারনেটে ping না যায়, তবে default route আছে কি না দেখুন। ip -6 route-এ default লাইন না থাকলে রাউটার advertisement আসছে না। যদি DNS নাম দিয়ে যেতে না পারেন কিন্তু IP ঠিকানায় যেতে পারেন, তবে DNS সমস্যা, IPv6 সমস্যা নয়। প্রতিটি ধাপে একটি করে অনুমান পরীক্ষা করুন, একসঙ্গে অনেক কিছু বদলাবেন না।

নিরাপত্তা বিবেচনা

IPv6-এ প্রতিটি যন্ত্রের global ঠিকানা থাকতে পারে, তাই firewall ছাড়া সার্ভার চালানো বিপজ্জনক। শুধু প্রয়োজনীয় port খোলা রাখুন, বাকি সব বন্ধ রাখুন। Link-local ঠিকানা ইন্টারনেটে যায় না, কিন্তু local নেটওয়ার্কে থাকা যন্ত্র তা দেখতে পারে। তাই local নেটওয়ার্কেও অপ্রয়োজনীয় সেবা চালু রাখবেন না। SLAAC-এ interface identifier MAC ঠিকানা থেকে তৈরি হলে যন্ত্রকে সহজে চেনা যায়, তাই privacy extension ব্যবহারের কথা ভাবুন। IPv6 বন্ধ করে দেওয়া নিরাপত্তা নয়, সঠিক firewall ও আপডেটই নিরাপত্তা।

সেরা অনুশীলন

- ঠিকানা লেখার সময় সবসময় :: নিয়ম মেনে চলুন এবং একটি ঠিকানায় একবারই :: ব্যবহার করুন।
- Documentation-এর উদাহরণে 2001:db8::/32 পরিসর ব্যবহার করুন, বাস্তব ঠিকানা উদাহরণে দেবেন না।
- Prefix length সবসময় ঠিকানার সঙ্গে লিখুন, যেমন 2001:db8::1/64।
- Dual-stack নেটওয়ার্কে IPv4 ও IPv6 দুটোরই firewall নিয়ম আলাদাভাবে পরীক্ষা করুন।
- পরিবর্তনের আগে ip -6 addr ও ip -6 route-এর আউটপুট সংরক্ষণ করুন, যাতে আগের অবস্থায় ফেরা যায়।

Knowledge Check — জ্ঞান যাচাই

1. IPv6 ঠিকানা কত bit-এর? A) 32-bit B) 64-bit C) 128-bit D) 256-bit
2. নিচের কোনটি IPv6 loopback ঠিকানা? A) 127.0.0.1 B) ::1 C) fe80::1 D) 2001:db8::1

3. 2001:0db8:0000:0000:0000:ff00:0042:8329 ঠিকানার সঠিক ছোট রূপ কোনটি? A) 2001:db8::ff00:42:8329 B) 2001:db8::ff00::42:8329 C) 2001:0db8:0:0:0:ff00:42:8329 D) 2001.db8.ff00.42.8329
4. fe80::/10 পরিসরের ঠিকানা কোন ধরনের? A) Global unicast B) Link-local C) Multicast D) Loopback
5. SLAAC-এর মূল কাজ কী? A) DNS server দেওয়া B) Router ছাড়া ঠিকানা তৈরি করা C) Firewall নিয়ম তৈরি করা D) IPv4 ঠিকানা বদলে IPv6 করা

স্বাধীন অ্যাসাইনমেন্ট

আপনার Ubuntu Server-এ ip -6 addr ও ip -6 route চালিয়ে একটি প্রতিবেদন লিখুন। প্রতিবেদনে loopback, link-local ও global ঠিকানা আলাদা করে লিখুন, প্রতিটি ঠিকানার পূর্ণ রূপ লিখুন, prefix length ব্যাখ্যা করুন এবং আপনার নেটওয়ার্কে SLAAC নাকি DHCPv6 ব্যবহৃত হচ্ছে তা যুক্তি দিয়ে লিখুন। যদি global ঠিকানা না থাকে, তবে কেন নেই তা সম্ভাব্য কারণসহ লিখুন। প্রতিবেদনটি এক পৃষ্ঠার বেশি লম্বা করবেন না, কিন্তু প্রতিটি দাবির পাশে কমান্ডের নাম লিখুন।

সারাংশ

IPv6 হলো 128-bit ঠিকানার প্রোটোকল, যা IPv4 ঠিকানার অভাব দূর করে। ঠিকানা আটটি 16-bit hexadecimal ভাগে লেখা হয়, :: দিয়ে শূন্যের ভাগ ছোট করা যায়, তবে :: একবারই ব্যবহার করা যায়। ::1 হলো loopback, fe80::/10 হলো link-local, 2000::/3 হলো global unicast। SLAAC যন্ত্রকে নিজে থেকে ঠিকানা তৈরি করতে দেয়, আর dual-stack IPv4 ও IPv6 একসঙ্গে চালায়। Ubuntu Server-এ ip -6 addr ও ip -6 route দিয়ে ঠিকানা ও রাউটিং যাচাই করা যায়। এই অধ্যায়ের ধারণাগুলো পরের অধ্যায়ে TCP ও UDP বোঝার ভিত্তি হবে।

Revision Checklist — পুনরালোচনার তালিকা

- আমি 128-bit, 8 groups, 16-bit hex গঠন ব্যাখ্যা করতে পারি।
- আমি :: shortening নিয়ম প্রয়োগ করে ঠিকানা ছোট ও বড় করতে পারি।
- আমি ::1, fe80::/10 ও 2000::/3 চিনতে পারি।
- আমি /64 prefix-এর অর্থ ব্যাখ্যা করতে পারি।
- আমি SLAAC ও dual-stack ব্যাখ্যা করতে পারি।
- আমি ip -6 addr, ip -6 route ও ping -6 চালিয়ে IPv6 যাচাই করতে পারি।

Chapter 10: TCP, UDP, Ports and Sockets — TCP, UDP, পোর্ট ও সকেট

অধ্যায় ১০ • Networking & Internet Infrastructure • Ubuntu Server 24.04 LTS • আনুমানিক পড়ার সময়: ৫০ মিনিট • হাতে-কলমে অনুশীলনসহ

শুরুতে: একটি বাস্তব সমস্যা

আপনার সার্ভারে একই সময়ে একটি ওয়েবসাইট, একটি ডাটাবেস এবং SSH সেবা চলছে। বাইরে থেকে একটি প্যাকেট এলে সার্ভার কীভাবে বুঝবে সেটি ওয়েবসাইটের জন্য, নাকি ডাটাবেসের জন্য, নাকি SSH-এর জন্য? IP ঠিকানা শুধু মেশিনকে চেনায়, মেশিনের ভেতরের সেবাকে চেনায় না। এই কাজটি করে port। Port হলো একটি সংখ্যা, যা মেশিনের ভেতরে কোন সেবা প্যাকেটটি পাবে তা ঠিক করে। আর TCP ও UDP হলো দুটি ভিন্ন পরিবহন পদ্ধতি: TCP নির্ভরযোগ্য ও ক্রম ঠিক রাখে, UDP দ্রুত কিন্তু নির্ভরযোগ্যতার নিশ্চয়তা দেয় না। এই অধ্যায়ে আপনি শিখবেন TCP ও UDP কখন ব্যবহার করবেন, port কীভাবে কাজ করে, socket কী এবং Ubuntu Server-এ ss কমান্ড দিয়ে listening সেবা কীভাবে দেখবেন।

উপমা: IP ঠিকানাকে ভাবুন একটি বড় ভবনের ঠিকানা হিসেবে, port হলো ভবনের ভেতরের ফ্ল্যাট নম্বর। একই ভবনে অনেক ফ্ল্যাট থাকতে পারে, প্রতিটি ফ্ল্যাটে আলাদা মানুষ থাকে। চিঠি ভবনে পৌঁছালেই হবে না, ফ্ল্যাট নম্বর ঠিক থাকতে হবে। Socket হলো একটি নির্দিষ্ট যোগাযোগের ঠিকানা: source IP, source port, destination IP ও destination port — এই চারটি মিলে একটি socket pair তৈরি হয়।

Learning Objectives — শেখার লক্ষ্য

- এই অধ্যায় শেষে আপনি TCP ও UDP-এর মধ্যে পার্থক্য ব্যাখ্যা করতে পারবেন এবং প্রতিটির উপযুক্ত ব্যবহার বলতে পারবেন।
- আপনি port-এর ধারণা ব্যাখ্যা করতে পারবেন এবং well-known, registered ও ephemeral port চিনতে পারবেন।
- আপনি socket ও socket pair ব্যাখ্যা করতে পারবেন এবং একটি সংযোগকে চারটি অংশ দিয়ে চিনতে পারবেন।
- আপনি TCP header-এর মূল field, যেমন source port, destination port, sequence number ও acknowledgement number, চিনতে পারবেন।
- আপনি Ubuntu Server 24.04-এ ss -tuln, ss -tnp ও ss -unp দিয়ে listening ও সংযুক্ত সেবা দেখতে পারবেন।
- আপনি nc ও curl দিয়ে একটি TCP সেবা পরীক্ষা করতে পারবেন এবং ফলাফল ব্যাখ্যা করতে পারবেন।

Prerequisites — পূর্বশর্ত

- অধ্যায় ৯ থেকে IPv4 ও IPv6 ঠিকানার ধারণা জানা থাকতে হবে।
- Linux কমান্ড লাইনে sudo দিয়ে কমান্ড চালানোর অভিজ্ঞতা থাকতে হবে।
- HTTP, SSH ও DNS নামগুলোর সঙ্গে প্রাথমিক পরিচয় থাকতে হবে।
- Ubuntu Server 24.04 LTS-এ পরীক্ষার জন্য একটি local মেশিন থাকতে হবে।

প্রথম নীতি: নির্ভরযোগ্যতা বনাম গতি

নেটওয়ার্কে তথ্য পাঠানোর সময় দুটি বিপরীত চাহিদা থাকে। একদিকে তথ্য যেন হারিয়ে না যায়, ক্রম ঠিক থাকে এবং সব অংশ পৌঁছায়। অন্যদিকে তথ্য যেন দ্রুত পৌঁছায়, দেরি না হয়। TCP প্রথম চাহিদাকে গুরুত্ব দেয়, UDP দ্বিতীয়টিকে। TCP সংযোগ তৈরি করে, প্রতিটি অংশের প্রাপ্তি নিশ্চিত করে, হারানো অংশ আবার পাঠায় এবং ক্রম ঠিক রাখে। UDP সংযোগ তৈরি করে না, প্রাপ্তির নিশ্চয়তা দেয় না এবং ক্রম ঠিক রাখে না। তাই TCP ধীর হতে পারে, কিন্তু নির্ভরযোগ্য। UDP দ্রুত, কিন্তু তথ্য হারাতে পারে। কোনটি ব্যবহার করবেন তা নির্ভর করে সেবার প্রয়োজনের ওপর।

একটি উপমা: TCP হলো registered ডাক, যেখানে প্রতিটি চিঠির প্রাপ্তি স্বীকার করা হয় এবং হারানো চিঠি আবার পাঠানো হয়। UDP হলো সাধারণ পোস্টকার্ড, যা দ্রুত পাঠানো হয়, কিন্তু পৌঁছেছে কি না তা জানা যায় না। ওয়েবসাইট, ইমেইল ও ফাইল পাঠানোর জন্য registered ডাক দরকার, কারণ একটি অক্ষর হারালেও অর্থ বদলে যায়। লাইভ ভিডিও বা অনলাইন খেলায় একটি ছোট অংশ হারালে তেমন ক্ষতি হয় না, কিন্তু দেরি হলে খেলা নষ্ট হয়, তাই সেখানে UDP উপযুক্ত।

TCP কীভাবে কাজ করে

TCP একটি connection-oriented প্রোটোকল। প্রথমে সংযোগ তৈরি হয়, তারপর তথ্য যায়, শেষে সংযোগ বন্ধ হয়। সংযোগ তৈরির প্রক্রিয়াকে three-way handshake বলা হয়, যা অধ্যায় ১১-এ বিস্তারিত আছে। TCP তথ্যকে segment-এ ভাগ করে, প্রতিটি segment-এ sequence number দেয় এবং প্রাপক acknowledgement পাঠায়। প্রাপক যদি কোনো segment না পায়, তবে প্রেরক তা আবার পাঠায়। এই প্রক্রিয়াকে retransmission বলা হয়। TCP flow control দিয়ে প্রাপকের গতি অনুযায়ী পাঠানোর গতি নিয়ন্ত্রণ করে, যাতে প্রাপক তথ্যে ডুবে না যায়। TCP congestion control দিয়ে নেটওয়ার্কের ভিড় অনুযায়ী গতি কমায় বা বাড়ায়। এই সব ব্যবস্থার কারণে TCP নির্ভরযোগ্য, কিন্তু এর জন্য অতিরিক্ত তথ্য ও সময় দরকার হয়।

UDP কীভাবে কাজ করে

UDP একটি connectionless প্রোটোকল। এতে সংযোগ তৈরি হয় না, শুধু datagram পাঠানো হয়। প্রতিটি datagram স্বাধীন, তার কোনো sequence number বা acknowledgement নেই। প্রেরক জানে না প্রাপক datagram পেয়েছে কি না। UDP header ছোট, মাত্র 8 byte, তাই এটি হালকা ও দ্রুত। DNS প্রায়ই UDP ব্যবহার করে, কারণ একটি ছোট প্রশ্ন ও ছোট উত্তর দ্রুত পাঠানো দরকার। ভিডিও কল, লাইভ সম্প্রচার ও অনলাইন খেলায় UDP ব্যবহৃত হয়। তবে UDP ব্যবহার করলে application-কেই প্রয়োজনে নির্ভরযোগ্যতা যোগ করতে হয়, যেমন হারানো অংশ আবার চাওয়া বা ভুল সহ্য করা।

Port: মেশিনের ভেতরের ঠিকানা

Port হলো 16-bit সংখ্যা, 0 থেকে 65535 পর্যন্ত। Well-known port 0 থেকে 1023 পর্যন্ত, যা পরিচিত সেবার জন্য ব্যবহৃত হয়: HTTP 80, HTTPS 443, SSH 22, DNS 53। Registered port 1024 থেকে 49151 পর্যন্ত, যা নির্দিষ্ট application-এর জন্য ব্যবহৃত হয়। Ephemeral port 49152 থেকে 65535 পর্যন্ত, যা client-এর অস্থায়ী port হিসেবে ব্যবহৃত হয়। যখন আপনার ব্রাউজার একটি ওয়েবসাইটে যায়, তখন ব্রাউজার একটি ephemeral port ব্যবহার করে, আর সার্ভার 443 port-এ শোনে। এই ভাগ বোঝা গেলে ss আউটপুট পড়া সহজ হয়।

একটি port-এ একসময়ে একটি সেবাই শুনতে পারে, যদি না বিশেষ ব্যবস্থা থাকে। দুটি সেবা একই port-এ শুনতে চাইলে দ্বিতীয়টি ব্যর্থ হয়। এটি একটি সাধারণ সমস্যা, যা troubleshooting অংশে আলোচনা করা হয়েছে। Port বন্ধ থাকলে সংযোগ প্রত্যাখ্যাত হয়, firewall port বন্ধ রাখলে প্যাকেট পৌঁছায় না। তাই port জানা মানে সেবা জানা, আর সেবা জানা মানে সমস্যার অর্ধেক সমাধান।

Socket ও Socket Pair

Socket হলো একটি যোগাযোগের শেষ প্রান্ত। একটি socket তৈরি হয় IP ঠিকানা ও port দিয়ে। উদাহরণস্বরূপ, 192.0.2.10:443 একটি socket। একটি সংযোগে দুটি socket থাকে: client socket ও server socket। এই চারটি অংশ —

source IP, source port, destination IP, destination port — মিলে socket pair তৈরি হয়। একই সার্ভার port-এ অনেক client সংযোগ করতে পারে, কারণ প্রতিটি client-এর source port আলাদা হয়। তাই socket pair-ই একটি সংযোগকে অনন্যভাবে চেনায়, শুধু port নয়।

ভেতরের পথচলা: ব্রাউজার থেকে সার্ভার পর্যন্ত

ধরুন, আপনি ব্রাউজারে https://example.com খুললেন। প্রথমে DNS example.com-এর IP ঠিকানা দেয়। তারপর ব্রাউজার একটি ephemeral port বেছে নেয়, যেমন 52341। ব্রাউজার সার্ভারের 443 port-এ TCP সংযোগ তৈরি করে। সংযোগ তৈরি হলে ব্রাউজার HTTP অনুরোধ পাঠায়, সার্ভার উত্তর পাঠায়। এই সময়ে socket pair হলো আপনার IP:52341 ও সার্ভারের IP:443। সংযোগ শেষ হলে socket বন্ধ হয়, ephemeral port মুক্ত হয়। একই সময়ে আপনি অন্য একটি সাইট খুললে আরেকটি ephemeral port ব্যবহৃত হয়। এই পথচলা বোঝা গেলে আপনি বুঝবেন কেন একই সার্ভারে অনেক সংযোগ একসঙ্গে চলতে পারে।

UDP-এর ক্ষেত্রে পথচলা আরও সহজ। DNS প্রশ্ন পাঠানোর সময় client একটি ephemeral port থেকে DNS সার্ভারের 53 port-এ datagram পাঠায়। সার্ভার উত্তর পাঠায়, কিন্তু কোনো সংযোগ তৈরি হয় না। উত্তর না এলে client আবার চেষ্টা করতে পারে, অথবা application সময় শেষ বলে ধরে নেয়। এই সরলতা UDP-কে দ্রুত করে, কিন্তু নির্ভরযোগ্যতার দায়িত্ব application-এর ওপর ছেড়ে দেয়।

পরিভাষা

পরিভাষা	অর্থ	উদাহরণ
TCP	Transmission Control Protocol, নির্ভরযোগ্য, connection-oriented প্রোটোকল	HTTP, SSH
UDP	User Datagram Protocol, দ্রুত, connectionless প্রোটোকল	DNS, ভিডিও কল
Port	মেশিনের ভেতরে সেবা চেনার 16-bit সংখ্যা	80, 443, 22
Socket	IP ঠিকানা ও port-এর সমন্বয়ে যোগাযোগের শেষ প্রান্ত	192.0.2.10:443
Socket pair	Source ও destination socket মিলে একটি সংযোগের পরিচয়	Client:52341 ও Server:443
Segment	TCP-এ ভাগ করা তথ্যের অংশ	Sequence number সহ অংশ
Datagram	UDP-এ পাঠানো স্বাধীন তথ্যের অংশ	DNS প্রশ্ন
Ephemeral port	Client-এর অস্থায়ী port	52341
Well-known port	পরিচিত সেবার জন্য নির্ধারিত port, 0-1023	80, 443, 22
Flow control	প্রাপকের গতি অনুযায়ী পাঠানোর গতি নিয়ন্ত্রণ	Window size

চিত্র ১০.১ — TCP বনাম UDP

TCP: সংযোগ তৈরি → তথ্য পাঠানো → স্বীকৃতি → হারানো অংশ আবার পাঠানো → সংযোগ বন্ধ

UDP: সংযোগ নেই → datagram পাঠানো → স্বীকৃতি নেই → হারালে হারালো

TCP header বড়, নির্ভরযোগ্য | UDP header ছোট (8 byte), দ্রুত

চিত্র ১০.২ — Socket pair

Client: 192.0.2.5 : 52341 (ephemeral) → Server: 203.0.113.10 : 443 (HTTPS)

একই সার্ভার port 443-এ অনেক client সংযোগ করতে পারে, কারণ প্রতিটি client-এর source port আলাদা।

IP ঠিকানা = ভবন | Port = ফ্ল্যাট নম্বর | Socket = ভবন + ফ্ল্যাট নম্বর

বাস্তব পরিস্থিতি

প্রথম পরিস্থিতি: একটি ওয়েবসাইট ধীরে খুলছে। আপনি ss দিয়ে দেখলেন সার্ভারে অনেক সংযোগ TIME_WAIT অবস্থায় আছে। এর মানে সংযোগ বন্ধ হওয়ার পরও socket কিছু সময় ধরে থাকে। দ্বিতীয় পরিস্থিতি: একটি DNS প্রশ্নের উত্তর আসছে না। UDP-তে উত্তর না এলে client আবার চেষ্টা করে, কিন্তু TCP-তে সংযোগ তৈরি না হলে স্পষ্ট ভুল দেখা যায়। তৃতীয় পরিস্থিতি: একটি নতুন সেবা চালু করতে গিয়ে port already in use ভুল দেখা গেল। এর মানে অন্য একটি সেবা সেই port-এ already শুনছে। এই তিনটি পরিস্থিতিতেই port ও socket বোঝা সমস্যা নির্ণয়ের প্রথম ধাপ।

কাজের উদাহরণ

উদাহরণ ১: আপনার সার্ভারে SSH 22 port-এ, ওয়েব সার্ভার 80 ও 443 port-এ শুনছে। ss -tuln-এ আপনি তিনটি লাইন দেখবেন। উদাহরণ ২: ব্রাউজার থেকে example.com-এ গেলে source port ephemeral, destination port 443। উদাহরণ ৩: DNS প্রশ্ন পাঠালে destination port 53, প্রোটোকল UDP। উদাহরণ ৪: একটি সেবা 8080 port-এ শুনছে, কিন্তু firewall 8080 বন্ধ রেখেছে। Local-এ curl কাজ করবে, বাইরে থেকে কাজ করবে না। এই পার্থক্য বোঝা গুরুত্বপূর্ণ, কারণ local সাফল্য মানেই বাইরে থেকে সাফল্য নয়।

কমান্ড

নিচের কমান্ডগুলো Ubuntu Server 24.04-এ TCP, UDP ও port দেখতে ব্যবহৃত হয়।

```
ss -tuln
ss -tnp
ss -unp
ss -tlnp
nc -vz example.com 443
curl -v http://127.0.0.1:8080/
```

ss -tuln কমান্ড TCP ও UDP listening port দেখায়, সংখ্যা আকারে। ss -tnp কমান্ড TCP সংযোগ ও process-এর নাম দেখায়। ss -unp কমান্ড UDP সংযোগ ও process দেখায়। ss -tlnp কমান্ড listening TCP port ও process দেখায়। nc -

vz example.com 443 কমান্ড 443 port-এ সংযোগ পরীক্ষা করে। curl -v http://127.0.0.1:8080/ কমান্ড local 8080 port-এ HTTP অনুরোধ পাঠায়।

নির্দেশিত অনুশীলন

> Lab 10.1 — Listening port খুঁজুন
>
> ১. টার্মিনালে ss -tuln চালান। ২. TCP ও UDP লাইন আলাদা করুন। ৩. কোন port-এ কোন সেবা শুনছে তা লিখুন। ৪. ss -tnp চালিয়ে কোন process কোন সংযোগ ব্যবহার করছে তা দেখুন। ৫. nc -vz example.com 443 চালিয়ে বাইরের সার্ভারের port পরীক্ষা করুন।

উদাহরণ আউটপুট (illustrative)

```
$ ss -tuln
State  Recv-Q Send-Q Local Address:Port  Peer Address:Port
LISTEN 0      4096   0.0.0.0:22         0.0.0.0:*
LISTEN 0      4096   0.0.0.0:80         0.0.0.0:*
LISTEN 0      4096   [::]:443            [::]:*
$ nc -vz example.com 443
Connection to example.com 443 port [tcp/https] succeeded!
```

উপরের আউটপুটে 0.0.0.0:22 মানে সব IPv4 interface-এ 22 port শুনছে, [::]:443 মানে সব IPv6 interface-এ 443 port শুনছে। আপনার মেশিনে port ও process আলাদা হতে পারে।

সাধারণ ভুল

- TCP ও UDP port-কে এক ভাবা। একই সংখ্যার TCP ও UDP port আলাদা সেবা হতে পারে।
- Port খোলা থাকলেই সেবা চলছে ভাবা। Port শুনছে, কিন্তু application ভুল উত্তর দিতে পারে।
- Ephemeral port-কে সার্ভার port ভাবা। Client port অস্থায়ী, সার্ভার port স্থির।
- ss আউটপুটে 0.0.0.0 ও 127.0.0.1-এর পার্থক্য না বোঝা। প্রথমটি সব interface, দ্বিতীয়টি শুধু local।
- UDP-তে সংযোগ আশা করা। UDP-তে সংযোগ নেই, তাই ss-এ UDP সংযোগের অর্থ আলাদা।

সমস্যা নির্ণয়

যদি ss -tuln-এ port না দেখা যায়, তবে সেবা চালু নেই বা ভিন্ন port-এ শুনছে। প্রথমে সেবার অবস্থা দেখুন, তারপর কনফিগারেশন দেখুন। যদি port দেখা যায় কিন্তু বাইরে থেকে সংযোগ না হয়, তবে firewall দেখুন। Local-এ curl কাজ করলে সেবা ঠিক আছে, সমস্যা নেটওয়ার্ক বা firewall-এ। যদি nc-তে connection refused আসে, তবে port-এ কেউ শুনছে না। যদি timeout আসে, তবে firewall প্যাকেট ফেলে দিচ্ছে বা রাউটিং সমস্যা আছে। এই পার্থক্য অধ্যায় ১১-এ আরও বিস্তারিত আছে।

নিরাপত্তা বিবেচনা

প্রতিটি খোলা port একটি সম্ভাব্য প্রবেশপথ। শুধু প্রয়োজনীয় port খোলা রাখুন, বাকি সব বন্ধ রাখুন। SSH-এর জন্য 22 port খোলা থাকলে শক্তিশালী প্রমাণীকরণ ব্যবহার করুন। অপ্রয়োজনীয় সেবা বন্ধ রাখুন, কারণ সেবা চললেই port খোলে।

UDP সেবায় rate limiting ভাবুন, কারণ UDP-তে সংযোগ নেই বলে অপব্যবহার সহজ হতে পারে। Port scan দেখলে আতঙ্কিত হবেন না, কিন্তু লগ দেখে অচেনা সেবা খোলা আছে কি না তা যাচাই করুন।

সেরা অনুশীলন

- সেবা চালু করার আগে কোন port ব্যবহার করবে তা নথিভুক্ত করুন।
- ss -tuln দিয়ে নিয়মিত listening port পরীক্ষা করুন।
- Production সার্ভারে শুধু প্রয়োজনীয় port firewall-এ খুলুন।
- Well-known port বদলালে তা দলের সবাইকে জানান।
- পরিবর্তনের আগে ও পরে ss আউটপুট সংরক্ষণ করুন।

Knowledge Check — জ্ঞান যাচাই

1. কোন প্রোটোকল নির্ভরযোগ্য ও connection-oriented? A) UDP B) TCP C) ICMP D) ARP
2. HTTPS সাধারণত কোন port ব্যবহার করে? A) 80 B) 22 C) 443 D) 53
3. Socket pair-এ কয়টি অংশ থাকে? A) 2টি B) 3টি C) 4টি D) 5টি
4. DNS প্রশ্ন সাধারণত কোন প্রোটোকল ব্যবহার করে? A) শুধু TCP B) শুধু UDP C) UDP, প্রয়োজনে TCP D) ICMP
5. ss -tuln কমান্ড কী দেখায়? A) শুধু process নাম B) Listening TCP ও UDP port C) শুধু routing table D) শুধু DNS উত্তর

স্বাধীন অ্যাসাইনমেন্ট

আপনার Ubuntu Server-এ ss -tuln ও ss -tnp চালিয়ে একটি তালিকা তৈরি করুন। প্রতিটি listening port-এর পাশে প্রোটোকল, port সংখ্যা, process-এর নাম (যদি দেখা যায়) এবং সেবাটি প্রয়োজনীয় কি না তা লিখুন। অপ্রয়োজনীয় মনে হলে কেন অপ্রয়োজনীয় তা যুক্তি দিয়ে লিখুন। তালিকাটি এক পৃষ্ঠায় লিখুন এবং প্রতিটি সিদ্ধান্তের পাশে কমান্ডের নাম লিখুন।

সারাংশ

TCP নির্ভরযোগ্য, connection-oriented প্রোটোকল; UDP দ্রুত, connectionless প্রোটোকল। Port মেশিনের ভেতরে সেবা চেনায়, socket IP ও port-এর সমন্বয়ে যোগাযোগের শেষ প্রান্ত তৈরি করে, socket pair একটি সংযোগকে অনন্যভাবে চেনায়। Well-known port 0-1023, ephemeral port client-এর অস্থায়ী port। Ubuntu Server-এ ss -tuln, ss -tnp ও ss -unp দিয়ে port ও সংযোগ দেখা যায়, nc ও curl দিয়ে সেবা পরীক্ষা করা যায়। এই ধারণাগুলো অধ্যায় ১১-এ সংযোগের জীবনচক্র বোঝার ভিত্তি।

Revision Checklist — পুনরালোচনার তালিকা

- আমি TCP ও UDP-এর পার্থক্য ব্যাখ্যা করতে পারি।
- আমি well-known, registered ও ephemeral port চিনতে পারি।
- আমি socket ও socket pair ব্যাখ্যা করতে পারি।
- আমি ss -tuln, ss -tnp ও ss -unp চালাতে পারি।
- আমি nc ও curl দিয়ে port পরীক্ষা করতে পারি।

- আমি 0.0.0.0 ও 127.0.0.1-এর পার্থক্য ব্যাখ্যা করতে পারি।

Chapter 11: TCP Connection Lifecycle and Troubleshooting — TCP সংযোগের জীবনচক্র ও সমস্যা নির্ণয়

অধ্যায় ১১ • Networking & Internet Infrastructure • Ubuntu Server 24.04 LTS • আনুমানিক পড়ার সময়: ৫৫ মিনিট • হাতে-কলমে অনুশীলনসহ

শুরুতে: একটি বাস্তব সমস্যা

আপনি একটি ওয়েবসাইট খুলতে গেলেন, কিন্তু পাতা খুলছে না। কখনো connection refused লেখা আসে, কখনো অনেকক্ষণ ধরে ঘুরতে থাকে, তারপর timeout হয়। দুটি ভুল দেখতে একই মনে হলেও তাদের কারণ সম্পূর্ণ আলাদা। Connection refused মানে সার্ভারে পৌঁছানো গেছে, কিন্তু সেই port-এ কেউ শুনছে না। Timeout মানে প্যাকেট পৌঁছায়নি, অথবা উত্তর ফিরে আসেনি। একজন DevOps প্রকৌশলী হিসেবে এই পার্থক্য বোঝা আপনার দৈনন্দিন কাজ। এই অধ্যায়ে আপনি শিখবেন TCP সংযোগ কীভাবে তৈরি হয়, কীভাবে তথ্য যায়, কীভাবে সংযোগ বন্ধ হয় এবং ss, ping, curl ও tcpdump-এর ধারণা দিয়ে সমস্যা কীভাবে নির্ণয় করবেন।

উপমা: TCP সংযোগকে ভাবুন একটি ফোন কলের মতো। প্রথমে ডায়াল করা হয়, অপর পক্ষ ফোন ধরে, তারপর কথা হয়, শেষে বিদায় জানিয়ে ফোন রাখা হয়। ফোন না ধরলে, ব্যস্ত থাকলে, অথবা লাইন কেটে গেলে প্রতিটি ক্ষেত্রে আলাদা অর্থ হয়। TCP-তেও SYN, SYN-ACK ও ACK দিয়ে কল শুরু হয়, FIN দিয়ে ভদ্রভাবে শেষ হয়, আর RST দিয়ে হঠাৎ কেটে যায়।

Learning Objectives — শেখার লক্ষ্য

- এই অধ্যায় শেষে আপনি three-way handshake (SYN, SYN-ACK, ACK) ব্যাখ্যা করতে পারবেন।
- আপনি sequence number ও acknowledgement number-এর কাজ ব্যাখ্যা করতে পারবেন।
- আপনি FIN ও RST দিয়ে সংযোগ বন্ধ হওয়ার পার্থক্য ব্যাখ্যা করতে পারবেন।
- আপনি retransmission, windowing ও flow control ব্যাখ্যা করতে পারবেন।
- আপনি congestion control-এর প্রাথমিক ধারণা ব্যাখ্যা করতে পারবেন।
- আপনি timeout ও connection refused-এর মধ্যে পার্থক্য করতে পারবেন এবং ss, ping, curl ও tcpdump ধারণা দিয়ে সমস্যা নির্ণয় করতে পারবেন।

Prerequisites — পূর্বশর্ত

- অধ্যায় ১০ থেকে TCP, UDP, port ও socket ধারণা জানা থাকতে হবে।
- অধ্যায় ৯ থেকে IPv4 ও IPv6 ঠিকানা জানা থাকতে হবে।
- Ubuntu Server 24.04-এ ss, ping ও curl চালানোর অভিজ্ঞতা থাকতে হবে।
- একটি পরীক্ষামূলক সার্ভার বা local মেশিন থাকতে হবে, যেখানে আপনি নিরাপদে পরীক্ষা করতে পারেন।

প্রথম নীতি: নির্ভরযোগ্য সংযোগের জন্য চুক্তি দরকার

দুটি যন্ত্রের মধ্যে নির্ভরযোগ্য যোগাযোগের জন্য প্রথমে একটি চুক্তি দরকার: আমরা কথা বলতে প্রস্তুত, আমরা একে অপরের কথা শুনছি, আমরা তথ্যের ক্রম ঠিক রাখব। TCP এই চুক্তি তৈরি করে three-way handshake দিয়ে। Handshake ছাড়া

প্রেরক জানত না প্রাপক প্রস্তুত কি না, প্রাপক জানত না প্রেরক কে। Handshake-এর পর দুই পক্ষই জানে সংযোগ তৈরি হয়েছে এবং তথ্য পাঠানো যাবে। এই চুক্তি না থাকলে তথ্য হারালে কেউ জানত না, ক্রম উল্টে গেলে কেউ ঠিক করত না। তাই handshake TCP-এর নির্ভরযোগ্যতার ভিত্তি।

Sequence number হলো প্রতিটি byte-এর ক্রমিক নম্বর। প্রেরক প্রথম byte-কে একটি নম্বর দেয়, তারপর প্রতিটি byte সেই নম্বর থেকে গোনা হয়। প্রাপক acknowledgement number দিয়ে জানায় সে কোন নম্বর পর্যন্ত পেয়েছে এবং পরের নম্বর কী আশা করছে। উদাহরণস্বরূপ, প্রাপক যদি acknowledgement 1001 পাঠায়, তবে এর অর্থ সে 1000 পর্যন্ত পেয়েছে এবং 1001 আশা করছে। এই সংখ্যা দিয়েই হারানো অংশ চেনা যায় এবং আবার পাঠানো যায়।

Three-Way Handshake

প্রথম ধাপে client SYN পাঠায়। SYN মানে synchronize, অর্থাৎ আমি সংযোগ করতে চাই এবং আমার প্রথম sequence number এইটি। দ্বিতীয় ধাপে server SYN-ACK পাঠায়। এর অর্থ, আমি তোমার SYN পেয়েছি, আমি সংযোগে রাজি এবং আমার প্রথম sequence number এইটি। তৃতীয় ধাপে client ACK পাঠায়। এর অর্থ, আমি তোমার SYN-ACK পেয়েছি, সংযোগ তৈরি। এই তিনটি ধাপের পর সংযোগ ESTABLISHED অবস্থায় যায় এবং তথ্য পাঠানো শুরু হয়। যদি কোনো ধাপে উত্তর না আসে, তবে client আবার চেষ্টা করে, তারপর সময় শেষ হলে timeout হয়।

Handshake-এর সময় দুই পক্ষ window size-ও জানায়। Window size বলে দেয় প্রাপক একসঙ্গে কত তথ্য নিতে পারে। এটি flow control-এর ভিত্তি। Handshake-এর সময় maximum segment size-ও আলোচনা হতে পারে, যা একটি segment কত বড় হতে পারে তা ঠিক করে। এই সব আলোচনা সংযোগ শুরুর আগেই হয়, যাতে তথ্য পাঠানোর সময় বিভ্রান্তি না হয়।

তথ্য পাঠানো, স্বীকৃতি ও Retransmission

সংযোগ তৈরি হলে প্রেরক তথ্য segment-এ ভাগ করে পাঠায়। প্রতিটি segment-এ sequence number থাকে। প্রাপক segment পেলে acknowledgement পাঠায়। প্রেরক যদি নির্দিষ্ট সময়ের মধ্যে acknowledgement না পায়, তবে ধরে নেয় segment হারিয়েছে এবং তা আবার পাঠায়। এই প্রক্রিয়াকে retransmission বলা হয়। Retransmission-এর সময় আবার পাঠানোর ব্যবধান ধীরে ধীরে বাড়ে, যাতে নেটওয়ার্কে ভিড় না বাড়ে। এই ব্যবস্থার কারণে TCP ধীর নেটওয়ার্কেও তথ্য পৌঁছে দেয়, তবে সময় বেশি লাগতে পারে।

Windowing ও Flow Control

Flow control-এর কাজ হলো প্রেরক যেন প্রাপকের চেয়ে দ্রুত তথ্য না পাঠায়। প্রাপক window size দিয়ে জানায় সে কত তথ্য নিতে পারে। প্রেরক সেই সীমার মধ্যে তথ্য পাঠায়। প্রাপক তথ্য পড়ে ফেললে window বড় হয়, আর প্রাপক ব্যস্ত থাকলে window ছোট হয়। Window শূন্য হলে প্রেরক অপেক্ষা করে। এই ব্যবস্থা না থাকলে দ্রুত প্রেরক ধীর প্রাপককে তথ্যে ডুবিয়ে দিত। Sliding window নামে একটি পদ্ধতিতে window ধীরে ধীরে এগিয়ে যায়, যাতে তথ্য ক্রমাগত পাঠানো যায়।

Congestion Control: একটি সংক্ষিপ্ত পরিচয়

Congestion control flow control থেকে আলাদা। Flow control প্রাপকের গতি দেখে, congestion control নেটওয়ার্কের ভিড় দেখে। নেটওয়ার্কে ভিড় বাড়লে প্যাকেট হারায়, দেরি বাড়ে। TCP তখন পাঠানোর গতি কমায়। ভিড় কমলে গতি ধীরে ধীরে বাড়ায়। এই প্রক্রিয়ায় slow start, congestion avoidance ও fast retransmit-এর মতো ধারণা আছে। এই অধ্যায়ে গভীর গণিত দরকার নেই, শুধু এই নীতিটি মনে রাখুন: TCP নেটওয়ার্কের অবস্থা অনুযায়ী গতি বদলায়, সবসময় একই গতিতে পাঠায় না। তাই একই সংযোগ কখনো দ্রুত, কখনো ধীর হতে পারে।

সংযোগ বন্ধ করা: FIN ও RST

সংযোগ শেষ করার ভদ্র উপায় হলো FIN। এক পক্ষ FIN পাঠায়, অর্থাৎ আমার আর তথ্য নেই। অপর পক্ষ ACK পাঠায়, তারপর নিজের FIN পাঠায়। প্রথম পক্ষ ACK পাঠালে সংযোগ বন্ধ হয়। এই প্রক্রিয়াকে four-way termination বলা হয়, কারণ চারটি বার্তা যায়। RST হলো হঠাৎ বন্ধ করা। RST মানে সংযোগ এখনই বন্ধ করো, আর কোনো তথ্য আশা কোরো না। RST তখন আসে যখন port-এ কেউ শুনছে না, সংযোগে ভুল হয়েছে, অথবা application হঠাৎ বন্ধ হয়ে গেছে। FIN ভদ্র বিদায়, RST হঠাৎ লাইন কেটে দেওয়া।

ভেতরের পথচলা: একটি HTTP অনুরোধের জীবন

ধরুন, আপনি curl দিয়ে একটি ওয়েবসাইটে অনুরোধ পাঠালেন। প্রথমে DNS IP ঠিকানা দেয়। তারপর TCP handshake হয়: SYN, SYN-ACK, ACK। সংযোগ ESTABLISHED হলে curl HTTP অনুরোধ পাঠায়। সার্ভার উত্তর পাঠায়, প্রতিটি অংশের acknowledgement যায়। উত্তর সম্পূর্ণ হলে সার্ভার বা client FIN পাঠায়, সংযোগ বন্ধ হয়। ss -tnp চালালে আপনি এই সংযোগের অবস্থা দেখতে পারেন: ESTABLISHED, TIME_WAIT, CLOSE_WAIT। TIME_WAIT মানে সংযোগ বন্ধ হয়েছে, কিন্তু socket কিছু সময় ধরে থাকে, যাতে দেরিতে আসা প্যাকেট বিভ্রান্তি না করে। এই অবস্থা স্বাভাবিক, তবে অনেক TIME_WAIT থাকলে সার্ভারে port শেষ হয়ে যেতে পারে।

পরিভাষা

পরিভাষা	অর্থ	উদাহরণ
SYN	Synchronize, সংযোগ শুরুর অনুরোধ	Client থেকে Server-এ SYN
SYN-ACK	Server-এর উত্তর, SYN গ্রহণ ও নিজের SYN	Server থেকে Client-এ SYN-ACK
ACK	Acknowledgement, তথ্য পাওয়ার স্বীকৃতি	ACK 1001
Sequence number	তথ্যের byte-এর ক্রমিক নম্বর	Seq 1000
FIN	Finish, ভদ্রভাবে সংযোগ বন্ধের অনুরোধ	FIN পাঠিয়ে সংযোগ বন্ধ
RST	Reset, হঠাৎ সংযোগ বন্ধ করা	Port বন্ধ থাকলে RST
Retransmission	হারানো segment আবার পাঠানো	ACK না পেলে আবার পাঠানো
Window	প্রাপক একসঙ্গে কত তথ্য নিতে পারে	Window size 65535
Timeout	নির্দিষ্ট সময়ে উত্তর না পাওয়া	Connection timed out
Connection refused	সার্ভারে পৌঁছানো গেছে, কিন্তু port-এ কেউ শুনছে না	Connection refused

চিত্র ১১.১ — Three-way handshake

Client → Server: SYN, Seq 1000

Server → Client: SYN-ACK, Seq 5000, Ack 1001

Client → Server: ACK, Seq 1001, Ack 5001

এরপর সংযোগ ESTABLISHED, তথ্য পাঠানো শুরু হয়।

চিত্র ১১.২ — সংযোগ বন্ধ করা

ভদ্রভাবে: FIN → ACK → FIN → ACK (four-way termination)

হঠাৎ: RST (কোনো বিদায় নেই, সংযোগ এখনই বন্ধ)

TIME_WAIT: সংযোগ বন্ধের পর socket কিছু সময় থাকে, দেরিতে আসা প্যাকেটের জন্য।

বাস্তব পরিস্থিতি

প্রথম পরিস্থিতি: একটি সার্ভারে curl করলে connection refused আসে। এর অর্থ সার্ভার চালু আছে, কিন্তু সেই port-এ সেবা শুনছে না। দ্বিতীয় পরিস্থিতি: curl করলে অনেকক্ষণ পর timeout হয়। এর অর্থ firewall প্যাকেট ফেলে দিচ্ছে, অথবা সার্ভারে পৌঁছানো যাচ্ছে না। তৃতীয় পরিস্থিতি: সংযোগ তৈরি হয়, কিন্তু তথ্য মাঝপথে আটকে যায়। এর কারণ হতে পারে window শূন্য হয়ে যাওয়া, অথবা নেটওয়ার্কে ভিড়। এই তিনটি পরিস্থিতিতে প্রথমে ss দিয়ে সংযোগের অবস্থা দেখুন, তারপর ping দিয়ে পৌঁছানো যায় কি না দেখুন, তারপর curl দিয়ে application পরীক্ষা করুন।

কাজের উদাহরণ

উদাহরণ ১: nc -vz 127.0.0.1 22 চালালে connection refused এলে বুঝবেন SSH সেবা বন্ধ আছে। উদাহরণ ২: ping কাজ করে, কিন্তু curl timeout হয়। এর অর্থ নেটওয়ার্ক ঠিক আছে, কিন্তু port firewall-এ বন্ধ আছে। উদাহরণ ৩: ss -tnp-এ অনেক সংযোগ TIME_WAIT অবস্থায় আছে। এর অর্থ অনেক সংযোগ বন্ধ হয়েছে, কিন্তু socket এখনো মুক্ত হয়নি। উদাহরণ ৪: একটি সংযোগ ESTABLISHED, কিন্তু তথ্য যাচ্ছে না। এর কারণ হতে পারে application আটকে আছে, অথবা window শূন্য। এই উদাহরণগুলো থেকে বোঝা যায়, একটি কমান্ডের ফলাফল দিয়ে সব বোঝা যায় না, কয়েকটি কমান্ড মিলিয়ে সিদ্ধান্ত নিতে হয়।

কমান্ড

নিচের কমান্ডগুলো Ubuntu Server 24.04-এ TCP সংযোগ দেখতে ও পরীক্ষা করতে ব্যবহৃত হয়। tcpdump একটি শক্তিশালী টুল, তবে তা শুধু অনুমতি থাকা নেটওয়ার্কে ও পরীক্ষামূলক পরিবেশে ব্যবহার করুন।

```
ss -tnp
ss -tuln
ss -tn state established
ping -c 4 example.com
curl -v http://127.0.0.1:8080/
```

```
sudo tcpdump -i eth0 -n tcp port 80
nc -vz example.com 443
```

ss -tnp কমান্ড TCP সংযোগ ও process দেখায়। ss -tuln কমান্ড listening port দেখায়। ss -tn state established কমান্ড শুধু ESTABLISHED সংযোগ দেখায়। ping -c 4 example.com কমান্ড চারটি ping পাঠায়। curl -v কমান্ড HTTP অনুরোধের বিস্তারিত দেখায়। sudo tcpdump -i eth0 -n tcp port 80 কমান্ড eth0 interface-এ 80 port-এর TCP প্যাকেট দেখায়, এখানে eth0 আপনার interface দিয়ে বদলাতে হবে। nc -vz কমান্ড port-এ সংযোগ পরীক্ষা করে।

নির্দেশিত অনুশীলন

> Lab 11.1 — Timeout বনাম Connection Refused
>
> ১. ss -tuln চালিয়ে কোন port শুনছে তা লিখুন। ২. nc -vz 127.0.0.1 22 চালান, ফলাফল লিখুন। ৩. একটি বন্ধ port-এ, যেমন 9999, nc -vz 127.0.0.1 9999 চালান, ফলাফল লিখুন। ৪. curl -v http://127.0.0.1:8080/ চালান, ফলাফল লিখুন। ৫. দুটি ফলাফলের পার্থক্য ব্যাখ্যা করুন: কোনটি refused, কোনটি timeout, কেন।

উদাহরণ আউটপুট (illustrative)

```
$ ss -tnp
State      Recv-Q Send-Q Local Address:Port  Peer Address:Port
ESTABLISHED 0      0      192.0.2.5:52341      203.0.113.10:443
$ nc -vz 127.0.0.1 9999
nc: connect to 127.0.0.1 port 9999 (tcp) failed: Connection refused
$ sudo tcpdump -i eth0 -n tcp port 80
tcpdump: listening on eth0
IP 192.0.2.5.52341 > 203.0.113.10.80: Flags [S], seq 1000
IP 203.0.113.10.80 > 192.0.2.5.52341: Flags [S.], seq 5000, ack 1001
```

উপরের tcpdump অংশটি illustrative, বাস্তব capture-এ সময়, IP ও sequence number আলাদা হবে। Flags [S] মানে SYN, [S.] মানে SYN-ACK। আপনার মেশিনে আউটপুট আলাদা হতে পারে, কিন্তু refused ও timeout-এর অর্থ একই থাকবে।

সাধারণ ভুল

- Timeout ও connection refused-কে এক ভাবা। প্রথমটি পৌঁছাতে না পারা, দ্বিতীয়টি port বন্ধ থাকা।
- Ping কাজ করলেই সেবা কাজ করবে ভাবা। Ping ICMP, সেবা TCP port-এ চলে।
- TIME_WAIT দেখে আতঙ্কিত হওয়া। এটি স্বাভাবিক অবস্থা, তবে অতিরিক্ত হলে পরীক্ষা করুন।
- tcpdump-এ সব প্যাকেট দেখে সিদ্ধান্ত নেওয়া। প্রথমে ss ও curl দিয়ে সহজ পরীক্ষা করুন।
- একটি সংযোগের অবস্থা দেখে পুরো সার্ভারের অবস্থা ধরে নেওয়া। কয়েকটি সংযোগ দেখে সিদ্ধান্ত নিন।

সমস্যা নির্ণয়

সমস্যা নির্ণয়ের ধাপগুলো ক্রমানুসারে করুন। প্রথমে ping দিয়ে দেখুন সার্ভারে পৌঁছানো যায় কি না। Ping না গেলে রাউটিং বা firewall সমস্যা। Ping গেলে ss -tuln দিয়ে দেখুন port শুনছে কি না। Port না শুনলে সেবা চালু করুন। Port শুনলে nc দিয়ে সংযোগ পরীক্ষা করুন। nc-তে refused এলে সেবা ভিন্ন interface-এ শুনছে কি না দেখুন। nc-তে timeout এলে

firewall দেখুন। সংযোগ তৈরি হলে curl দিয়ে application পরীক্ষা করুন। curl-এ ভুল এলে application লগ দেখুন। tcpdump শুধু তখনই ব্যবহার করুন, যখন ss, ping ও curl দিয়ে কারণ পাওয়া যাচ্ছে না। প্রতিটি ধাপের ফলাফল লিখে রাখুন, যাতে পরে তুলনা করা যায়।

নিরাপত্তা বিবেচনা

tcpdump দিয়ে প্যাকেট দেখা যায়, তাই তা শুধু নিজের মেশিনে ও অনুমতি থাকা নেটওয়ার্কে ব্যবহার করুন। অন্যের প্যাকেট দেখা গোপনীয়তার লঙ্ঘন হতে পারে। SYN flood আক্রমণে অনেক SYN পাঠিয়ে সার্ভারকে ব্যস্ত রাখা হয়, তাই SYN backlog ও rate limiting-এর কথা ভাবুন। RST আক্রমণে সংযোগ জোর করে বন্ধ করা যায়, তাই গুরুত্বপূর্ণ সংযোগে TLS ব্যবহার করুন। খোলা port-এর তালিকা নিয়মিত পরীক্ষা করুন, অপ্রয়োজনীয় port বন্ধ রাখুন। লগে বারবার refused সংযোগ দেখলে তা আক্রমণের চেষ্টা হতে পারে, তবে একবারের refused স্বাভাবিকও হতে পারে।

সেরা অনুশীলন

- সমস্যা নির্ণয়ে সবসময় সহজ পরীক্ষা আগে করুন: ping, ss, nc, curl, তারপর tcpdump।
- Timeout ও refused-এর পার্থক্য লিখে রাখুন এবং দলের সবাইকে জানান।
- পরিবর্তনের আগে ss -tnp আউটপুট সংরক্ষণ করুন।
- Production সার্ভারে tcpdump চালানোর আগে অনুমতি নিন ও সময় সীমিত রাখুন।
- Application লগ ও নেটওয়ার্ক পরীক্ষা একসঙ্গে দেখুন, শুধু একটি দিয়ে সিদ্ধান্ত নেবেন না।

Knowledge Check — জ্ঞান যাচাই

1. Three-way handshake-এর সঠিক ক্রম কোনটি? A) SYN, ACK, SYN-ACK B) SYN, SYN-ACK, ACK C) ACK, SYN, SYN-ACK D) SYN-ACK, SYN, ACK
2. Acknowledgement number কী জানায়? A) পরের byte-এর নম্বর B) Window size C) Port সংখ্যা D) IP ঠিকানা
3. Connection refused-এর অর্থ কী? A) Firewall প্যাকেট ফেলেছে B) Port-এ কেউ শুনছে না C) DNS কাজ করছে না D) নেটওয়ার্কে ভিড় আছে
4. FIN-এর কাজ কী? A) সংযোগ হঠাৎ বন্ধ করা B) ভদ্রভাবে সংযোগ বন্ধ করা C) সংযোগ তৈরি করা D) তথ্য আবার পাঠানো
5. ss -tn state established কমান্ড কী দেখায়? A) সব listening port B) শুধু ESTABLISHED TCP সংযোগ C) শুধু UDP সংযোগ D) Routing table

স্বাধীন অ্যাসাইনমেন্ট

আপনার সার্ভারে একটি সেবা বেছে নিন, যেমন SSH বা একটি local ওয়েব সেবা। প্রথমে ss -tuln দিয়ে port দেখুন, তারপর nc দিয়ে সংযোগ পরীক্ষা করুন, তারপর curl দিয়ে application পরীক্ষা করুন। একটি পরীক্ষা ইচ্ছাকৃতভাবে ব্যর্থ করুন, যেমন বন্ধ port-এ সংযোগ করে connection refused দেখুন। প্রতিটি ধাপের কমান্ড, ফলাফল ও ব্যাখ্যা এক পৃষ্ঠায় লিখুন। শেষে লিখুন, timeout ও refused-এর মধ্যে পার্থক্য আপনি কীভাবে বুঝলেন।

সারাংশ

TCP সংযোগ three-way handshake (SYN, SYN-ACK, ACK) দিয়ে তৈরি হয়, sequence ও acknowledgement number দিয়ে তথ্যের ক্রম ঠিক রাখা হয়, retransmission দিয়ে হারানো অংশ আবার পাঠানো হয়, windowing ও flow control দিয়ে প্রাপকের গতি নিয়ন্ত্রণ করা হয় এবং congestion control দিয়ে নেটওয়ার্কের ভিড় অনুযায়ী গতি বদলানো হয়। সংযোগ FIN দিয়ে ভদ্রভাবে বন্ধ হয়, RST দিয়ে হঠাৎ বন্ধ হয়। Connection refused মানে port-এ কেউ শুনছে না, timeout মানে উত্তর আসেনি। ss, ping, curl ও tcpdump ধারণা দিয়ে ধাপে ধাপে সমস্যা নির্ণয় করা যায়। এই অধ্যায়ের দক্ষতা Vol 02-এর পরের অধ্যায়ে DNS, HTTP ও firewall বোঝার ভিত্তি হবে।

Revision Checklist — পুনরালোচনার তালিকা

- আমি SYN, SYN-ACK, ACK ক্রম ব্যাখ্যা করতে পারি।
- আমি sequence ও acknowledgement number-এর কাজ ব্যাখ্যা করতে পারি।
- আমি FIN ও RST-এর পার্থক্য ব্যাখ্যা করতে পারি।
- আমি retransmission, windowing ও congestion control ব্যাখ্যা করতে পারি।
- আমি timeout ও connection refused-এর পার্থক্য করতে পারি।
- আমি ss, ping, curl ও tcpdump ধারণা দিয়ে সমস্যা নির্ণয় করতে পারি।

PART III • DEVOPS ENGINEERING MASTERY

Internet Protocols

ইন্টারনেট প্রোটোকল — রাউটিং, DNS, DHCP, HTTP, TLS ও রিভার্স প্রক্সি।

অধ্যায় ১২ • Part III — Internet Protocols

Chapter 12: Routing, Gateways and NAT — রাউটিং, গেটওয়ে ও NAT

শুরুতে: একটি বাস্তব দৃশ্য

ঢাকার একটি ছোট সফটওয়্যার কোম্পানির অফিসে একটি ল্যাপটপ থেকে একজন developer `github.com`-এ কোড push করলেন। ল্যাপটপটি অফিসের Wi-Fi-তে যুক্ত, তার IP ঠিকানা `192.168.1.50` — একটি private ঠিকানা, যা ইন্টারনেটে কোথাও রাউট করা যায় না। তবু push সফল হলো। কীভাবে? প্যাকেটটি প্রথমে গেল অফিসের router-এ, router সিদ্ধান্ত নিল কোন পথে পাঠাবে, তারপর NAT-এর মাধ্যমে private ঠিকানা বদলে public ঠিকানা হলো, এবং উত্তর ফেরার সময় আবার ঠিক ল্যাপটপটিতেই পৌঁছাল। এই অধ্যায়ে আমরা সেই সিদ্ধান্তের যুক্তি শিখব: routing কী, gateway কী, routing table কীভাবে কাজ করে, আর NAT কেন ইন্টারনেটকে চালু রেখেছে।

Learning Objectives — শেখার উদ্দেশ্য

- Routing কী এবং কেন দরকার, তা প্রথম নীতি থেকে ব্যাখ্যা করতে পারবেন।
- Routing table পড়তে ও ব্যাখ্যা করতে পারবেন: destination, gateway, interface, metric।
- Default gateway-এর ভূমিকা ব্যাখ্যা করতে পারবেন।
- Static ও dynamic routing-এর পার্থক্য বলতে পারবেন।
- NAT ও PAT কীভাবে private-to-public যোগাযোগ সম্ভব করে, তা ব্যাখ্যা করতে পারবেন।
- Port forwarding-এর ধারণা ও ঝুঁকি ব্যাখ্যা করতে পারবেন।
- `ip route`, `ping` ও `traceroute` দিয়ে রাউটিং সমস্যা নির্ণয় করতে পারবেন।

Prerequisites — পূর্বজ্ঞান

অধ্যায় ৬–৭-এর MAC address, Ethernet frame ও IPv4 address-এর ধারণা, এবং Vol 01-এর Linux terminal ও `ip` কমান্ডের প্রাথমিক অভিজ্ঞতা। Subnet ও CIDR না জানলে অধ্যায় ৮ আবার দেখে নিন — routing table পড়তে CIDR লাগবেই।

১২.১ প্রথম নীতি: প্যাকেট নিজে পথ জানে না

একটি IP প্যাকেটে শুধু source ও destination ঠিকানা লেখা থাকে; কোন পথে যাবে, তা লেখা থাকে না। প্রতিটি hop-এ — প্রতিটি router-এ — কেউ একজন সিদ্ধান্ত নেয়: “এই destination-এর জন্য পরের ধাপ কোনটি?” এই সিদ্ধান্ত নেওয়ার কাজই routing। উপমা: ডাকঘর। চিঠিতে শুধু প্রাপকের ঠিকানা লেখা থাকে; প্রতিটি ডাকঘর ঠিকানা দেখে সিদ্ধান্ত নেয় চিঠিটি কোন শহরের বস্তায় যাবে। কোনো ডাকঘরই পুরো পথ জানে না — শুধু পরের ধাপ জানে। ইন্টারনেটও ঠিক এভাবেই চলে: hop-by-hop forwarding।

কেন এভাবে? কারণ ইন্টারনেটে কোটি কোটি নেটওয়ার্ক; কোনো একটি যন্ত্রের পক্ষে সব পথ মনে রাখা অসম্ভব। তাই প্রতিটি router শুধু তার আশপাশের নেটওয়ার্কগুলো সম্পর্কে জানে, আর বাকি সবকিছুর জন্য একটি default পথের ওপর ভরসা করে। এই বিকেন্দ্রীকরণই ইন্টারনেটকে টিকিয়ে রেখেছে: একটি পথ ভেঙে গেলে প্যাকেট অন্য পথে যায়।

> **CONCEPT:** Routing হলো সিদ্ধান্ত — কোন পথে পাঠাব। Forwarding হলো কাজ — সিদ্ধান্ত অনুযায়ী প্যাকেটটি আসলেই পাঠানো। Router দুটোই করে: routing table দেখে সিদ্ধান্ত নেয়, তারপর interface দিয়ে প্যাকেট পাঠায়।

১২.২ Routing Table: রাউটারের সিদ্ধান্তের খাতা

প্রতিটি Linux হোস্ট ও router-এর একটি routing table থাকে। প্রতিটি entry বলে: "এই destination network-এ যেতে হলে এই gateway দিয়ে, এই interface দিয়ে যাও।" সবচেয়ে গুরুত্বপূর্ণ entry হলো default route — `default via 192.168.1.1` — যার অর্থ, "যে destination-এর জন্য আলাদা কোনো entry নেই, তাকে এই gateway-তে পাঠাও।"

সিদ্ধান্তের নিয়মটি সহজ: **longest prefix match**। প্যাকেটের destination ঠিকানার সঙ্গে যে entry-র network prefix সবচেয়ে বেশি মিলে, সেই entry জেতে। যেমন `10.0.0.0/8` ও `10.1.0.0/16` — দুটোই `10.1.5.20`-এর সঙ্গে মিলে, কিন্তু `/16` বেশি নির্দিষ্ট বলে সেটিই জিতবে। আর কোনো entry না মিললে default route; default route-ও না থাকলে প্যাকেট ফেলে দেওয়া হয় — `Network is unreachable`।

চিত্র ১২.১ — Routing সিদ্ধান্তের প্রবাহ

প্যাকেট আসে → Destination IP পড়া হয় → Routing table-এ longest prefix match খোঁজা হয় → মিল পাওয়া entry-র gateway ও interface পাওয়া যায় → ARP দিয়ে gateway-র MAC address বের করা হয় → Ethernet frame-এ ভরে interface দিয়ে পাঠানো হয় → কোনো মিল না পেলে default route, তাও না থাকলে প্যাকেট বাতিল

চিত্র ১২.২ — একটি অফিস নেটওয়ার্কের রাউটিং

ল্যাপটপ (192.168.1.50) → অফিস router / default gateway (192.168.1.1) → ISP router → ইন্টারনেট backbone → সার্ভারের নেটওয়ার্কের router → সার্ভার। প্রতিটি তীরের মাথায় একটি routing সিদ্ধান্ত হয়; প্রতিটি router শুধু পরের hop জানে।

১২.৩ Default Gateway: "বাকি সবকিছু" যেখানে যায়

আপনার ল্যাপটপ যখন `192.168.1.0/24` নেটওয়ার্কের বাইরের কোনো ঠিকানায় প্যাকেট পাঠায়, সে নিজে জানে না পথ। সে প্যাকেটটি default gateway-কে দেয় — সাধারণত আপনার router। Gateway হলো সেই দরজা, যা দিয়ে local network থেকে বাইরে যাওয়া যায়। DHCP-র মাধ্যমে (অধ্যায় ১৪) এই gateway ঠিকানাটি স্বয়ংক্রিয়ভাবে পাওয়া যায়। Gateway ভুল থাকলে লক্ষণটি খুব চেনা: local নেটওয়ার্কে সব চলে (printer, file share), কিন্তু ইন্টারনেট চলে না।

১২.৪ Static বনাম Dynamic Routing

Static routing মানে মানুষ হাতে entry লেখে: "`10.2.0.0/16`-এ যেতে `192.168.1.254` দিয়ে যাও।" ছোট, স্থির নেটওয়ার্কে এটি সহজ ও অনুমেয়। কিন্তু নেটওয়ার্ক বড় হলে বা পথ বদলালে হাতে বদলানো অসম্ভব হয়ে যায়। **Dynamic routing**-এ router-রা নিজেদের মধ্যে protocol-এর মাধ্যমে (যেমন OSPF, BGP) নেটওয়ার্কের খবর আদান-প্রদান করে ও table নিজে নিজে তৈরি করে। ISP ও বড় প্রতিষ্ঠানে dynamic routing চলে; আপনার বাড়ি বা ছোট অফিসে static default route-ই যথেষ্ট। এই বইয়ে আমরা ধারণাটি বুঝব, protocol-এর গভীরে যাব না — তবে জানবেন: ইন্টারনেটের backbone চলে BGP-র ওপর, যা স্বায়ত্তশাসিত নেটওয়ার্কগুলোর (Autonomous System) মধ্যে পথ বিনিময় করে।

১২.৫ NAT ও PAT: একটি public ঠিকানায় অনেক যন্ত্র

IPv4 ঠিকানা সীমিত (অধ্যায় ৭)। সমাধান: ভেতরে সবাই private ঠিকানা (`192.168.x.x`, `10.x.x.x`) ব্যবহার করে, আর router বাইরে যাওয়ার সময় source ঠিকানা বদলে নিজের public ঠিকানা বসিয়ে দেয় — এটাই **NAT (Network Address Translation)**। উত্তরের প্যাকেট ফিরলে router মনে রাখা mapping দেখে আবার আসল যন্ত্রে পৌঁছে দেয়।

একটি public IP-তে একসঙ্গে অনেক যন্ত্র চালাতে NAT-এর সঙ্গে **PAT (Port Address Translation)** ব্যবহৃত হয়: প্রতিটি connection-কে আলাদা source port দিয়ে চিহ্নিত করা হয়। ল্যাপটপের connection port `50001`, ফোনেরটি `50002` — উত্তরের প্যাকেটের destination port দেখেই router বোঝে সেটি কার। আপনার বাড়ির router প্রতিদিন এটাই করে। NAT নিরাপত্তা দেয় না — এটি ঠিকানা বাঁচানোর কৌশল; ভেতরের যন্ত্র সরাসরি ইন্টারনেট থেকে পৌঁছানো যায় না বলে আকস্মিক সুরক্ষা মেলে, কিন্তু সেটিকে firewall ভাবা ভুল।

> **REAL WORLD:** একটি স্টার্টআপের অফিসে নতুন developer-এর ল্যাপটপে ইন্টারনেট চলছিল না, অথচ Wi-Fi যুক্ত ছিল। তদন্তে দেখা গেল DHCP থেকে ভুল default gateway এসেছে — পুরোনো router-এর ঠিকানা, যেটি আর চালু নেই। Gateway ঠিক করতেই সব ঠিক। শিক্ষা: “ইন্টারনেট নেই” মানেই ISP-র দোষ নয় — প্রথমে নিজের routing table দেখুন।

১২.৬ Port Forwarding

বাইরে থেকে ভেতরের একটি সার্ভারে পৌঁছাতে হলে router-কে বলতে হয়: “public IP-র port `8080`-এ আসা traffic ভেতরের `192.168.1.10:80`-এ পাঠাও।” এটাই port forwarding। বাড়িতে গেম সার্ভার বা পরীক্ষামূলক ওয়েব সার্ভার চালাতে এটি ব্যবহৃত হয়। ঝুঁকি: প্রতিটি forwarding একটি দরজা খোলে — যা ইন্টারনেটের যে কেউ চেষ্টা করতে পারে। তাই শুধু দরকারি port খুলুন, শক্তিশালী authentication রাখুন, আর সম্ভব হলে VPN বা reverse proxy ব্যবহার করুন (অধ্যায় ১৭, ১৯)।

১২.৭ পরিভাষা — Terminology

Term	বাংলা ব্যাখ্যা
Routing	প্যাকেট কোন পথে যাবে, সেই সিদ্ধান্ত নেওয়া।
Forwarding	সিদ্ধান্ত অনুযায়ী প্যাকেট আসলেই পাঠানো।
Routing table	Destination network, gateway ও interface-এর তালিকা।
Default gateway	Local network-এর বাইরের traffic যেখানে পাঠানো হয়।
Next hop	প্যাকেটের পরের গন্তব্য router।
Longest prefix match	সবচেয়ে নির্দিষ্ট (দীর্ঘতম) prefix-এর entry জেতার নিয়ম।
NAT	প্যাকেটের IP ঠিকানা বদলে দেওয়া — private থেকে public।
PAT	Port নম্বর দিয়ে অনেক connection-কে এক IP-তে চিহ্নিত করা।
Static / Dynamic routing	হাতে লেখা entry / protocol-এর মাধ্যমে স্বয়ংক্রিয় entry।

Port forwarding	বাইরের port-এর traffic ভেতরের নির্দিষ্ট যন্ত্রে পাঠানো।

১২.৮ বাস্তব উদাহরণ: প্যাকেটের যাত্রা, ধাপে ধাপে

ল্যাপটপ (`192.168.1.50`) → `203.0.113.10` সার্ভারে একটি TCP প্যাকেট পাঠাচ্ছে: (১) ল্যাপটপ দেখে destination নিজের subnet-এ নেই, তাই প্যাকেট default gateway `192.168.1.1`-কে দেয়; (২) gateway-র MAC address ARP দিয়ে বের করা হয়; (৩) router routing table দেখে ISP-র দিকে পাঠায়; (৪) প্রতিটি router একইভাবে পরের hop বেছে নেয়; (৫) সার্ভারের নেটওয়ার্কের router প্যাকেটটি সার্ভারে পৌঁছে দেয়; (৬) উত্তরের প্যাকেট একইভাবে — হয়তো অন্য পথে — ফেরে। লক্ষ করুন: প্রেরক কখনো পুরো পথ জানে না।

১২.৯ ব্যবহারিক কমান্ড

```
$ ip route
# routing table দেখায়: default via কোন gateway, কোন interface।
# উদাহরণ আউটপুট (illustrative):
# default via 192.168.1.1 dev eth0 proto dhcp metric 100
# 192.168.1.0/24 dev eth0 proto kernel scope link src 192.168.1.50

$ ip route get 8.8.8.8
# একটি নির্দিষ্ট destination-এর জন্য কোন route, gateway ও interface
# ব্যবহার হবে, তা দেখায় — প্যাকেট না পাঠিয়েই সিদ্ধান্ত যাচাই করা যায়।

$ traceroute 8.8.8.8
# প্যাকেট কোন কোন hop পেরিয়ে যায়, তা দেখায়। প্রতিটি hop-এ সময়ও দেখায়।
# নিজের নেটওয়ার্কের বাইরের hop-এ * দেখা গেলে সেটি সাধারণত ICMP উত্তর না দেওয়া,
# সবসময় সমস্যা নয়।
```

`ip route` পড়ার নিয়ম: `default via` হলো default gateway; `dev eth0` হলো interface; `proto dhcp` মানে entry-টি DHCP থেকে এসেছে; `metric` কম হলে সেই route-কে অগ্রাধিকার দেওয়া হয়।

১২.১০ Hands-on Exercise — নিজের routing table পড়া

LAB: (১) VM-এ `ip route` চালিয়ে default gateway ও local subnet লিখুন। (২) `ip route get 1.1.1.1` চালিয়ে দেখুন কোন interface দিয়ে যাবে। (৩) `ping -c 3 <gateway>` দিয়ে gateway-তে পৌঁছানো যায় কি না যাচাই করুন। (৪) `traceroute 8.8.8.8` চালিয়ে hop-গুলো গুনুন। যাচাই: gateway-তে ping সফল হলে local নেটওয়ার্ক ঠিক আছে; traceroute-এর প্রথম hop-ই gateway হওয়া উচিত।

উদাহরণ আউটপুট (illustrative):

```
$ ip route get 1.1.1.1
1.1.1.1 via 192.168.1.1 dev eth0 src 192.168.1.50 uid 1000
# অর্থ: 1.1.1.1-এর প্যাকেট 192.168.1.1 gateway দিয়ে eth0 interface থেকে যাবে।
```

১২.১১ Common Mistakes ও Troubleshooting

- **ভুল:** "ইন্টারনেট নেই" শুনেই ISP-কে দোষ দেওয়া। **সংশোধন:** প্রথমে `ping` gateway, তারপর `ping 8.8.8.8`, তারপর `ping google.com` — কোন ধাপে ভাঙে, তা দেখুন (অধ্যায় ২২-এর পদ্ধতি)।
- **ভুল:** NAT-কে firewall মনে করা। **সংশোধন:** NAT ঠিকানা বদলায়; firewall নিয়ম করে traffic আটকায়। দুটো আলাদা কাজ।
- **ভুল:** একই নেটওয়ার্কে দুটি gateway বসানো। **সংশোধন:** একটি default gateway রাখুন; বিশেষ subnet-এর জন্য আলাদা static route লিখুন।
- **লক্ষণ:** `Network is unreachable` — default route নেই বা interface down। `ip route` ও `ip link` দেখুন।
- **লক্ষণ:** কিছু সাইট চলে, কিছু চলে না — MTU বা নির্দিষ্ট route-এর সমস্যা হতে পারে; `traceroute` দিয়ে কোন hop-এ আটকায়, দেখুন।

TROUBLESHOOTING: Gateway-তে ping যায় কিন্তু ইন্টারনেটে যায় না — সম্ভবত router-এর NAT বা ISP-র পথের সমস্যা, অথবা DNS-এর সমস্যা (অধ্যায় ১৩)। Gateway-তে ping-ই যায় না — local নেটওয়ার্ক, কেবল বা gateway ঠিকানা পরীক্ষা করুন।

১২.১২ Security Considerations

Routing table বদলাতে পারে এমন কাউকে বিশ্বাস করবেন না: একটি ভুল বা দুষ্ট static route traffic-কে ভুল জায়গায় পাঠাতে পারে। Port forwarding খুললে সেই সার্ভিসটি ইন্টারনেটের সামনে উন্মুক্ত হয় — authentication, update ও log নজরদারি ছাড়া খুলবেন না। NAT-এর আড়ালে থাকা মানে নিরাপদ থাকা নয়; ভেতর থেকে বেরোনো malware-ও NAT পেরিয়ে বাইরে যেতে পারে।

১২.১৩ Best Practices

- IP পরিকল্পনা লিখে রাখুন: কোন subnet কোথায়, gateway কোনটি (অধ্যায় ৮, ২৪)।
- Default route-এর ওপর নির্ভর করুন; অকারণে static route বাড়াবেন না।
- পরিবর্তনের আগে `ip route` সংরক্ষণ করুন, যাতে ফিরিয়ে নেওয়া যায়।
- Production-এ routing বদলানোর আগে lab-এ পরীক্ষা করুন।

Knowledge Check — যাচাই করুন

CHECK YOUR UNDERSTANDING: উত্তর Answer Guide-এ আছে — আগে নিজে চেষ্টা করুন।

1. Routing table-এ দুটি entry-ই একটি destination-এর সঙ্গে মিললে কোনটি ব্যবহৃত হয়? A) যেটি প্রথমে লেখা B) যেটির prefix সবচেয়ে দীর্ঘ (longest prefix match) C) যেটির metric সবচেয়ে বেশি D) যেটি সবচেয়ে পুরোনো
2. Default gateway-এর কাজ কী? A) DNS নাম মেলানো B) Local subnet-এর বাইরের traffic-কে পরের hop-এ পাঠানো C) IP ঠিকানা বরাদ্দ করা D) প্যাকেট encrypt করা
3. NAT-এর প্রধান উদ্দেশ্য কী? A) গতি বাড়ানো B) সীমিত public IPv4 ঠিকানা দিয়ে অনেক private যন্ত্রকে ইন্টারনেটে যুক্ত করা C) ভাইরাস আটকানো D) DNS দ্রুত করা

4. PAT-এ একাধিক যন্ত্রের connection আলাদা করা হয় কী দিয়ে? A) MAC address দিয়ে B) Source port নম্বর দিয়ে C) Hostname দিয়ে D) কেবলের রং দিয়ে
5. `ip route get 8.8.8.8` কমান্ডটি কী করে? A) 8.8.8.8-এ ping পাঠায় B) ওই destination-এর জন্য কোন route ও gateway ব্যবহৃত হবে, তা দেখায় C) DNS lookup করে D) Route মুছে ফেলে

Practical Assignment

নিজের VM-এর routing table-এর একটি চিত্র আঁকুন: local subnet, default gateway, interface ও metric লিখুন। তারপর ব্যাখ্যা করুন — (ক) একই subnet-এর যন্ত্রে প্যাকেট কীভাবে যায়, (খ) ইন্টারনেটের ঠিকানায় প্যাকেট কীভাবে যায়, (গ) gateway বন্ধ থাকলে কী লক্ষণ দেখা যাবে।

Chapter Summary

প্যাকেট নিজে পথ জানে না; প্রতিটি router routing table দেখে পরের hop বেছে নেয়। Longest prefix match সিদ্ধান্তের নিয়ম, default gateway বাকি সব traffic-এর দরজা। NAT ও PAT সীমিত public IPv4 ঠিকানা দিয়ে অনেক private যন্ত্রকে ইন্টারনেটে যুক্ত করে। রাউটিং সমস্যা নির্ণয়ের শুরু `ip route` দিয়ে।

Revision Checklist

- ☐ Longest prefix match ব্যাখ্যা করতে পারি
- ☐ `ip route` পড়ে gateway ও interface চিনতে পারি
- ☐ NAT ও PAT-এর পার্থক্য বলতে পারি
- ☐ Port forwarding-এর ঝুঁকি ব্যাখ্যা করতে পারি
- ☐ Gateway-তে ping করে local সমস্যা আলাদা করতে পারি

অধ্যায় ১৩ • Part III — Internet Protocols

Chapter 13: DNS — How Domain Name Resolution Works — DNS: ডোমেইন নাম কীভাবে ঠিকানায় বদলায়

শুরুতে: একটি বাস্তব দৃশ্য

আপনি ব্রাউজারে `www.example.com` লিখে Enter চাপলেন। এক সেকেন্ডের ভগ্নাংশের মধ্যে ব্রাউজার জেনে গেল সার্ভারের IP ঠিকানা। কিন্তু ব্রাউজার তো আগে থেকে তা জানত না! পর্দার আড়ালে একটি প্রশ্নোত্তর পর্ব চলেছে: আপনার কম্পিউটার প্রথমে জিজ্ঞেস করেছে একটি resolver-কে, সে জিজ্ঞেস করেছে root server-কে, তারপর TLD server-কে, তারপর authoritative server-কে — এবং প্রত্যেকে শুধু পরের ঠিকানাটি জানিয়ে দিয়েছে। এই অধ্যায়ে আমরা সেই যাত্রা ধাপে ধাপে দেখব। DNS না বুঝলে “সাইট খুলছে না” সমস্যার অর্ধেকই বোঝা যায় না।

Learning Objectives — শেখার উদ্দেশ্য

- DNS কেন দরকার, তা ব্যাখ্যা করতে পারবেন।
- Domain hierarchy — root, TLD, authoritative — ব্যাখ্যা করতে পারবেন।
- Recursive ও iterative resolution-এর পার্থক্য বলতে পারবেন।
- A, AAAA, CNAME, MX, TXT, NS, PTR ও SRV রেকর্ডের কাজ ব্যাখ্যা করতে পারবেন।
- Caching ও TTL কীভাবে কাজ করে, তা ব্যাখ্যা করতে পারবেন।
- DNS propagation নিয়ে ভুল ধারণা শনাক্ত করতে পারবেন।
- `dig` ও `nslookup` দিয়ে DNS সমস্যা নির্ণয় করতে পারবেন।

Prerequisites — পূর্বজ্ঞান

IPv4 ঠিকানা (অধ্যায় ৭), routing ও gateway (অধ্যায় ১২), এবং UDP-র ধারণা (অধ্যায় ১০)। DNS মূলত UDP port 53-এ চলে — কেন, তা এই অধ্যায়েই বুঝবেন।

১৩.১ প্রথম নীতি: মানুষ নাম মনে রাখে, কম্পিউটার সংখ্যা

কম্পিউটার যোগাযোগ করে IP ঠিকানা দিয়ে; মানুষ মনে রাখে নাম। DNS হলো সেই অনুবাদ ব্যবস্থা — ফোনবুকের মতো। নাম না থাকলে প্রতিটি ওয়েবসাইটের সংখ্যা মনে রাখতে হতো, যা অসম্ভব। কিন্তু ফোনবুকের সঙ্গে একটি পার্থক্য: DNS একটি একক বই নয়, এটি একটি বিকেন্দ্রীকৃত শ্রেণিবদ্ধ ব্যবস্থা। কোনো একটি সার্ভার সব নাম জানে না; প্রত্যেকে নিজের অংশের উত্তর দেয়।

ডোমেইন নাম ডানদিক থেকে বামদিকে পড়া হয়: `www.example.com`-এ `com` হলো Top-Level Domain (TLD), `example` second-level domain, `www` subdomain বা host। শেষে একটি অদৃশ্য ডট থাকে — root — যা পুরো গাছের শিকড়।

> **CONCEPT:** DNS একটি database নয়, একটি distributed database। প্রতিটি zone-এর জন্য একটি authoritative server দায়িত্বশীল; বাকিরা জিজ্ঞেস করে করে উত্তর খুঁজে নেয়। এই বিভাজনই DNS-কে কোটি কোটি নাম সামলাতে দেয়।

১৩.২ Resolution-এর যাত্রা: ধাপে ধাপে

`www.example.com` খুলতে যা ঘটে: (১) ব্রাউজার প্রথমে নিজের cache, তারপর OS-এর cache দেখে — থাকলে সেখানেই শেষ। (২) না থাকলে প্রশ্ন যায় **recursive resolver**-এর কাছে (সাধারণত ISP-র বা `1.1.1.1` / `8.8.8.8`-এর মতো public resolver)। (৩) Resolver root server-কে জিজ্ঞেস করে: “ `com` কে সামলায়?” Root উত্তর দেয় TLD server-এর ঠিকানা — আসল উত্তর নয়, শুধু পরের ঠিকানা। (৪) Resolver TLD server-কে জিজ্ঞেস করে: “`example.com` কে সামলায়?” উত্তর: authoritative nameserver-এর ঠিকানা। (৫) Resolver authoritative server-কে জিজ্ঞেস করে: “`www.example.com`-এর A রেকর্ড কী?” এবার আসল উত্তর আসে: IP ঠিকানা। (৬) Resolver উত্তরটি cache-এ রেখে আপনার কম্পিউটারকে দেয়।

এখানে দুটি ভূমিকা আলাদা করুন। **Recursive** resolver আপনার হয়ে পুরো যাত্রাটি করে; **iterative** প্রশ্নে প্রতিটি server শুধু “আমি জানি না, তবে ওকে জিজ্ঞেস করো” বলে। Root ও TLD server-রা iterative উত্তর দেয়; আপনার resolver recursive কাজ করে।

চিত্র ১৩.১ — DNS Hierarchy (গাছ)
Root (.) → TLD: .com, .org, .bd, .net → Second-level: example.com → Subdomain/host: www.example.com, mail.example.com। প্রতিটি স্তর নিচের স্তরের জন্য দায়ী server-কে চেনে।

চিত্র ১৩.২ — Resolution-এর ধাপ
ব্রাউজার → (cache?) → Recursive resolver → Root server (“com-এর server কে?”) → TLD server (“example.com-এর server কে?”) → Authoritative server (“www-এর IP কী?”) → উত্তর ফিরে আসে → Resolver cache করে → ব্রাউজার IP পায় → HTTP request শুরু

১৩.৩ DNS রেকর্ড: প্রতিটি রেকর্ড একেকটি কাজ

রেকর্ড	কাজ	উদাহরণ
A	নাম → IPv4 ঠিকানা	www.example.com → 203.0.113.10
AAAA	নাম → IPv6 ঠিকানা	www.example.com → 2001:db8::1
CNAME	এক নামের উপনাম — অন্য নামের দিকে নির্দেশ	blog.example.com → example.com
MX	ওই ডোমেইনের মেইল কোন সার্ভার নেবে	example.com → mail.example.com
NS	ওই zone-এর authoritative nameserver কে	example.com → ns1.example.com
TXT	যেকোনো লেখা — SPF, DKIM, যাচাইকরণ	v=spf1 ...
PTR	IP → নাম (reverse DNS)	203.0.113.10 → www.example.com

SRV	কোনো সেবা কোন host ও port-এ চলে	_https._tcp.example.com

মনে রাখবেন: CNAME একটি নামকে আরেক নামে পাঠায়, IP দেয় না; চূড়ান্ত নামের A রেকর্ডেই IP থাকে। MX রেকর্ড না থাকলে মেইল আসে না — ওয়েবসাইট চললেও ইমেইল বন্ধ থাকতে পারে।

১৩.৪ Caching ও TTL: কেন উত্তর বদলাতে সময় লাগে

প্রতিটি DNS উত্তরের সঙ্গে একটি **TTL (Time To Live)** থাকে — সেকেন্ডে মাপা, এই উত্তরটি কতক্ষণ cache-এ রাখা যাবে। TTL শেষ না হওয়া পর্যন্ত resolver আবার জিজ্ঞেস করে না। তাই আপনি A রেকর্ড বদলালেও পুরোনো উত্তর TTL শেষ হওয়া পর্যন্ত অনেক জায়গায় দেখা যাবে — এটাই “propagation”-এর আসল অর্থ। DNS-এর কোনো জাদুকরী “ছড়িয়ে পড়া” নেই; শুধু বিভিন্ন cache-এ পুরোনো উত্তরের মেয়াদ শেষ হওয়ার অপেক্ষা। বড় পরিবর্তনের আগে TTL কমিয়ে রাখা (যেমন ৩০০ সেকেন্ড) একটি পেশাদার অভ্যাস।

> **REAL WORLD:** একটি কোম্পানি সার্ভার বদলে নতুন IP-তে A রেকর্ড দিল, কিন্তু অর্ধেক ব্যবহারকারী পুরোনো সার্ভারেই যাচ্ছিলেন। কারণ: পুরোনো রেকর্ডের TTL ছিল ৮৬৪০০ সেকেন্ড (এক দিন)। কারও দোষ নয় — TTL-ই নিয়ম। শিক্ষা: মাইগ্রেশনের এক দিন আগে TTL কমান; মাইগ্রেশনের পর আবার বাড়ান।

১৩.৫ DoH, DoT ও নিরাপত্তা

সাধারণ DNS প্রশ্ন plaintext-এ যায় — নেটওয়ার্কে যে কেউ দেখতে পারে আপনি কোন নাম জিজ্ঞেস করছেন, এমনকি বদলেও দিতে পারে। **DNS over HTTPS (DoH)** প্রশ্নটি HTTPS-এর ভেতরে পাঠায়, **DNS over TLS (DoT)** TLS-এর ভেতরে — দুটোই গোপনীয়তা বাড়ায়। তবে মনে রাখবেন: এগুলো প্রশ্নের পথটি সুরক্ষিত করে, উত্তরটি সত্য কি না তা যাচাই করে না। উত্তরের সত্যতা যাচাইয়ের ব্যবস্থা **DNSSEC** — তা আলাদা, এবং এখনো সর্বত্র চালু নয়।

১৩.৬ পরিভাষা — Terminology

Term	বাংলা ব্যাখ্যা
Resolver	আপনার হয়ে DNS প্রশ্নের পুরো যাত্রা করা সার্ভার।
Authoritative server	একটি zone-এর আসল, চূড়ান্ত উত্তরের মালিক।
Root / TLD server	শ্রেণিবিন্যাসের ওপরের স্তরের নির্দেশক সার্ভার।
Recursive / Iterative	পুরো যাত্রা করে উত্তর দেওয়া / শুধু পরের ঠিকানা জানানো।
TTL	উত্তরটি cache-এ কতক্ষণ রাখা যাবে (সেকেন্ডে)।
Zone	একটি ডোমেইনের প্রশাসনিক অংশ।
Propagation	আসলে cache-এর মেয়াদ শেষ হওয়ার অপেক্ষা; কোনো জাদু নয়।
DoH / DoT	HTTPS / TLS-এর ভেতরে DNS প্রশ্ন পাঠানোর পদ্ধতি।

১৩.৭ বাস্তব উদাহরণ: একটি ডোমেইন কেনা থেকে সাইট চালু

রাফি `rafi-portfolio.com` ডোমেইন কিনলেন, VPS-এর IP `203.0.113.25`। তিনি registrar-এ A রেকর্ড বসালেন: `@ → 203.0.113.25`, আর `www → @`-এর CNAME। কিন্তু সাইট খুলল না — কারণ তিনি NS রেকর্ড বদলে ফেলেছিলেন ভুল nameserver-এ। তদন্তে `dig NS rafi-portfolio.com` দেখাল registrar-এর nameserver-ই উত্তর দিচ্ছে না। NS ঠিক করতেই, TTL-এর মেয়াদ শেষে, সাইট চালু হলো। শিক্ষা: DNS সমস্যায় প্রথমে NS ও A — দুটোই আলাদা করে যাচাই করুন।

১৩.৮ ব্যবহারিক কমান্ড

```
$ dig example.com
# A রেকর্ড জিজ্ঞেস করে। ANSWER SECTION-এ IP, QUERY TIME ও
# SERVER (কোন resolver উত্তর দিয়েছে) দেখুন।

$ dig example.com MX +short
# শুধু MX রেকর্ডের সংক্ষিপ্ত উত্তর।

$ dig @1.1.1.1 example.com
# নির্দিষ্ট resolver (1.1.1.1) ব্যবহার করে জিজ্ঞেস করা —
# নিজের resolver সন্দেহ হলে তুলনা করার উপায়।

$ nslookup example.com
# dig-এর সহজ বিকল্প; তবে আউটপুট কম বিস্তারিত।

$ getent hosts example.com
# OS-এর resolver (cache সহ) কী উত্তর দেয়, তা দেখায়।
```

`dig`-এর আউটপুটে `status: NXDOMAIN` মানে নামটির অস্তিত্ব নেই; `SERVFAIL` মানে resolver-ই উত্তর পায়নি; `NOERROR` কিন্তু খালি ANSWER মানে নাম আছে, ওই ধরনের রেকর্ড নেই।

১৩.৯ Hands-on Exercise — DNS যাত্রা অনুসরণ

> **LAB:** (১) `dig example.com` চালিয়ে ANSWER SECTION-এর IP ও TTL লিখুন। (২) `dig example.com +trace` চালিয়ে root → TLD → authoritative ধাপগুলো দেখুন। (৩) `dig @1.1.1.1 example.com` ও `dig @8.8.8.8 example.com`-এর উত্তর মিলিয়ে দেখুন। (৪) `dig example.com MX +short` দিয়ে মেইল সার্ভার বের করুন। যাচাই: `+trace`-এর শেষ ধাপে authoritative server-এর নাম দেখা যাবে।

উদাহরণ আউটপুট (illustrative):

```
$ dig example.com +short
203.0.113.10
# মাত্র IP — সহজ উত্তরের জন্য +short ব্যবহার করুন।
```

১৩.১০ Common Mistakes ও Troubleshooting

- **ভুল:** “DNS propagate হতে ৪৮ ঘণ্টা লাগে।” **সংশোধন:** আসলে TTL-এর মেয়াদ; TTL ৩০০ হলে পাঁচ মিনিটেই অধিকাংশ cache বদলায়।

- **ভুল:** CNAME-কে IP ভাবা। **সংশোধন:** CNAME নামের দিকে পাঠায়; IP থাকে চূড়ান্ত নামের A রেকর্ডে।
- **ভুল:** ব্রাউজার cache না সাফ করে DNS পরীক্ষা করা। **সংশোধন:** ব্রাউজার ও OS cache আলাদা; `dig` সরাসরি resolver-কে জিজ্ঞেস করে, ব্রাউজারকে নয়।
- **লক্ষণ:** IP-তে সাইট চলে, নামে চলে না — DNS সমস্যা। `dig` দিয়ে শুরু করুন।
- **লক্ষণ:** কখনো চলে, কখনো চলে না — একাধিক nameserver-এর একটিতে ভুল রেকর্ড থাকতে পারে; প্রতিটি NS আলাদা করে যাচাই করুন।

> **TROUBLESHOOTING:** `dig` উত্তর দিচ্ছে কিন্তু ব্রাউজার খুলছে না — সমস্যা DNS-এ নয়, HTTP/TLS-এ (অধ্যায় ১৫–১৬)। `dig` -ই উত্তর দিচ্ছে না — resolver বা নেটওয়ার্কে সমস্যা; অন্য resolver (`@1.1.1.1`) দিয়ে পরীক্ষা করুন।

১৩.১১ Security Considerations

DNS hijacking — ভুল উত্তরের মাধ্যমে ব্যবহারকারীকে নকল সাইটে পাঠানো — বাস্তব আক্রমণ। Public Wi-Fi-তে DNS উত্তর বদলে দেওয়া সম্ভব; তাই গুরুত্বপূর্ণ সাইটে HTTPS (অধ্যায় ১৬) আবশ্যক, কারণ সার্টিফিকেট যাচাই নকল সাইট ধরে ফেলে। নিজের ডোমেইনের registrar অ্যাকাউন্টে two-factor authentication চালু রাখুন — ডোমেইন চুরি হলে মেইলসহ সবকিছু চুরি যায়।

১৩.১২ Best Practices

- পরিবর্তনের আগে TTL কমান; পরে আবার বাড়ান।
- অন্তত দুটি nameserver রাখুন, ভিন্ন নেটওয়ার্কে।
- SPF, DKIM ও DMARC TXT রেকর্ড দিয়ে মেইল সুরক্ষিত করুন।
- DNS পরিবর্তনের পর `dig +trace` ও একাধিক resolver দিয়ে যাচাই করুন।

Knowledge Check — যাচাই করুন

> **CHECK YOUR UNDERSTANDING:** উত্তর Answer Guide-এ আছে — আগে নিজে চেষ্টা করুন।

1. Recursive resolver-এর কাজ কী? A) শুধু পরের server-এর ঠিকানা জানানো B) ব্যবহারকারীর হয়ে root থেকে authoritative পর্যন্ত পুরো যাত্রা করে উত্তর দেওয়া C) IP ঠিকানা বরাদ্দ করা D) ওয়েবসাইট host করা
2. MX রেকর্ডের কাজ কী? A) IPv4 ঠিকানা দেওয়া B) ডোমেইনের মেইল কোন সার্ভার নেবে, তা জানানো C) নামের উপনাম দেওয়া D) সার্টিফিকেট দেওয়া
3. DNS রেকর্ড বদলানোর পর পুরোনো উত্তর কিছুক্ষণ দেখা যাওয়ার কারণ কী? A) ইন্টারনেট ধীর B) TTL শেষ না হওয়া পর্যন্ত resolver পুরোনো উত্তর cache-এ রাখে C) সার্ভার বন্ধ থাকে D) ব্রাউজার ভুল
4. CNAME রেকর্ড কী করে? A) সরাসরি IP ঠিকানা দেয় B) এক নামকে আরেক নামের দিকে নির্দেশ করে C) মেইল সার্ভার জানায় D) IPv6 ঠিকানা দেয়
5. কোন কমান্ডে DNS resolution-এর পুরো ধাপ (root → TLD → authoritative) দেখা যায়? A) ping B) dig +trace C) curl D) traceroute

Practical Assignment

একটি পরিচিত ডোমেইনের (যেমন `example.com`) জন্য `dig` দিয়ে A, AAAA, MX ও NS রেকর্ড বের করে একটি টেবিলে লিখুন। তারপর `dig +trace`-এর ধাপগুলো নিজের ভাষায় ব্যাখ্যা করুন: প্রতিটি ধাপে কে কাকে কী জিজ্ঞেস করল, আর কে কী উত্তর দিল।

১৩.৭ক CNAME Chain, Wildcard ও Apex-এর সীমা

CNAME একাধিক ধাপে চলতে পারে: `www.example.com` → `cdn.example.net` → `cdn.example.net`-এর A রেকর্ড। একে chain বলে; প্রতিটি ধাপে একটি করে প্রশ্ন বাড়ে, তাই chain লম্বা করবেন না। **Wildcard** রেকর্ড (`*.example.com`) যেকোনো subdomain-কে একই উত্তরে পাঠায় — পরীক্ষামূলক subdomain-এর জন্য সুবিধাজনক, কিন্তু ভুল টাইপ করা নামও সাইটে পৌঁছে যায় বলে সাবধানে ব্যবহার করুন। একটি গুরুত্বপূর্ণ সীমা: ডোমেইনের apex-এ (`example.com` নিজে, `www` ছাড়া) CNAME বসানো যায় না — কারণ apex-এ NS ও SOA রেকর্ডও থাকে, আর DNS নিয়মে একটি নামে CNAME থাকলে আর কোনো রেকর্ড থাকতে পারে না। তাই apex-কে IP-তে পাঠাতে A/AAAA রেকর্ড ব্যবহার করুন, অথবা registrar-এর ALIAS/ANAME সুবিধা থাকলে তা ব্যবহার করুন।

Chapter Summary

DNS নামকে IP-তে বদলায় একটি শ্রেণিবদ্ধ, বিকেন্দ্রীকৃত ব্যবস্থায়: recursive resolver root, TLD ও authoritative server ধাপে ধাপে জিজ্ঞেস করে। প্রতিটি রেকর্ডের আলাদা কাজ — A ঠিকানা দেয়, MX মেইল জানায়, CNAME নামের দিকে পাঠায়। TTL cache-এর মেয়াদ ঠিক করে; “propagation” আসলে সেই মেয়াদের অপেক্ষা।

Revision Checklist

- ☐ Root → TLD → authoritative যাত্রা ব্যাখ্যা করতে পারি
- ☐ Recursive ও iterative-এর পার্থক্য বলতে পারি
- ☐ A, MX, CNAME, NS, TXT রেকর্ডের কাজ জানি
- ☐ TTL ও propagation-এর সম্পর্ক ব্যাখ্যা করতে পারি
- ☐ `dig +trace` দিয়ে DNS সমস্যা নির্ণয় করতে পারি

অধ্যায় ১৪ • Part III — Internet Protocols

Chapter 14: DHCP, ARP, ICMP and Essential Protocols — DHCP, ARP, ICMP ও অপরিহার্য প্রোটোকল

শুরুতে: একটি বাস্তব দৃশ্য

নতুন ল্যাপটপ Wi-Fi-তে যুক্ত হতেই ইন্টারনেট চলতে শুরু করল — কেউ IP ঠিকানা টাইপ করেনি, gateway লেখেনি, DNS বসায়নি। কীভাবে? ল্যাপটপটি চিৎকার করে জিজ্ঞেস করেছিল, “আমাকে একটি ঠিকানা দাও!” — DHCP সার্ভার উত্তর দিয়েছিল ঠিকানা, gateway ও DNS-সহ। তারপর ল্যাপটপ gateway-র MAC address জানতে ARP করল, আর সংযোগ যাচাই করতে ping (ICMP) চালাল। এই অধ্যায়ের তিন নায়ক — DHCP, ARP, ICMP — নেটওয়ার্কের দৈনন্দিন কাজগুলো চুপচাপ সামলায়। এদের না বুঝলে “Wi-Fi যুক্ত, কিন্তু ইন্টারনেট নেই” সমস্যার মূলে পৌঁছানো যায় না।

Learning Objectives — শেখার উদ্দেশ্য

- DHCP DORA প্রক্রিয়া ধাপে ধাপে ব্যাখ্যা করতে পারবেন।
- DHCP lease কী এবং কেন মেয়াদ থাকে, তা বলতে পারবেন।
- ARP কীভাবে IP ঠিকানাকে MAC address-এ বদলায়, তা ব্যাখ্যা করতে পারবেন।
- ICMP ও ping কীভাবে কাজ করে, তা বলতে পারবেন।
- Traceroute ও TTL-এর সম্পর্ক ব্যাখ্যা করতে পারবেন।
- NTP ও SSH-এর নেটওয়ার্কিং ভূমিকা ব্যাখ্যা করতে পারবেন।
- সাধারণ সার্ভিসের port নম্বর চিনতে ও সমস্যা নির্ণয় করতে পারবেন।

Prerequisites — পূর্বজ্ঞান

IPv4 address ও subnet (অধ্যায় ৭–৮), MAC address ও Ethernet (অধ্যায় ৬), UDP বনাম TCP (অধ্যায় ১০), এবং DNS-এর প্রাথমিক ধারণা (অধ্যায় ১৩)।

১৪.১ DHCP: ঠিকানা বরাদ্দের স্বয়ংক্রিয় ব্যবস্থা

প্রতিটি যন্ত্রকে হাতে IP ঠিকানা দিলে ভুল হবেই — একই ঠিকানা দুজনে পেয়ে যাবে, gateway ভুল লেখা হবে। **DHCP (Dynamic Host Configuration Protocol)** এই কাজটি স্বয়ংক্রিয় করে: নতুন যন্ত্র নেটওয়ার্কে এলে DHCP সার্ভার তাকে IP ঠিকানা, subnet mask, default gateway ও DNS server-এর ঠিকানা দেয়। বাড়ির router, অফিসের সার্ভার — সবাই DHCP সার্ভার চালায়। সার্ভারগুলোতে সাধারণত static ঠিকানা ব্যবহার করা হয়, যাতে ঠিকানা কখনো না বদলায়; ল্যাপটপ ও ফোনে dynamic ঠিকানা চলে।

১৪.২ DORA: চার ধাপের কথোপকথন

DHCP-র যাত্রা মনে রাখার উপায় **DORA**: **Discover** — ক্লায়েন্ট broadcast করে জিজ্ঞেস করে, “DHCP সার্ভার আছ?” (সে নিজের ঠিকানাও জানে না, তাই broadcast)। **Offer** — সার্ভার উত্তর দেয়, “এই ঠিকানাটি নিতে পারো।” **Request** — ক্লায়েন্ট বলে, “ঠিক আছে, এটাই নিলাম।” **Acknowledge** — সার্ভার নিশ্চিত করে, “ঠিকানাটি তোমার, এই

মেয়াদের জন্য।” এই মেয়াদই **lease** — যেমন ২৪ ঘণ্টা। মেয়াদ শেষ হওয়ার আগেই ক্লায়েন্ট নবায়নের (renew) অনুরোধ করে; সার্ভার না থাকলে ঠিকানাটি ছেড়ে দিতে হয়।

চিত্র ১৪.১ — DHCP DORA
ক্লায়েন্ট → (broadcast) DISCOVER: “DHCP সার্ভার আছ?” → সার্ভার → OFFER: “192.168.1.50 নাও” → ক্লায়েন্ট → REQUEST: “এটাই নিলাম” → সার্ভার → ACKNOWLEDGE: “নিশ্চিত, lease ২৪ ঘণ্টা”। সঙ্গে আসে: subnet mask, gateway, DNS server।

চিত্র ১৪.২ — ARP: IP থেকে MAC
“192.168.1.1-এর MAC কে?” (broadcast প্রশ্ন) → শুধু 192.168.1.1 উত্তর দেয়: “আমার MAC aa:bb:cc:dd:ee:ff” → প্রশ্নকারী উত্তরটি ARP table-এ রেখে দেয় → পরের বার আর জিজ্ঞেস করতে হয় না

১৪.৩ ARP: শেষ ধাপের অনুবাদ

IP ঠিকানা দিয়ে রাউটিং হয়, কিন্তু Ethernet frame পাঠাতে দরকার MAC address। **ARP (Address Resolution Protocol)** সেই অনুবাদ করে: “`192.168.1.1`-এর MAC address কী?” — broadcast প্রশ্ন, শুধু ওই ঠিকানার মালিক উত্তর দেয়। উত্তরটি **ARP table**-এ (বা neighbour table-এ) কিছুক্ষণ রাখা হয়, যাতে প্রতিবার জিজ্ঞেস করতে না হয়। লক্ষ করুন: ARP শুধু একই local নেটওয়ার্কে চলে; দূরের ঠিকানার MAC জানার দরকার নেই — দূরের প্যাকেট gateway-র MAC-এ যায় (অধ্যায় ১২)। ARP-র কোনো যাচাই নেই — যে কেউ ভুল উত্তর দিতে পারে; এটাই ARP spoofing আক্রমণের ভিত্তি।

১৪.৪ ICMP, Ping ও Traceroute

ICMP (Internet Control Message Protocol) তথ্য বহন করে না; এটি নেটওয়ার্কের “বার্তা” পাঠায় — “গন্তব্যে পৌঁছানো যাচ্ছে না”, “সময় শেষ হয়ে গেছে”। `ping` ICMP Echo Request পাঠায়, গন্তব্য Echo Reply দেয়; সময় মেপে বোঝা যায় সংযোগ আছে কি না, আর কত দেরি। `traceroute` একটি চতুর কৌশল ব্যবহার করে: প্রথমে TTL=1 দিয়ে প্যাকেট পাঠায় — প্রথম router TTL শূন্য করে “Time Exceeded” ফেরত পাঠায়, ফলে প্রথম hop জানা যায়। তারপর TTL=2, 3... বাড়িয়ে প্রতিটি hop বের করা হয়। তাই traceroute-এর প্রতিটি লাইন একটি router-এর উত্তর।

> **CONCEPT:** ping সফল মানে শুধু ICMP পৌঁছেছে — ওয়েব সার্ভার চলছে কি না, তা ping বলে না। উল্টোদিকে ping ব্যর্থ মানেই সার্ভার বন্ধ নয়; অনেক firewall ICMP আটকায়। ping হলো প্রথম পরীক্ষা, শেষ কথা নয়।

১৪.৫ NTP: সবার ঘড়ি এক রাখা

Log-এর সময়, সার্টিফিকেটের মেয়াদ, Kerberos authentication — সবকিছু সঠিক সময়ের ওপর নির্ভর করে। **NTP (Network Time Protocol)** কম্পিউটারের ঘড়িকে নির্ভরযোগ্য সময়-সার্ভারের সঙ্গে মেলায়। Ubuntu-তে `systemd-timesyncd` এই কাজটি করে। সার্ভারের ঘড়ি কয়েক মিনিট এগিয়ে-পিছিয়ে থাকলে TLS handshake ব্যর্থ হতে পারে — একটি চেনা কিন্তু অপ্রত্যাশিত সমস্যা।

১৪.৬ SSH Networking ও সাধারণ পোর্ট

SSH (port 22, TCP) শুধু remote login নয় — এটি encrypted channel, যার ভেতর দিয়ে file transfer (SCP/SFTP) ও port forwarding (অধ্যায় ১৯) চলে। মনে রাখার মতো port: DNS 53, HTTP 80, HTTPS 443, SSH 22, SMTP 25/587, NTP 123, DHCP 67/68। সমস্যা নির্ণয়ে প্রথম প্রশ্ন: "কোন port-এ কথা হচ্ছে, আর সেটি খোলা কি না?" — উত্তর দেয় `ss -tlnp` (অধ্যায় ২১)।

১৪.৭ পরিভাষা — Terminology

Term	বাংলা ব্যাখ্যা
DHCP	IP ঠিকানা, gateway ও DNS স্বয়ংক্রিয়ভাবে বরাদ্দের প্রোটোকল।
DORA	Discover–Offer–Request–Acknowledge: DHCP-র চার ধাপ।
Lease	DHCP ঠিকানা ব্যবহারের মেয়াদ; শেষে নবায়ন করতে হয়।
ARP	IP ঠিকানাকে MAC address-এ বদলানোর প্রোটোকল।
ARP table	IP–MAC জোড়ার সাময়িক তালিকা।
ICMP	নেটওয়ার্কের নিয়ন্ত্রণ ও ত্রুটি-বার্তার প্রোটোকল।
TTL (IP header)	প্যাকেট কত hop যেতে পারবে; প্রতি router-এ ১ কমে।
NTP	কম্পিউটারের ঘড়ি মেলানোর প্রোটোকল।
Broadcast	একই নেটওয়ার্কের সবাইকে পাঠানো বার্তা।

১৪.৮ বাস্তব উদাহরণ: "Wi-Fi যুক্ত, ইন্টারনেট নেই"

একটি ক্যাফেতে একজনের ল্যাপটপ Wi-Fi-তে যুক্ত, কিন্তু কিছুই খুলছে না। ধাপে ধাপে: (১) `ip addr` দেখাল `169.254.x.x` ঠিকানা — এটি APIPA, অর্থাৎ DHCP উত্তর দেয়নি, ল্যাপটপ নিজেই একটি ঠিকানা বানিয়েছে। (২) কারণ: ক্যাফের DHCP pool শেষ হয়ে গেছে — অনেক যন্ত্র যুক্ত। (৩) সমাধান: ক্যাফের router restart করে pool খালি করা, অথবা lease সময় কমানো। শিক্ষা: `169.254.0.0/16` দেখলেই বুঝবেন DHCP ব্যর্থ হয়েছে।

১৪.৯ ব্যবহারিক কমান্ড

```
$ ip addr show
# interface-গুলোর IP ঠিকানা দেখায়। 169.254.x.x দেখলে DHCP ব্যর্থ।

$ ip neighbour
# ARP / neighbour table: কোন IP-র MAC কী, তা দেখায়।

$ ping -c 4 8.8.8.8
# ICMP Echo: সংযোগ ও round-trip সময় যাচাই। -c 4 মানে ৪টি প্যাকেট।

$ ping -c 4 example.com
# নামে ping: DNS ও সংযোগ — দুটোই একসঙ্গে পরীক্ষা হয়।
```

```
$ traceroute example.com
# কোন কোন hop পেরিয়ে গন্তব্যে যাওয়া যায়, তা দেখায়।

$ timedatectl
# সিস্টেমের সময় ও NTP sync অবস্থা দেখায়।
```

১৪.১০ Hands-on Exercise — DORA ও ARP নিজের চোখে দেখা

LAB: (১) `ip addr` দিয়ে নিজের IP, আর `ip route` দিয়ে gateway ও DNS লিখুন। (২) `ping -c 3 <gateway>` দিয়ে gateway-তে পৌঁছান, তারপর `ip neighbour` দিয়ে gateway-র MAC দেখুন। (৩) `traceroute 1.1.1.1` চালিয়ে প্রথম তিনটি hop লিখুন। (৪) `timedatectl` দিয়ে NTP sync যাচাই করুন। যাচাই: gateway-র MAC ARP table-এ থাকতে হবে; traceroute-এর প্রথম hop gateway-ই হবে।

উদাহরণ আউটপুট (illustrative):

```
$ ip neighbour
192.168.1.1 dev eth0 lladdr aa:bb:cc:dd:ee:ff REACHABLE
# অর্থ: gateway 192.168.1.1-এর MAC aa:bb:cc:dd:ee:ff, সংযোগ সচল।
```

১৪.১১ Common Mistakes ও Troubleshooting

- **ভুল:** ping ব্যর্থ হলেই “সার্ভার বন্ধ” ভাবা। **সংশোধন:** অনেক সার্ভার ICMP বন্ধ রাখে; port-এ সংযোগ পরীক্ষা করুন (`curl`, `ss`)।
- **ভুল:** DHCP-র দেওয়া ঠিকানাকে স্থায়ী ভাবা। **সংশোধন:** lease শেষে ঠিকানা বদলাতে পারে; সার্ভারে static ঠিকানা বা DHCP reservation ব্যবহার করুন।
- **ভুল:** এক নেটওয়ার্কে দুটি DHCP সার্ভার চালানো। **সংশোধন:** দুটি সার্ভার পরস্পরবিরোধী ঠিকানা দেবে — একটি রাখুন।
- **লক্ষণ:** `169.254.x.x` ঠিকানা — DHCP ব্যর্থ।
- **লক্ষণ:** নামে ping ব্যর্থ, IP-তে সফল — DNS সমস্যা (অধ্যায় ১৩)।
- **লক্ষণ:** ঘড়ি ভুল — TLS ও log-এ সমস্যা; NTP যাচাই করুন।

TROUBLESHOOTING: সংযোগ সমস্যায় ক্রমটি মনে রাখুন: IP আছে কি (`ip addr`) → gateway-তে ping যায় কি → বাইরের IP-তে ping যায় কি → নামে ping যায় কি। যে ধাপে ভাঙে, সমস্যা সেই স্তরে।

১৪.১২ Security Considerations

DHCP-র কোনো authentication নেই — যে কেউ ভুয়া DHCP সার্ভার বসিয়ে ভুল gateway ও DNS দিতে পারে, আর সব traffic নিজের দিকে টানতে পারে (rogue DHCP)। Public নেটওয়ার্কে এটি বাস্তব ঝুঁকি; তাই অপরিচিত Wi-Fi-তে VPN ব্যবহার করুন। ARP-ও যাচাই করে না — ARP spoofing-এর মাধ্যমে একই নেটওয়ার্কের traffic আড়ি পাতা যায়। সুরক্ষা: গুরুত্বপূর্ণ নেটওয়ার্কে DHCP snooping ও Dynamic ARP Inspection (managed switch-এর বৈশিষ্ট্য) ব্যবহার করা হয়।

১৪.১৩ Best Practices

- সার্ভারে static IP বা DHCP reservation ব্যবহার করুন; ঠিকানা যেন না বদলায়।
- DHCP scope-এ gateway ও DNS ঠিকানা সঠিক রাখুন; scope শেষ হওয়ার আগেই সতর্ক হোন।
- গুরুত্বপূর্ণ সার্ভারের সময় NTP দিয়ে sync রাখুন।
- ping-কে একমাত্র পরীক্ষা ভাববেন না — port ও সার্ভিস আলাদা করে যাচাই করুন।

Knowledge Check — যাচাই করুন

> **CHECK YOUR UNDERSTANDING:** উত্তর Answer Guide-এ আছে — আগে নিজে চেষ্টা করুন।

1. DHCP DORA-তে ক্লায়েন্ট প্রথমে কী পাঠায়? A) Request B) Discover (broadcast) C) Acknowledge D) Offer
2. ARP-এর কাজ কী? A) ডোমেইন নামকে IP-তে বদলানো B) IP ঠিকানাকে MAC address-এ বদলানো C) IP ঠিকানা বরাদ্দ করা D) প্যাকেট encrypt করা
3. একটি যন্ত্রের IP ঠিকানা 169.254.10.5 দেখা গেলে এর অর্থ কী? A) DHCP সফল হয়েছে B) DHCP ব্যর্থ হয়েছে, যন্ত্রটি নিজেই APIPA ঠিকানা নিয়েছে C) সার্ভারটি public IP পেয়েছে D) DNS ঠিক আছে
4. traceroute কীভাবে প্রতিটি hop বের করে? A) প্রতিবার TTL বাড়িয়ে Time Exceeded বার্তা সংগ্রহ করে B) DNS জিজ্ঞেস করে C) MAC address পড়ে D) প্যাকেটের আকার বাড়িয়ে
5. ping ব্যর্থ হলেও ওয়েবসাইট চলতে পারে কেন? A) ping ভুল কমান্ড B) অনেক firewall ICMP আটকায়, কিন্তু HTTP port খোলা থাকে C) DNS বন্ধ থাকে D) ping শুধু local নেটওয়ার্কে চলে

Practical Assignment

নিজের VM-এ `ip addr`, `ip route`, `ip neighbour` ও `timedatectl`-এর আউটপুট সংগ্রহ করে একটি “নেটওয়ার্ক পরিচয়পত্র” তৈরি করুন: আমার IP, subnet, gateway, gateway-র MAC, DNS server ও NTP অবস্থা। তারপর একটি কাল্পনিক সমস্যা লিখুন: “হঠাৎ IP 169.254.x.x হয়ে গেল” — কারণ ও সমাধানের ধাপ লিখুন।

১৪.৬ক DHCP Relay ও Lease নবায়নের সময়সূচী

বড় নেটওয়ার্কে প্রতিটি subnet-এ আলাদা DHCP সার্ভার বসানো হয় না; router-এর **DHCP relay agent** ক্লায়েন্টের broadcast অনুরোধ ধরে দূরের DHCP সার্ভারে পাঠিয়ে দেয়। Lease নবায়নের সময়সূচীও নির্দিষ্ট: lease-এর অর্ধেক সময়ে (T1, ৫০%) ক্লায়েন্ট প্রথমে আসল সার্ভারকে নবায়নের অনুরোধ করে; সাড়া না পেলে ৮৭.৫% সময়ে (T2) যেকোনো DHCP সার্ভারকে জিজ্ঞেস করে; মেয়াদ শেষ হলে ঠিকানাটি ছেড়ে দিতে হয়। তাই একটি সার্ভার কয়েক ঘণ্টা বন্ধ থাকলেও অধিকাংশ ক্লায়েন্টের ঠিকানা বদলায় না — তারা আগেই নবায়ন করে নিয়েছে। এই সময়সূচী জানলে “হঠাৎ সবার IP বদলে গেল” সমস্যার কারণ বোঝা সহজ হয়: সম্ভবত relay বন্ধ ছিল বা scope শেষ হয়ে গিয়েছিল।

১৪.৬খ Static বনাম Dynamic: কখন কোনটি?

সব যন্ত্রে DHCP ব্যবহার করলে কি চলে? না — সার্ভার, router, printer ও network যন্ত্রে static ঠিকানা (বা DHCP reservation) ব্যবহার করা হয়, কারণ এদের ঠিকানা বদলালে অন্যরা আর খুঁজে পাবে না: DNS রেকর্ড ভুল ঠিকানায় পাঠাবে, firewall নিয়ম ভেঙে যাবে, মনিটরিং সতর্কবার্তা দেবে। ল্যাপটপ, ফোন ও অতিথি যন্ত্রে dynamic ঠিকানা চলে —

এদের ঠিকানা কেউ মনে রাখে না। মাঝামাঝি পথ হলো **DHCP reservation**: MAC address দেখে DHCP সার্ভার সবসময় একই ঠিকানা দেয় — কেন্দ্রীয় ব্যবস্থাপনার সুবিধা, static-এর স্থিরতা। একটি ছোট অফিসের নিয়ম হতে পারে: সার্ভার ও নেটওয়ার্ক যন্ত্রের জন্য `192.168.1.1–192.168.1.49` সংরক্ষিত, DHCP pool `192.168.1.100–192.168.1.200` — এভাবে ঠিকানার সংঘর্ষ (conflict) এড়ানো যায়।

Chapter Summary

DHCP DORA প্রক্রিয়ায় ঠিকানা, gateway ও DNS দেয়; lease-এর মেয়াদ থাকে। ARP IP-কে MAC-এ বদলায়, শুধু local নেটওয়ার্কে। ICMP দিয়ে ping ও traceroute কাজ করে — traceroute TTL বাড়িয়ে প্রতিটি hop বের করে। NTP সময় মেলায়; SSH port 22-এ নিরাপদ remote কাজ দেয়। সমস্যা নির্ণয়ের ক্রম: IP → gateway → বাইরের IP → নাম।

Revision Checklist

- ☐ DORA-র চার ধাপ ক্রমসহ বলতে পারি
- ☐ ARP table পড়তে পারি
- ☐ 169.254.x.x দেখলে কারণ বলতে পারি
- ☐ traceroute-এর TTL কৌশল ব্যাখ্যা করতে পারি
- ☐ ping-এর সীমা জানি

অধ্যায় ১৫ • Part III — Internet Protocols

Chapter 15: HTTP and HTTPS — How the Web Works — HTTP ও HTTPS: ওয়েব কীভাবে কাজ করে

শুরুতে: একটি বাস্তব দৃশ্য

আপনি ব্রাউজারে একটি নিউজ সাইট খুললেন। পর্দার আড়ালে কী ঘটল? ব্রাউজার সার্ভারকে একটি লেখা বার্তা পাঠাল — “আমাকে হোমপেজ দাও” — সার্ভার উত্তর দিল আরেকটি লেখা বার্তায়: “নাও, সঙ্গে এই ছবিগুলোও নাও।” এই বার্তা বিনিময়ের ভাষার নাম **HTTP**। আপনার লগইন, কেনাকাটা, API কল — সবকিছুই এই ভাষায় লেখা। DevOps engineer হিসেবে আপনি প্রতিদিন HTTP দেখবেন: log-এ, curl-এ, ব্রাউজারের developer tools-এ, আর সার্ভার বন্ধ হলে প্রথমে HTTP status code-ই বলবে কী হয়েছে।

Learning Objectives — শেখার উদ্দেশ্য

- HTTP request–response মডেল ব্যাখ্যা করতে পারবেন।
- GET, POST, PUT, PATCH, DELETE method-এর কাজ ও পার্থক্য বলতে পারবেন।
- Request ও response header পড়তে ও ব্যাখ্যা করতে পারবেন।
- HTTP status code-এর পাঁচটি শ্রেণি চিনতে ও সাধারণ code-এর অর্থ বলতে পারবেন।
- Cookie ও session কীভাবে কাজ করে, তা ব্যাখ্যা করতে পারবেন।
- Keep-alive ও HTTP/2, HTTP/3-এর ধারণা দিতে পারবেন।
- `curl` ও browser developer tools দিয়ে request পরীক্ষা করতে পারবেন।
- ব্রাউজার থেকে সার্ভার পর্যন্ত সম্পূর্ণ request lifecycle বর্ণনা করতে পারবেন।

Prerequisites — পূর্বজ্ঞান

DNS resolution (অধ্যায় ১৩), TCP connection (অধ্যায় ১০–১১), TLS-এর প্রাথমিক ধারণা (অধ্যায় ১৬-এর সঙ্গে পড়ুন)। একটি ওয়েবসাইট খুলে developer tools (F12 → Network) দেখার অভিজ্ঞতা থাকলে ভালো।

১৫.১ প্রথম নীতি: ওয়েব হলো বার্তা বিনিময়

HTTP একটি **application-layer protocol** — মানুষের পড়ার মতো লেখা বার্তায় client সার্ভারকে কিছু চায়, সার্ভার উত্তর দেয়। এটি **stateless**: প্রতিটি request আলাদা; সার্ভার আগের request মনে রাখে না। “লগইন থাকা” তাহলে কীভাবে হয়? Cookie ও session-এর মাধ্যমে — ক্লায়েন্ট প্রতিবার একটি চিহ্ন (session ID) পাঠায়, সার্ভার তা দেখে চিনে নেয়। Stateless হওয়ার সুবিধা: যেকোনো সার্ভার যেকোনো request সামলাতে পারে — load balancing-এর (অধ্যায় ১৭) ভিত্তি এটাই।

একটি request-এর গঠন: প্রথম লাইনে method, path ও version (`GET /index.html HTTP/1.1`), তারপর header (Host, User-Agent, Accept...), একটি ফাঁকা লাইন, তারপর প্রয়োজনে body। Response-এর গঠন একই রকম: প্রথম লাইনে status (`HTTP/1.1 200 OK`), header, ফাঁকা লাইন, body (HTML, JSON...)।

> **CONCEPT:** HTTP নিজে জানে না আপনি কে, কোথা থেকে এসেছেন, বা আগে কী করেছেন। প্রতিটি request সম্পূর্ণ — Host header ছাড়া সার্ভার বুঝবেই না আপনি কোন সাইট চাইছেন, কারণ একই সার্ভারে অনেক সাইট থাকতে পারে (virtual hosting)।

১৫.২ HTTP Method: কী করতে চান?

Method	কাজ	উদাহরণ
GET	তথ্য আনা; body থাকে না; নিরাপদ ও idempotent	পাতা খোলা, ছবি আনা
POST	নতুন তথ্য তৈরি করা	ফর্ম জমা, নতুন পোস্ট
PUT	পুরো resource বদলে দেওয়া	প্রোফাইল সম্পূর্ণ আপডেট
PATCH	resource-এর অংশবিশেষ বদলানো	শুধু নাম বদলানো
DELETE	resource মোছা	পোস্ট মুছে ফেলা
HEAD / OPTIONS	শুধু header / কী করা যায়, তা জানা	স্বাস্থ্য পরীক্ষা, CORS

Idempotent মানে একই request বারবার করলেও ফল একই: GET দুবার করলে দুবার একই পাতা আসে; POST দুবার করলে দুটি পোস্ট তৈরি হতে পারে। তাই ব্রাউজার refresh-এ POST আবার পাঠাতে সতর্ক করে।

১৫.৩ Status Code: সার্ভারের সংক্ষিপ্ত উত্তর

শ্রেণি	অর্থ	পরিচিত উদাহরণ
2xx	সফল	200 OK, 201 Created, 204 No Content
3xx	অন্য কোথাও যান (redirect)	301 Moved Permanently, 302 Found, 304 Not Modified
4xx	ক্লায়েন্টের ভুল	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found
5xx	সার্ভারের ভুল	500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable

DevOps-এ এই পার্থক্য জীবন-মরণ: 4xx মানে request-এ সমস্যা — সার্ভার ঠিক আছে; 5xx মানে সার্ভার বা তার পেছনের সেবায় সমস্যা — আপনাকেই ঠিক করতে হবে। 502 মানে reverse proxy (অধ্যায় ১৭) পেছনের সার্ভারের সঙ্গে কথা বলতে পারেনি; 503 মানে সার্ভার ইচ্ছে করে সেবা বন্ধ রেখেছে বা overload-এ আছে।

১৫.৪ Header, Cookie ও Session

Header হলো request/response-এর সঙ্গে যাওয়া অতিরিক্ত তথ্য: `Host` (কোন সাইট), `User-Agent` (কোন ব্রাউজার), `Accept` (কী ধরনের উত্তর চাই), `Content-Type` (body কী ধরনের — `application/json` না `text/html`), `Authorization` (পরিচয়)। Response-এ `Set-Cookie` দিয়ে সার্ভার ব্রাউজারকে একটি ছোট টুকরো তথ্য রাখতে বলে; ব্রাউজার পরের request-এ `Cookie` header-এ তা ফেরত পাঠায়। Session ID এই cookie-তেই থাকে। নিরাপত্তার জন্য

cookie-তে `HttpOnly` (JavaScript পড়তে পারবে না), `Secure` (শুধু HTTPS-এ যাবে) ও `SameSite` flag ব্যবহার করা হয়।

১৫.৫ Keep-Alive, HTTP/2 ও HTTP/3

HTTP/1.1-এ **keep-alive** মানে একটি TCP connection-এ একাধিক request পাঠানো যায় — প্রতিবার নতুন handshake নয়, ফলে দ্রুত। **HTTP/2** একই connection-এ অনেক request একসঙ্গে (multiplexing) পাঠায় ও header সংকুচিত করে। **HTTP/3** TCP-র বদলে UDP-র ওপর QUIC ব্যবহার করে — connection দ্রুত শুরু হয়, আর নেটওয়ার্ক বদলালেও (Wi-Fi থেকে মোবাইল ডেটা) connection টিকে থাকে। আপনার সার্ভারে এগুলো চালু আছে কি না, তা Nginx কনফিগারেশনেই ঠিক হয় (অধ্যায় ১৭)।

চিত্র ১৫.১ — সম্পূর্ণ Browser-to-Server Lifecycle

URL লেখা → DNS-এ IP বের করা → TCP handshake (port 443) → TLS handshake → HTTP request (GET /) → সার্ভারে routing / application → Response (200 + HTML) → ব্রাউজার CSS/JS-এর জন্য আরও request → পাতা দেখানো

চিত্র ১৫.২ — HTTP Request ও Response-এর গঠন

Request: [Method Path Version] → [Host, User-Agent, Accept...] → [খালি লাইন] → [Body]। Response: [Version Status Code] → [Content-Type, Content-Length, Set-Cookie...] → [খালি লাইন] → [Body: HTML/JSON]।

১৫.৬ REST API: HTTP-কে সেবার ভাষা হিসেবে ব্যবহার

REST API-তে প্রতিটি resource-এর একটি URL থাকে (`/api/users/42`), আর method-ই বলে কী করতে হবে: GET মানে পড়া, POST মানে তৈরি, PUT/PATCH মানে বদলানো, DELETE মানে মোছা। উত্তরে status code ও JSON body থাকে। API debug করার সময় প্রথমে দেখুন: status code কী, Content-Type কী, আর body-তে আসল বার্তা কী লেখা আছে — অধিকাংশ API ভুলের ব্যাখ্যা body-তেই দেয়।

REAL WORLD: একটি মোবাইল অ্যাপ হঠাৎ লগইন করতে পারছিল না। API log-এ দেখা গেল সব request-এ `401 Unauthorized`। কারণ: সার্ভারের ঘড়ি NTP sync হারিয়ে দুই মিনিট এগিয়ে গিয়েছিল, আর token-এর মেয়াদ যাচাইয়ে ব্যর্থ হচ্ছিল। HTTP status code সমস্যার দিক নির্দেশ করেছিল — সমাধান ছিল ঘড়ি ঠিক করা, কোড বদলানো নয়।

১৫.৭ পরিভাষা — Terminology

Term	বাংলা ব্যাখ্যা
Request / Response	ক্লায়েন্টের চাওয়া / সার্ভারের উত্তর।
Method	Request-এর উদ্দেশ্য: GET, POST, PUT, PATCH, DELETE।
Header	Request/response-এর সঙ্গে যাওয়া অতিরিক্ত তথ্য।
Status code	উত্তরের সংক্ষিপ্ত ফলাফল: 2xx, 3xx, 4xx, 5xx।

Stateless	সার্ভার আগের request মনে রাখে না।
Cookie / Session	ব্রাউজারে রাখা চিহ্ন / সার্ভারে রাখা লগইন অবস্থা।
Keep-alive	একটি connection-এ একাধিক request পাঠানো।
REST	HTTP method ও URL দিয়ে resource পরিচালনার ধরন।
Idempotent	একই request বারবার করলেও ফল একই।

১৫.৮ বাস্তব উদাহরণ: 502 Bad Gateway-এর তদন্ত

ব্যবহারকারীরা সাইটে `502 Bad Gateway` দেখছেন। তদন্তের ক্রম: (১) Nginx চলছে কি? `systemctl status nginx` — চলছে। (২) Nginx-এর error log-এ কী লেখা? `connect() failed (111: Connection refused) while connecting to upstream` — পেছনের application সার্ভার শুনছে না। (৩) Application সার্ভার চলছে কি? বন্ধ — deploy-এর সময় crash করেছে। (৪) Application চালু করতেই 502 চলে গেল। শিক্ষা: 502 মানে proxy ঠিক আছে, upstream-ই সমস্যা।

১৫.৯ ব্যবহারিক কমান্ড

```
$ curl -v https://example.com
# -v (verbose) দিয়ে দেখা যায়: DNS, TCP, TLS handshake, পাঠানো
# request header, আর পাওয়া response header — সব ধাপ একসঙ্গে।

$ curl -i https://example.com
# response header + body একসঙ্গে দেখায়।

$ curl -s -o /dev/null -w "%{http_code} %{time_total}s\n" https://example.com
# শুধু status code ও সময় — স্বাস্থ্য পরীক্ষার জন্য উপযোগী।

$ curl -X POST https://example.com/api/login \
    -H "Content-Type: application/json" \
    -d '{"user":"rafi"}'
# POST request: -H দিয়ে header, -d দিয়ে JSON body পাঠানো।

# ব্রাউজারে: F12 → Network ট্যাব → পাতা reload করে প্রতিটি request-এর
# status, সময়, header ও body দেখুন।
```

`curl -v`-এর আউটপুটে `>` দিয়ে শুরু লাইন আপনি পাঠাচ্ছেন, `<` দিয়ে শুরু লাইন সার্ভার পাঠাচ্ছে — এই চিহ্ন দুটি মনে রাখুন।

১৫.১০ Hands-on Exercise — curl দিয়ে ওয়েব পড়া

LAB: (১) `curl -i https://example.com` চালিয়ে status code, Content-Type ও Server header লিখুন। (২) `curl -v https://example.com 2>&1 | head -30` দিয়ে TLS handshake-এর লাইনগুলো খুঁজুন। (৩) ব্রাউজারে F12 → Network খুলে একটি সাইট reload করুন; সবচেয়ে ধীর request কোনটি, তা লিখুন। (৪) `curl -s -o /dev/null -w "%{http_code}\n" https://example.com/nonexistent` দিয়ে 404 যাচাই করুন।

উদাহরণ আউটপুট (illustrative):

```
$ curl -s -o /dev/null -w "%{http_code}\n" https://example.com/
200
# অর্থ: request সফল হয়েছে (200 OK)।
```

১৫.১১ Common Mistakes ও Troubleshooting

- **ভুল:** 404 দেখলেই সার্ভার বন্ধ ভাবা। **সংশোধন:** 404 মানে সার্ভার চলছে, কিন্তু ওই path-এ কিছু নেই — URL বা routing দেখুন।
- **ভুল:** POST দিয়ে তথ্য আনা। **সংশোধন:** পড়ার কাজে GET ব্যবহার করুন; POST cache হয় না ও refresh-এ আবার পাঠাতে চায়।
- **ভুল:** Cookie ছাড়া "লগইন থাকে না কেন" ভাবা। **সংশোধন:** ব্রাউজার cookie block করলে session কাজ করে না।
- **লক্ষণ:** 301 loop — A থেকে B, B থেকে A redirect; redirect নিয়ম পরীক্ষা করুন।
- **লক্ষণ:** API-তে 415 Unsupported Media Type — Content-Type header ভুল; JSON পাঠাচ্ছেন কি না দেখুন।

TROUBLESHOOTING: সাইট খুলছে না? ক্রম: DNS ঠিক কি (`dig`) → TCP সংযোগ হয় কি (`curl -v`-এ connection) → TLS ঠিক কি → HTTP status কী। যে ধাপে আটকায়, সমস্যা সেখানেই।

১৫.১২ Security Considerations

HTTP-এ সবকিছু plaintext-এ যায় — পাসওয়ার্ড, cookie, সব। Public Wi-Fi-তে HTTP সাইটে লগইন করা মানে তথ্য সবার সামনে খুলে দেওয়া। তাই production-এ সবকিছু HTTPS-এ চালান (অধ্যায় ১৬), আর HTTP request-কে HTTPS-এ redirect করুন। Cookie-তে `Secure` ও `HttpOnly` flag ব্যবহার করুন। সার্ভারের version জানানো header (`Server: nginx/1.24.0`) লুকিয়ে রাখা ভালো অভ্যাস — আক্রমণকারীকে তথ্য দেবেন কেন?

১৫.১৩ Best Practices

- API-তে সঠিক method ও status code ব্যবহার করুন; সব ভুলে 200 দেবেন না।
- বড় response-এ cache header (`Cache-Control`) ঠিক করুন।
- Log-এ প্রতিটি request-এর method, path, status ও সময় রাখুন।
- স্বাস্থ্য পরীক্ষার জন্য একটি হালকা endpoint (`/health`) রাখুন, যা 200 দেয়।

Knowledge Check — যাচাই করুন

CHECK YOUR UNDERSTANDING: উত্তর Answer Guide-এ আছে — আগে নিজে চেষ্টা করুন।

1. HTTP stateless হওয়ার অর্থ কী? A) সার্ভার কখনো বন্ধ হয় না B) সার্ভার আগের request মনে রাখে না; প্রতিটি request আলাদা C) ব্রাউজার কিছু মনে রাখে না D) Cookie ব্যবহার করা যায় না
2. একই POST request দুবার করলে দুটি পোস্ট তৈরি হতে পারে কেন? A) POST idempotent নয় B) সার্ভার ধীর C) নেটওয়ার্ক ভুল D) Cookie নেই বলে

3. 502 Bad Gateway-এর সবচেয়ে সম্ভাব্য অর্থ কী? A) URL ভুল B) Reverse proxy পেছনের upstream সার্ভারের সঙ্গে যোগাযোগ করতে পারেনি C) পাসওয়ার্ড ভুল D) DNS নেই
4. curl -v-এর আউটপুটে > দিয়ে শুরু লাইন কী বোঝায়? A) সার্ভারের উত্তর B) ক্লায়েন্ট (আপনি) যা পাঠাচ্ছেন C) ভুল বার্তা D) DNS উত্তর
5. Cookie-তে Secure flag-এর কাজ কী? A) Cookie encrypt করা B) Cookie শুধু HTTPS সংযোগে পাঠানো হবে C) Cookie মুছে ফেলা D) Cookie বড় করা

Practical Assignment

`curl -v` দিয়ে একটি পরিচিত HTTPS সাইটে request পাঠিয়ে ধাপগুলো লিখুন: DNS, TCP, TLS, request header, response status ও header। তারপর ব্রাউজারের Network ট্যাবে একই সাইটের পাঁচটি request-এর status, আকার ও সময়ের একটি টেবিল তৈরি করুন। সবচেয়ে ধীর request-টি কেন ধীর হতে পারে, তা অনুমান করে লিখুন।

১৫.৪ক Caching: Cache-Control, ETag ও 304

প্রতিটি request সার্ভারে যাওয়া জরুরি নয়। `Cache-Control: max-age=3600` header বললে ব্রাউজার এক ঘণ্টা উত্তরটি নিজের কাছে রেখে আবার জিজ্ঞেস করে না — সাইট দ্রুত লাগে, সার্ভারের চাপ কমে। `ETag` হলো একটি resource-এর সংস্করণ-চিহ্ন; ব্রাউজার পরের বার `If-None-Match` দিয়ে জিজ্ঞেস করে “এটি কি বদলেছে?” — না বদলালে সার্ভার `304 Not Modified` দিয়ে body ছাড়াই উত্তর দেয়। ছবি, CSS ও JavaScript-এর মতো বদলায় না এমন ফাইলে দীর্ঘ cache ঠিক করা হয়; কিন্তু ফাইলের নামে সংস্করণ (যেমন `style.v2.css`) না বদলালে ব্যবহারকারী পুরোনো ফাইলই দেখতে থাকবে — তাই deploy-এ ফাইলের নাম বদলানো একটি পেশাদার অভ্যাস।

Chapter Summary

HTTP হলো লেখা বার্তায় request–response বিনিময়: method বলে কী করতে হবে, status code বলে ফল কী হলো, header অতিরিক্ত তথ্য দেয়। HTTP stateless — লগইন টিকে থাকে cookie ও session-এর মাধ্যমে। 4xx মানে ক্লায়েন্টের ভুল, 5xx মানে সার্ভারের ভুল। সম্পূর্ণ lifecycle: DNS → TCP → TLS → HTTP request → response।

Revision Checklist

- ☐ GET/POST/PUT/PATCH/DELETE-এর পার্থক্য বলতে পারি
- ☐ 200, 301, 404, 502, 503-এর অর্থ জানি
- ☐ Cookie ও session-এর সম্পর্ক ব্যাখ্যা করতে পারি
- ☐ `curl -v` পড়তে পারি
- ☐ 502 দেখলে কোথায় তাকাতে হয়, জানি

অধ্যায় ১৬ • Part III — Internet Protocols

Chapter 16: TLS, SSL Certificates and Secure Communication — TLS, SSL সার্টিফিকেট ও নিরাপদ যোগাযোগ

শুরুতে: একটি বাস্তব দৃশ্য

আপনি একটি অনলাইন দোকানে কার্ডের তথ্য দিয়ে কেনাকাটা করলেন। ব্রাউজারের ঠিকানা বারে তালা চিহ্ন দেখা গেল — HTTPS। কিন্তু তালাটির মানে কী? আপনার ব্রাউজার ও সার্ভার প্রথমে পরস্পরকে চিনেছে, একটি গোপন চাবি বানিয়েছে, তারপর সেই চাবি দিয়ে সব কথা গোপন ভাষায় বলেছে। মাঝখানে কেউ শুনলেও কিছু বুঝবে না, বদলাতে চাইলেও ধরা পড়বে। এই অধ্যায়ে আমরা সেই গোপন ভাষার যুক্তি শিখব: encryption, সার্টিফিকেট, আর TLS handshake।

Learning Objectives — শেখার উদ্দেশ্য

- Symmetric ও asymmetric encryption-এর পার্থক্য ব্যাখ্যা করতে পারবেন।
- Public ও private key-এর ভূমিকা বলতে পারবেন।
- Digital signature কীভাবে সত্যতা যাচাই করে, তা ব্যাখ্যা করতে পারবেন।
- Certificate Authority (CA) ও X.509 সার্টিফিকেটের কাজ ব্যাখ্যা করতে পারবেন।
- TLS handshake ধাপে ধাপে বর্ণনা করতে পারবেন।
- Certificate validation কীভাবে হয়, তা বলতে পারবেন।
- Certificate expiration ও mismatch সমস্যা নির্ণয় করতে পারবেন।
- `openssl s_client` দিয়ে সার্টিফিকেট পরীক্ষা করতে পারবেন।

Prerequisites — পূর্বজ্ঞান

HTTP request–response (অধ্যায় ১৫), TCP handshake (অধ্যায় ১১), DNS (অধ্যায় ১৩)। গাণিতিক গভীরে যাওয়া হবে না — ধারণাটিই মূল লক্ষ্য।

১৬.১ প্রথম নীতি: তিনটি প্রশ্ন, তিনটি উত্তর

নিরাপদ যোগাযোগে তিনটি প্রশ্নের উত্তর দরকার: (১) **গোপনীয়তা** — মাঝখানে কেউ পড়তে পারবে না তো? (২) **সত্যতা** — আমি কি আসল সার্ভারের সঙ্গেই কথা বলছি, নকল কারও সঙ্গে নয়? (৩) **অখণ্ডতা** — পথে বার্তা বদলে দেওয়া হয়নি তো? Encryption প্রথমটির উত্তর দেয়; certificate ও signature দ্বিতীয়টির; আর প্রতিটি বার্তার সঙ্গে যাওয়া যাচাই-সংখ্যা (MAC) তৃতীয়টির। এই তিনটি একসঙ্গে না থাকলে যোগাযোগ নিরাপদ নয়।

১৬.২ Symmetric বনাম Asymmetric Encryption

Symmetric encryption-এ একটি চাবি দিয়েই তালা লাগানো ও খোলা হয় — দ্রুত, কিন্তু চাবিটি দুই পক্ষের কাছে পৌঁছাতে হয় নিরাপদে। **Asymmetric encryption**-এ দুটি চাবি: **public key** সবাই জানতে পারে, **private key** শুধু মালিকের কাছে থাকে। Public key দিয়ে তালা লাগানো বার্তা শুধু private key দিয়ে খোলা যায়। উপমা: খোলা তালা সবাইকে দেওয়া যায়;

তালা লাগানো চিঠি শুধু চাবির মালিকই খুলতে পারে। TLS দুটোই ব্যবহার করে: handshake-এ asymmetric দিয়ে নিরাপদে একটি session key বিনিময় হয়, তারপর আসল তথ্য symmetric দিয়ে দ্রুত encrypt করা হয় — গতির ও নিরাপত্তার সেরা সমন্বয়।

> **CONCEPT:** Private key কখনো নেটওয়ার্কে পাঠানো হয় না — কখনোই না। যে সার্ভার তার private key পাঠায়, সেটি ভুয়া বা ভুল কনফিগার করা। আপনার ব্রাউজার সার্ভারের public key দিয়েই কাজ করে।

১৬.৩ Digital Signature: কে বলেছে, তা যাচাই

Digital signature হলো উল্টো ব্যবহার: বার্তাটিকে private key দিয়ে "সই" করা হয়; যে কেউ public key দিয়ে যাচাই করতে পারে সইটি আসল কি না। সই মিললে বোঝা যায় বার্তাটি private key-র মালিকেরই লেখা, আর পথে বদলানো হয়নি। সার্টিফিকেট আসলে একটি signed দাবি: "এই public key-টি এই ডোমেইনের" — আর তাতে সই করেছে একটি বিশ্বস্ত প্রতিষ্ঠান।

১৬.৪ Certificate Authority ও X.509

কেউ নিজেই দাবি করতে পারে "আমি example.com" — তা বিশ্বাসযোগ্য নয়। তাই **Certificate Authority (CA)** নামের বিশ্বস্ত প্রতিষ্ঠান যাচাই করে সার্টিফিকেট দেয়: "হ্যাঁ, এই public key-র মালিক example.com-এর নিয়ন্ত্রক।" আপনার ব্রাউজার ও OS-এ CA-দের তালিকা আগে থেকেই আছে; সার্টিফিকেটের সই সেই তালিকার কোনো CA-র সঙ্গে মিললেই ব্রাউজার বিশ্বাস করে। **X.509** হলো সার্টিফিকেটের মানক গঠন: subject (ডোমেইন নাম), issuer (কোন CA দিয়েছে), validity (কবে থেকে কবে পর্যন্ত), public key, আর signature। Chain-টি এরকম: আপনার সার্টিফিকেট ← intermediate CA ← root CA (ব্রাউজারে আগে থেকে বিশ্বস্ত)।

১৬.৫ TLS Handshake: ধাপে ধাপে

ব্রাউজার সার্ভারে সংযোগ করলে যা ঘটে: (১) **ClientHello** — ব্রাউজার বলে, "আমি TLS 1.3 পারি, এই cipher পছন্দ করি, আর একটি random সংখ্যা নাও।" (২) **ServerHello** — সার্ভার cipher বেছে নেয়, নিজের random সংখ্যা দেয়, আর নিজের **সার্টিফিকেট** পাঠায়। (৩) ব্রাউজার সার্টিফিকেট যাচাই করে: নাম মিলছে কি, মেয়াদ আছে কি, CA-র সই ঠিক কি? (৪) দুই পক্ষ key exchange-এর মাধ্যমে একটি shared session key বানায় — TLS 1.3-এ এটি forward secrecy দেয়: session key চুরি গেলেও আগের session পড়া যায় না। (৫) এরপর সব HTTP বার্তা ওই session key দিয়ে encrypt হয়। পুরো handshake সাধারণত কয়েকশো মিলিসেকেন্ডে শেষ হয়।

চিত্র ১৬.১ — TLS Handshake (সরলীকৃত)

ClientHello (TLS version, cipher, random) → ServerHello (নির্বাচিত cipher, random) + Certificate (X.509) + Key Exchange → ব্রাউজার: সার্টিফিকেট যাচাই (নাম, মেয়াদ, CA সই) → Session key তৈরি → [এরপর সবকিছু encrypted] → HTTP request/response

চিত্র ১৬.২ — Certificate Chain

Root CA (ব্রাউজারে আগে থেকে বিশ্বস্ত) → Intermediate CA → সার্ভারের সার্টিফিকেট (example.com)। যাচাই মানে শিকলটি ধরে ওপরে ওঠা: প্রতিটি সই পরেরটির সঙ্গে মিলতে হবে।

১৬.৬ Certificate Validation: ব্রাউজার কী কী যাচাই করে

তালা চিহ্ন পেতে চারটি যাচাই পাস করতে হয়: (১) **নাম মিলছে কি** — সার্টিফিকেটের নাম (CN/SAN) ঠিকানার বারের ডোমেইনের সঙ্গে মিলতে হবে; `example.com`-এর সার্টিফিকেট দিয়ে `evil.com` চালানো যায় না। (২) **মেয়াদ আছে কি** — মেয়াদ শেষ সার্টিফিকেট অগ্রহণযোগ্য; Let's Encrypt-এর সার্টিফিকেট ৯০ দিনের হয়, তাই auto-renewal আবশ্যক। (৩) **CA বিশ্বস্ত কি** — self-signed সার্টিফিকেট ব্রাউজার বিশ্বাস করে না, কারণ কেউ যাচাই করেনি। (৪) **বাতিল হয়নি তো** — চুরি যাওয়া সার্টিফিকেট CA বাতিল (revoke) করতে পারে। যেকোনো একটিতে ব্যর্থ হলেই ব্রাউজার সতর্কবার্তা দেখায় — সেই সতর্কবার্তা উপেক্ষা করে “Proceed” চাপা বিপজ্জনক অভ্যাস।

> **REAL WORLD:** একটি জনপ্রিয় সাইট এক সকালে “Your connection is not private” দেখাতে শুরু করল। কারণ: সার্টিফিকেটের মেয়াদ রাতেই শেষ হয়েছিল, আর auto-renewal ব্যর্থ হয়েছিল — renewal স্ক্রিপ্টে ভুল পথ লেখা ছিল। মনিটরিংয়ে সার্টিফিকেটের মেয়াদ নজরদারি ছিল না বলেই কেউ আগে জানতে পারেনি। শিক্ষা: সার্টিফিকেটের মেয়াদ মনিটর করুন; মেয়াদ শেষ হওয়ার ৩০ দিন আগে সতর্কবার্তা আসা উচিত।

১৬.৭ পরিভাষা — Terminology

Term	বাংলা ব্যাখ্যা
Symmetric encryption	একটি চাবিতেই তালা লাগানো ও খোলা; দ্রুত।
Asymmetric encryption	Public key দিয়ে তালা, private key দিয়ে খোলা।
Public / Private key	সবাই জানতে পারে / শুধু মালিকের কাছে থাকে।
Digital signature	Private key দিয়ে সই; public key দিয়ে যাচাই।
CA	সার্টিফিকেট যাচাই করে দেওয়া বিশ্বস্ত প্রতিষ্ঠান।
X.509	সার্টিফিকেটের মানক গঠন।
TLS handshake	পরিচয় যাচাই ও session key তৈরির প্রাথমিক কথোপকথন।
Forward secrecy	এক session-এর key চুরি গেলেও আগের session নিরাপদ থাকে।
Self-signed	নিজেই নিজেকে দেওয়া সার্টিফিকেট; ব্রাউজার বিশ্বাস করে না।
Revocation	CA-র মাধ্যমে সার্টিফিকেট বাতিল করা।

১৬.৮ বাস্তব উদাহরণ: “Certificate Mismatch” তদন্ত

ব্যবহারকারীরা `app.example.com`-এ সতর্কবার্তা পাচ্ছেন, কিন্তু `example.com` ঠিক আছে। তদন্ত: সার্টিফিকেটে শুধু `example.com` ও `www.example.com` আছে — `app.example.com` নেই। সমাধান: wildcard (`*.example.com`) বা আলাদা SAN-সহ সার্টিফিকেট নেওয়া। শিক্ষা: subdomain যোগ করার সময় সার্টিফিকেটের নামের তালিকা (SAN) পরীক্ষা করুন।

১৬.৯ ব্যবহারিক কমান্ড

```
$ openssl s_client -connect example.com:443 -servername example.com
# সার্ভারের সার্টিফিকেট দেখায়: issuer, validity, subject।
# -servername (SNI) জরুরি — একই IP-তে অনেক সাইট থাকলে সার্ভার
# জানতে পারে আপনি কোন সাইটের সার্টিফিকেট চাইছেন।

$ echo | openssl s_client -connect example.com:443 -servername example.com 2>/dev/null \
    | openssl x509 -noout -dates -subject -issuer
# শুধু মেয়াদ, subject ও issuer — দ্রুত পরীক্ষার জন্য।

$ curl -v https://example.com 2>&1 | grep -i "subject\|issuer\|expire"
# curl-এর মাধ্যমে সার্টিফিকেটের মূল তথ্য দেখা।
```

এই কমান্ডগুলো শুধু পড়ে (read-only inspection) — কিছুই বদলায় না, তাই নিরাপদ। Private key কখনো এইভাবে দেখাবেন না বা কোথাও পাঠাবেন না।

১৬.১০ Hands-on Exercise — সার্টিফিকেট পরীক্ষা

LAB: (১) `openssl s_client -connect example.com:443 -servername example.com` চালিয়ে issuer ও মেয়াদের তারিখ লিখুন। (২) একটি পরিচিত সাইটের সার্টিফিকেটের SAN-এ কোন কোন নাম আছে, তা খুঁজুন। (৩) ব্রাউজারে একই সাইটের তালা চিহ্নে ক্লিক করে সার্টিফিকেটের তথ্য মিলিয়ে দেখুন। (৪) `curl -v https://expired.badssl.com` (পরীক্ষামূলক সাইট) চালিয়ে দেখুন curl কী বার্তা দেয় — এটি শুধু পর্যবেক্ষণ, কোনো কিছু বাইপাস করবেন না।

উদাহরণ আউটপুট (illustrative):

```
notBefore=Nov  1 00:00:00 2026 GMT
notAfter=Jan 30 23:59:59 2027 GMT
subject=CN = example.com
issuer=C = US, O = Let's Encrypt, CN = R3
# অর্থ: সার্টিফিকেট ১ নভেম্বর ২০২৬ থেকে ৩০ জানুয়ারি ২০২৭ পর্যন্ত বৈধ,
# দিয়েছে Let's Encrypt।
```

১৬.১১ Common Mistakes ও Troubleshooting

- **ভুল:** ব্রাউজারের সতর্কবার্তা উপেক্ষা করে এগিয়ে যাওয়া। **সংশোধন:** সতর্কবার্তা মানেই কিছু ভুল — কারণ খুঁজুন, বাইপাস নয়।
- **ভুল:** Self-signed সার্টিফিকেট production-এ ব্যবহার করা। **সংশোধন:** Lab-এ ঠিক আছে; production-এ বিশ্বস্ত CA-র সার্টিফিকেট নিন (Let's Encrypt বিনামূল্যে দেয়)।
- **ভুল:** Private key Git-এ commit করা। **সংশোধন:** Private key কখনো repository-তে যাবে না; ফাঁস হলে সার্টিফিকেট বাতিল করে নতুন নিন।
- **লক্ষণ:** “Certificate expired” — মেয়াদ শেষ; renewal যাচাই করুন।
- **লক্ষণ:** “Hostname mismatch” — SAN-এ ডোমেইন নেই; সার্টিফিকেটের নামের তালিকা দেখুন।
- **লক্ষণ:** হঠাৎ TLS ব্যর্থ — সিস্টেমের ঘড়ি ভুল কি না দেখুন (অধ্যায় ১৪)।

TROUBLESHOOTING: `curl` বলছে "certificate verify failed" — প্রথমে ঘড়ি, তারপর সার্টিফিকেটের মেয়াদ ও chain দেখুন। `curl -k` দিয়ে যাচাই বন্ধ করা শুধু পরীক্ষার জন্য, কখনোই সমাধান নয় — এটি সুরক্ষাই বন্ধ করে দেয়।

১৬.১২ Security Considerations

সার্টিফিকেট থাকলেই সাইট নিরাপদ নয় — সার্টিফিকেট শুধু বলে "আপনি আসল example.com-এর সঙ্গে কথা বলছেন"; ওই সাইটটি নিজে ক্ষতিকর কি না, তা বলে না। TLS 1.0 ও 1.1 এখন অনিরাপদ — বন্ধ রাখুন। Private key-র অনুমতি কঠোর রাখুন (`600`, root-এর মালিকানায়)। Renewal স্বয়ংক্রিয় করুন ও মেয়াদ মনিটর করুন।

১৬.১৩ Best Practices

- Production-এ সব traffic HTTPS-এ চালান; HTTP-কে HTTPS-এ redirect করুন।
- সার্টিফিকেট auto-renewal করুন (যেমন certbot) ও মেয়াদের সতর্কবার্তা সেট করুন।
- সার্টিফিকেটের SAN-এ সব subdomain রাখুন, অথবা wildcard ব্যবহার করুন।
- Private key আলাদা, নিরাপদ জায়গায় রাখুন; backup encrypt করে রাখুন।

Knowledge Check — যাচাই করুন

CHECK YOUR UNDERSTANDING: উত্তর Answer Guide-এ আছে — আগে নিজে চেষ্টা করুন।

1. TLS-এ আসল তথ্য encrypt করতে সাধারণত কোনটি ব্যবহৃত হয়? A) শুধু asymmetric encryption B) Handshake-এ asymmetric দিয়ে session key বানিয়ে, তারপর symmetric encryption C) কোনো encryption নয় D) শুধু hashing
2. Private key সম্পর্কে কোনটি সত্য? A) সবাইকে দেওয়া যায় B) কখনো নেটওয়ার্কে পাঠানো হয় না; শুধু মালিকের কাছে থাকে C) প্রতিটি session-এ বদলায় D) ব্রাউজারকে পাঠাতে হয়
3. ব্রাউজার সার্টিফিকেট যাচাইয়ের সময় কী পরীক্ষা করে না? A) নাম মিলছে কি না B) মেয়াদ আছে কি না C) সার্ভারের RAM কত D) CA-র সই ঠিক কি না
4. Self-signed সার্টিফিকেটে ব্রাউজার সতর্কবার্তা দেয় কেন? A) সার্টিফিকেটটি ধীর বলে B) কোনো বিশ্বস্ত CA তা যাচাই করেনি বলে C) সার্টিফিকেটটি বড় বলে D) সার্ভার বন্ধ বলে
5. Forward secrecy-র অর্থ কী? A) চাবি কখনো বদলায় না B) এক session-এর key চুরি গেলেও আগের session-এর তথ্য পড়া যায় না C) সার্টিফিকেট চিরকাল বৈধ থাকে D) Private key public হয়ে যায়

Practical Assignment

দুটি সাইটের সার্টিফিকেট `openssl s_client` দিয়ে পরীক্ষা করে একটি টেবিল তৈরি করুন: issuer, notBefore, notAfter, subject। তারপর ব্যাখ্যা করুন — (ক) কোন সার্টিফিকেটটি কবে মেয়াদ শেষ হবে, (খ) renewal কবে করা উচিত, (গ) একটি সার্টিফিকেট "mismatch" হলে ব্রাউজার কী দেখাবে ও কেন।

১৬.৫ক TLS 1.2 বনাম TLS 1.3

TLS 1.3 আগের সংস্করণের চেয়ে দুটি দিক থেকে ভালো। প্রথমত, handshake দ্রুত: TLS 1.2-এ সাধারণত দুটি round-trip লাগত, TLS 1.3-এ একটি — কারণ key exchange-এর তথ্য ClientHello-তেই পাঠানো হয়। দ্বিতীয়ত, নিরাপত্তা: 1.3 দুর্বল ও পুরোনো cipher সরিয়ে দিয়েছে, আর handshake-এর বেশিরভাগ অংশই encrypt করা থাকে, ফলে মাঝখানে কেউ সার্ভারের সার্টিফিকেট পর্যন্ত সহজে দেখতে পায় না। আপনার সার্ভারে TLS 1.2 ও 1.3 দুটোই চালু রাখা এখনো সাধারণ অভ্যাস — খুব পুরোনো ক্লায়েন্টের জন্য 1.2 দরকার হতে পারে — কিন্তু TLS 1.0 ও 1.1 বন্ধ রাখুন, সেগুলোতে জানা দুর্বলতা আছে। Nginx-এ `ssl_protocols TLSv1.2 TLSv1.3;` দিয়ে এটি ঠিক করা হয়।

Chapter Summary

TLS তিনটি নিশ্চয়তা দেয়: গোপনীয়তা, সত্যতা ও অখণ্ডতা। Asymmetric encryption দিয়ে handshake-এ session key বানানো হয়, তারপর symmetric encryption-এ আসল তথ্য যায়। CA সার্টিফিকেটে সই করে ডোমেইনের মালিকানা যাচাই করে; ব্রাউজার নাম, মেয়াদ ও CA-র সই পরীক্ষা করে। মেয়াদ ও নামের ভুলই সবচেয়ে সাধারণ TLS সমস্যা।

Revision Checklist

- ☐ Symmetric ও asymmetric-এর পার্থক্য বলতে পারি
- ☐ TLS handshake-এর ধাপগুলো বলতে পারি
- ☐ সার্টিফিকেট যাচাইয়ের চারটি শর্ত জানি
- ☐ `openssl s_client` দিয়ে মেয়াদ দেখতে পারি
- ☐ Private key কখনো শেয়ার করা যায় না — জানি

অধ্যায় ১৭ • Part III — Internet Protocols

Chapter 17: Proxies, Reverse Proxies and Load Balancing — প্রক্সি, রিভার্স প্রক্সি ও লোড ব্যালান্সিং

শুরুতে: একটি বাস্তব দৃশ্য

একটি জনপ্রিয় সংবাদ সাইটে হঠাৎ লাখ লাখ পাঠক ঢুকলেন। সাইটটি কিন্তু পড়ল না। কেন? পাঠকরা সরাসরি application সার্ভারে যাননি — সামনে দাঁড়িয়েছিল Nginx, একটি reverse proxy। সে request-গুলো তিনটি application সার্ভারে ভাগ করে দিল, TLS-এর কাজ নিজে সামলাল, আর static ছবিগুলো নিজেই দিয়ে দিল। Application সার্ভারগুলো শুধু নিজের কাজটুকু করল। এই অধ্যায়ে আমরা শিখব কীভাবে একটি সার্ভারের সামনে দাঁড়িয়ে থাকা এই “দারোয়ান” পুরো production স্থাপত্য বদলে দেয়।

Learning Objectives — শেখার উদ্দেশ্য

- Forward proxy ও reverse proxy-র পার্থক্য ব্যাখ্যা করতে পারবেন।
- Reverse proxy কেন ব্যবহার করা হয়, তা বলতে পারবেন।
- Nginx-এর মূল ধারণা ও কনফিগারেশন পড়তে পারবেন।
- Upstream সার্ভার ও load balancing algorithm ব্যাখ্যা করতে পারবেন।
- TLS termination কী, তা বলতে পারবেন।
- WebSocket proxying ও rate limiting-এর ধারণা দিতে পারবেন।
- Nginx দিয়ে reverse proxy ও load balancer কনফিগার করতে পারবেন।
- 502/504 ত্রুটির কারণ নির্ণয় করতে পারবেন।

Prerequisites — পূর্বজ্ঞান

HTTP ও status code (অধ্যায় ১৫), TLS handshake ও সার্টিফিকেট (অধ্যায় ১৬), DNS (অধ্যায় ১৩), এবং Vol 01-এর systemd ও Linux ফাইল অনুমতির ধারণা।

১৭.১ প্রথম নীতি: মাঝখানে দাঁড়ানো সার্ভার

Proxy মানে মাঝখানে দাঁড়ানো সার্ভার — ক্লায়েন্টের হয়ে সার্ভারের সঙ্গে, বা সার্ভারের হয়ে ক্লায়েন্টের সঙ্গে কথা বলে। পার্থক্যটি দিকের: **forward proxy** ক্লায়েন্টের পাশে দাঁড়ায় — ক্লায়েন্টের request নিয়ে ইন্টারনেটে যায় (অফিসের proxy, যা কিছু সাইট আটকায়)। **Reverse proxy** সার্ভারের পাশে দাঁড়ায় — ইন্টারনেটের request নিয়ে ভেতরের সার্ভারে যায়। ব্যবহারকারী জানে না পেছনে কয়টি সার্ভার আছে; সে শুধু reverse proxy-কেই দেখে। এই বইয়ের মূল বিষয় reverse proxy — কারণ DevOps-এ এটাই দৈনন্দিন কাজ।

> **CONCEPT:** Forward proxy ক্লায়েন্টকে লুকায়; reverse proxy সার্ভারকে লুকায়। দুটোই মাঝখানে দাঁড়ায়, কিন্তু কার স্বার্থে, তা উল্টো।

১৭.২ Reverse Proxy কেন? পাঁচটি কারণ

(১) **Load balancing** — request অনেক সার্ভারে ভাগ করে দেওয়া। (২) **TLS termination** — TLS-এর ভারী কাজটি proxy সামলায়; ভেতরের সার্ভারগুলো সাধারণ HTTP-তে কথা বলে। (৩) **নিরাপত্তা** — ভেতরের সার্ভার ইন্টারনেটে সরাসরি উন্মুক্ত থাকে না; শুধু proxy-ই দেখা যায়। (৪) **Static content** — ছবি, CSS, JS proxy নিজেই দিয়ে দেয়; application সার্ভারকে বিরক্ত করে না। (৫) **একটি দরজা** — অনেক সার্ভিস (`/api`, `/blog`, `/`) একই ডোমেইনের আড়ালে ভিন্ন ভিন্ন সার্ভারে পাঠানো যায়।

১৭.৩ Nginx: কীভাবে কাজ করে

Nginx একটি হালকা, দ্রুত web server ও reverse proxy। এর মূল ধারণা: `server` block বলে কোন ডোমেইনের request নেবে; `location` বলে কোন path-এর request কোথায় যাবে; `proxy_pass` বলে পেছনের সার্ভারের ঠিকানা; `upstream` block বলে একাধিক পেছনের সার্ভারের তালিকা। Nginx event-driven — অল্প memory-তে অনেক concurrent connection সামলাতে পারে, তাই reverse proxy হিসেবে এত জনপ্রিয়।

চিত্র ১৭.১ — Reverse Proxy স্থাপত্য

ব্যবহারকারী → [Nginx: TLS termination, static file, rate limit] → Upstream: App Server 1 / App Server 2 / App Server 3 → Database (শুধু app server-ই পৌঁছাতে পারে)। ব্যবহারকারী কখনো app server বা database সরাসরি দেখে না।

চিত্র ১৭.২ — Load Balancing অ্যালগরিদম

Round-robin: ১ → ২ → ৩ → ১ → ২ → ৩...। Least connections: যার চলমান connection সবচেয়ে কম, সে পায়। IP hash: একই ক্লায়েন্ট সবসময় একই সার্ভারে যায় (session ধরে রাখতে)।

১৭.৪ Load Balancing অ্যালগরিদম

Round-robin সবচেয়ে সহজ: request পর্যায়ক্রমে প্রতিটি সার্ভারে যায়। সার্ভারগুলো সমান শক্তির হলে এটাই যথেষ্ট। **Least connections** ব্যস্ত সার্ভারকে এড়িয়ে কম ব্যস্তটিকে বেছে নেয় — দীর্ঘ request থাকলে ভালো। **IP hash** একই ক্লায়েন্টকে সবসময় একই সার্ভারে পাঠায় — session memory-তে থাকলে দরকারি (তবে sticky session-এর ওপর নির্ভর না করাই ভালো অভ্যাস; session shared store-এ রাখুন)। **Weighted** round-robin শক্তিশালী সার্ভারকে বেশি request দেয়। Health check-এর ধারণাও জানুন: proxy নিয়মিত পরীক্ষা করে কোন সার্ভার জীবিত; মৃত সার্ভারে request পাঠায় না।

১৭.৫ TLS Termination ও WebSocket

TLS termination মানে proxy-তেই TLS খোলা হয়: বাইরে থেকে HTTPS আসে, proxy সার্টিফিকেট দিয়ে decrypt করে, ভেতরে সাধারণ HTTP-তে পাঠায়। সুবিধা: একটি জায়গায় সার্টিফিকেট রাখা ও নবায়ন করা যায়। ঝুঁকি: proxy ও app server-এর মাঝের নেটওয়ার্কটি বিশ্বস্ত হতে হবে — সেটি private নেটওয়ার্ক হওয়া উচিত। **WebSocket** (live chat, real-time update) একটি দীর্ঘস্থায়ী connection — proxy-কে বলতে হয় connection upgrade করতে দিতে (`Upgrade` ও `Connection` header পাঠাতে); না বললে WebSocket বারবার ভেঙে যাবে।

১৭.৬ Rate Limiting: বন্যা আটকানো

Rate limiting মানে প্রতি ক্লায়েন্টের request-এর হার সীমিত করা — যেমন প্রতি সেকেন্ডে ১০টির বেশি নয়। এতে একটি ভুল বা দুষ্ট ক্লায়েন্ট পুরো সার্ভার দখল করতে পারে না। লগইন endpoint-এ rate limit brute-force আক্রমণও ধীর করে দেয়। মনে রাখবেন: rate limit সুরক্ষার একটি স্তর মাত্র, সম্পূর্ণ সমাধান নয়।

১৭.৭ পরিভাষা — Terminology

Term	বাংলা ব্যাখ্যা
Forward proxy	ক্লায়েন্টের পাশে দাঁড়ানো মধ্যবর্তী সার্ভার।
Reverse proxy	সার্ভারের পাশে দাঁড়ানো মধ্যবর্তী সার্ভার।
Upstream	Reverse proxy-র পেছনের আসল application সার্ভার।
Load balancing	Request অনেক সার্ভারে ভাগ করে দেওয়া।
TLS termination	Proxy-তেই TLS খুলে ভেতরে HTTP পাঠানো।
Health check	Upstream সার্ভার জীবিত কি না, নিয়মিত পরীক্ষা।
Sticky session	একই ক্লায়েন্টকে বারবার একই সার্ভারে পাঠানো।
Rate limiting	প্রতি ক্লায়েন্টের request-এর হার সীমিত করা।
WebSocket	একটি দীর্ঘস্থায়ী, দ্বিমুখী সংযোগ।

১৭.৮ বাস্তব উদাহরণ: একটি সাইটের স্থাপত্য

TechNova-র সাইট: ব্যবহারকারী → Cloudflare (DNS ও CDN) → Nginx (VPS-এ, TLS termination) → দুটি app server (Node.js, port 3000) → PostgreSQL (শুধু private নেটওয়ার্কে)। Static ফাইল Nginx নিজেই দেয়। `/api` যায় app server-এ, `/` যায় একই সার্ভারে। একদিন app server ১ বন্ধ হলো — Nginx health check-এ তা ধরে দ্বিতীয়টিতেই সব request পাঠাল; ব্যবহারকারীরা কিছুই টের পেলেন না। এটাই reverse proxy-র মূল্য: ব্যর্থতা লুকানো নয়, ব্যর্থতা সামলানো।

১৭.৯ ব্যবহারিক কনফিগারেশন

```
# /etc/nginx/sites-available/technova — উদাহরণ (নিজের ডোমেইন ও IP বসান)
upstream app_servers {
    # Round-robin (ডিফল্ট): request পর্যায়ক্রমে যাবে
    server 10.0.1.10:3000;
    server 10.0.1.11:3000;
}

server {
    listen 80;
    server_name example.com www.example.com;
    # HTTP-কে HTTPS-এ পাঠানো (অধ্যায় ১৬)
    return 301 https://$host$request_uri;
```

```
}

server {
    listen 443 ssl;
    server_name example.com;
    ssl_certificate     /etc/ssl/certs/example.com.crt;
    ssl_certificate_key /etc/ssl/private/example.com.key;

    location / {
        proxy_pass http://app_servers;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    location /static/ {
        # Static ফাইল Nginx নিজেই দেবে
        alias /var/www/technova/static/;
    }
}
```

ব্যাখ্যা: `proxy_set_header Host` না দিলে app server জানবে না আসল ডোমেইন কী ছিল; `X-Real-IP` ও `X-Forwarded-For` না দিলে app server সব ক্লায়েন্টকে proxy-র IP ভাববে — log ও rate limit ভুল হবে। `X-Forwarded-Proto` জানায় আসল request HTTPS ছিল কি না।

```
$ sudo nginx -t
# কনফিগারেশন পরীক্ষা — ভুল থাকলে এখানেই ধরা পড়ে, reload-এর আগে।

$ sudo systemctl reload nginx
# reload: connection না ভেঙে নতুন কনফিগারেশন নেয় (restart নয়)।

$ curl -i http://localhost/
# proxy দিয়ে উত্তর আসছে কি না, তা যাচাই করুন।
```

WARNING: কনফিগারেশন বদলের পর `nginx -t` না করে কখনো reload করবেন না — ভুল কনফিগারেশনে Nginx বন্ধ হয়ে যেতে পারে। আর সার্টিফিকেটের পথ ভুল হলে reload ব্যর্থ হবে; error log দেখুন (`journalctl -u nginx`)।

১৭.১০ Hands-on Exercise — নিজের Reverse Proxy

LAB: (১) VM-এ Nginx ইনস্টল করুন (`sudo apt update && sudo apt install nginx`)। (২) port 3000-এ একটি সহজ HTTP সার্ভার চালান (`python3 -m http.server 3000 --directory /tmp/demo` — শুধু lab-এর জন্য)। (৩) উপরের উদাহরণ অনুযায়ী (TLS ছাড়া, শুধু port 80) একটি server block লিখে `proxy_pass http://127.0.0.1:3000;` বসান। (৪) `nginx -t` ও reload করে `curl -i http://localhost/` চালান। যাচাই: উত্তরে আপনার demo ফাইলের তালিকা আসতে হবে। Cleanup: পরীক্ষা শেষে site disable করুন ও demo সার্ভার বন্ধ করুন।

উদাহরণ আউটপুট (illustrative):

```
$ sudo nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
# অর্থ: কনফিগারেশনে syntax ভুল নেই; reload করা নিরাপদ।
```

১৭.১১ Common Mistakes ও Troubleshooting

- **ভুল:** `proxy_pass` -এ trailing slash-এর ভুল। **সংশোধন:** `proxy_pass http://app/;` বনাম `proxy_pass http://app;` — path বদলে যায়; Nginx ডকুমেন্টেশনে নিয়মটি পড়ে নিন।
- **ভুল:** Header না পাঠানো। **সংশোধন:** Host ও X-Forwarded-* header সবসময় পাঠান, নইলে app ভুল IP ও ভুল scheme দেখবে।
- **ভুল:** Upstream-এর IP public নেটওয়ার্কে রাখা। **সংশোধন:** Upstream private নেটওয়ার্কে রাখুন; শুধু proxy-ই public থাকবে।
- **লক্ষণ:** 502 Bad Gateway — upstream শুনছে না বা বন্ধ (অধ্যায় ১৫)।
- **লক্ষণ:** 504 Gateway Timeout — upstream উত্তর দিতে দেরি করছে; app ধীর বা ডেটাবেস আটকে আছে।
- **লক্ষণ:** WebSocket বারবার ভাঙে — Upgrade header proxy করছে কি না দেখুন।

TROUBLESHOOTING: 502 দেখলে প্রথমে upstream-এ সরাসরি যান: `curl http://10.0.1.10:3000/` — উত্তর এলে proxy-র কনফিগারেশনে সমস্যা; না এলে app সার্ভারেই সমস্যা।

১৭.১২ Security Considerations

Reverse proxy নিরাপত্তা বাড়ায়, কিন্তু নিজেই আক্রমণের লক্ষ্য হয়। Nginx সবসময় update রাখুন; অপ্রয়োজনীয় location খুলবেন না; `server_tokens off;` দিয়ে version লুকান। Upstream-এর সঙ্গে যোগাযোগ private নেটওয়ার্কে রাখুন — public IP-তে app server খোলা থাকলে proxy-র সুরক্ষা অর্থহীন। Rate limit লগইন ও API endpoint-এ বসান।

১৭.১৩ Best Practices

- একটি জায়গায় TLS রাখুন (proxy), সার্টিফিকেট সেখানেই নবায়ন করুন।
- Static content proxy থেকে দিন; application শুধু dynamic কাজ করুক।
- Health check ও log নজরদারি চালু রাখুন; upstream মারা গেলে জানতে হবে।
- কনফিগারেশন version control-এ রাখুন; `nginx -t` ছাড়া reload নয়।

Knowledge Check — যাচাই করুন

> **CHECK YOUR UNDERSTANDING:** উত্তর Answer Guide-এ আছে — আগে নিজে চেষ্টা করুন।

1. Forward proxy ও reverse proxy-র মূল পার্থক্য কী? A) কোনো পার্থক্য নেই B) Forward proxy ক্লায়েন্টের পাশে, reverse proxy সার্ভারের পাশে দাঁড়ায় C) Forward proxy দ্রুত, reverse proxy ধীর D) Reverse proxy শুধু HTTP-তে চলে
2. TLS termination-এর অর্থ কী? A) TLS বন্ধ করে দেওয়া B) Proxy-তেই TLS খুলে ভেতরে সাধারণ HTTP পাঠানো C) সার্টিফিকেট মুছে ফেলা D) HTTPS বন্ধ করা
3. Round-robin load balancing কীভাবে কাজ করে? A) সব request একটি সার্ভারে যায় B) Request পর্যায়ক্রমে প্রতিটি upstream সার্ভারে যায় C) সবচেয়ে ধীর সার্ভারকে বেছে নেয় D) এলোমেলোভাবে একটি সার্ভার বন্ধ করে
4. Nginx কনফিগারেশনে proxy_set_header X-Real-IP না পাঠালে কী হয়? A) কিছুই হয় না B) App server সব ক্লায়েন্টকে proxy-র IP হিসেবে দেখে — আসল ক্লায়েন্ট IP হারিয়ে যায় C) সাইট বন্ধ হয়ে যায় D) TLS ভেঙে যায়
5. 504 Gateway Timeout-এর সবচেয়ে সম্ভাব্য অর্থ কী? A) URL ভুল B) Upstream সার্ভার সময়মতো উত্তর দেয়নি C) DNS নেই D) পাসওয়ার্ড ভুল

Practical Assignment

একটি কাল্পনিক সাইটের স্থাপত্য আঁকুন: দুটি app server, একটি Nginx reverse proxy, একটি database। দেখান — (ক) TLS কোথায় শেষ হয়, (খ) static ফাইল কে দেয়, (গ) একটি app server বন্ধ হলে কী হয়, (ঘ) database কেন public নেটওয়ার্কে নেই। তারপর উপরের উদাহরণ অনুযায়ী Nginx কনফিগারেশন লিখুন (নিজের ডোমেইন ও private IP ব্যবহার করে)।

১৭.৪ক Sticky Session বনাম Shared Session

IP hash বা cookie-ভিত্তিক sticky session একই ব্যবহারকারীকে সবসময় একই app server-এ পাঠায় — session memory-তে থাকলে এটি কাজ করে। কিন্তু সার্ভারটি মারা গেলে ওই ব্যবহারকারীর session-ও হারিয়ে যায়, আর load-ও অসম হয়: কিছু সার্ভার ব্যস্ত, কিছু ফাঁকা। পেশাদার সমাধান **shared session store**: session Redis বা ডেটাবেসে রাখা হয়, যাতে যেকোনো app server যেকোনো request সামলাতে পারে। তখন round-robin নির্ভয়ে ব্যবহার করা যায়, আর একটি সার্ভার মারা গেলেও ব্যবহারকারী লগইন থাকেন। নিয়মটি মনে রাখুন: application server-কে stateless রাখুন, state-কে shared store-এ রাখুন — তাহলেই load balancing সত্যিকার অর্থে কাজ করে।

Chapter Summary

Reverse proxy সার্ভারের পাশে দাঁড়িয়ে request গ্রহণ করে: load balancing করে, TLS termination সামলায়, static content দেয়, আর ভেতরের সার্ভারকে আড়াল করে। Nginx-এ `upstream`, `server`, `location` ও `proxy_pass` দিয়ে এটি কনফিগার করা হয়। 502 মানে upstream শুনছে না; 504 মানে upstream দেরি করছে। Header ঠিকমতো না পাঠালে app ভুল IP দেখবে।

Revision Checklist

- ☐ Forward ও reverse proxy-র পার্থক্য বলতে পারি

- ☐ TLS termination ব্যাখ্যা করতে পারি
- ☐ Round-robin, least connections ও IP hash বলতে পারি
- ☐ Nginx-এ reverse proxy লিখতে পারি
- ☐ 502 ও 504-এর পার্থক্য জানি

PART IV • DEVOPS ENGINEERING MASTERY

Production Networking

প্রোডাকশন নেটওয়ার্কিং — ফায়ারওয়াল, VPN, CDN, ক্লাউড ও ট্রাবলশুটিং।

অধ্যায় ১৮ • Part IV — Production Networking

Chapter 18: Firewalls, Network Security and Segmentation — ফায়ারওয়াল, নেটওয়ার্ক নিরাপত্তা ও সেগমেন্টেশন

শুরুতে: একটি বাস্তব দৃশ্য

ঢাকার একটি ছোট সফটওয়্যার কোম্পানি তাদের প্রথম production সার্ভার চালু করল। ওয়েবসাইট চলছে, গ্রাহক আসছে — সব ঠিকঠাক। তিন সপ্তাহ পর এক সকালে দেখা গেল সার্ভারের CPU ১০০%, লগে হাজার হাজার ব্যর্থ SSH লগইন চেষ্টা, আর ডেটাবেস পোর্টটি ইন্টারনেট থেকে সরাসরি খোলা। কেউ ইচ্ছে করে আক্রমণ করেনি — কোম্পানিটি শুধু দরজা খোলা রেখেছিল। ইন্টারনেটে একটি সার্ভার চালু করার কয়েক মিনিটের মধ্যেই স্বয়ংক্রিয় স্ক্যানার সেটিকে খুঁজে পায়। এই অধ্যায়ের প্রশ্ন তাই খুব ব্যবহারিক: কোন দরজা খোলা থাকবে, কোনটি বন্ধ, আর কে কোন দরজা দিয়ে ঢুকতে পারবে — এই সিদ্ধান্ত কীভাবে নেওয়া হয়? ফায়ারওয়াল সেই সিদ্ধান্ত কার্যকর করার যন্ত্র।

Learning Objectives — শেখার উদ্দেশ্য

- ফায়ারওয়াল কী এবং কেন দরকার, তা প্রথম নীতি থেকে ব্যাখ্যা করতে পারবেন।
- Stateless ও stateful packet filtering-এর পার্থক্য ব্যাখ্যা করতে পারবেন।
- Host-based ও network firewall-এর ভূমিকা আলাদা করতে পারবেন।
- Ubuntu-তে UFW দিয়ে নিরাপদ নিয়ম লিখতে, যাচাই করতে ও ভুল শুধরাতে পারবেন।
- nftables কীভাবে কাজ করে, তা ধারণাগতভাবে বুঝতে পারবেন।
- VLAN ও network segmentation দিয়ে blast radius কমানোর নকশা করতে পারবেন।
- Least exposure নীতি অনুযায়ী একটি ছোট সার্ভারের ফায়ারওয়াল নীতি লিখতে পারবেন।

Prerequisites — পূর্বজ্ঞান

অধ্যায় ৬ (Ethernet ও switching), অধ্যায় ১০ (TCP/UDP ও port), অধ্যায় ১২ (routing ও NAT) এবং Vol 01-এর অধ্যায় ১৯ (Linux নিরাপত্তা)। `ss -tlnp` দিয়ে listening port দেখা ও `ip addr` জানা থাকলে ভালো হয়।

১৮.১ প্রথম নীতি: সীমানা না থাকলে নিরাপত্তা নেই

একটি নেটওয়ার্ক মূলত অনেকগুলো যন্ত্রের মধ্যে প্যাকেট আদান-প্রদান। কোনো প্যাকেট কোথা থেকে আসছে, কোথায় যাচ্ছে, কোন port-এ যাচ্ছে — এসব তথ্য প্যাকেটের header-এ লেখা থাকে। ফায়ারওয়াল হলো সেই সীমানায় বসা প্রহরী, যে প্রতিটি প্যাকেটের header পড়ে একটি নিয়মের তালিকার সঙ্গে মিলিয়ে সিদ্ধান্ত নেয়: যেতে দেবে, নাকি ফেলে দেবে। প্রহরী প্যাকেটের ভেতরের তথ্য বদলায় না; সে শুধু পথ আটকায় বা ছেড়ে দেয়। এই সরল ধারণাটিই পুরো ফায়ারওয়াল প্রযুক্তির ভিত্তি।

কেন সীমানা দরকার? কারণ প্রতিটি খোলা port একটি আক্রমণের পৃষ্ঠ (attack surface)। একটি ওয়েব সার্ভারের শুধু ৮০ ও ৪৪৩ খোলা থাকা উচিত, SSH শুধু নির্দিষ্ট উৎস থেকে। ডেটাবেস port (যেমন PostgreSQL-এর ৫৪৩২) কখনোই ইন্টারনেটে খোলা থাকা উচিত নয় — ডেটাবেসের সঙ্গে শুধু application সার্ভার কথা বলবে, তাও private নেটওয়ার্কে। এই নিয়মের নাম **least exposure**: যত কম প্রকাশ, তত কম ঝুঁকি।

> **CONCEPT:** ফায়ারওয়াল নিরাপত্তার একমাত্র স্তর নয় — এটি একটি স্তর মাত্র। শক্তিশালী পাসওয়ার্ড, আপডেট, key-based SSH, সঠিক permission — সব মিলিয়ে defense in depth হয়। ফায়ারওয়াল থাকলেই অ্যাপ্লিকেশনের দুর্বলতা ঢাকা পড়ে না।

১৮.২ Packet Filtering: Stateless বনাম Stateful

Stateless filtering প্রতিটি প্যাকেটকে আলাদাভাবে দেখে: source IP, destination IP, protocol, port — নিয়ম মিললে ছাড়ে, না মিললে ফেলে। প্যাকেটটি কোনো চলমান সংযোগের অংশ কি না, তা সে জানে না। ফলে ফেরত প্যাকেটের জন্য আলাদা নিয়ম লিখতে হয় — ভুল হওয়ার সুযোগ বেশি।

Stateful filtering সংযোগের অবস্থা (state) মনে রাখে। আপনি যখন বাইরে একটি অনুরোধ পাঠান, ফায়ারওয়াল সেই সংযোগটি একটি state table-এ লিখে রাখে। ফেরত প্যাকেট এলে দেখে — এটি কি পরিচিত সংযোগের উত্তর? হ্যাঁ হলে ছেড়ে দেয়, আলাদা নিয়ম লাগে না। প্রায় সব আধুনিক ফায়ারওয়াল stateful। Linux-এর netfilter (যার ওপর UFW ও nftables দাঁড়িয়ে) connection tracking (conntrack) দিয়ে এই state রাখে। বাস্তব অর্থ: `ufw allow out 443` না লিখেও আপনি HTTPS ব্যবহার করতে পারেন, কারণ ফেরত প্যাকেট established সংযোগের অংশ হিসেবে অনুমোদিত হয়।

তিনটি সিদ্ধান্ত মনে রাখুন: **ACCEPT** (যেতে দেওয়া), **DROP** (চুপচাপ ফেলে দেওয়া — প্রেরক কোনো উত্তর পায় না, সময় নষ্ট হয়) ও **REJECT** (ফেলে দেওয়া ও প্রেরককে জানানো যে বন্ধ)। বাইরের আক্রমণকারীর কাছে DROP সাধারণত ভালো, কারণ তাতে port-টি আছে কি না, তা নিশ্চিত হওয়া যায় না। ভেতরের বৈধ ব্যবহারকারীর জন্য REJECT ভালো, কারণ সে দ্রুত বুঝতে পারে সংযোগ বন্ধ, অপেক্ষা করতে হয় না।

চিত্র ১৮.১ — ফায়ারওয়াল কোথায় বসে

Internet → [Network Firewall / Cloud Security Group] → [Host Firewall (UFW/nftables)] → Application (Nginx :443, SSH :22)
→ Database tier (private network, কোনো public port নেই) — প্রতিটি স্তরে শুধু দরকারি port খোলা

১৮.৩ Host-Based বনাম Network Firewall

ধরন	কোথায় থাকে	কী রক্ষা করে	উদাহরণ
Host-based	প্রতিটি সার্ভারে	সেই একটি হোস্ট	UFW, nftables, Windows Firewall
Network	নেটওয়ার্কের সীমানায়	পুরো subnet বা VPC	Cloud security group, network ACL, hardware firewall

দুটোই দরকার। Network firewall ভুল করে ডেটাবেস port খুলে দিলেও host firewall তা আটকাতে পারে — এটাই defense in depth। উল্টোভাবে, host firewall ভুল করে বন্ধ হয়ে গেলেও network firewall বাইরের আক্রমণ ঠেকায়। Vol 01-এ আপনি UFW দেখেছেন; এখানে আমরা নীতি লেখার দিকে মনোযোগ দেব।

১৮.৪ UFW on Ubuntu: নিরাপদ ক্রম

UFW (Uncomplicated Firewall) হলো nftables/netfilter-এর ওপর একটি সহজ frontend। সবচেয়ে গুরুত্বপূর্ণ নিয়মটি মনে রাখুন: **SSH-এর নিয়ম যোগ করার আগে UFW enable করবেন না** — নইলে নিজেই সার্ভার থেকে বেরিয়ে যাবেন। নিরাপদ ক্রমটি হলো:

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow OpenSSH
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw enable
sudo ufw status verbose
```

ব্যাখ্যা: প্রথম দুই লাইনে ডিফল্ট নীতি — বাইরে থেকে আসা সবকিছু বন্ধ, ভেতর থেকে বাইরে যাওয়া সব খোলা (আপডেট, DNS ইত্যাদির জন্য)। `allow OpenSSH` SSH (port 22) খোলে। তারপর HTTP/HTTPS। `ufw enable` চালু করে। `status verbose` দিয়ে যাচাই করুন — যা লিখেছেন, তা-ই আছে কি না।

নির্দিষ্ট উৎস থেকে SSH সীমিত করা আরও নিরাপদ: `sudo ufw allow from 203.0.113.10 to any port 22` — শুধু ওই IP থেকে SSH আসতে পারবে। এখানে 203.0.113.10 একটি উদাহরণ IP (TEST-NET-3 রেঞ্জ); নিজের আসল IP বসান। কোনো নিয়ম মুছতে `sudo ufw delete allow 80/tcp`। নিয়মের নম্বর দেখে মুছতে `sudo ufw status numbered` তারপর `sudo ufw delete 2`।

> **WARNING:** Cloud VPS-এ UFW enable করার আগে cloud security group-এও SSH খোলা আছে কি না যাচাই করুন। দুই স্তরেই বন্ধ থাকলে সার্ভারে ঢোকার পথ থাকবে না — তখন শুধু provider-এর console (VNC) ভরসা। কোনো পরিবর্তনের আগে console access পরীক্ষা করে রাখুন।

১৮.৫ nftables Overview

nftables হলো Linux kernel-এর আধুনিক packet filtering framework — পুরোনো iptables-এর উত্তরসূরি। UFW-এর নিচে আসলে nftables/iptables নিয়মই তৈরি হয়। সরাসরি nftables লেখা UFW-এর চেয়ে শক্তিশালী: একটিমাত্র নিয়মে একাধিক port, set (IP-র তালিকা), rate limit লেখা যায়। তবে শেখার পর্যায়ে UFW যথেষ্ট; nftables তখনই ধরুন যখন UFW দিয়ে প্রকাশ করা যায় না এমন নীতি দরকার হয়। মূল ধারণাটি একই: table → chain (input/output/forward) → rule (match → verdict)। `nft list ruleset` দিয়ে বর্তমান নিয়ম দেখা যায়।

চিত্র ১৮.২ — Segmentation: blast radius কমানো

[Public subnet: Nginx reverse proxy] → [Private subnet: App servers] → [Isolated subnet: Database — শুধু app subnet থেকে 5432]

আক্রমণকারী proxy ভেদ করলেও ডেটাবেসে সরাসরি পৌঁছাতে পারে না — প্রতিটি সীমানায় ফায়ারওয়াল নিয়ম আছে

১৮.৬ VLAN ও Network Segmentation

Segmentation মানে নেটওয়ার্ককে ছোট ছোট অংশে ভাগ করা, যাতে একটি অংশ আক্রান্ত হলে ক্ষতি সেখানেই আটকে থাকে — একে **blast radius** কমানো বলে। **VLAN** (Virtual LAN) একই physical switch-এ একাধিক logical নেটওয়ার্ক তৈরি করে: HR বিভাগের কম্পিউটার ও সার্ভার এক VLAN-এ, অতিথি Wi-Fi আরেক VLAN-এ — তারা একে অপরকে দেখতেই পায় না, যদি না router/firewall অনুমতি দেয়। Cloud-এ একই কাজ করে VPC-র public/private subnet ও security group। নীতি একই: ওয়েব স্তর শুধু ইন্টারনেট থেকে ৪৪৩ পায়; অ্যাপ স্তর শুধু ওয়েব স্তর থেকে পায়; ডেটাবেস স্তর শুধু অ্যাপ স্তর থেকে পায়। কোনো স্তরই তার দরকারের বেশি কিছু দেখতে পায় না।

১৮.৭ বাস্তব উদাহরণ: একটি ছোট কোম্পানির নীতি

TechNova-র (অধ্যায় ২৪-এ বিস্তারিত) একটিমাত্র public সার্ভার: Nginx (443), SSH শুধু অফিস IP থেকে, আর কিছু নয়। `ss -tlnp` দিয়ে দেখা গেল ৩৩০৬ (MySQL) শুনছে 0.0.0.0-তে — অর্থাৎ সারা ইন্টারনেট থেকে পৌঁছানো যায়। সমাধান দুটি: প্রথমে MySQL-কে শুধু localhost (127.0.0.1) বা private IP-তে শোনানো, তারপর UFW-এ ৩৩০৬-এর কোনো allow নিয়ম না রাখা। শিক্ষা: ফায়ারওয়াল নিয়ম লেখার আগে প্রথমে জানুন কী শুনছে — `ss -tlnp` ছাড়া ফায়ারওয়াল নীতি অন্ধকারে লেখা।

১৮.৮ হাতে-কলমে অনুশীলন: সার্ভারের ফায়ারওয়াল নীতি

LAB: নিজের VM-এ (snapshot নিয়ে) এই ধাপগুলো করুন: (১) `ss -tlnp` দিয়ে listening port-এর তালিকা লিখুন; (২) `sudo ufw default deny incoming` ও `allow outgoing` দিন; (৩) `sudo ufw allow OpenSSH` দিন; (৪) `sudo ufw enable` করার আগে `ufw show added` দিয়ে নিয়ম যাচাই করুন; (৫) `sudo ufw status verbose` দিয়ে চূড়ান্ত অবস্থা লিখুন; (৬) অন্য টার্মিনাল থেকে SSH করে দেখুন এখনও ঢোকা যায় কি না। যাচাই: SSH কাজ করে, কিন্তু `nc -zv <VM-IP> 3306` (যদি MySQL না থাকে, যেকোনো বন্ধ port) ব্যর্থ হয়। Cleanup: `sudo ufw disable` ও `sudo ufw reset` — তবে শুধু নিজের lab VM-এ।

উদাহরণ আউটপুট (illustrative):

```
$ sudo ufw status verbose
Status: active
Default: deny (incoming), allow (outgoing)
To                         Action      From
--                         ------      ----
22/tcp (OpenSSH)           ALLOW IN    Anywhere
80/tcp                     ALLOW IN    Anywhere
443/tcp                    ALLOW IN    Anywhere
```

১৮.৯ Common Mistakes ও Troubleshooting

- **ভুল:** SSH নিয়ম ছাড়া `ufw enable` — নিজেকে বাইরে আটকে ফেলা। **সংশোধন:** Console/VNC দিয়ে `ufw disable` করুন; ভবিষ্যতে আগে `allow OpenSSH`।
- **ভুল:** `ufw allow 5432` দিয়ে ডেটাবেস সারা বিশ্বের জন্য খোলা। **সংশোধন:** শুধু app সার্ভারের IP থেকে অনুমতি দিন, বা private network-এ রাখুন।
- **ভুল:** ফায়ারওয়াল চালু আছে ভেবে নিশ্চিন্ত থাকা, কিন্তু নিয়মের ক্রম ভুল — প্রথম মেলা নিয়মই কার্যকর হয়। **সংশোধন:** `ufw status numbered` দিয়ে ক্রম দেখুন।
- **সমস্যা:** সার্ভিসে পৌঁছানো যাচ্ছে না — প্রথমে দেখুন সার্ভিস চলছে কি না (`systemctl status`), তারপর port শুনছে কি না (`ss -tlnp`), তারপর ফায়ারওয়াল (`ufw status`)। তিন স্তর আলাদা করে পরীক্ষা করুন।

১৮.১০ Security Considerations ও Best Practices

- Default deny incoming — শুধু যা দরকার, তা-ই খুলুন; প্রতিটি নিয়মের পাশে কেন খোলা, তা নোটে লিখুন।
- SSH-এ key-based login, password login বন্ধ (Vol 01, অধ্যায় ১৯), আর সম্ভব হলে SSH port-টি নির্দিষ্ট IP থেকে সীমিত করুন।

- ফায়ারওয়াল নিয়ম version control-এ রাখুন (একটি স্ক্রিপ্ট বা ডকুমেন্ট হিসেবে) — হাতে লেখা নিয়ম হারিয়ে যায়, audit করা যায় না।
- নিয়মিত `ss -tlnp` দিয়ে audit করুন: অপ্রত্যাশিত listening port মানেই তদন্ত।
- Log দেখুন: `sudo ufw logging on` দিয়ে denied প্যাকেট লগ করা যায়; অস্বাভাবিক scan চোখে পড়ে।

Knowledge Check — যাচাই করুন

1. Stateful firewall-এর প্রধান সুবিধা কী? A) প্রতিটি প্যাকেট আলাদাভাবে দেখে B) সংযোগের অবস্থা মনে রাখে বলে ফেরত প্যাকেটের জন্য আলাদা নিয়ম লাগে না C) প্যাকেটের ভেতরের তথ্য বদলে দেয় D) শুধু UDP ফিল্টার করে
2. UFW enable করার আগে সবচেয়ে গুরুত্বপূর্ণ ধাপ কোনটি? A) 443 খোলা B) SSH-এর allow নিয়ম যোগ করা C) সার্ভার reboot করা D) সব port বন্ধ করা
3. DROP ও REJECT-এর মধ্যে পার্থক্য কী? A) কোনো পার্থক্য নেই B) DROP চুপচাপ ফেলে দেয়, REJECT প্রেরককে জানিয়ে ফেলে দেয় C) DROP শুধু TCP-তে কাজ করে D) REJECT প্যাকেট গ্রহণ করে
4. ডেটাবেস port ইন্টারনেটে খোলা রাখা কেন বিপজ্জনক? A) ডেটাবেস ধীর হয় B) আক্রমণকারী সরাসরি ডেটাবেসে পৌঁছে যেতে পারে C) ব্যাকআপ বন্ধ হয় D) DNS কাজ করে না
5. VLAN-এর মূল উদ্দেশ্য কী? A) ইন্টারনেটের গতি বাড়ানো B) একই physical নেটওয়ার্কে আলাদা logical নেটওয়ার্ক তৈরি করে ভাগ করা C) Wi-Fi-এর রেঞ্জ বাড়ানো D) IP ঠিকানা বাঁচানো

Practical Assignment

আপনার lab VM-এর জন্য একটি ফায়ারওয়াল নীতি ডকুমেন্ট লিখুন: কোন port খোলা, কেন, কার জন্য — টেবিল আকারে। তারপর UFW-তে তা প্রয়োগ করে `ufw status verbose` ও `ss -tlnp`-এর আউটপুট দিয়ে যাচাই করুন। একটি ইচ্ছাকৃত ভুল করুন (যেমন ভুল port খোলা) ও তা ধরে শুধরানোর প্রক্রিয়াটি লিখুন। সমাধান Answer Guide-এ দেখুন।

Chapter Summary

ফায়ারওয়াল প্যাকেটের header পড়ে নিয়ম অনুযায়ী ছাড়ে বা ফেলে। Stateful firewall সংযোগের অবস্থা মনে রাখে। নিরাপদ নীতি হলো default deny incoming, শুধু দরকারি port খোলা, SSH আগে খুলে তারপর enable করা। Host ও network firewall একসঙ্গে defense in depth তৈরি করে, আর VLAN/segmentation blast radius কমায়। ফায়ারওয়াল লেখার আগে `ss -tlnp` দিয়ে জানুন আসলে কী শুনছে।

Revision Checklist

- ☐ Stateful বনাম stateless ব্যাখ্যা করতে পারি
- ☐ UFW-এ নিরাপদ ক্রমে নিয়ম লিখতে পারি
- ☐ DROP বনাম REJECT-এর পার্থক্য জানি
- ☐ Segmentation দিয়ে blast radius কমানোর নকশা করতে পারি
- ☐ `ss -tlnp` দিয়ে listening port audit করতে পারি

অধ্যায় ১৯ • Part IV — Production Networking

Chapter 19: VPN, Tunnels and Secure Remote Connectivity — VPN, টানেল ও নিরাপদ দূরবর্তী সংযোগ

শুরুতে: একটি বাস্তব দৃশ্য

চট্টগ্রামের একটি অফিসের ডেভেলপার রিমা ঢাকা থেকে বাড়িতে বসে কাজ করছেন। তার অ্যাপ্লিকেশন সার্ভার ও ডেটাবেস আছে কোম্পানির private নেটওয়ার্কে — ইন্টারনেট থেকে সরাসরি পৌঁছানো যায় না, ইচ্ছে করেই। তবু রিমা নিজের ল্যাপটপ থেকে ডেটাবেসে সংযোগ দিতে পারছেন, যেন তিনি অফিসের ভেতরেই বসে আছেন। কীভাবে? তার ল্যাপটপ ও অফিসের গেটওয়ের মধ্যে একটি এনক্রিপ্ট করা সুড়ঙ্গ (tunnel) তৈরি হয়েছে — পাবলিক ইন্টারনেটের ওপর দিয়ে যাওয়া প্যাকেটগুলো এমনভাবে মোড়ানো যে মাঝখানে কেউ পড়তে পারে না। এই সুড়ঙ্গের নাম VPN। এই অধ্যায়ে আমরা বুঝব সুড়ঙ্গ আসলে কী, কীভাবে তৈরি হয়, আর কখন VPN দরকার — আর কখন শুধু SSH-ই যথেষ্ট।

Learning Objectives — শেখার উদ্দেশ্য

- VPN ও tunneling-এর ধারণা প্রথম নীতি থেকে ব্যাখ্যা করতে পারবেন।
- Site-to-site ও remote-access VPN-এর পার্থক্য বলতে পারবেন।
- WireGuard-এর স্থাপত্য ও নিরাপত্তা মডেল ব্যাখ্যা করতে পারবেন।
- SSH tunneling (local port forwarding) দিয়ে নিরাপদে private সার্ভিসে পৌঁছাতে পারবেন।
- Split tunneling ও full tunneling-এর সুবিধা-অসুবিধা তুলনা করতে পারবেন।
- VPN-এর মধ্যে routing কীভাবে কাজ করে, তা ব্যাখ্যা করতে পারবেন।
- সাধারণ VPN সংযোগ সমস্যা নির্ণয় করতে পারবেন।

Prerequisites — পূর্বজ্ঞান

অধ্যায় ১২ (routing, NAT), অধ্যায় ১৬ (TLS, encryption, public/private key) ও অধ্যায় ১৮ (ফায়ারওয়াল)। SSH key-based login জানা থাকতে হবে (Vol 01, অধ্যায় ১৯)।

১৯.১ প্রথম নীতি: Tunnel মানে কী?

কল্পনা করুন আপনি একটি চিঠি পাঠাচ্ছেন। সাধারণ ডাকে চিঠিটি খোলা পোস্টকার্ডের মতো — পথে যে কেউ পড়তে পারে। Tunnel মানে চিঠিটিকে একটি সিল করা খামের ভেতরে ভরে, তার ওপর আরেকটি ঠিকানা লিখে পাঠানো। বাইরের খামে লেখা গন্তব্য শুধু সুড়ঙ্গের অপর প্রান্ত; ভেতরের আসল চিঠি শুধু প্রাপকই খুলতে পারে। নেটওয়ার্কে একই কাজ হয়: আসল প্যাকেটটিকে এনক্রিপ্ট করে আরেকটি প্যাকেটের ভেতরে ভরা হয় — বাইরের header-এ শুধু tunnel-এর দুই প্রান্তের ঠিকানা থাকে। মাঝের router-গুলো ভেতরের কিছু দেখে না, শুধু বাইরের প্যাকেটটি এগিয়ে দেয়।

এই মোড়ানো (encapsulation) ও এনক্রিপশন দুটো আলাদা জিনিস। Encapsulation ছাড়া সুড়ঙ্গ হয় না; encryption ছাড়া সুড়ঙ্গ নিরাপদ হয় না। ভালো VPN দুটোই করে।

> **CONCEPT:** VPN আপনার ট্রাফিককে "অদৃশ্য" করে না — শুধু পথের মাঝখানের পর্যবেক্ষকের কাছে অপাঠ্য করে। Tunnel-এর দুই প্রান্ত (আপনার ডিভাইস ও VPN সার্ভার) ভেতরের তথ্য দেখতে পায়। তাই VPN provider-কে বিশ্বাস করার প্রশ্নটি বাস্তব — নিজের সার্ভারে নিজের WireGuard চালানো আর অচেনা ফ্রি VPN ব্যবহার করা এক কথা নয়।

১৯.২ VPN-এর ধরন

ধরন	কে কার সঙ্গে যুক্ত হয়	ব্যবহার
Remote-access VPN	একজন ব্যবহারকারী ↔ কোম্পানির নেটওয়ার্ক	বাড়ি থেকে অফিসের private সার্ভিসে কাজ
Site-to-site VPN	দুটি অফিসের নেটওয়ার্ক ↔ একে অপরের সঙ্গে	ঢাকা অফিস ↔ চট্টগ্রাম অফিস, সবসময় চালু

Remote-access VPN সাধারণত চাহিদা অনুযায়ী চালু হয়; site-to-site সবসময় চালু থাকে, দুই পাশের gateway নিজেরাই tunnel রক্ষা করে। দুটোতেই মূল প্রশ্ন একই: কোন ট্রাফিক tunnel-এ যাবে, কোনটি যাবে না?

১৯.৩ Split Tunneling বনাম Full Tunneling

Full tunneling-এ আপনার সব ট্রাফিক VPN-এর ভেতর দিয়ে যায় — এমনকি YouTube দেখাও। সুবিধা: কোম্পানির নীতি সব ট্রাফিকে প্রযোজ্য হয়। অসুবিধা: VPN সার্ভার বোতল-ঘাড় (bottleneck) হয়, আর ব্যক্তিগত ট্রাফিকও কোম্পানির সার্ভার দিয়ে যায়। **Split tunneling**-এ শুধু কোম্পানির নেটওয়ার্কের (যেমন 10.0.0.0/8) ট্রাফিক tunnel-এ যায়; বাকিটা সরাসরি ইন্টারনেটে যায়। সুবিধা: দ্রুত, সার্ভারের চাপ কম। ঝুঁকি: ভুল routing নিয়মে কোম্পানির ট্রাফিক tunnel-এর বাইরে চলে যেতে পারে। কোনটি ঠিক, তা নির্ভর করে নিরাপত্তা নীতির ওপর — ব্যাংক সাধারণত full, ছোট স্টার্টআপ সাধারণত split বেছে নেয়।

চিত্র ১৯.১ — VPN tunnel কীভাবে কাজ করে

[ল্যাপটপ 10.8.0.2] —(এনক্রিপ্ট করা tunnel, ইন্টারনেটের ওপর দিয়ে)→ [VPN Gateway] —(private নেটওয়ার্ক)→ [App Server 10.0.1.20]

মাঝের router শুধু বাইরের প্যাকেট দেখে; ভেতরের গন্তব্য জানে না

১৯.৪ WireGuard Architecture

WireGuard একটি আধুনিক, ছোট ও দ্রুত VPN প্রোটোকল — Linux kernel-এর অংশ (kernel 5.6 থেকে, Ubuntu 24.04-এ অন্তর্ভুক্ত)। পুরোনো VPN-গুলোর তুলনায় এর কোডবেস অনেক ছোট, ফলে audit করা সহজ। মূল ধারণা: প্রতিটি peer-এর একটি public/private key জোড়া থাকে। সংযোগের আগে দুই পক্ষ একে অপরের public key জানে — এই পরিচয়ই প্রমাণীকরণ। কোনো username/password নয়, কোনো certificate authority নয়: key-ই পরিচয়।

WireGuard UDP-তে চলে (ডিফল্ট port 51820), প্রতিটি প্যাকেট এনক্রিপ্ট করে, আর কোনো ট্রাফিক না থাকলে চুপচাপ থাকে — "silent" থাকায় স্ক্যানার সহজে খুঁজে পায় না। প্রতিটি peer-এর একটি virtual interface (যেমন `wg0`) ও একটি tunnel IP (যেমন 10.8.0.2/32) থাকে। Routing নিয়ম বলে দেয় কোন destination tunnel-এ যাবে: `AllowedIPs = 10.0.0.0/8` মানে ওই রেঞ্জের ট্রাফিক এই peer-এর দিকে যাবে।

```
# /etc/wireguard/wg0.conf — সার্ভার পক্ষের উদাহরণ (illustrative)
[Interface]
```

```
PrivateKey = <সার্ভারের private key — কখনো শেয়ার করবেন না>
Address = 10.8.0.1/24
ListenPort = 51820

[Peer]
PublicKey = <ক্লায়েন্টের public key>
AllowedIPs = 10.8.0.2/32
```

ব্যাখ্যা: `[Interface]` নিজের পরিচয় ও ঠিকানা; `[Peer]` বিপরীত পক্ষের public key ও তাকে কোন IP দেওয়া হয়েছে। Key তৈরি করতে `wg genkey | tee privatekey | wg pubkey > publickey` — private key কখনো ফাইলের বাইরে যাবে না, কখনো চ্যাটে বা Git-এ দেবেন না।

> **WARNING:** WireGuard-এর private key হারানো মানে পরিচয় হারানো — যে key পাবে, সে-ই ওই peer সেজে ঢুকতে পারবে। Key ফাইলের permission `600` রাখুন, ব্যাকআপ এনক্রিপ্ট করে রাখুন। Lab-এর বাইরে production key কখনো উদাহরণ হিসেবে শেয়ার করবেন না।

১৯.৫ SSH Tunneling: ছোট কাজের বড় হাতিয়ার

পূর্ণ VPN সাজানোর আগে অনেক কাজ SSH tunnel-এই হয়ে যায়। **Local port forwarding**: আপনার ল্যাপটপের 5433 port-এ আসা সংযোগ SSH সার্ভারের ভেতর দিয়ে private ডেটাবেসের 5432-তে পৌঁছে যায়।

```
ssh -L 5433:10.0.1.30:5432 -N user@bastion.example.com
# তারপর নিজের ল্যাপটপে: psql -h localhost -p 5433
```

ব্যাখ্যা: `-L 5433:10.0.1.30:5432` মানে localhost:5433 → SSH সার্ভারের দৃষ্টিতে 10.0.1.30:5432। `-N` মানে shell নয়, শুধু tunnel। ডেটাবেস কখনো ইন্টারনেটে খোলা হয়নি — তবু আপনি নিরাপদে কাজ করলেন। এটাই bastion host প্যাটার্ন: একটি মাত্র সার্ভার SSH-এর জন্য খোলা, বাকি সব private।

১৯.৬ Routing Through VPN

VPN চালু হলে আপনার routing table বদলায়: tunnel-এর destination রেঞ্জের জন্য `wg0` interface-এর দিকে একটি route যোগ হয়। `ip route` দিয়ে দেখা যায়। সাধারণ ভুল: VPN সংযুক্ত, কিন্তু route যোগ হয়নি — তখন প্যাকেট tunnel-এ যায় না, সরাসরি ইন্টারনেটে গিয়ে ব্যর্থ হয়। নির্ণয়ের প্রথম ধাপ সবসময় `ip route` ও `ip addr show wg0`। আরেকটি সাধারণ সমস্যা MTU: tunnel header যোগ হওয়ায় প্যাকেট বড় হয়ে যায়; 1420-এর মতো ছোট MTU ব্যবহার করা হয়। `ping -M do -s 1400` দিয়ে MTU সমস্যা পরীক্ষা করা যায়।

চিত্র ১৯.২ — Split বনাম Full tunneling

Split: [কোম্পানির ট্রাফিক 10.0.0.0/8 → VPN] + [সাধারণ ইন্টারনেট → সরাসরি]

Full: [সব ট্রাফিক → VPN Gateway → ইন্টারনেট] — নীতি কঠোর, গতি তুলনামূলক কম

১৯.৭ বাস্তব উদাহরণ: অফিস ছাড়া অফিসের নেটওয়ার্ক

একটি স্টার্টআপের ডেটাবেস শুধু private subnet-এ। ডেভেলপাররা বাড়ি থেকে SSH tunnel দিয়ে কাজ করতেন, কিন্তু প্রত্যেকে আলাদা port মনে রাখতে হতো। WireGuard চালু করার পর প্রত্যেকে একটি tunnel IP পেলেন (10.8.0.x);

ডেটাবেসের ফায়ারওয়াল নিয়ম শুধু 10.8.0.0/24 থেকে সংযোগ নেয়। ফল: কোনো public port নেই, কোনো port মনে রাখা লাগে না, আর প্রতিটি সংযোগ এনক্রিপ্ট করা। শিক্ষা: VPN শুধু "দূর থেকে ঢোকা" নয় — এটি নেটওয়ার্ক ডিজাইনের অংশ, যা public port-এর সংখ্যা শূন্যের কাছাকাছি নিয়ে আসে।

১৯.৮ হাতে-কলমে অনুশীলন: SSH tunnel দিয়ে private সার্ভিসে পৌঁছানো

LAB: নিজের lab-এ দুটি VM নিন (বা একটি VM ও হোস্ট): (১) VM-এ একটি সার্ভিস শুধু localhost-এ চালান — যেমন `python3 -m http.server 8000 --bind 127.0.0.1`; (২) হোস্ট থেকে সরাসরি `curl http://VM-IP:8000` ব্যর্থ হয় কি না দেখুন; (৩) `ssh -L 8000:127.0.0.1:8000 -N user@VM-IP` দিয়ে tunnel করুন; (৪) হোস্টে `curl http://localhost:8000` চালিয়ে দেখুন কাজ করে। যাচাই: সরাসরি সংযোগ ব্যর্থ, tunnel-এর সংযোগ সফল। Cleanup: tunnel বন্ধ করতে SSH প্রসেসটি থামান (Ctrl+C), VM-এর http server থামান।

উদাহরণ আউটপুট (illustrative):

```
$ curl http://localhost:8000
<!DOCTYPE html>
<title>Directory listing for /</title>
# tunnel দিয়ে localhost-এর ভেতর দিয়ে VM-এর সার্ভিসে পৌঁছানো গেল
```

১৯.৯ Common Mistakes ও Troubleshooting

- **ভুল:** Private key শেয়ার করা বা Git-এ তোলা। **সংশোধন:** নতুন key জোড়া তৈরি করে পুরোনো peer মুছে ফেলুন।
- **সমস্যা:** VPN সংযুক্ত কিন্তু কিছুই খোলে না — প্রথমে `ip route` দেখুন: route আছে কি না। তারপর `ping` tunnel IP-তে।
- **সমস্যা:** বড় ফাইল আপলোড আটকে যায় — MTU সমস্যা সন্দেহ করুন; `wg0`-এর MTU 1420-এ নামিয়ে দেখুন।
- **সমস্যা:** DNS কাজ করে না — VPN-এর DNS সেটিং দেখুন; split tunnel-এ কোম্পানির DNS শুধু কোম্পানির ডোমেইনের জন্য ব্যবহার করুন।
- **ভুল:** ফ্রি VPN-কে নিরাপত্তার সমাধান ভাবা — ফ্রি VPN-এর ব্যবসায়িক মডেল প্রায়ই আপনার তথ্য। নিজের সার্ভারে WireGuard চালানোই পেশাদার পথ।

১৯.১০ Security Considerations ও Best Practices

- VPN gateway-এর ফায়ারওয়ালে শুধু VPN port (যেমন UDP 51820) খোলা রাখুন; বাকি সব বন্ধ।
- প্রতিটি ব্যবহারকারীর আলাদা key — একজন চলে গেলে শুধু তার peer মুছুন, সবার key বদলাতে হবে না।
- VPN-এর ভেতরেও ডেটাবেসে সরাসরি নয় — অ্যাপ্লিকেশন স্তর দিয়েই যান; least privilege tunnel-এর ভেতরেও প্রযোজ্য।
- Key rotation: নিয়মিত ব্যবধানে key বদলানোর নীতি রাখুন ও লিখে রাখুন।
- Log: কে কখন সংযুক্ত হলো, তা লগ করুন — তবে ব্যবহারকারীর ভেতরের ট্রাফিকের বিষয়বস্তু লগ করবেন না।

১৯.১১ WireGuard বনাম SSH Tunnel: কখন কোনটি?

দুটো হাতিয়ারই এনক্রিপ্ট করা পথ দেয়, কিন্তু ব্যবহারের জায়গা আলাদা। SSH tunnel একটি নির্দিষ্ট port-এর জন্য, সাময়িক কাজের জন্য সেরা — একটি ডেটাবেসে একবার ঢোকা, একটি অভ্যন্তরীণ ওয়েব পাতা দেখা। এটি চালু করতে কোনো সার্ভার কনফিগারেশন লাগে না, শুধু SSH access থাকলেই হয়। WireGuard পুরো নেটওয়ার্কের জন্য: একবার সংযুক্ত হলে সব private সার্ভিসে পৌঁছানো যায়, যেন আপনি অফিসের ভেতরে বসে আছেন। দলের সবাই নিয়মিত দূর থেকে কাজ করলে WireGuard; একবারের জরুরি কাজে SSH tunnel। আরেকটি পার্থক্য: SSH tunnel বন্ধ করলে সংযোগ শেষ, কিন্তু WireGuard সবসময় চালু থাকে — তাই WireGuard-এর key পরিচালনা ও ফায়ারওয়াল নীতি আরও সতর্কভাবে করতে হয়। ছোট দলের জন্য পরামর্শ: প্রথমে SSH tunnel দিয়ে bastion প্যাটার্ন শিখুন, তারপর WireGuard-এ যান — দুটোই জানা থাকলে পরিস্থিতি অনুযায়ী বেছে নিতে পারবেন।

Knowledge Check — যাচাই করুন

1. VPN tunnel-এ আসল প্যাকেটের কী হয়? A) মুছে ফেলা হয় B) এনক্রিপ্ট করে আরেকটি প্যাকেটের ভেতরে ভরা হয় C) সরাসরি পাঠানো হয় D) ছোট করে ভাগ করা হয়
2. WireGuard-এ পরিচয় (authentication) কীভাবে হয়? A) Username/password দিয়ে B) Public/private key জোড়া দিয়ে C) SMS কোড দিয়ে D) IP ঠিকানা দিয়ে
3. Split tunneling-এর অর্থ কী? A) সব ট্রাফিক VPN-এ যায় B) শুধু নির্দিষ্ট নেটওয়ার্কের ট্রাফিক VPN-এ যায়, বাকিটা সরাসরি যায় C) VPN দুই ভাগে ভাগ হয় D) দুটি VPN একসঙ্গে চলে
4. SSH local port forwarding-এ `-L 5433:10.0.1.30:5432`-এর অর্থ কী? A) 5433 port বন্ধ করা B) localhost:5433-এ আসা সংযোগ 10.0.1.30:5432-তে পাঠানো C) 5433 port-এ সার্ভার চালানো D) 5432 port মুছে ফেলা
5. VPN সংযুক্ত কিন্তু কিছু খুলছে না — প্রথমে কী দেখবেন? A) ব্রাউজার বদলাবেন B) `ip route` দিয়ে route আছে কি না দেখবেন C) কম্পিউটার restart দেবেন D) পাসওয়ার্ড বদলাবেন

Practical Assignment

আপনার lab-এর জন্য একটি ছোট VPN নকশা লিখুন: কে কে সংযুক্ত হবে, তাদের tunnel IP কী হবে, কোন রেঞ্জ tunnel-এ যাবে (split না full — কেন), ফায়ারওয়ালে কোন port খোলা থাকবে। তারপর SSH tunnel দিয়ে একটি localhost-only সার্ভিসে পৌঁছে তার প্রমাণ (কমান্ড ও আউটপুট) লিখুন। সমাধান Answer Guide-এ দেখুন।

Chapter Summary

VPN হলো পাবলিক নেটওয়ার্কের ওপর এনক্রিপ্ট করা সুড়ঙ্গ: আসল প্যাকেট আরেকটি প্যাকেটের ভেতরে ভরা থাকে। WireGuard key-ভিত্তিক, ছোট ও দ্রুত — আধুনিক পছন্দ। SSH tunneling ছোট কাজের জন্য যথেষ্ট। Split বনাম full tunneling নীতির সিদ্ধান্ত; routing ও MTU-ই সবচেয়ে সাধারণ সমস্যার জায়গা। ভালো VPN নকশা public port কমিয়ে আনে।

Revision Checklist

- ☐ Tunnel ও encapsulation ব্যাখ্যা করতে পারি
- ☐ WireGuard-এর key মডেল বুঝি
- ☐ SSH `-L` দিয়ে tunnel করতে পারি

- ☐ Split বনাম full tunneling তুলনা করতে পারি
- ☐ `ip route` দিয়ে VPN সমস্যা নির্ণয় করতে পারি

অধ্যায় ২০ • Part IV — Production Networking

Chapter 20: CDN, Edge Networks and Global Traffic Delivery — CDN, এজ নেটওয়ার্ক ও বৈশ্বিক ট্রাফিক সরবরাহ

শুরুতে: একটি বাস্তব দৃশ্য

ঢাকার একজন ব্যবহারকারী একটি আন্তর্জাতিক সংবাদ সাইট খুললেন — পাতা এলো এক সেকেন্ডেরও কমে। সাইটটির মূল সার্ভার (origin) যুক্তরাষ্ট্রে। আলোর গতিতে গেলেও ঢাকা থেকে যুক্তরাষ্ট্রে প্যাকেট যেতে-আসতে ২০০ মিলিসেকেন্ডের বেশি লাগে — তবু পাতা এত দ্রুত এলো কীভাবে? কারণ পাতাটি যুক্তরাষ্ট্র থেকে আসেনি। এসেছে ঢাকার কাছের একটি সার্ভার থেকে — একটি CDN edge location থেকে, যেখানে পাতার কপি আগেই রাখা ছিল। এই অধ্যায়ে আমরা বুঝব: ইন্টারনেট কীভাবে "কাছে" আনা হয়, cache কীভাবে কাজ করে, আর CDN কখন সাহায্য করে, কখন বিপদে ফেলে।

Learning Objectives — শেখার উদ্দেশ্য

- CDN কেন দরকার — latency ও origin load-এর দৃষ্টিতে — ব্যাখ্যা করতে পারবেন।
- Edge location, origin ও cache-এর সম্পর্ক ব্যাখ্যা করতে পারবেন।
- Cache-Control header পড়ে কতক্ষণ cache থাকবে, তা বলতে পারবেন।
- DNS কীভাবে ব্যবহারকারীকে কাছের edge-এ পাঠায়, তা ব্যাখ্যা করতে পারবেন।
- Anycast-এর ধারণা ব্যাখ্যা করতে পারবেন।
- Cache invalidation ও origin protection-এর নীতি বলতে পারবেন।
- Cloudflare-এর মতো CDN পরিষেবার ভূমিকা মূল্যায়ন করতে পারবেন।

Prerequisites — পূর্বজ্ঞান

অধ্যায় ১৩ (DNS), অধ্যায় ১৫ (HTTP, header, status code) ও অধ্যায় ১৭ (reverse proxy)। `curl -I` দিয়ে header দেখা জানা থাকলে ভালো।

২০.১ প্রথম নীতি: দূরত্বই latency

পদার্থবিজ্ঞানের সীমা কেউ বদলাতে পারে না: ফাইবারে আলো প্রায় ২০০,০০০ কিমি/সেকেন্ডে চলে। ঢাকা থেকে সিঙ্গাপুর প্রায় ৩,০০০ কিমি — শুধু যাওয়া-আসাতেই ৩০ মিলিসেকেন্ড, তার ওপর router-এর প্রতিটি hop-এ প্রক্রিয়াকরণ সময়। একটিমাত্র HTTP অনুরোধে DNS, TCP handshake, TLS handshake — কয়েকটি round trip লাগে। দূরত্ব কমানো ছাড়া latency কমানোর উপায় নেই। CDN-এর মূল কৌশল তাই একটাই: **বিষয়বস্তুকে ব্যবহারকারীর কাছে নিয়ে যাওয়া**, ব্যবহারকারীকে বিষয়বস্তুর কাছে নয়।

দ্বিতীয় কারণ: origin সুরক্ষা। লাখ লাখ ব্যবহারকারী সরাসরি origin সার্ভারে আঘাত করলে সেটি ভেঙে পড়ে। CDN মাঝখানে দাঁড়িয়ে বেশিরভাগ অনুরোধ নিজেই মেটায় — origin-এ পৌঁছায় শুধু cache-এ না থাকা অনুরোধ।

> **CONCEPT:** CDN মূলত একটি বৈশ্বিক reverse proxy + cache। অধ্যায় ১৭-এর Nginx reverse proxy যেমন আপনার সার্ভারের সামনে বসে, CDN তেমনই পুরো বিশ্বজুড়ে বসে — পার্থক্য শুধু পরিসর ও DNS-ভিত্তিক routing-এ।

২০.২ Edge, Origin ও Cache কীভাবে কাজ করে

ব্যবহারকারী `example.com/logo.png` চাইলে: (১) DNS তাকে কাছের edge server-এর IP দেয়; (২) edge দেখে — এই ফাইল cache-এ আছে কি না? থাকলে সরাসরি দেয় (cache HIT); (৩) না থাকলে edge origin থেকে ফাইলটি এনে (cache MISS), কপি রেখে, তারপর দেয়; (৪) পরের ব্যবহারকারী একই ফাইল চাইলে HIT পায়। `Cache-Control: max-age=3600` মানে ফাইলটি ৩৬০০ সেকেন্ড (১ ঘণ্টা) পর্যন্ত cache-এ বৈধ। সময় শেষ হলে edge আবার origin যাচাই করে (revalidate)।

কী cache করা যায়? ছবি, CSS, JavaScript, ভিডিও — স্থির (static) বিষয়বস্তু। কী cache করা যায় না? লগইন করা ব্যবহারকারীর ব্যক্তিগত পাতা, শপিং কার্ট, `Cache-Control: private, no-store` বা `Set-Cookie`-যুক্ত উত্তর। ভুল করে ব্যক্তিগত তথ্য cache করলে এক ব্যবহারকারীর তথ্য আরেকজন দেখতে পায় — গুরুতর নিরাপত্তা ভুল।

চিত্র ২০.১ — CDN অনুরোধের প্রবাহ

[User, ঢাকা] → DNS → [Edge, সিঙ্গাপুর — cache HIT হলে এখানেই উত্তর] → (MISS হলে) → [Origin, যুক্তরাষ্ট্র] → Edge কপি রাখে → User

দ্বিতীয় ব্যবহারকারী একই ফাইল চাইলে origin-এ আর যাওয়া লাগে না

২০.৩ DNS-Based Traffic Delivery ও Anycast

CDN কীভাবে জানে ব্যবহারকারী কোথায়? DNS উত্তরের সময়: একই ডোমেইনের জন্য ভিন্ন অঞ্চলের ব্যবহারকারীকে ভিন্ন edge IP দেওয়া হয় — একে geo-DNS বলে। আরও গভীরে আছে **anycast**: একই IP ঠিকানা বিশ্বের অনেক জায়গায় ঘোষণা করা হয়; BGP routing স্বয়ংক্রিয়ভাবে ব্যবহারকারীকে সবচেয়ে কাছের (routing-এর দৃষ্টিতে সস্তা) location-এ পাঠায়। DNS সার্ভারগুলো (যেমন 1.1.1.1, 8.8.8.8) এভাবেই কাজ করে। Anycast-এর সুবিধা: একটি location ভেঙে গেলেও ট্রাফিক স্বয়ংক্রিয়ভাবে অন্য location-এ যায়।

২০.৪ Cache-Control Header পড়া

```
curl -I https://example.com/style.css
# উদাহরণ আউটপুট (illustrative):
# HTTP/2 200
# cache-control: public, max-age=31536000, immutable
# cf-cache-status: HIT
# age: 4821
```

ব্যাখ্যা: `max-age=31536000` মানে ১ বছর cache বৈধ — ফাইলের নামে version/hash থাকলে (যেমন `style.a1b2.css`) এটি নিরাপদ, কারণ ফাইল বদলালে নামই বদলে যায়। `immutable` মানে ফাইলটি কখনো বদলাবে না। `cf-cache-status: HIT` বলছে উত্তরটি CDN cache থেকে এসেছে; `age: 4821` মানে ফাইলটি ৪৮২১ সেকেন্ড ধরে cache-এ আছে। MISS হলে origin থেকে আনা হয়েছে।

২০.৫ Invalidation: ভুল cache শুধরানো

Cache-এর সবচেয়ে কঠিন সমস্যা: পুরোনো কপি। ফাইল বদলে গেলেও edge পুরোনো কপি দিতে থাকতে পারে — যতক্ষণ max-age শেষ না হয়। সমাধান তিনটি: (১) ফাইলের নামে hash/version দেওয়া (সবচেয়ে ভালো); (২) ছোট max-age রাখা (ঘন ঘন বদলানো বিষয়বস্তুর জন্য); (৩) **purge/invalidation** — CDN-কে বলা যে নির্দিষ্ট URL-এর cache মুছে ফেলো। Purge বিশ্বজুড়ে ছড়াতে কিছু সময় লাগে — "instant" বলে কিছু নেই। তাই জরুরি সংশোধনের জন্য purge-এর ওপর ভরসা না করে versioned filename ব্যবহার করাই পেশাদার অভ্যাস।

২০.৬ Origin Protection ও নিরাপত্তা

CDN শুধু গতির জন্য নয় — নিরাপত্তার ঢালও। DDoS আক্রমণের ট্রাফিক edge-এ শোষিত হয়, origin পর্যন্ত পৌঁছায় না। আরও একটি নিয়ম: origin সার্ভারের আসল IP গোপন রাখুন — কেউ সরাসরি origin-এ আক্রমণ করতে না পারে। Origin শুধু CDN-এর IP রেঞ্জ থেকে সংযোগ নেবে (ফায়ারওয়াল নিয়ম দিয়ে), আর সবাই CDN দিয়ে আসবে। নিজের origin IP GitHub-এ, পুরোনো DNS রেকর্ডে বা ইমেইল header-এ ফাঁস হলে এই সুরক্ষা ভেঙে যায়।

চিত্র ২০.২ — CDN ছাড়া বনাম CDN সহ

ছাড়া: সব User → সরাসরি Origin (দূরত্ব বেশি, load বেশি, আক্রমণ সরাসরি)

সহ: User → কাছের Edge (HIT: দ্রুত উত্তর) → প্রয়োজনে Origin (শুধু CDN-এর IP থেকে গ্রহণ করে)

২০.৭ Cloudflare Ecosystem Overview

Cloudflare সবচেয়ে পরিচিত CDN/DNS পরিষেবাগুলোর একটি — তবে একমাত্র নয় (Akamai, Fastly, AWS CloudFront-ও আছে)। Cloudflare-এর বিনামূল্যের স্তরে DNS, CDN cache, TLS সার্টিফিকেট ও মৌলিক DDoS সুরক্ষা পাওয়া যায় — ছোট প্রকল্পের জন্য যথেষ্ট। গুরুত্বপূর্ণ বোঝাপড়া: Cloudflare ব্যবহার করলে আপনার ট্রাফিক Cloudflare-এর সার্ভারের ভেতর দিয়ে যায় — তারা TLS শেষ করে (terminate) আবার নতুন TLS-এ origin-এ পাঠায়। ফলে origin-এর সঙ্গে সংযোগটিও HTTPS হতে হবে (Full Strict মোড), নইলে মাঝের অংশটি এনক্রিপ্ট করা থাকে না। কোনো provider-ই "জাদু" নয় — প্রতিটির সীমা ও খরচের মডেল পড়ে বুঝে নিন।

২০.৮ বাস্তব উদাহরণ: ছবি বদলাল কিন্তু পাতায় পুরোনো ছবি

একটি ই-কমার্স সাইট নতুন পণ্যের ছবি আপলোড করল, কিন্তু গ্রাহকরা পুরোনো ছবি দেখছেন। কারণ: ছবির URL একই, আর `max-age=86400` (১ দিন) cache করা ছিল। সমাধান: ছবির নামে version যোগ করা (`product-v2.jpg`) ও CDN purge করা। শিক্ষা: cache নীতি লেখার সময়ই ভাবুন — "এই ফাইল বদলালে কী হবে?" বদলায় এমন ফাইলে লম্বা max-age দেওয়া ভুল।

২০.৯ হাতে-কলমে অনুশীলন: Header পড়ে cache বোঝা

LAB: নিজের কম্পিউটারে: (১) `curl -I https://example.com` চালিয়ে status ও header লিখুন; (২) `cache-control`, `age`, `cf-cache-status` বা `x-cache` header খুঁজুন; (৩) একই কমান্ড দুবার চালিয়ে দ্বিতীয়বার `age` বেড়েছে কি না দেখুন; (৪) একটি ছবির URL-এ `curl -I` চালিয়ে তার cache নীতি লিখুন; (৫) নিজের Nginx সার্ভারে (অধ্যায় ১৭) একটি static ফাইলে `Cache-Control: max-age=3600` header যোগ করে যাচাই করুন। যাচাই: HIT/MISS-এর অর্থ ব্যাখ্যা করতে পারেন। Cleanup: কিছু মোছার নেই — শুধু পর্যবেক্ষণ।

উদাহরণ আউটপুট (illustrative):

```
$ curl -I https://example.com/
HTTP/2 200
content-type: text/html
cache-control: max-age=604800
# 604800 সেকেন্ড = 7 দিন — এই উত্তরটি 7 দিন পর্যন্ত cache-এ থাকতে পারে
```

২০.১০ Common Mistakes ও Troubleshooting

- **ভুল:** ব্যক্তিগত পাতায় লম্বা cache — এক ব্যবহারকারীর তথ্য আরেকজন দেখে। **সংশোধন:** `Cache-Control: private, no-store`।
- **ভুল:** "DNS propagation" বলে ৪৮ ঘণ্টা অপেক্ষা করা — আসলে পুরোনো রেকর্ডের TTL শেষ না হওয়া পর্যন্ত cache থাকে। **সংশোধন:** পরিবর্তনের আগে TTL কমিয়ে রাখুন।
- **সমস্যা:** CDN-এর পেছনে ব্যবহারকারীর আসল IP দেখা যায় না — `X-Forwarded-For` header পড়ুন ও Nginx-এ `real_ip` কনফিগার করুন।
- **সমস্যা:** HTTPS কাজ করে কিন্তু origin-এ HTTP — Cloudflare-এ "Flexible" মোডের ফাঁদ; "Full (Strict)" ব্যবহার করুন।

২০.১১ Security Considerations ও Best Practices

- Origin IP গোপন রাখুন; origin শুধু CDN-এর IP রেঞ্জ থেকে সংযোগ নেবে।
- Static ফাইলে versioned filename + লম্বা max-age; বদলায় এমন বিষয়ে ছোট max-age।
- Cookie-যুক্ত উত্তরে cache সতর্কভাবে ব্যবহার করুন; `Vary: Cookie` বুঝে লিখুন।
- CDN-এর WAF/rate limit-কে একমাত্র নিরাপত্তা ভাববেন না — origin-এর নিজের ফায়ারওয়ালও থাকতে হবে।
- খরচ: CDN-এর egress ও request-ভিত্তিক খরচ পড়ুন; cache hit ratio বাড়ানো মানেই খরচ কমানো।

২০.১২ CDN ছাড়া নিজের Cache: Nginx-এ Static Caching

CDN না থাকলেও cache-এর ধারণা কাজ করে — নিজের Nginx সার্ভারেই। একটি ছবি বা CSS ফাইল প্রথমবার ডিস্ক থেকে পড়া হয়, তারপর Nginx-এর cache-এ থাকে; পরের অনুরোধে ডিস্ক ছোঁয়া লাগে না। `proxy_cache_path /var/cache/nginx levels=1:2 keys_zone=mycache:10m max_size=1g inactive=60m;` দিয়ে cache-এর জায়গা নির্ধারণ করা যায়, আর `location`-এ `proxy_cache mycache;` ও `add_header Cache-Control "public, max-age=3600";` দিয়ে নীতি লেখা যায়। তবে মনে রাখুন: নিজের cache শুধু একটি সার্ভারে কাজ করে, বিশ্বজুড়ে নয় — CDN-এর বিকল্প নয়, বরং তার আগের ধাপ। শেখার জন্য এটি চমৎকার: নিজের সার্ভারে cache HIT ও MISS দেখে CDN-এর আচরণ হাতে-কলমে বোঝা যায়। আর cache-এর আকারের দিকে খেয়াল রাখুন — `max_size` ছাড়া cache ডিস্ক ভরে ফেলতে পারে, যা অধ্যায় ১৬-এর (Vol 01) disk-full সমস্যারই আরেক রূপ।

২০.১৩ CDN-এর সীমা: কখন CDN সাহায্য করে না

CDN সব সমস্যার সমাধান নয়। প্রথমত, dynamic বিষয়বস্তু — প্রতিটি ব্যবহারকারীর জন্য আলাদা তৈরি হওয়া পাতা — cache করা যায় না, তাই CDN সেখানে শুধু TLS termination ও DDoS সুরক্ষা দেয়, গতির সুবিধা সীমিত। দ্বিতীয়ত, প্রথমবারের অনুরোধ (cold cache) সবসময় origin পর্যন্ত যায় — হঠাৎ ভাইরাল হওয়া বিষয়বস্তুর প্রথম ঢেউ origin-কেই

সামলাতে হয়। তৃতীয়ত, CDN-এর নিজের বিভ্রাট হলে আপনার সাইটও প্রভাবিত হতে পারে — তাই CDN-কে bypass করে সরাসরি origin-এ যাওয়ার জরুরি পথ (DNS পরিবর্তনের মাধ্যমে) আগে থেকে পরিকল্পনা করে রাখুন। সবশেষে, আইনি প্রশ্ন: কিছু দেশের তথ্য সেই দেশের ভেতরেই রাখতে হয় — CDN-এর edge সেই দেশের বাইরে হলে নিয়ম ভাঙা হতে পারে। সিদ্ধান্ত: CDN ব্যবহার করুন, কিন্তু তার সীমা জেনে — আর origin-কে সবসময় নিজের পায়ে দাঁড়াতে সক্ষম রাখুন।

Knowledge Check — যাচাই করুন

1. CDN-এর প্রধান কাজ কী? A) ডেটাবেস চালানো B) বিষয়বস্তু ব্যবহারকারীর কাছের edge server থেকে দিয়ে latency ও origin load কমানো C) DNS রেকর্ড তৈরি করা D) পাসওয়ার্ড সংরক্ষণ করা
2. `Cache-Control: max-age=3600`-এর অর্থ কী? A) 3600 বার cache করা যাবে B) 3600 সেকেন্ড পর্যন্ত cache বৈধ C) 3600 জন ব্যবহারকারী দেখতে পারবে D) ফাইল 3600 byte
3. Cache HIT বলতে কী বোঝায়? A) Origin সার্ভার বন্ধ B) Edge server cache থেকে উত্তর দিয়েছে, origin-এ যায়নি C) ফাইল মুছে গেছে D) DNS ব্যর্থ হয়েছে
4. Anycast-এর সুবিধা কী? A) একটি IP অনেক জায়গায় থাকে বলে ব্যবহারকারী কাছের location-এ যায় B) IP ঠিকানা বাঁচে C) DNS দ্রুত হয় D) TLS সস্তা হয়
5. ব্যক্তিগত পাতা cache করা কেন বিপজ্জনক? A) পাতা ধীর হয় B) এক ব্যবহারকারীর তথ্য আরেক ব্যবহারকারী দেখতে পারে C) সার্ভার বন্ধ হয় D) ছবি আসে না

Practical Assignment

একটি কাল্পনিক সংবাদ সাইটের জন্য cache নীতি লিখুন: হোমপেজ, ছবি, CSS/JS, লগইন করা ব্যবহারকারীর পাতা — প্রতিটির `Cache-Control` মান কী হবে, কেন? তারপর `curl -I` দিয়ে তিনটি বাস্তব ওয়েবসাইটের header পরীক্ষা করে তাদের নীতি আপনার নকশার সঙ্গে তুলনা করুন। সমাধান Answer Guide-এ দেখুন।

Chapter Summary

CDN বিষয়বস্তুকে ব্যবহারকারীর কাছে নিয়ে আসে: edge cache HIT দ্রুত উত্তর দেয়, MISS হলে origin থেকে আনে। Cache-Control বলে দেয় কতক্ষণ cache বৈধ; versioned filename-ই invalidation-এর নির্ভরযোগ্য পথ। Anycast ও geo-DNS ব্যবহারকারীকে কাছের edge-এ পাঠায়। CDN গতির পাশাপাশি DDoS ঢাল ও origin সুরক্ষা দেয় — তবে origin IP গোপন রাখা ও সঠিক TLS মোড জরুরি।

Revision Checklist

- ☐ Edge, origin, HIT/MISS ব্যাখ্যা করতে পারি
- ☐ Cache-Control header পড়তে পারি
- ☐ Anycast-এর ধারণা বুঝি
- ☐ ব্যক্তিগত বিষয়বস্তু কেন cache করা যাবে না, জানি
- ☐ Origin protection-এর নীতি বলতে পারি

অধ্যায় ২১ • Part IV — Production Networking

Chapter 21: Linux Network Configuration and Diagnostics — Linux নেটওয়ার্ক কনফিগারেশন ও রোগনির্ণয়

শুরুতে: একটি বাস্তব দৃশ্য

নতুন সার্ভারে SSH করতে গিয়ে দেখলেন সংযোগ হচ্ছে না। আগের অধ্যায়গুলোতে আপনি শিখেছেন OSI মডেল, TCP handshake, DNS, ফায়ারওয়াল — এখন প্রশ্ন: এই জ্ঞান দিয়ে হাতে-কলমে কীভাবে খুঁজে বের করবেন কোথায় আটকে আছে? উত্তরটি একগুচ্ছ ছোট কমান্ডে: `ip addr`, `ip route`, `ss`, `ping`, `dig`, `curl`, `tcpdump`। প্রতিটি কমান্ড একটি নির্দিষ্ট স্তরে প্রশ্ন করে — আর তাদের উত্তরগুলো একসঙ্গে মিলিয়ে সমস্যার জায়গাটি সংকুচিত হয়। এই অধ্যায়টি আপনার দৈনন্দিন হাতিয়ারের বাক্স: কোন কমান্ড কখন, কেন, আর আউটপুটের অর্থ কী।

Learning Objectives — শেখার উদ্দেশ্য

- `ip addr`, `ip link`, `ip route` দিয়ে interface, ঠিকানা ও routing দেখতে ও ব্যাখ্যা করতে পারবেন।
- `ss` দিয়ে listening port ও সংযোগের অবস্থা নির্ণয় করতে পারবেন।
- `ping`, `traceroute` / `tracepath`, `mtr` দিয়ে পথ ও latency পরীক্ষা করতে পারবেন।
- `dig` ও `nslookup` দিয়ে DNS যাচাই করতে পারবেন।
- `curl` ও `wget` দিয়ে HTTP স্তর পরীক্ষা করতে পারবেন।
- `tcpdump` দিয়ে নিজের lab interface-এ প্যাকেট দেখতে পারবেন।
- Netplan দিয়ে Ubuntu-তে স্থায়ী নেটওয়ার্ক কনফিগারেশন লিখতে পারবেন।

Prerequisites — পূর্বজ্ঞান

অধ্যায় ৬–১৪ (Ethernet, IP, subnetting, routing, DNS, ICMP) ও Vol 01-এর টার্মিনাল অধ্যায়গুলো। নিজের VM-এ root/sudo অধিকার থাকতে হবে।

২১.১ প্রথম নীতি: প্রতিটি কমান্ড একটি প্রশ্ন

Troubleshooting-এর সময় এলোমেলো কমান্ড চালাবেন না — প্রতিটি কমান্ড একটি নির্দিষ্ট প্রশ্নের উত্তর দেয়। `ip addr` জিজ্ঞেস করে: "আমার interface-গুলো কী, তাদের IP কী?" `ip route`: "প্যাকেট কোন পথে যাবে?" `ss -tlnp`: "কোন port-এ কে শুনছে?" `ping`: "ওই হোস্টে পৌঁছানো যায় কি?" `dig`: "নামটি কোন IP-তে মেলে?" `curl -v`: "HTTP স্তরে কী ঘটছে?" এই প্রশ্নগুলো নিচের স্তর থেকে ওপরের দিকে সাজান — Link → IP → Route → DNS → TCP → HTTP। নিচের স্তর ভাঙা থাকলে ওপরের পরীক্ষা অর্থহীন।

> **CONCEPT:** পুরোনো `ifconfig`, `route`, `netstat` কমান্ডগুলো এখনও অনেক টিউটোরিয়ালে দেখা যায়, কিন্তু Ubuntu-তে এগুলো deprecated — আধুনিক হাতিয়ার `ip` ও `ss`। `ip` কমান্ড iproute2 প্যাকেজের অংশ; `ss` socket-এর তথ্য সরাসরি kernel থেকে পড়ে, তাই `netstat`-এর চেয়ে দ্রুত ও নির্ভুল।

২১.২ ip: Interface, Address ও Route

```
ip addr show          # interface ও IP ঠিকানা
ip link show          # interface-এর অবস্থা (UP/DOWN, MTU, MAC)
ip route show         # routing table
ip route get 8.8.8.8  # এই গন্তব্যে কোন পথে যাবে, কোন interface দিয়ে
```

ব্যাখ্যা: `ip addr`-এ `inet 192.168.1.10/24` মানে interface-এর IPv4 ঠিকানা ও prefix; `scope global` মানে এটি সাধারণ ব্যবহারের ঠিকানা, `scope host` (127.0.0.1) মানে শুধু নিজের ভেতরে। `ip link`-এ `state UP` মানে interface চালু; `mtu 1500` মানে সর্বোচ্চ প্যাকেট আকার। `ip route get` সবচেয়ে উপকারী — এটি routing table-এর হিসাব করে বলে দেয় কোন interface ও gateway ব্যবহার হবে, কোনো প্যাকেট না পাঠিয়েই।

অস্থায়ী পরিবর্তন (reboot-এ মুছে যায়): `sudo ip addr add 10.0.0.5/24 dev eth0`, `sudo ip link set eth0 up`। স্থায়ী পরিবর্তন Netplan-এ লিখতে হয় (নিচে দেখুন)।

২১.৩ ss: কে শুনছে, কে সংযুক্ত

```
ss -tlnp   # TCP listening port, process-সহ
ss -tulpn  # TCP + UDP listening
ss -tn state established  # চলমান TCP সংযোগ
```

ব্যাখ্যা: `-t` TCP, `-u` UDP, `-l` listening, `-n` নাম নয়, সংখ্যা দেখাও (DNS lookup এড়িয়ে দ্রুত হয়), `-p` কোন process। `0.0.0.0:80` মানে সব interface-এ শোনা হচ্ছে; `127.0.0.1:5432` মানে শুধু localhost — বাইরে থেকে পৌঁছানো যাবে না (ইচ্ছাকৃত নিরাপত্তা)। `ss -tn`-এ `ESTAB` মানে সংযোগ চলছে, `TIME-WAIT` মানে বন্ধ হওয়ার পর অপেক্ষা করছে — স্বাভাবিক। হাজার হাজার `SYN-RECV` দেখলে SYN flood আক্রমণ সন্দেহ করুন।

চিত্র ২১.১ — নির্ণয়ের সিঁড়ি (নিচ থেকে ওপরে)

1. ip link — কেবল/interface চালু? → 2. ip addr — IP আছে? → 3. ip route — gateway আছে? → 4. ping gateway → 5. ping 8.8.8.8 → 6. dig — DNS কাজ করে? → 7. curl — HTTP কাজ করে?

যে ধাপে প্রথম ব্যর্থতা, সমস্যা সেখানেই

২১.৪ ping, traceroute, mtr

`ping` ICMP Echo পাঠায় — পৌঁছানো যায় কি না, আর কত সময় লাগে (RTT)। `ping -c 4 example.com` চারটি প্যাকেট পাঠায়। 0% loss ও কম RTT ভালো; 100% loss মানে পৌঁছানো যাচ্ছে না — তবে সতর্ক থাকুন: অনেক ফায়ারওয়াল ICMP বন্ধ করে দেয়, তাই ping ব্যর্থ হলেও TCP সার্ভিস চলতে পারে। ping ব্যর্থতা প্রমাণ নয়, শুধু ইঙ্গিত।

`traceroute` (বা `tracepath`, sudo ছাড়া চলে) TTL বাড়িয়ে প্রতিটি hop-এর ঠিকানা ও সময় দেখায় — প্যাকেট কোথায় আটকাচ্ছে, তা বোঝা যায়। `mtr` ping ও traceroute একসঙ্গে করে ক্রমাগত আপডেট দেয় — কোন hop-এ packet loss হচ্ছে, তা চোখে পড়ে। Ubuntu-তে `sudo apt install traceroute mtr` দিয়ে ইনস্টল করা যায়।

২১.৫ dig ও nslookup: DNS যাচাই

```
dig example.com
dig example.com MX +short
dig @8.8.8.8 example.com   # নির্দিষ্ট resolver দিয়ে জিজ্ঞেস করা
```

ব্যাখ্যা: `dig`-এর উত্তরে `ANSWER SECTION`-এ A রেকর্ড, `Query time`-এ কত সময় লাগল, `SERVER`-এ কোন resolver উত্তর দিল — এগুলো দেখুন। `+short` শুধু উত্তরটি দেখায়। `SERVFAIL` মানে resolver-এর সমস্যা; `NXDOMAIN` মানে নামটি নেই। `nslookup` পুরোনো, সহজ হাতিয়ার — একই কাজ করে, তবে কম তথ্য দেয়। DNS সমস্যায় প্রথমে `cat /etc/resolv.conf` দিয়ে দেখুন কোন nameserver ব্যবহার হচ্ছে।

২১.৬ curl ও wget: HTTP স্তরের পরীক্ষা

```
curl -I https://example.com        # শুধু header
curl -v https://example.com       # পুরো অনুরোধ-উত্তর, TLS-সহ
curl -w "%{http_code} %{time_total}\n" -o /dev/null -s https://example.com
```

ব্যাখ্যা: `-I` HEAD অনুরোধ — বডি না নামিয়ে header দেখা। `-v` verbose — DNS, TCP, TLS handshake-এর প্রতিটি ধাপ দেখায়; TLS সমস্যায় এটাই প্রথম হাতিয়ার। `-w` দিয়ে status code ও মোট সময় মাপা যায় — মনিটরিং স্ক্রিপ্টে কাজে লাগে। `wget` ফাইল নামানোর জন্য সুবিধাজনক, তবে পরীক্ষায় `curl`-ই বেশি নমনীয়।

২১.৭ tcpdump: প্যাকেট দেখা (নিজের lab-এ)

`tcpdump` একটি interface-এর প্যাকেট ধরে দেখায় — Wireshark-এর কমান্ড-লাইন ভাই। শুধু নিজের lab নেটওয়ার্কে ব্যবহার করবেন, অন্যের নেটওয়ার্কে নয়।

```
sudo tcpdump -i eth0 -c 5 icmp        # 5টি ICMP প্যাকেট দেখাও
sudo tcpdump -i eth0 -c 5 port 80 -A   # port 80-এর প্যাকেট, ASCII-সহ
sudo tcpdump -i eth0 -w /tmp/cap.pcap # ফাইলে সংরক্ষণ, পরে Wireshark-এ খুলুন
```

ব্যাখ্যা: `-i` interface, `-c` কয়টি প্যাকেট, `-A` ASCII দেখাও, `-w` ফাইলে লেখো। `tcpdump` চালানোর সময় অন্য টার্মিনালে `ping` করুন — প্যাকেটগুলো চোখের সামনে দেখা যাবে। বিস্তারিত বিশ্লেষণ অধ্যায় ২২-এ।

২১.৮ Netplan: স্থায়ী কনফিগারেশন

Ubuntu Server 24.04-এ নেটওয়ার্ক কনফিগারেশন Netplan YAML ফাইলে লেখা হয় (`/etc/netplan/`)। `ip` কমান্ডের পরিবর্তন reboot-এ মুছে যায়; Netplan-এর পরিবর্তন স্থায়ী হয়।

```
# /etc/netplan/50-cloud-init.yaml — উদাহরণ (illustrative)
network:
  version: 2
  ethernets:
    eth0:
      dhcp4: false
      addresses: [192.168.1.10/24]
      routes:
        - to: default
          via: 192.168.1.1
```

```
      nameservers:
        addresses: [8.8.8.8, 1.1.1.1]
```

প্রয়োগ: `sudo netplan try` (ভুল হলে স্বয়ংক্রিয়ভাবে ফিরে আসে — দূরবর্তী সার্ভারে এটাই নিরাপদ) তারপর `sudo netplan apply`। YAML-এ indentation ভুল হলে `netplan try` ব্যর্থ হবে — ফাইলটি `cat` দিয়ে যাচাই করুন। Cloud VPS-এ cloud-init Netplan ফাইল পরিচালনা করে — হাতে সম্পাদনার আগে provider-এর ডকুমেন্ট পড়ুন।

চিত্র ২১.২ — একটি সম্পূর্ণ নির্ণয়-প্রবাহ
curl ব্যর্থ → ping gateway সফল? না → ip addr/route দেখুন → হ্যাঁ → ping 8.8.8.8 → ব্যর্থ → gateway/NAT দেখুন → সফল → dig → ব্যর্থ → resolv.conf/DNS → সফল → curl -v → TLS/HTTP স্তরে সমস্যা

২১.৯ বাস্তব উদাহরণ: "ইন্টারনেট নেই" — আসলে DNS নেই

এক ব্যবহারকারী বললেন "ইন্টারনেট নেই"। পরীক্ষায় দেখা গেল: `ping 8.8.8.8` কাজ করে, কিন্তু `ping google.com` ব্যর্থ — `dig` বলল `SERVFAIL`। অর্থাৎ ইন্টারনেট আছে, DNS নেই। `/etc/resolv.conf`-এ ভুল nameserver ছিল। Nameserver বদলে (8.8.8.8) সমস্যা মিটল। শিক্ষা: ব্যবহারকারীর বর্ণনা ("ইন্টারনেট নেই") আর প্রযুক্তিগত কারণ এক নয় — সিঁড়ি ধরে নিচ থেকে পরীক্ষা করলে আসল স্তরটি ধরা পড়ে।

২১.১০ হাতে-কলমে অনুশীলন: নিজের সার্ভারের স্বাস্থ্য পরীক্ষা

LAB: নিজের VM-এ এই ক্রমটি চালান ও প্রতিটি আউটপুটের অর্থ এক লাইনে লিখুন: (১) `ip -br addr`; (২) `ip route`; (৩) `ss -tlnp`; (৪) `ping -c 3 8.8.8.8`; (৫) `dig example.com +short`; (৬) `curl -s -o /dev/null -w "%{http_code}\n" https://example.com`; (৭) `sudo tcpdump -i eth0 -c 3 icmp` (অন্য টার্মিনালে ping চালিয়ে)। যাচাই: প্রতিটি ধাপের আউটপুট ব্যাখ্যা করতে পারেন। Cleanup: কিছু মোছার নেই।

উদাহরণ আউটপুট (illustrative):

```
$ ip -br addr
lo               UNKNOWN        127.0.0.1/8
eth0             UP             192.168.1.10/24
$ ip route
default via 192.168.1.1 dev eth0 proto dhcp
192.168.1.0/24 dev eth0 proto kernel scope link src 192.168.1.10
$ ss -tlnp
State  Recv-Q  Send-Q  Local Address:Port  Process
LISTEN 0       4096    0.0.0.0:22          sshd
```

২১.১১ Common Mistakes ও Troubleshooting

- **ভুল:** `ping` ব্যর্থ দেখে "নেটওয়ার্ক নেই" বলা — অনেক সার্ভার ICMP বন্ধ রাখে। **সংশোধন:** `curl` বা `ss` দিয়ে TCP স্তরে পরীক্ষা করুন।
- **ভুল:** `ip` কমান্ডে পরিবর্তন করে স্থায়ী ভাবা — reboot-এ মুছে যায়। **সংশোধন:** Netplan-এ লিখুন।
- **ভুল:** DNS সমস্যায় `/etc/hosts` এডিট করে "ঠিক" করা — সাময়িক জোড়াতালি, আসল সমস্যা থেকে যায়।

- **সমস্যা:** `curl: (7) Failed to connect` — TCP স্তরে ব্যর্থ: ফায়ারওয়াল, সার্ভিস বন্ধ, বা ভুল port। `curl: (6) Could not resolve host` — DNS সমস্যা। কোডটি পড়লেই স্তরটি বোঝা যায়।

২১.১২ Security Considerations ও Best Practices

- `tcpdump` শুধু নিজের lab interface-এ চালান; অন্যের ট্রাফিক ধরা অনৈতিক ও অনেক জায়গায় বেআইনি।
- `tcpdump -w` ফাইলে পাসওয়ার্ড বা token থাকতে পারে — capture ফাইল নিরাপদে রাখুন ও কাজ শেষে মুছে ফেলুন।
- দূরবর্তী সার্ভারে নেটওয়ার্ক বদলের আগে `netplan try` ব্যবহার করুন; console access হাতে রাখুন।
- নির্ণয়ের ফলাফল লিখে রাখুন — একই সমস্যা আবার এলে নোটই সবচেয়ে দ্রুত হাতিয়ার।

২১.১৩ একটি নির্ণয়-চেকলিস্ট (কেটে রাখার মতো)

প্রতিবার সমস্যায় এই ক্রমটি অনুসরণ করুন: (১) `ip link` — interface UP কি না; (২) `ip addr` — IP ঠিক আছে কি না; (৩) `ip route` — default gateway আছে কি না; (৪) `ping` gateway, তারপর 8.8.8.8; (৫) `dig` — DNS উত্তর দেয় কি না; (৬) `ss -tlnp` — সার্ভিস শুনছে কি না; (৭) `curl -v` — HTTP/TLS স্তর; (৮) `journalctl -u <service>` — সার্ভিসের নিজের লগ। প্রতিটি ধাপের ফল এক লাইনে লিখুন। যে ধাপে প্রথম ব্যর্থতা, সেখানেই থামুন — তার নিচের ধাপগুলো আবার পরীক্ষা করার দরকার নেই, কারণ নিচের স্তর ঠিক না থাকলে ওপরের পরীক্ষা অর্থহীন। এই চেকলিস্টটি প্রিন্ট করে ডেস্কে রাখুন; চাপের মুহূর্তে মানুষ সবচেয়ে বেশি ভুল করে, আর লেখা চেকলিস্ট সেই ভুল কমায়।

Knowledge Check — যাচাই করুন

1. `ip route get 8.8.8.8` কী বলে দেয়? A) DNS উত্তর B) ওই গন্তব্যে কোন interface ও gateway ব্যবহার হবে C) প্যাকেটের বিষয়বস্তু D) ফায়ারওয়াল নিয়ম
2. `ss -tlnp`-এ `127.0.0.1:5432` মানে কী? A) সবাই সংযোগ দিতে পারে B) শুধু localhost থেকে সংযোগ দেওয়া যায় C) port বন্ধ D) UDP port
3. ping সফল কিন্তু `curl` ব্যর্থ হলে সমস্যা সম্ভবত কোন স্তরে? A) শুধু DNS B) TCP/HTTP স্তরে — ফায়ারওয়াল বা সার্ভিস C) কেবল ছেঁড়া D) MAC ঠিকানা ভুল
4. `dig`-এ `NXDOMAIN` মানে কী? A) সার্ভার বন্ধ B) ওই নামের কোনো রেকর্ড নেই C) ইন্টারনেট নেই D) TLS ব্যর্থ
5. Netplan-এর সুবিধা কী? A) দ্রুত ইন্টারনেট B) reboot-এর পরেও টিকে থাকা স্থায়ী কনফিগারেশন C) ফ্রি VPN D) DNS বদলানো

Practical Assignment

নিজের VM-এর একটি "নেটওয়ার্ক স্বাস্থ্য প্রতিবেদন" তৈরি করুন: `ip addr`, `ip route`, `ss -tlnp`, `ping`, `dig`, `curl -I`-এর আউটপুটসহ প্রতিটির এক লাইন ব্যাখ্যা। তারপর ইচ্ছে করে একটি ভুল করুন (যেমন ভুল DNS সার্ভার বসানো) ও নির্ণয়ের সিঁড়ি ধরে তা খুঁজে বের করার প্রক্রিয়াটি লিখুন। সমাধান Answer Guide-এ দেখুন।

Chapter Summary

প্রতিটি নির্ণয়-কমান্ড একটি নির্দিষ্ট স্তরে প্রশ্ন করে: `ip` interface ও route, `ss` listening port, `ping` / `traceroute` পথ, `dig` DNS, `curl` HTTP, `tcpdump` প্যাকেট। নিচের স্তর থেকে ওপরের দিকে পরীক্ষা করলে সমস্যার জায়গা সংকুচিত হয়। স্থায়ী কনফিগারেশন Netplan-এ লেখা হয়; `ip` কমান্ডের পরিবর্তন অস্থায়ী।

Revision Checklist

- ☐ `ip addr/route` পড়ে ব্যাখ্যা করতে পারি
- ☐ `ss -tlnp` দিয়ে listening port নির্ণয় করতে পারি
- ☐ নির্ণয়ের সিঁড়ি ধরে সমস্যার স্তর খুঁজতে পারি
- ☐ `dig` ও `curl -v` ব্যবহার করতে পারি
- ☐ Netplan-এ স্থায়ী কনফিগারেশন লিখতে পারি

অধ্যায় ২২ • Part IV — Production Networking

Chapter 22: Network Troubleshooting and Packet Analysis — নেটওয়ার্ক ট্রাবলশুটিং ও প্যাকেট বিশ্লেষণ

শুরুতে: একটি বাস্তব দৃশ্য

শুক্রবার রাত ১১টা। TechNova-র ওয়েবসাইট মাঝে মাঝে খুলছে, মাঝে মাঝে "connection timed out"। সার্ভার চলছে, CPU স্বাভাবিক, ডিস্ক ফাঁকা। সাপোর্ট টিকিটে গ্রাহকরা লিখছেন "সাইট স্লো"। কী করবেন? এলোমেলোভাবে সার্ভার restart দেওয়া? নাকি পদ্ধতিগতভাবে খুঁজে বের করা — প্যাকেট কোথায় যাচ্ছে, কোথায় আটকাচ্ছে, কোন স্তরে ব্যর্থ হচ্ছে? এই অধ্যায়টি সেই পদ্ধতি শেখায়: structured troubleshooting, আর তার সবচেয়ে শক্তিশালী হাতিয়ার — প্যাকেট নিজে দেখা, Wireshark ও tcpdump দিয়ে।

Learning Objectives — শেখার উদ্দেশ্য

- Structured troubleshooting methodology প্রয়োগ করতে পারবেন।
- OSI স্তর ধরে ধরে সমস্যা সংকুচিত করতে পারবেন।
- Wireshark-এ capture, filter ও মৌলিক বিশ্লেষণ করতে পারবেন।
- TCP handshake, DNS ও HTTP প্যাকেট পড়ে সমস্যা নির্ণয় করতে পারবেন।
- TLS handshake পর্যবেক্ষণ করে সার্টিফিকেট সমস্যা চিনতে পারবেন।
- Packet loss, latency, timeout, DNS failure ও firewall blockage আলাদা করতে পারবেন।
- Case study থেকে কারণ ও নিরাপদ সমাধান লিখতে পারবেন।

Prerequisites — পূর্বজ্ঞান

অধ্যায় ৩–৪ (OSI/TCP-IP), ১০–১১ (TCP lifecycle), ১৩ (DNS), ১৫–১৬ (HTTP/TLS), ২১ (নির্ণয়-কমান্ড)। নিজের lab VM ও Wireshark ইনস্টল করা কম্পিউটার দরকার।

২২.১ প্রথম নীতি: অনুমান নয়, প্রমাণ

Troubleshooting-এর সবচেয়ে বড় ভুল হলো প্রথম অনুমানেই ঝাঁপিয়ে পড়া। পেশাদার পদ্ধতিটি ছয় ধাপের: (১) **লক্ষণ লিখুন** — কী হচ্ছে, কখন থেকে, কাদের হচ্ছে? (২) **সমস্যার পরিধি নির্ণয় করুন** — একজন না সবাই? একটি সার্ভিস না সব? (৩) **অনুমান তৈরি করুন** — সবচেয়ে সম্ভাব্য কারণ প্রথমে। (৪) **পরীক্ষা করুন** — একটি পরিবর্তন, একটি পরীক্ষা। (৫) **সমাধান করুন ও যাচাই করুন** — সমস্যা গেল কি না, অন্য কিছু ভাঙল কি না। (৬) **লিখে রাখুন** — কারণ, সমাধান, সময়। এই লেখাটিই পরের incident-এর সবচেয়ে মূল্যবান সম্পদ।

দ্বিতীয় নিয়ম: **একবারে একটি পরিবর্তন**। একসঙ্গে পাঁচটি জিনিস বদলালে কোনটিতে কাজ হলো, তা জানা যায় না — আর সমস্যা ফিরে এলে আবার অন্ধকারে হাতড়াতে হয়।

> **CONCEPT:** "এটি কাজ করছিল, হঠাৎ বন্ধ হলো" — এই বাক্যটিই তদন্তের শুরু। প্রশ্ন করুন: হঠাৎ-এর আগে কী বদলেছে? Deploy? আপডেট? কনফিগ পরিবর্তন? সার্টিফিকেটের মেয়াদ? বেশিরভাগ incident-এর পেছনে একটি সাম্প্রতিক পরিবর্তন থাকে।

২২.২ Layer-by-Layer Diagnosis

OSI মডেল শুধু পরীক্ষার বিষয় নয় — এটি নির্ণয়ের মানচিত্র। নিচ থেকে ওপরে যান: L1 কেবল/interface চালু? L2 MAC/ARP ঠিক? L3 IP/route/gateway? L4 TCP সংযোগ হয়? L7 DNS/HTTP/TLS? প্রতিটি স্তরে একটি করে পরীক্ষা আছে (অধ্যায় ২১)। একটি স্তর সফল হলে তার নিচের সব স্তর ঠিক আছে বলে ধরে নেওয়া যায় — এভাবেই সমস্যার জায়গা দ্রুত সংকুচিত হয়। মনে রাখুন: "ওয়েবসাইট খুলছে না" একটি L7 লক্ষণ; কারণ L1 থেকে L7-এর যেকোনো স্তরে থাকতে পারে।

২২.৩ Wireshark Fundamentals

Wireshark একটি graphical packet analyzer — প্যাকেট ধরে (capture), রঙিনভাবে দেখায়, আর filter দিয়ে খুঁজতে দেয়। নিজের কম্পিউটার বা lab VM-এর ট্রাফিকেই ব্যবহার করবেন — অন্যের নেটওয়ার্কে capture করা অনৈতিক ও অনেক জায়গায় বেআইনি। মূল ধারণা: capture filter (কী ধরবেন) বনাম display filter (ধরা প্যাকেটের মধ্যে কী দেখবেন)। Display filter-এর উদাহরণ: `tcp.port == 443`, `dns`, `http.request`, `tcp.flags.syn == 1`।

একটি প্যাকেট খুললে তিনটি অংশ দেখা যায়: Ethernet/IP/TCP header-এর স্তর, আর নিচে hex ও ASCII। প্রথমে শিখুন header পড়তে: source/destination IP ও port, TCP flags (SYN, ACK, FIN, RST), sequence number। বাকিটা অভিজ্ঞতার সঙ্গে আসে।

```
# tcpdump দিয়ে capture নিয়ে Wireshark-এ খোলা (নিজের lab-এ)
sudo tcpdump -i eth0 -w /tmp/lab.pcap host 192.168.1.10 and port 80
# তারপর lab.pcap ফাইলটি নিজের কম্পিউটারে Wireshark দিয়ে খুলুন
# Display filter উদাহরণ: tcp.flags.syn == 1  (শুধু SYN প্যাকেট)
```

চিত্র ২২.১ — TCP handshake প্যাকেটে

1. Client → Server: SYN (Seq=1000) — "সংযোগ চাই"
2. Server → Client: SYN-ACK (Seq=5000, Ack=1001) — "রাজি, তোমার 1001 থেকে শুনব"
3. Client → Server: ACK (Ack=5001) — "বুঝেছি, শুরু করো"

এরপর ডেটা। SYN-এর উত্তর না এলে: ফায়ারওয়াল বা সার্ভার বন্ধ। RST এলে: port বন্ধ।

২২.৪ TCP Handshake Analysis

Wireshark-এ handshake দেখে অনেক কিছু বোঝা যায়: SYN পাঠিয়ে কোনো উত্তর নেই — প্যাকেট কোথাও আটকেছে (ফায়ারওয়াল DROP) বা সার্ভার পৌঁছানো যাচ্ছে না। SYN-এর উত্তরে RST এলো — সার্ভার পৌঁছানো গেছে, কিন্তু ওই port-এ কিছু শুনছে না। SYN-ACK এলো কিন্তু ACK গেল না — ক্লায়েন্ট পক্ষের সমস্যা। বারবার retransmission (একই প্যাকেট আবার পাঠানো) মানে প্যাকেট হারাচ্ছে — নেটওয়ার্ক congestion বা ত্রুটিপূর্ণ পথ। `tcp.analysis.retransmission` filter দিয়ে retransmission খুঁজুন।

২২.৫ DNS, HTTP ও TLS প্যাকেট

DNS: `dns` filter-এ দেখুন — query গেল কি? উত্তর এলো কি? `NXDOMAIN` নাকি `SERVFAIL`? Query গিয়ে উত্তর না এলে resolver বা পথের সমস্যা। **HTTP:** `http` filter-এ request method, URL, status code দেখা যায় — 404 মানে পথ ভুল, 500 মানে সার্ভারের ভেতরের ভুল, 301/302 মানে redirect। **TLS:** handshake এনক্রিপ্ট করা হয় না এমন অংশে দেখা যায়: ClientHello (ক্লায়েন্ট কী চায়), ServerHello, Certificate (সার্ভারের সার্টিফিকেট)। সার্টিফিকেটের মেয়াদ শেষ হলে বা নাম না মিললে handshake ব্যর্থ হয় — Wireshark-এ alert প্যাকেট দেখা যায়। তবে TLS-এর ভেতরের HTTP ডেটা পড়া যায় না — এটাই TLS-এর উদ্দেশ্য।

চিত্র ২২.২ — পাঁচটি সাধারণ ব্যর্থতার চিহ্ন

Timeout: SYN যায়, উত্তর আসে না → ফায়ারওয়াল DROP / রুট নেই
Refused: RST আসে → port-এ কিছু শুনছে না
DNS failure: query-র উত্তর NXDOMAIN/SERVFAIL → নামের সমস্যা
Packet loss: retransmission বারবার → congestion / ত্রুটিপূর্ণ লিংক
TLS failure: handshake-এ alert → সার্টিফিকেটের মেয়াদ/নাম

২২.৬ বাস্তব কেস স্টাডি

কেস ১ — "মাঝে মাঝে সাইট খোলে না": `mtr`-এ দেখা গেল একটি নির্দিষ্ট hop-এ 40% packet loss — ISP-র একটি লিংক ত্রুটিপূর্ণ। সার্ভারের দোষ নয়। সমাধান: provider-কে জানানো, আর সাময়িকভাবে বিকল্প পথ। **কেস ২ — "API হঠাৎ 500 দেয়":** HTTP প্যাকেটে 500, কিন্তু TCP ঠিক আছে — সার্ভারের ভেতরের সমস্যা; application log দেখতে হয়, নেটওয়ার্ক নয়। **কেস ৩ — "নতুন deploy-এর পর কেউ ঢুকতে পারছে না":** Nginx-এর upstream IP বদলে গেছে, কিন্তু কনফিগে পুরোনো IP — `curl -v` থেকে 502, log-এ "connection refused"। শিক্ষা: লক্ষণ এক হলেও কারণ ভিন্ন স্তরে হতে পারে — প্যাকেট দেখলে স্তরটি ধরা পড়ে।

২২.৭ হাতে-কলমে অনুশীলন: নিজের handshake দেখা

LAB: নিজের lab-এ: (১) `sudo tcpdump -i eth0 -w /tmp/handshake.pcap -c 30 host example.com` চালান (অন্য টার্মিনালে `curl https://example.com` চালান); (২) ফাইলটি Wireshark-এ খুলে `tcp.flags.syn == 1` filter দিন — SYN, SYN-ACK, ACK তিনটি খুঁজুন; (৩) `dns` filter দিয়ে query ও উত্তর দেখুন; (৪) একটি ইচ্ছাকৃত ব্যর্থতা করুন — বন্ধ port-এ `curl http://localhost:9999` চালিয়ে capture-এ RST দেখুন। যাচাই: প্রতিটি প্যাকেটের অর্থ এক লাইনে লিখতে পারেন। Cleanup: `rm /tmp/handshake.pcap` — capture ফাইলে সংবেদনশীল তথ্য থাকতে পারে।

উদাহরণ আউটপুট (illustrative):

```
$ sudo tcpdump -i eth0 -c 4 port 443
10:01:02.100 IP 192.168.1.10.54321 > 93.184.216.34.443: Flags [S]
10:01:02.150 IP 93.184.216.34.443 > 192.168.1.10.54321: Flags [S.]
10:01:02.151 IP 192.168.1.10.54321 > 93.184.216.34.443: Flags [.]
# [S]=SYN, [S.]=SYN-ACK, [.]=ACK — তিন ধাপের handshake সম্পূর্ণ
```

২২.৮ Common Mistakes ও Troubleshooting

- **ভুল:** লক্ষণ দেখেই restart দেওয়া — কারণ না জেনে restart করলে সমস্যা ফিরে আসে, আর প্রমাণ নষ্ট হয়। **সংশোধন:** প্রথমে capture ও log সংগ্রহ করুন।
- **ভুল:** "সবাই বলছে স্লো" শুনে সার্ভারের CPU বাড়ানো — আসল কারণ packet loss বা DNS হলে CPU বাড়িয়ে লাভ নেই।
- **ভুল:** Wireshark-এ সব প্যাকেট দেখে অভিভূত হওয়া — filter ছাড়া capture অর্থহীন। প্রথমে প্রশ্ন ঠিক করুন, তারপর filter লিখুন।
- **সমস্যা:** Intermittent (মাঝে মাঝে) ব্যর্থতা — দীর্ঘ capture নিন; `mtr` কিছুক্ষণ চালিয়ে loss-এর ধরন দেখুন।

২২.৯ Security Considerations ও Best Practices

- Packet capture শুধু নিজের বা অনুমতি পাওয়া নেটওয়ার্কে — অন্যের ট্রাফিক ধরা গুরুতর লঙ্ঘন।
- Capture ফাইলে পাসওয়ার্ড, token, ব্যক্তিগত তথ্য থাকতে পারে — এনক্রিপ্ট করে রাখুন, কাজ শেষে মুছে ফেলুন, কখনো শেয়ার করবেন না।
- Production-এ capture নেওয়ার আগে অনুমতি নিন ও সময় সীমিত রাখুন — বড় capture ডিস্ক ভরে ফেলতে পারে।
- Incident-এর নোট লিখুন: সময়, লক্ষণ, পরীক্ষা, কারণ, সমাধান — পরের বার এই নোটই সবচেয়ে দ্রুত হাতিয়ার।
- Blameless postmortem: দোষ নয়, কারণ খুঁজুন — নইলে পরের বার কেউ সমস্যা লুকাবে।

২২.১০ একটি পূর্ণ Incident Walkthrough

ঘটনা: সোমবার সকাল ৯টা, TechNova-র সাইট খুলছে না। ধাপ ১ — লক্ষণ লেখা: "সব ব্যবহারকারী, সব পাতা, ৯:০২ থেকে"। ধাপ ২ — পরিধি: শুধু ওয়েবসাইট; SSH-এ সার্ভারে ঢোকা যাচ্ছে, তাই সার্ভার চালু আছে। ধাপ ৩ — সিঁড়ি: `systemctl status nginx` — চলছে; `ss -tlnp` — 443 শুনছে; `curl -v https://localhost` সার্ভারের ভেতর থেকে — 502 Bad Gateway; অর্থাৎ Nginx চলছে কিন্তু upstream app-এ পৌঁছাতে পারছে না। ধাপ ৪ — log: `journalctl -u app`-এ দেখা গেল রাতের deploy-এ app crash করেছে, আর systemd restart-এর চেষ্টা করছে। ধাপ ৫ — সমাধান: আগের সংস্করণে ফিরে যাওয়া (rollback), তারপর স্বাস্থ্য পরীক্ষা সফল। ধাপ ৬ — লেখা: কারণ (deploy-এ নতুন environment variable অনুপস্থিত), সমাধান, আর ভবিষ্যতের ব্যবস্থা (deploy-এর আগে স্বাস্থ্য পরীক্ষা বাধ্যতামূলক)। পুরো ঘটনা ৪০ মিনিটে মিটল — কারণ কেউ এলোমেলো restart দেয়নি, সিঁড়ি ধরে এগিয়েছে।

২২.১১ Wireshark Filter চিট-শিট

যা খুঁজছেন	Display filter
একটি হোস্টের সব প্যাকেট	`ip.addr == 192.168.1.10`
শুধু DNS	`dns`
শুধু SYN প্যাকেট	`tcp.flags.syn == 1 and tcp.flags.ack == 0`
Retransmission	`tcp.analysis.retransmission`
HTTP error (4xx/5xx)	`http.response.code >= 400`
	`tls.handshake.type == 1`

TLS handshake	

Filter লেখার নিয়ম: প্রথমে প্রশ্নটি নির্দিষ্ট করুন ("SYN-এর উত্তর আসছে কি?"), তারপর সেই প্রশ্নের filter লিখুন। Wireshark-এ filter বাক্স সবুজ হলে syntax ঠিক, লাল হলে ভুল — এই প্রতিক্রিয়াই শেখার দ্রুততম পথ।

Knowledge Check — যাচাই করুন

1. SYN পাঠিয়ে কোনো উত্তর না এলে সবচেয়ে সম্ভাব্য কারণ কী? A) Port বন্ধ B) ফায়ারওয়াল প্যাকেট DROP করছে বা পথ নেই C) DNS ভুল D) TLS মেয়াদ শেষ
2. SYN-এর উত্তরে RST এলে কী বোঝায়? A) সংযোগ সফল B) সার্ভারে পৌঁছানো গেছে কিন্তু ওই port-এ কিছু শুনছে না C) প্যাকেট হারিয়েছে D) DNS ব্যর্থ
3. Wireshark-এ `tcp.analysis.retransmission` কী খুঁজে দেয়? A) DNS query B) বারবার পাঠানো প্যাকেট — packet loss-এর ইঙ্গিত C) TLS সার্টিফিকেট D) HTTP status
4. Troubleshooting-এ একবারে একটি পরিবর্তন করার নিয়ম কেন? A) সময় বাঁচে B) কোন পরিবর্তনে কাজ হলো, তা জানা যায় C) লগ ছোট থাকে D) সার্ভার দ্রুত হয়
5. TLS handshake-এ সার্টিফিকেটের মেয়াদ শেষ হলে কী দেখা যায়? A) HTTP 404 B) Handshake ব্যর্থতা / alert প্যাকেট C) DNS NXDOMAIN D) ICMP unreachable

Practical Assignment

তিনটি কেস স্টাডি লিখুন (প্রতিটি ৫–৬ লাইন): (ক) "সবাই সাইটে ঢুকতে পারছে না, ping-ও ব্যর্থ" — আপনার নির্ণয়ের ধাপ; (খ) "শুধু একজন ব্যবহারকারীর DNS ব্যর্থ" — কোথায় দেখবেন; (গ) "API মাঝে মাঝে timeout" — কোন capture ও filter ব্যবহার করবেন। প্রতিটিতে কারণের অনুমান, পরীক্ষা ও সমাধান লিখুন। সমাধান Answer Guide-এ দেখুন।

Chapter Summary

ভালো troubleshooting অনুমান নয়, প্রমাণের পদ্ধতি: লক্ষণ লেখা, পরিধি নির্ণয়, একবারে একটি পরীক্ষা, লিখে রাখা। OSI স্তর ধরে নিচ থেকে ওপরে গেলে সমস্যার জায়গা সংকুচিত হয়। Wireshark/tcpdump-এ handshake, DNS, HTTP ও TLS প্যাকেট পড়তে পারলে timeout, refused, DNS failure, packet loss ও TLS failure আলাদা করা যায়। Capture শুধু নিজের lab-এ, আর ফাইল নিরাপদে মুছে ফেলুন।

Revision Checklist

- ☐ ছয় ধাপের troubleshooting পদ্ধতি বলতে পারি
- ☐ SYN/SYN-ACK/ACK ও RST পড়ে সমস্যা চিনতে পারি
- ☐ Wireshark display filter লিখতে পারি
- ☐ পাঁচটি সাধারণ ব্যর্থতার চিহ্ন আলাদা করতে পারি
- ☐ Incident নোট লেখার অভ্যাস জানি

অধ্যায় ২৩ • Part IV — Production Networking

Chapter 23: Cloud Networking Fundamentals — ক্লাউড নেটওয়ার্কিংয়ের মৌলিক বিষয়

শুরুতে: একটি বাস্তব দৃশ্য

TechNova তাদের সার্ভার নিজেদের অফিসের একটি র‍্যাক থেকে ক্লাউডে নিতে চায়। প্রথম প্রশ্নটি সার্ভারের নয় — নেটওয়ার্কের: ক্লাউডে "নেটওয়ার্ক" মানে কী? সেখানে কি কেবল লাগাতে হয়? Switch কোথায়? ফায়ারওয়াল কোথায়? উত্তর: ক্লাউডে নেটওয়ার্ক একটি সফটওয়্যার-সংজ্ঞায়িত (software-defined) জিনিস — আপনি কনসোল বা কোড দিয়ে virtual নেটওয়ার্ক, subnet, route table, ফায়ারওয়াল নিয়ম তৈরি করেন; তার নিচের physical হার্ডওয়্যার provider পরিচালনা করে। ধারণাগুলো (subnet, route, firewall) আপনি ইতিমধ্যে জানেন — এখানে শুধু ক্লাউডের ভাষা ও পার্থক্যগুলো শিখবেন।

Learning Objectives — শেখার উদ্দেশ্য

- Virtual network (VPC/VNet) কী, তা ব্যাখ্যা করতে পারবেন।
- Public ও private subnet-এর পার্থক্য ও ব্যবহার বলতে পারবেন।
- Internet gateway, NAT gateway ও route table-এর ভূমিকা ব্যাখ্যা করতে পারবেন।
- Security group ও network ACL-এর পার্থক্য ব্যাখ্যা করতে পারবেন।
- Load balancer-এর ধরন ও ব্যবহার বলতে পারবেন।
- AWS, Azure, GCP ও IBM-এর নেটওয়ার্কিং ধারণার পার্থক্য চিহ্নিত করতে পারবেন।
- Hybrid connectivity-র ধারণা ব্যাখ্যা করতে পারবেন।

Prerequisites — পূর্বজ্ঞান

অধ্যায় ৭–৮ (IP, subnetting, CIDR), ১২ (routing, NAT), ১৭ (load balancing), ১৮ (ফায়ারওয়াল, segmentation)। Subnetting-এ দুর্বল থাকলে অধ্যায় ৮ আবার দেখুন — ক্লাউড নেটওয়ার্কিং মূলত subnetting-এর প্রয়োগ।

২৩.১ প্রথম নীতি: একই ধারণা, নতুন ভাষা

নিজের অফিসে আপনি router, switch, ফায়ারওয়াল কিনে তার লাগান। ক্লাউডে সেই যন্ত্রগুলো virtual: provider-এর বিশাল ডেটা সেন্টারে software-ই packet forwarding করে। আপনার কাজ: ঠিক করা — কোন ঠিকানা-রেঞ্জ ব্যবহার হবে, কোন subnet public হবে, কোনটি private, কোন পথে প্যাকেট যাবে, কোন port খোলা থাকবে। এই সিদ্ধান্তগুলোই ক্লাউড নেটওয়ার্কিং। Physical কেবল নিয়ে ভাবতে হয় না; কিন্তু IP পরিকল্পনা, routing ও ফায়ারওয়াল নীতি আগের চেয়ে আরও গুরুত্বপূর্ণ — কারণ ভুল করলে তা কয়েক ক্লিকে পুরো বিশ্বের সামনে খুলে যায়।

> **CONCEPT:** ক্লাউডে "নেটওয়ার্ক" একটি যৌক্তিক (logical) সীমানা — আপনার VPC-র ভেতরের যন্ত্রগুলো একে অপরকে দেখে, বাইরের কেউ দেখে না, যদি না আপনি পথ খুলে দেন। এই বিচ্ছিন্নতাই (isolation) ক্লাউড নিরাপত্তার ভিত্তি।

২৩.২ Virtual Network: VPC ও VNet

AWS-এ VPC (Virtual Private Cloud), **Azure-এ VNet** (Virtual Network), **GCP-তে VPC** — তিনটিই একই ধারণা: ক্লাউডে আপনার নিজের আলাদা নেটওয়ার্ক, নিজের IP রেঞ্জ (যেমন 10.0.0.0/16) নিয়ে। পার্থক্য আছে, এবং সেগুলো গুরুত্বপূর্ণ: AWS VPC একটি region-এর মধ্যে থাকে; Azure VNet-ও region-ভিত্তিক, তবে peering-এর নিয়ম ভিন্ন; GCP VPC স্বভাবগতভাবে global — একটি VPC একাধিক region জুড়ে বিস্তৃত হতে পারে, subnet region-ভিত্তিক। এই পার্থক্য না জেনে এক provider-এর নকশা আরেক provider-এ হুবহু বসানো যায় না।

২৩.৩ Public বনাম Private Subnet

একটি subnet **public** হয় যখন তার route table-এ internet gateway-এর দিকে একটি route থাকে (0.0.0.0/0 → IGW) — সেখানকার যন্ত্রের public IP থাকলে ইন্টারনেট থেকে সরাসরি পৌঁছানো যায়। **Private** subnet-এ IGW route নেই — বাইরে থেকে সরাসরি পৌঁছানো যায় না। তবে private subnet-এর যন্ত্রগুলোরও আপডেট নামাতে ইন্টারনেটে যেতে হয় — তার জন্য **NAT gateway**: বাইরে যাওয়া ট্রাফিক NAT gateway-এর public IP দিয়ে বেরোয়, কিন্তু বাইরে থেকে ভেতরে আসার পথ খোলে না। মানচিত্রটি মনে রাখুন: public subnet-এ load balancer ও bastion; private subnet-এ app ও database।

উপাদান	কাজ	মনে রাখার কথা
Internet Gateway (IGW)	VPC ↔ ইন্টারনেট সংযোগ	একটি VPC-তে একটি; শুধু route থাকলেই কাজ করে
NAT Gateway	Private subnet থেকে বাইরে যাওয়া	Public subnet-এ বসে; খরচ আছে — ঘণ্টা ও ডেটা হিসেবে
Route Table	কোন গন্তব্যে কোন পথ	প্রতিটি subnet একটি route table-এর সঙ্গে যুক্ত
Security Group	Instance-স্তরের stateful ফায়ারওয়াল	শুধু allow নিয়ম; default deny
Network ACL	Subnet-স্তরের stateless ফায়ারওয়াল	Allow ও deny দুটোই; নম্বর অনুযায়ী ক্রম

২৩.৪ Security Group বনাম Network ACL

এই দুটির পার্থক্য পরীক্ষায় ও বাস্তবে — দুই জায়গাতেই আসে। **Security group** stateful ও instance-স্তরের: আপনি একটি EC2 instance-এর সঙ্গে যুক্ত করেন; ফেরত ট্রাফিক স্বয়ংক্রিয়ভাবে অনুমোদিত হয়; শুধু allow নিয়ম লেখা যায়, deny নয়; নিয়ম বদলালে সঙ্গে সঙ্গে কার্যকর হয়। **Network ACL** stateless ও subnet-স্তরের: allow ও deny দুটোই লেখা যায়; নিয়মের নম্বর অনুযায়ী ক্রমে মূল্যায়ন হয়; ফেরত ট্রাফিকের জন্য আলাদা নিয়ম লাগে। পেশাদার নকশা: NACL-এ মোটা নিয়ম (যেমন নির্দিষ্ট রেঞ্জ থেকে সব বন্ধ), security group-এ সূক্ষ্ম নিয়ম (শুধু app সার্ভার থেকে ডেটাবেসে 5432)।

> **WARNING:** Security group-এ `0.0.0.0/0` দিয়ে SSH (22) বা ডেটাবেস (5432/3306) খোলা সবচেয়ে সাধারণ ও বিপজ্জনক ভুল। কয়েক মিনিটের মধ্যেই স্বয়ংক্রিয় স্ক্যানার খুঁজে পায়। SSH শুধু নিজের IP বা bastion থেকে, ডেটাবেস শুধু app security group থেকে — এই নিয়মটি কখনো ভাঙবেন না।

চিত্র ২৩.১ — একটি সাধারণ ক্লাউড নেটওয়ার্ক

Internet → [Internet Gateway] → [Public subnet: Load Balancer + NAT Gateway]
→ [Private subnet A: App servers] → [Private subnet B: Database — শুধু app থেকে]
Route table: public → 0.0.0.0/0 via IGW; private → 0.0.0.0/0 via NAT Gateway

২৩.৫ Load Balancer ও Private Connectivity

ক্লাউড load balancer (AWS ALB/NLB, Azure Load Balancer, GCP Cloud Load Balancing) অধ্যায় ১৭-এর ধারণারই পরিচালিত রূপ: ট্রাফিক নিয়ে স্বাস্থ্য পরীক্ষা করে সুস্থ instance-এ পাঠায়। Application Load Balancer (L7) HTTP path/host দেখে route করে; Network Load Balancer (L4) দ্রুত, TCP/UDP স্তরে। Private connectivity মানে আপনার VPC থেকে অন্য পরিষেবায় (যেমন managed database বা storage) ইন্টারনেটে না গিয়ে provider-এর ভেতরের পথে পৌঁছানো — AWS PrivateLink, Azure Private Link, GCP Private Service Connect। এতে ট্রাফিক কখনো public ইন্টারনেটে ওঠে না।

২৩.৬ Provider-ভিত্তিক পার্থক্য (স্পষ্টভাবে চিহ্নিত)

- **AWS:** VPC region-ভিত্তিক; subnet availability zone-ভিত্তিক; Internet Gateway ও NAT Gateway আলাদা উপাদান; security group stateful; VPC peering non-transitive (A↔B ও B↔C থাকলেও A↔C হয় না)।
- **Azure:** VNet-এ subnet-এর মধ্যে network security group (NSG) যুক্ত হয়; Azure Firewall একটি পরিচালিত পরিষেবা; VNet peering-এ transitive routing সীমিত; private endpoint দিয়ে PaaS পরিষেবায় private সংযোগ।
- **GCP:** VPC global — একটি VPC অনেক region জুড়ে; firewall rule VPC-স্তরে, tag/service account দিয়ে লক্ষ্য নির্ধারণ; Cloud NAT-এ NAT gateway-এর মতো কাজ; Cloud Load Balancing global anycast IP দেয়।
- **IBM Cloud:** VPC-তে security group ও ACL ধারণা একই রকম, তবে নাম ও সীমা (quota, region) ভিন্ন — কোনো নকশা চালু করার আগে IBM-এর বর্তমান ডকুমেন্ট পড়ুন।

মূল কথা: ধারণা (subnet, route, stateful firewall) সব provider-এ একই; নাম, সীমা ও ডিফল্ট আচরণ ভিন্ন। "সব ক্লাউড এক" ভাবা ভুল — প্রতিটি provider-এর ডকুমেন্টই সত্যের উৎস।

২৩.৭ Hybrid Connectivity

অনেক কোম্পানির নিজস্ব অফিস/ডেটা সেন্টার ও ক্লাউড — দুটোই থাকে। তাদের যুক্ত করার পথ: **VPN** (ইন্টারনেটের ওপর এনক্রিপ্ট করা tunnel — সস্তা, দ্রুত চালু হয়) বা **dedicated connection** (AWS Direct Connect, Azure ExpressRoute, GCP Cloud Interconnect — provider-এর ডেটা সেন্টারে সরাসরি লাইন, বেশি স্থিতিশীল ও নিরাপদ, তবে খরচ ও সময় বেশি)। ছোট কাজে VPN যথেষ্ট; স্থায়ী, ভারী ট্রাফিকে dedicated connection। Hybrid-এ routing-এর দিকে খেয়াল রাখুন: অফিসের ও ক্লাউডের IP রেঞ্জ যেন overlap না করে — overlap হলে প্যাকেট কোথায় যাবে, তা অনিশ্চিত হয়ে যায়।

চিত্র ২৩.২ — Hybrid সংযোগ

[অফিস নেটওয়ার্ক 192.168.0.0/16] —VPN/Direct Connect→ [Cloud VPC 10.0.0.0/16]
দুই রেঞ্জ আলাদা — overlap নেই; প্রতিটি পাশের route table-এ অপর পাশের রেঞ্জের route আছে

২৩.৮ বাস্তব উদাহরণ: ভুল রেঞ্জের খরচ

একটি স্টার্টআপ VPC-র জন্য 10.0.0.0/16 নিল, কিন্তু পরে অফিসের নেটওয়ার্কও 10.0.0.0/16 দেখা গেল — overlap। VPN চালু করার পর কিছু প্যাকেট ভুল পথে যেতে লাগল। সমাধান: ক্লাউডের রেঞ্জ বদলানো — যা চালু সিস্টেমে কঠিন ও ব্যয়বহুল। শিক্ষা: IP পরিকল্পনা প্রথম দিনের কাজ; অফিস, ক্লাউড, VPN — সবার রেঞ্জ একসঙ্গে পরিকল্পনা করুন (অধ্যায় ৮ ও ২৪)।

২৩.৯ হাতে-কলমে অনুশীলন: কাগজে একটি VPC নকশা

LAB: কোনো খরচ ছাড়াই, কাগজে/ডায়াগ্রামে: (১) VPC 10.0.0.0/16 নিন; (২) দুটি availability zone-এ public subnet (10.0.1.0/24, 10.0.2.0/24) ও private subnet (10.0.11.0/24, 10.0.12.0/24) ভাগ করুন; (৩) প্রতিটি subnet-এর route table লিখুন (public → IGW, private → NAT); (৪) তিনটি security group লিখুন: ALB (443 সবার থেকে), app (ALB থেকে 8080), DB (app থেকে 5432); (৫) একটি NACL নিয়ম লিখুন যা DB subnet-এ 22 বন্ধ করে। যাচাই: প্রতিটি প্যাকেটের পথ (ইন্টারনেট → ALB → app → DB) ধাপে ধাপে বলতে পারেন। Cleanup: কিছু চালু করা হয়নি — খরচ শূন্য।

উদাহরণ আউটপুট (illustrative):

```
VPC: 10.0.0.0/16
Public 10.0.1.0/24  (AZ-a): 0.0.0.0/0 -> igw-xxxx
Private 10.0.11.0/24 (AZ-a): 0.0.0.0/0 -> nat-xxxx
SG-ALB: 443 from 0.0.0.0/0
SG-App: 8080 from SG-ALB
SG-DB:  5432 from SG-App
```

২৩.১০ Common Mistakes ও Troubleshooting

- **ভুল:** Private subnet-এর instance-এ public IP দিয়ে SSH করার চেষ্টা — private মানে বাইরে থেকে পৌঁছানো যায় না। **সংশোধন:** Bastion host বা VPN ব্যবহার করুন।
- **ভুল:** NAT gateway ছাড়া private instance-এ `apt update` ব্যর্থ — বাইরে যাওয়ার পথ নেই। **সংশোধন:** Route table-এ NAT gateway-এর route আছে কি না দেখুন।
- **ভুল:** Security group ঠিক, কিন্তু NACL-এ ফেরত ট্রাফিক বন্ধ — stateless বলে return rule লাগে। **সংশোধন:** NACL-এ ephemeral port (1024–65535) ফেরত নিয়ম যোগ করুন।
- **সমস্যা:** দুটি VPC-র মধ্যে সংযোগ নেই — peering আছে কি না, route table-এ peer CIDR আছে কি না, security group অনুমতি দেয় কি না — তিনটি আলাদা করে দেখুন।

২৩.১১ Security Considerations ও Best Practices

- সবচেয়ে কম প্রকাশ: public subnet-এ শুধু load balancer; app ও DB সবসময় private।
- Security group-এ IP নয়, অন্য security group-কে উৎস হিসেবে ব্যবহার করুন — instance বদলালেও নিয়ম ভাঙে না।
- সব নেটওয়ার্ক পরিবর্তন Infrastructure as Code-এ (Terraform ইত্যাদি) লিখুন — হাতে ক্লিক করা নিয়ম audit করা যায় না।

- Flow log চালু করুন (VPC Flow Logs) — কোন সংযোগ প্রত্যাখ্যাত হলো, তা দেখা যায়।
- খরচ: NAT gateway ও ডেটা transfer-এর খরচ আগে হিসাব করুন; অপ্রয়োজনীয় NAT gateway বসাবেন না।

২৩.১২ খরচের হিসাব: একটি ছোট বাজেটের উদাহরণ

ক্লাউড নেটওয়ার্কিং-এর খরচ প্রায়ই অবহেলিত হয় — আর বিল দেখে চমক লাগে। একটি ছোট সেটআপের মাসিক খরচের উপাদান: NAT gateway (প্রতি ঘণ্টা + প্রক্রিয়াকৃত ডেটা প্রতি GB), load balancer (ঘণ্টা + LCU), ডেটা transfer (বাইরে যাওয়া প্রতি GB), public IP (ব্যবহার না হলে)। উদাহরণস্বরূপ, একটি NAT gateway সারাক্ষণ চালু থাকলে শুধু ঘণ্টার খরচই মাসে উল্লেখযোগ্য অঙ্ক হয় — তাই একটি ছোট lab-এ NAT gateway-এর বদলে একটি ছোট bastion instance দিয়ে বাইরে যাওয়ার পথ তৈরি করা যায়, যদিও তা production-এর জন্য উপযুক্ত নয়। নিয়ম: প্রতিটি উপাদান চালু করার আগে তার মূল্য-তালিকা পড়ুন, আর মাসের শুরুতে বিলের alert বসান। সবচেয়ে ব্যয়বহুল ভুল হলো ভুলে চালু রাখা resource — lab শেষে মুছে ফেলার চেকলিস্ট রাখুন।

Knowledge Check — যাচাই করুন

1. Private subnet-এর instance ইন্টারনেটে যেতে চাইলে কী দরকার? A) Internet Gateway-এর সরাসরি route B) NAT gateway-এর দিকে route C) Public IP D) কিছুই না
2. Security group ও Network ACL-এর মূল পার্থক্য কী? A) কোনো পার্থক্য নেই B) Security group stateful ও instance-স্তরের; NACL stateless ও subnet-স্তরের C) NACL শুধু AWS-এ আছে D) Security group শুধু deny করে
3. GCP VPC-এর একটি বৈশিষ্ট্য কী? A) শুধু একটি region-এ থাকে B) Global — একাধিক region জুড়ে বিস্তৃত হতে পারে C) কোনো firewall নেই D) Subnet থাকে না
4. Hybrid সংযোগে IP রেঞ্জ overlap হলে কী হয়? A) গতি বাড়ে B) প্যাকেট ভুল পথে যেতে পারে — routing অনিশ্চিত হয় C) খরচ কমে D) নিরাপত্তা বাড়ে
5. Load balancer-এর সামনে শুধু 443 খোলা রাখার নীতি কী বলে? A) সব port খোলা B) Least exposure — শুধু যা দরকার, তা-ই খোলা C) সব বন্ধ D) শুধু SSH খোলা

Practical Assignment

একটি কাল্পনিক ওয়েব অ্যাপের জন্য AWS-এ একটি VPC নকশা আঁকুন: CIDR, দুটি AZ-তে public/private subnet, route table, তিনটি security group, একটি NACL নিয়ম। তারপর একই নকশা Azure-এ কী নামে কী হবে, তা পাশে লিখুন (VNet, NSG ইত্যাদি)। কোন জায়গায় provider-এর পার্থক্য নকশা বদলে দেয়, তা এক অনুচ্ছেদে লিখুন। সমাধান Answer Guide-এ দেখুন।

Chapter Summary

ক্লাউড নেটওয়ার্কিং হলো পরিচিত ধারণার (subnet, route, firewall) software-defined রূপ। VPC/VNet আপনার আলাদা নেটওয়ার্ক; public subnet IGW দিয়ে, private subnet NAT gateway দিয়ে বাইরে যায়। Security group stateful ও instance-স্তরের, NACL stateless ও subnet-স্তরের। AWS, Azure, GCP-র ধারণা এক, নাম ও আচরণ ভিন্ন — provider-এর ডকুমেন্টই সত্যের উৎস। Hybrid-এ IP overlap এড়ানো প্রথম দিনের কাজ।

Revision Checklist

- ☐ VPC/VNet ও public/private subnet ব্যাখ্যা করতে পারি
- ☐ IGW বনাম NAT gateway-এর ভূমিকা জানি
- ☐ Security group বনাম NACL-এর পার্থক্য বলতে পারি
- ☐ AWS/Azure/GCP-র পার্থক্য চিহ্নিত করতে পারি
- ☐ একটি ছোট VPC নকশা আঁকতে পারি

অধ্যায় ২৪ • Part IV — Production Networking

Chapter 24: Designing and Operating Production Network Infrastructure — প্রোডাকশন নেটওয়ার্ক অবকাঠামো নকশা ও পরিচালনা

শুরুতে: একটি বাস্তব দৃশ্য

TechNova একটি ছোট সফটওয়্যার কোম্পানি। তাদের ওয়েব অ্যাপ্লিকেশন এখন একটি মাত্র সার্ভারে চলে — ওয়েব, অ্যাপ, ডেটাবেস সব এক জায়গায়, সব port খোলা, ব্যাকআপ হাতে করা হয়, আর সাইট ডাউন হলে কেউ জানে না কেন। গ্রাহক বাড়ছে, আর এই অবস্থায় আর চলবে না। আপনাকে বলা হলো: পুরো নেটওয়ার্কটি নতুন করে নকশা করুন — নিরাপদ, সহজে বোঝা যায় এমন, আর ভাঙলে দ্রুত ঠিক করা যায় এমন। এই অধ্যায়টি পুরো বইয়ের সমন্বয়: আগের ২৩টি অধ্যায়ের প্রতিটি ধারণা এখানে একটি বাস্তব নকশায় এসে মিলবে।

Learning Objectives — শেখার উদ্দেশ্য

- Requirements থেকে নেটওয়ার্ক নকশা বের করতে পারবেন।
- একটি multi-tier অ্যাপের জন্য IP ও subnet পরিকল্পনা করতে পারবেন।
- Public/private segmentation, DNS, TLS, reverse proxy ও load balancing একসঙ্গে নকশা করতে পারবেন।
- Firewall নীতি ও ডেটাবেস isolation লিখতে পারবেন।
- Availability, redundancy ও monitoring-এর সিদ্ধান্ত ব্যাখ্যা করতে পারবেন।
- Incident response ও পরিচালনার চেকলিস্ট তৈরি করতে পারবেন।
- খরচ, কর্মক্ষমতা ও নিরাপত্তার মধ্যে trade-off মূল্যায়ন করতে পারবেন।

Prerequisites — পূর্বজ্ঞান

এই বইয়ের সব অধ্যায় — বিশেষ করে ৮ (subnetting), ১২ (routing/NAT), ১৩ (DNS), ১৬–১৮ (TLS, proxy, firewall), ২০ (CDN), ২১–২৩ (নির্ণয়, troubleshooting, cloud)। Vol 01-এর systemd, ব্যাকআপ ও নিরাপত্তা অধ্যায়গুলোও দরকার হবে।

২৪.১ প্রথম নীতি: নকশা শুরু হয় প্রশ্ন দিয়ে, যন্ত্র দিয়ে নয়

নতুন সার্ভার কেনার আগে প্রশ্ন করুন: কত ব্যবহারকারী? কত ট্রাফিক? ডেটা কত সংবেদনশীল? কতক্ষণ ডাউন থাকা সহ্য করা যায় (RTO/RPO)? বাজেট কত? এই উত্তরগুলো ছাড়া নকশা অনুমান মাত্র। TechNova-র উত্তর: দিনে ~১০,০০০ ব্যবহারকারী, পিক আওয়ারে ৫০০ সমবর্তী (concurrent), গ্রাহকের ব্যক্তিগত তথ্য আছে (তাই ডেটাবেস কখনো public নয়), এক ঘণ্টার বেশি ডাউনটাইম সহ্য করা যায় না, বাজেট সীমিত — তাই পরিচালিত (managed) পরিষেবা কম, নিজের সার্ভার বেশি।

দ্বিতীয় নীতি: **সরলতা একটি বৈশিষ্ট্য**। জটিল নকশা দেখতে ভালো লাগে, কিন্তু ভাঙলে কেউ ঠিক করতে পারে না। প্রতিটি উপাদানের জন্য জিজ্ঞেস করুন: এটি না থাকলে কী ক্ষতি? উত্তর "খুব কম" হলে সেটি বাদ দিন।

> **CONCEPT:** ভালো নেটওয়ার্ক নকশার পরীক্ষা: রাত ৩টায় ঘুম থেকে উঠে একজন জুনিয়র অ্যাডমিন কি ডায়াগ্রাম দেখে বুঝতে পারবে প্যাকেট কোথা দিয়ে যায়? না পারলে নকশাটি খুব জটিল।

২৪.২ IP Address Planning

TechNova VPC: `10.0.0.0/16` (৬৫,৫৩৬টি ঠিকানা — ভবিষ্যতের জন্য যথেষ্ট, আর অফিসের 192.168.0.0/16-এর সঙ্গে overlap নেই)। দুটি availability zone-এ ভাগ:

Subnet	CIDR	Zone	যা থাকে
Public-A	10.0.1.0/24	AZ-a	Load balancer, NAT gateway
Public-B	10.0.2.0/24	AZ-b	Load balancer, NAT gateway
Private-App-A	10.0.11.0/24	AZ-a	App servers, Nginx
Private-App-B	10.0.12.0/24	AZ-b	App servers, Nginx
Private-DB-A	10.0.21.0/24	AZ-a	Database (primary)
Private-DB-B	10.0.22.0/24	AZ-b	Database (replica)

হিসাব: /24-এ ২৫৬টি ঠিকানা, ব্যবহারযোগ্য ২৫১টি (নেটওয়ার্ক, broadcast ও cloud-এর সংরক্ষিত ঠিকানা বাদে) — প্রতিটি স্তরের জন্য যথেষ্ট। দুটি zone-এ ভাগ করার কারণ: একটি zone-এর সমস্যায় অন্যটি চালু থাকে (অধ্যায় ২৩)।

২৪.৩ Tier Design: Web, App, Database

চিত্র ২৪.১ — TechNova multi-tier architecture

[User] → [CDN edge] → [Load Balancer, public subnet, :443] → [Nginx reverse proxy, private app subnet] → [App servers ×2, private] → [Database primary + replica, isolated subnet]

প্রতিটি তীরের মাঝে ফায়ারওয়াল নিয়ম; ডেটাবেসে শুধু app subnet থেকে 5432

প্রতিটি স্তরের কাজ আলাদা: **Load balancer** ট্রাফিক ভাগ করে ও স্বাস্থ্য পরীক্ষা করে; **Nginx** reverse proxy হিসেবে TLS শেষ করে, static ফাইল দেয়, rate limit করে; **App server** ব্যবসায়িক যুক্তি চালায়; **Database** শুধু app-এর প্রশ্নের উত্তর দেয়। কোনো স্তরই তার পরের স্তরের ভেতরের কথা জানে না — এটাই tier-এর অর্থ।

২৪.৪ DNS ও TLS Design

DNS: `technova.example` — root ও www রেকর্ড CDN/load balancer-এর দিকে; `api.technova.example` আলাদা; internal নাম (`db.internal`) শুধু VPC-র private DNS-এ — ইন্টারনেটে নেই। পরিবর্তনের আগে TTL ৩০০ সেকেন্ডে নামিয়ে রাখা হয় (অধ্যায় ১৩, ২০)। TLS: CDN/load balancer-এ public সার্টিফিকেট (Let's Encrypt বা পরিচালিত পরিষেবা — স্বয়ংক্রিয় নবায়নসহ), আর load balancer থেকে app পর্যন্তও TLS (internal সার্টিফিকেট) — "ভেতরে HTTP ঠিক আছে" ভাবা ভুল, কারণ ভেতরের ট্রাফিকও আক্রান্ত হতে পারে। সার্টিফিকেটের মেয়াদ মনিটর করা হয় — মেয়াদ শেষে সাইট বন্ধ হওয়া সবচেয়ে লজ্জাজনক incident।

২৪.৫ Reverse Proxy, Load Balancing ও Firewall Policy

```
# Nginx upstream — দুটি app server (illustrative)
upstream app {
    least_conn;
    server 10.0.11.10:8080 max_fails=3 fail_timeout=30s;
    server 10.0.12.10:8080 max_fails=3 fail_timeout=30s;
}
server {
    listen 443 ssl;
    server_name technova.example;
    location / {
        proxy_pass http://app;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
```

ব্যাখ্যা: `least_conn` কম সংযোগ থাকা সার্ভারে পাঠায়; `max_fails=3` মানে তিনবার ব্যর্থ হলে ৩০ সেকেন্ডের জন্য সার্ভারটিকে বাদ দেওয়া হয় — এটাই health check-এর সরল রূপ। Firewall নীতি (security group): ALB: 443 সবার থেকে; Nginx/App: 8080 শুধু ALB থেকে, 22 শুধু bastion থেকে; DB: 5432 শুধু app security group থেকে। NACL-এ DB subnet-এ 22 সম্পূর্ণ বন্ধ।

২৪.৬ Availability, Monitoring ও Incident Response

Availability আসে redundancy থেকে: দুটি zone, দুটি app server, ডেটাবেস replica, আর প্রতিটি স্তরে health check। কিন্তু redundancy-ই যথেষ্ট নয় — জানতে হবে কখন কিছু ভাঙল। Monitoring: প্রতিটি স্তরে `curl` স্বাস্থ্য পরীক্ষা, `ping` / latency, ডিস্ক ও CPU, TLS মেয়াদ, আর VPC Flow Logs। Alert: স্বাস্থ্য পরীক্ষা দুবার ব্যর্থ হলে বার্তা — কিন্তু alert এত বেশি নয় যে মানুষ তা উপেক্ষা করতে শুরু করে (alert fatigue)। Incident-এর সময়: (১) status পাতায় জানান, (২) নির্ণয়ের সিঁড়ি ধরে স্তর খুঁজুন (অধ্যায় ২২), (৩) একবারে একটি পরিবর্তন, (৪) ঠিক হলে blameless postmortem লিখুন।

চিত্র ২৪.২ — প্যাকেটের সম্পূর্ণ যাত্রা (বইয়ের সমন্বয়)

User → DNS (Ch 13) → CDN edge (Ch 20) → Load Balancer → Nginx (Ch 17, TLS Ch 16) → App → Database

প্রতিটি ধাপে: Ethernet/IP (Ch 6–8) → TCP (Ch 10–11) → Routing/NAT (Ch 12) → Firewall (Ch 18) → Log (Ch 22)

২৪.৭ খরচ, কর্মক্ষমতা ও নিরাপত্তার Trade-off

সিদ্ধান্ত	সুবিধা	খরচ/ঝুঁকি
CDN ব্যবহার	দ্রুত, origin সুরক্ষিত	মাসিক খরচ; cache নীতি ভুল হলে পুরোনো তথ্য
দুটি AZ	এক zone ভাঙলেও সাইট চলে	দ্বিগুণ সার্ভার ও NAT খরচ
সব স্তরে TLS	ভেতরের ট্রাফিকও নিরাপদ	সার্টিফিকেট পরিচালনা ও সামান্য CPU
Managed database	ব্যাকআপ ও patching provider করে	বেশি খরচ; নিয়ন্ত্রণ কম

TechNova-র সিদ্ধান্ত: CDN হ্যাঁ, দুটি AZ হ্যাঁ, সব স্তরে TLS হ্যাঁ, managed database না (বাজেটের জন্য নিজেরা চালাবে — তবে ব্যাকআপ ও replica বাধ্যতামূলক)। প্রতিটি trade-off লিখে রাখা হয় — পরে কেউ প্রশ্ন করলে উত্তর থাকে।

২৪.৮ TechNova Capstone: সম্পূর্ণ নকশা

Requirements: ১০,০০০ দৈনিক ব্যবহারকারী; ব্যক্তিগত তথ্য সুরক্ষিত; ১ ঘণ্টার বেশি ডাউনটাইম নয়; বাজেট সীমিত। **Milestones:** (১) VPC ও subnet তৈরি, IP পরিকল্পনা লেখা; (২) DNS ও TLS চালু; (৩) Nginx + load balancer; (৪) App ও DB স্তর, firewall নীতি; (৫) ব্যাকআপ ও restore পরীক্ষা; (৬) simulated failure: একটি app server বন্ধ করে দেখা ট্রাফিক অন্যটিতে যায় কি না; একটি AZ-এর সমস্যা অনুকরণ; DNS ভুল করে শুধরানো। **Deliverables:** নেটওয়ার্ক ডায়াগ্রাম, IP পরিকল্পনা, firewall নীতি ডকুমেন্ট, Nginx কনফিগ, ব্যাকআপ/restore প্রতিবেদন, incident runbook। **Acceptance criteria:** একটি app server বন্ধ করলে সাইট চলে; DB subnet-এ বাইরে থেকে পৌঁছানো যায় না (`ss` ও port scan-এর বদলে `curl` timeout দিয়ে প্রমাণ); ব্যাকআপ থেকে restore করে ডেটা পড়া যায়; TLS মেয়াদ মনিটরে দেখা যায়। **Evaluation rubric:** নকশার সম্পূর্ণতা ৩০, নিরাপত্তা (segmentation, least exposure) ৩০, যাচাই ও প্রমাণ ২৫, ডকুমেন্টেশন ১৫।

২৪.৯ হাতে-কলমে অনুশীলন: নিজের নকশার প্রথম খসড়া

> **LAB:** TechNova-র নকশা দেখে নিজের একটি ছোট অ্যাপের (যেমন একটি ব্লগ) নেটওয়ার্ক নকশা আঁকুন: (১) VPC CIDR বাছুন ও দুটি subnet-এ ভাগ করুন; (২) কোন স্তরে কী থাকবে, লিখুন; (৩) তিনটি firewall নিয়ম লিখুন; (৪) DNS রেকর্ডের তালিকা লিখুন; (৫) একটি ব্যর্থতার দৃশ্য লিখুন (যেমন একটি সার্ভার বন্ধ) ও কী হবে, তা বলুন। যাচাই: অন্য একজনকে ডায়াগ্রাম দেখিয়ে প্যাকেটের পথ বলতে বলুন — সে পারলে নকশাটি যথেষ্ট স্পষ্ট। Cleanup: কিছু চালু করা হয়নি — খরচ শূন্য।

উদাহরণ আউটপুট (illustrative):

```
$ curl -s -o /dev/null -w "%{http_code} %{time_total}s\n" https://technova.example/
200 0.42s
# একটি app server বন্ধ করার পরও 200 — load balancer ট্রাফিক সুস্থ সার্ভারে পাঠিয়েছে
```

২৪.১০ Common Mistakes ও Troubleshooting

- **ভুল:** সবকিছু একটি সার্ভারে রাখা — একটি ভাঙলে সব যায়। **সংশোধন:** স্তর আলাদা করুন, এমনকি ছোট হলেও।
- **ভুল:** নকশা শুধু মাথায় রাখা — কেউ চলে গেলে জ্ঞান চলে যায়। **সংশোধন:** ডায়াগ্রাম ও runbook লিখে version control-এ রাখুন।
- **ভুল:** ব্যাকআপ নেওয়া কিন্তু restore পরীক্ষা না করা — প্রয়োজনের দিন দেখা যায় ব্যাকআপ অকেজো।
- **সমস্যা:** "হঠাৎ সাইট ডাউন" — প্রথমে দেখুন কী বদলেছে (deploy? সার্টিফিকেট? DNS TTL?) — অধ্যায় ২২-এর পদ্ধতি।

২৪.১১ Security Considerations ও Best Practices

- ডেটাবেস কখনো public নয় — এই নিয়মের কোনো ব্যতিক্রম নেই।
- সব পরিবর্তন Infrastructure as Code-এ; হাতে ক্লিক করা পরিবর্তন audit করা যায় না।
- Secrets (API key, ডেটাবেস পাসওয়ার্ড) কখনো কোডে বা Git-এ নয় — secret manager বা environment-এ, সীমিত অধিকারসহ।

- নিয়মিত game day: ইচ্ছে করে একটি সার্ভার বন্ধ করে দেখুন দল কীভাবে সাড়া দেয় — অনুশীলন ছাড়া incident response শেখা যায় না।
- নকশা প্রতিবছর পুনর্বিবেচনা করুন — ব্যবসা বদলায়, ট্রাফিক বাড়ে, পুরোনো নকশা ছোট হয়ে যায়।

২৪.১২ পরিচালনার চেকলিস্ট: দৈনিক, সাপ্তাহিক, মাসিক

নকশা যত ভালোই হোক, পরিচালনা ছাড়া তা টেকে না। **দৈনিক:** স্বাস্থ্য পরীক্ষার dashboard দেখা, ব্যর্থ alert আছে কি না, ব্যাকআপ সফল হয়েছে কি না। **সাপ্তাহিক:** লগ পর্যালোচনা (ব্যর্থ SSH চেষ্টা, 5xx ত্রুটির হার), ডিস্ক ব্যবহার, সার্টিফিকেটের মেয়াদ (৯০ দিনের কম হলে ব্যবস্থা), আপডেটের তালিকা। **মাসিক:** ফায়ারওয়াল নিয়মের audit (`ss -tlnp` দিয়ে অপ্রত্যাশিত port আছে কি না), ব্যাকআপ থেকে restore পরীক্ষা, খরচের পর্যালোচনা, game day (একটি সার্ভার বন্ধ করে সাড়া পরীক্ষা), ডকুমেন্ট আপডেট। এই চেকলিস্টটি একটি ফাইলে লিখে রাখুন ও প্রতিটি কাজের পাশে তারিখ দিন — পরিচালনার প্রমাণই audit-এর সময় কাজে আসে। মনে রাখুন: production পরিচালনা একটি অভ্যাস, একটি ঘটনা নয়।

Knowledge Check — যাচাই করুন

1. Multi-tier নকশায় ডেটাবেস স্তর কেন আলাদা subnet-এ থাকে? A) গতি বাড়াতে B) শুধু app স্তর থেকে পৌঁছানো যায় — আক্রমণের পৃষ্ঠ কমাতে C) খরচ কমাতে D) DNS দ্রুত করতে
2. দুটি availability zone ব্যবহারের প্রধান কারণ কী? A) IP ঠিকানা বাঁচানো B) একটি zone-এর সমস্যায় অন্যটি চালু থাকে C) ইন্টারনেট সস্তা হয় D) DNS দ্রুত হয়
3. Load balancer-এ health check-এর কাজ কী? A) সার্ভারের তাপমাত্রা মাপা B) অসুস্থ সার্ভারে ট্রাফিক না পাঠানো C) ব্যাকআপ নেওয়া D) লগ লেখা
4. সার্টিফিকেটের মেয়াদ মনিটর করা কেন জরুরি? A) খরচ কমে B) মেয়াদ শেষ হলে সাইট বন্ধ হয়ে যায় C) DNS দ্রুত হয় D) ফায়ারওয়াল বন্ধ হয়
5. নকশা ডকুমেন্টে লিখে রাখা কেন গুরুত্বপূর্ণ? A) পরীক্ষায় নম্বর পাওয়া যায় B) দলের যে কেউ প্যাকেটের পথ বুঝতে পারে ও incident-এ দ্রুত সাড়া দিতে পারে C) সার্ভার দ্রুত হয় D) খরচ কমে

Practical Assignment

TechNova capstone-এর সম্পূর্ণ নকশা ডকুমেন্ট লিখুন: requirements, IP পরিকল্পনা টেবিল, tier ডায়াগ্রাম, DNS ও TLS নকশা, firewall নীতি, ব্যাকআপ পরিকল্পনা, monitoring তালিকা, একটি incident runbook (যেমন "ডেটাবেসে সংযোগ যাচ্ছে না" — ধাপে ধাপে নির্ণয়)। তারপর একটি simulated failure বেছে নিয়ে (একটি app server বন্ধ) কী ঘটবে, তা লিখুন। সমাধান Answer Guide-এ দেখুন।

Chapter Summary

ভালো production নকশা শুরু হয় requirements দিয়ে, যন্ত্র দিয়ে নয়। IP পরিকল্পনা, tier segmentation, DNS ও TLS নকশা, reverse proxy ও load balancing, কঠোর firewall নীতি, ডেটাবেস isolation, redundancy ও monitoring — সব মিলিয়ে একটি সিস্টেম হয় যা ভাঙলেও চলে, আর ভাঙলে দ্রুত ঠিক করা যায়। সবচেয়ে গুরুত্বপূর্ণ সম্পদ নকশা ডকুমেন্ট ও runbook — যন্ত্র নয়, জ্ঞানই টিকে থাকে। এই বইয়ের যাত্রা এখানেই শেষ নয়: পরের খণ্ডগুলোতে এই নেটওয়ার্কের ওপর সার্ভার, কন্টেইনার ও CI/CD গড়ে উঠবে।

Revision Checklist

- ☐ Requirements থেকে নকশা বের করতে পারি
- ☐ IP ও subnet পরিকল্পনা করতে পারি
- ☐ Multi-tier ডায়াগ্রাম আঁকতে পারি
- ☐ Firewall নীতি ও DB isolation লিখতে পারি
- ☐ Availability ও monitoring-এর সিদ্ধান্ত ব্যাখ্যা করতে পারি
- ☐ একটি incident runbook লিখতে পারি

পরিশিষ্ট A — Guided Labs (গাইডেড ল্যাব)

এই পরিশিষ্টের ২৪টি ল্যাব এই বইয়ের ব্যবহারিক হৃদয়। প্রতিটি ল্যাব একটি বাস্তব পরিস্থিতি থেকে শুরু হয়, একটি নিরাপদ বিচ্ছিন্ন ল্যাব নেটওয়ার্কে চলে, এবং যাচাই দিয়ে শেষ হয়। সব ল্যাবে ব্যবহৃত ঠিকানা (যেমন 192.168.56.0/24) শুধু উদাহরণের ল্যাব ঠিকানা — কোনো বাস্তব ব্যবহারকারীর নেটওয়ার্কের তথ্য নয়। নিজের নয় এমন কোনো নেটওয়ার্ক স্ক্যান বা আক্রমণ করবেন না; সব পরীক্ষা নিজের VM ও নিজের তৈরি সার্ভিসের ওপর করুন।

> **WARNING:** ল্যাবের কমান্ড শুধু আপনার নিজের VirtualBox VM বা বিচ্ছিন্ন ল্যাব নেটওয়ার্কে চালান। `tcpdump`, `nmap` বা firewall নিয়ম কখনও অফিস, বিশ্ববিদ্যালয় বা অন্যের নেটওয়ার্কে চালাবেন না। UFW চালু করার আগে SSH-এর নিয়ম যোগ করুন, নইলে নিজেই সার্ভারের বাইরে আটকে যাবেন।

Lab 01 — Interface ও IP Address চেনা: নিজের সার্ভারের পরিচয়পত্র

পরিস্থিতি: আপনি TechNova-র নতুন জুনিয়র ইঞ্জিনিয়ার। প্রথম দিন সার্ভারে লগইন করে জানতে হবে — এই মেশিনের interface কী কী, কোন IP-তে চলছে, gateway কোনটি।

উদ্দেশ্য: `ip` কমান্ড দিয়ে interface, address ও route পড়তে পারা।

পূর্বজ্ঞান: অধ্যায় ১, ৬, ৭। **টুল:** Ubuntu Server 24.04 VM। **প্রাথমিক অবস্থা:** VM চালু, NAT বা Host-only নেটওয়ার্কে যুক্ত।

ধাপ:

1. `ip -brief addr` চালান — প্রতিটি interface-এর সংক্ষিপ্ত তালিকা পাবেন। `ip` হলো আধুনিক Linux নেটওয়ার্ক টুল; পুরোনো `ifconfig` আর ডিফল্টভাবে থাকে না।
2. `ip addr show` চালান — প্রতিটি interface-এর MAC, IPv4/IPv6, broadcast ও state (UP/DOWN) দেখুন।
3. `ip route` চালান — default gateway কোনটি, তা লিখে রাখুন।
4. `ip link show` চালান — link-এর MTU ও state যাচাই করুন।

```
ip -brief addr
ip addr show
ip route
ip link show
```

উদাহরণ আউটপুট (illustrative):

```
lo               UNKNOWN        127.0.0.1/8
enp0s3           UP             192.168.56.10/24
default via 192.168.56.1 dev enp0s3 proto dhcp
```

যাচাই: `ip -brief addr`-এ অন্তত একটি UP interface ও একটি IPv4 ঠিকানা আছে; `ip route`-এ default লাইন আছে। **সাধারণ ব্যর্থতা:** interface DOWN থাকলে `sudo ip link set enp0s3 up` (নিজের VM-এই শুধু) চেষ্টা করুন; DHCP না পেলে VM-এর নেটওয়ার্ক সেটিংস দেখুন। **Cleanup:** কোনো পরিবর্তন না করলে কিছু মুছতে হবে না।

Lab 02 — Binary ও Subnetting হাতে-কলমে: কাগজ, তারপর টার্মিনাল

পরিস্থিতি: TechNova-র নেটওয়ার্ক পরিকল্পনায় 192.168.56.0/24 নেটওয়ার্ককে চারটি সমান subnet-এ ভাগ করতে হবে। প্রথমে কাগজে, তারপর `ipcalc` দিয়ে যাচাই।

উদ্দেশ্য: /24 থেকে /26 subnet-এর network, broadcast ও usable host হিসাব করা।

পূর্বজ্ঞান: অধ্যায় ৭, ৮। **টুল:** কাগজ-কলম, VM। **প্রাথমিক অবস্থা:** কোনো পরিবর্তন নেই।

1. কাগজে লিখুন: /26 মানে ২টি host bit ধার নেওয়া, ৪টি subnet, প্রতিটিতে ৬২টি usable host। প্রথম subnet: 192.168.56.0/26, broadcast 192.168.56.63।
2. VM-এ `sudo apt update && sudo apt install -y ipcalc` দিয়ে ipcalc ইনস্টল করুন।
3. `ipcalc 192.168.56.0/26` চালিয়ে নিজের হিসাবের সঙ্গে মেলান।
4. `ipcalc 192.168.56.64/26` ও `ipcalc 192.168.56.128/26` চালিয়ে বাকি দুটি subnet যাচাই করুন।

```
sudo apt update && sudo apt install -y ipcalc
ipcalc 192.168.56.0/26
ipcalc 192.168.56.64/26
ipcalc 10.0.0.0/22
```

উদাহরণ আউটপুট (illustrative): ipcalc দেখাবে Address, Netmask 255.255.255.192, Network, Broadcast 192.168.56.63, Hosts/Net: 62। **যাচাই:** আপনার কাগজের হিসাবের সঙ্গে ipcalc-এর Network/Broadcast/Hosts মিলেছে। **সাধারণ ভুল:** network ও broadcast ঠিকানাকে host ধরা; /26-এ ৬৪ নয়, ৬২টি usable host। **Cleanup:** ipcalc রেখে দিতে পারেন; শেখার টুল হিসেবে কাজে লাগবে।

Lab 03 — Routing Table পড়া ও Static Route যোগ করা

পরিস্থিতি: আপনার VM-এ দ্বিতীয় একটি নেটওয়ার্ক (192.168.99.0/24) যুক্ত হয়েছে, কিন্তু সেদিকে কোনো route নেই। Static route যোগ করে পৌঁছাতে হবে।

উদ্দেশ্য: routing table পড়া, static route যোগ ও মুছে ফেলা।

পূর্বজ্ঞান: অধ্যায় ১২। **টুল:** VM, sudo। **প্রাথমিক অবস্থা:** শুধু default route আছে।

1. `ip route show` দিয়ে বর্তমান table লিখে রাখুন।
2. `sudo ip route add 192.168.99.0/24 via 192.168.56.1 dev enp0s3` চালান — `via` মানে next-hop, `dev` মানে বেরোনোর interface।
3. `ip route show` দিয়ে নতুন লাইনটি যাচাই করুন।
4. `ping -c 2 192.168.99.1` চালান — উত্তরের আশা করবেন না, কারণ ওই নেটওয়ার্ক আসলে নেই; উদ্দেশ্য route lookup যাচাই, `ip route get 192.168.99.5` দিয়ে দেখুন কোন পথ বাছা হচ্ছে।
5. `sudo ip route del 192.168.99.0/24` দিয়ে route মুছে ফেলুন।

```
ip route show
sudo ip route add 192.168.99.0/24 via 192.168.56.1 dev enp0s3
ip route get 192.168.99.5
sudo ip route del 192.168.99.0/24
```

যাচাই: `ip route get` দেখায় 192.168.99.5-এর জন্য via 192.168.56.1; মুছে ফেলার পর লাইনটি আর নেই। **সাধারণ ব্যর্থতা:** ভুল gateway দিলে `Network is unreachable` আসে; `ip route` দিয়ে next-hop নিজের subnet-এ আছে কি না দেখুন। **Cleanup:** route মুছে ফেলা হয়েছে; reboot-এ এমন temporary route নিজে থেকেই মুছে যায়।

Lab 04 — TCP ও UDP সংযোগ: কোনটি কখন, নিজের হাতে দেখা

পরিস্থিতি: একটি web সার্ভিস TCP-তে চলে, একটি DNS-জাতীয় সার্ভিস UDP-তে। দুটি সংযোগের আচরণের পার্থক্য নিজে দেখতে হবে।

উদ্দেশ্য: `nc` (netcat) দিয়ে TCP listener ও UDP পরীক্ষা করা।

পূর্বজ্ঞান: অধ্যায় ১০, ১১। **টুল:** VM, `nc`। **প্রাথমিক অবস্থা:** কোনো listener নেই।

1. `sudo apt install -y netcat-openbsd` দিয়ে nc ইনস্টল করুন।
2. একটি টার্মিনালে `nc -l 8080` চালান — এটি TCP port 8080-এ শোনে।
3. আরেকটি টার্মিনালে `echo hello | nc 127.0.0.1 8080` চালান — প্রথম টার্মিনালে hello দেখা যাবে।
4. UDP-তে `nc -u -l 9090` ও `echo hi | nc -u 127.0.0.1 9090` দিয়ে একই পরীক্ষা করুন।
5. `ss -tlnp` দিয়ে দেখুন কোন TCP port শুনছে; UDP-র জন্য `ss -ulnp`।

```
sudo apt install -y netcat-openbsd
nc -l 8080
echo hello | nc 127.0.0.1 8080
ss -tlnp
ss -ulnp
```

যাচাই: TCP-তে বার্তা পৌঁছেছে; `ss`-এ 8080 LISTEN দেখা গেছে। **সাধারণ ব্যর্থতা:** port ইতিমধ্যে ব্যবহৃত হলে অন্য port (যেমন 8081) নিন; `-l` ছাড়া nc ক্লায়েন্ট হিসেবে চলে। **Cleanup:** Ctrl+C দিয়ে listener বন্ধ করুন; `ss -tlnp` দিয়ে নিশ্চিত হোন।

Lab 05 — Port Inspection: কোন সার্ভিস কোন পোর্টে শুনছে

পরিস্থিতি: TechNova-র সার্ভারে একটি সার্ভিস চালু হচ্ছে না বলে অভিযোগ। প্রথমে জানতে হবে — কোন port খোলা, কোন process সেখানে বাঁধা।

উদ্দেশ্য: `ss` দিয়ে listening port ও process শনাক্ত করা।

পূর্বজ্ঞান: অধ্যায় ১০, ২১। **টুল:** VM। **প্রাথমিক অবস্থা:** SSH চলছে।

1. `ss -tlnp` চালান — TCP, listening, সংখ্যায় port, process-সহ দেখাবে।
2. `ss -tlnp | grep 22` দিয়ে SSH (port 22) যাচাই করুন।
3. `ss -tulpn` দিয়ে TCP ও UDP দুটোই দেখুন।
4. `ss -tn state established` দিয়ে বর্তমান সক্রিয় সংযোগগুলো দেখুন।
5. একটি port-এ কিছু না শুনলে `curl -v http://127.0.0.1:8080/` চালিয়ে `Connection refused` বার্তাটি দেখুন — এটি প্রমাণ করে listener নেই, firewall নয়।

```
ss -tlnp
ss -tulpn
```

```
ss -tn state established
curl -v http://127.0.0.1:8080/
```

উদাহরণ আউটপুট (illustrative): `LISTEN 0 4096 0.0.0.0:22 0.0.0.0:* users:(("sshd",pid=812,fd=3))` — 0.0.0.0 মানে সব interface-এ শোনা হচ্ছে। **যাচাই:** port 22 LISTEN দেখা গেছে; `curl` বন্ধ port-এ refused দিয়েছে। **সাধারণ ভুল:** `Connection refused` আর `Connection timed out` গুলিয়ে ফেলা — প্রথমটি listener নেই, দ্বিতীয়টি পথে বাধা (firewall/routing)। **Cleanup:** কিছু বদলানো হয়নি।

Lab 06 — DNS Lookup: একটি নাম কীভাবে ঠিকানা হয়

পরিস্থিতি: ব্যবহারকারী বলছে example.com খুলছে না। DNS-ই কি দোষী, নাকি সার্ভার? `dig` দিয়ে ধাপে ধাপে দেখুন।

উদ্দেশ্য: `dig` দিয়ে A, AAAA, MX, NS রেকর্ড পড়া ও cache/TTL বোঝা।

পূর্বজ্ঞান: অধ্যায় ১৩। **টুল:** VM, ইন্টারনেট। **প্রাথমিক অবস্থা:** ইন্টারনেট আছে।

1. `dig example.com` চালান — ANSWER SECTION-এ A রেকর্ড, QUERY TIME, SERVER দেখুন।
2. `dig example.com AAAA` ও `dig example.com MX` চালান।
3. `dig +trace example.com` চালান — root থেকে authoritative পর্যন্ত iterative যাত্রা দেখুন।
4. `dig example.com +short` দিয়ে শুধু ঠিকানাটি নিন।
5. `cat /etc/resolv.conf` দিয়ে আপনার resolver কোনটি, তা দেখুন।

```
dig example.com
dig example.com MX
dig +trace example.com
dig example.com +short
cat /etc/resolv.conf
```

যাচাই: A রেকর্ডে একটি IP ফিরেছে; `+trace`-এ root, TLD ও authoritative সার্ভারের ধাপ দেখা গেছে। **সাধারণ ব্যর্থতা:** `NXDOMAIN` মানে নামটি নেই; `SERVFAIL` মানে resolver সমস্যা — তখন `dig @8.8.8.8 example.com` দিয়ে অন্য resolver পরীক্ষা করুন (শুধু পরীক্ষার জন্য)। **Cleanup:** কিছু বদলানো হয়নি।

Lab 07 — DHCP, ARP ও ICMP: ল্যানের ভেতরের কথোপকথন

পরিস্থিতি: নতুন VM নেটওয়ার্কে এলো। DHCP থেকে ঠিকানা পেল কি না, ARP table-এ প্রতিবেশী কারা, ping আসলে কী করে — নিজে দেখুন।

উদ্দেশ্য: DHCP lease, ARP cache ও ICMP যাচাই করা।

পূর্বজ্ঞান: অধ্যায় ১৪। **টুল:** VM। **প্রাথমিক অবস্থা:** DHCP-তে ঠিকানা পাওয়া।

1. `ip addr show` দিয়ে DHCP-পাওয়া ঠিকানা দেখুন; `cat /var/lib/dhcp/dhclient.leases` থাকলে lease-এর বিবরণ দেখুন (ফাইল না থাকলে systemd-networkd-এর lease `networkctl status` দিয়ে দেখুন)।
2. `ping -c 4 192.168.56.1` চালান — gateway-তে ICMP echo পৌঁছেছে কি না দেখুন।
3. `ip neigh show` চালান — ARP cache-এ gateway-এর MAC ঠিকানা দেখা যাবে।
4. `ping -c 2 8.8.8.8` দিয়ে ইন্টারনেট পথ যাচাই করুন (ইন্টারনেট থাকলে)।

5. `traceroute 8.8.8.8` বা `tracepath 8.8.8.8` চালিয়ে hop-গুলো দেখুন (না থাকলে `sudo apt install -y traceroute`)।

```
ping -c 4 192.168.56.1
ip neigh show
ping -c 2 8.8.8.8
tracepath 8.8.8.8
```

যাচাই: ping-এ 0% packet loss; `ip neigh`-এ gateway-এর REACHABLE এন্ট্রি আছে। **সাধারণ ব্যর্থতা:** ping ব্যর্থ হলে প্রথমে `ip route` দিয়ে default gateway আছে কি না দেখুন; gateway-তে ping না গেলে VM-এর নেটওয়ার্ক মোড দেখুন। **Cleanup:** কিছু বদলানো হয়নি।

Lab 08 — HTTP Request হাতে-কলমে: curl দিয়ে ওয়েবের ভেতর দেখা

পরিস্থিতি: একটি ওয়েবসাইট 404 দিচ্ছে। ব্রাউজার শুধু পৃষ্ঠা দেখায়; `curl` দিয়ে আসল request/response দেখতে হবে।

উদ্দেশ্য: `curl -v` দিয়ে method, header, status code পড়া।

পূর্বজ্ঞান: অধ্যায় ১৫। **টুল:** VM, ইন্টারনেট। **প্রাথমিক অবস্থা:** ইন্টারনেট আছে।

1. `curl -I https://example.com` চালান — শুধু response header দেখুন (HTTP/2 200, content-type)।
2. `curl -v https://example.com 2>&1 | head -30` চালিয়ে DNS, TLS ও header ধাপগুলো পড়ুন।
3. `curl -s -o /dev/null -w "%{http_code} %{time_total}s\n" https://example.com` দিয়ে status ও সময় মাপুন।
4. `curl -I https://example.com/nonexistent-page` দিয়ে 404 প্রতিক্রিয়া দেখুন।

```
curl -I https://example.com
curl -v https://example.com 2>&1 | head -30
curl -s -o /dev/null -w "%{http_code} %{time_total}s\n" https://example.com
```

যাচাই: 200 status পাওয়া গেছে; 404 পৃষ্ঠায় 404 দেখা গেছে; `-v`-এ TLS handshake লাইনগুলো দেখা গেছে। **সাধারণ ভুল:** `-I` মানে HEAD request — body আসে না, এটি স্বাভাবিক। **Cleanup:** কিছু বদলানো হয়নি।

পরিশিষ্ট A — Guided Labs (চলমান)

Lab 09 — TLS Certificate পরীক্ষা: openssl দিয়ে সার্টিফিকেট পড়া

পরিস্থিতি: একটি সাইটের certificate কবে মেয়াদ শেষ হবে, কে দিয়েছে, কোন নামের জন্য — ব্রাউজার না খুলে টার্মিনালেই যাচাই করতে হবে।

উদ্দেশ্য: `openssl s_client` দিয়ে certificate chain, মেয়াদ ও issuer পড়া।

পূর্বজ্ঞান: অধ্যায় ১৬। **টুল:** VM, openssl। **প্রাথমিক অবস্থা:** ইন্টারনেট আছে।

1. `echo | openssl s_client -connect example.com:443 -servername example.com 2>/dev/null | openssl x509 -noout -subject -issuer -dates` চালান — `-servername` SNI পাঠায়, নইলে ভুল certificate আসতে পারে।
2. `echo | openssl s_client -connect example.com:443 2>/dev/null | grep -E "Verify return code|Protocol"` দিয়ে validation ও TLS সংস্করণ দেখুন।
3. মেয়াদ শেষের তারিখ (notAfter) লিখে রাখুন — production-এ এই তারিখই monitoring-এর বিষয়।

```
echo | openssl s_client -connect example.com:443 -servername example.com 2>/dev/null | openssl x509 -noout -subject -issuer -dates
```

যাচাই: subject-এ example.com, issuer-এ একটি পরিচিত CA, notAfter ভবিষ্যতের তারিখ। **সাধারণ ব্যর্থতা:** `Verify return code: 21` মানে self-signed বা অচেনা chain; `unable to get local issuer certificate` মানে intermediate certificate অনুপস্থিত। **নিরাপত্তা:** কখনও private key (`.key` ফাইল) কারও সঙ্গে শেয়ার করবেন না, Git-এ তুলবেন না। **Cleanup:** কিছু বদলানো হয়নি।

Lab 10 — Nginx Reverse Proxy: নিজের ওয়েবে দরজা বসানো

পরিস্থিতি: TechNova-র অ্যাপ চলে port 3000-এ, কিন্তু ব্যবহারকারী আসবে port 80-এ। Nginx reverse proxy হিসেবে সামনে বসবে।

উদ্দেশ্য: Nginx ইনস্টল করে একটি upstream-এ proxy_pass করা।

পূর্বজ্ঞান: অধ্যায় ১৭, ২১। **টুল:** VM, sudo। **প্রাথমিক অবস্থা:** port 3000-এ একটি test অ্যাপ চলছে (Lab 04-এর `nc -l 3000` বা Python-এর `python3 -m http.server 3000`)।

1. `sudo apt update && sudo apt install -y nginx` দিয়ে Nginx ইনস্টল করুন।
2. `sudo nano /etc/nginx/sites-available/reverse-proxy` খুলে লিখুন: `server { listen 80; location / { proxy_pass http://127.0.0.1:3000; proxy_set_header Host $host; proxy_set_header X-Real-IP $remote_addr; } }`।
3. `sudo ln -s /etc/nginx/sites-available/reverse-proxy /etc/nginx/sites-enabled/` ও `sudo nginx -t` দিয়ে syntax যাচাই করুন।
4. `sudo systemctl reload nginx` চালিয়ে `curl -v http://127.0.0.1/` দিয়ে পরীক্ষা করুন — উত্তরে upstream-এর জবাব আসা উচিত।

```
sudo apt install -y nginx
sudo nginx -t
sudo systemctl reload nginx
curl -v http://127.0.0.1/
```

যাচাই: `nginx -t` সফল; `curl` localhost:80-এ upstream-এর জবাব ফেরত দিয়েছে। **সাধারণ ব্যর্থতা:** 502 Bad Gateway মানে upstream বন্ধ — প্রথমে `curl http://127.0.0.1:3000` দিয়ে upstream সরাসরি পরীক্ষা করুন; `nginx -t` ব্যর্থ হলে `journalctl -u nginx` দেখুন। **Cleanup:** `sudo rm /etc/nginx/sites-enabled/reverse-proxy && sudo systemctl reload nginx`।

Lab 11 — Load Balancing: দুটি Upstream-এ ট্রাফিক ভাগ করা

পরিস্থিতি: TechNova-র অ্যাপের দুটি কপি চলছে — port 3001 ও 3002-এ। Nginx দুজনের মধ্যে অনুরোধ ভাগ করবে।

উদ্দেশ্য: Nginx upstream block ও round-robin load balancing।

পূর্বজ্ঞান: অধ্যায় ১৭, Lab 10। **টুল:** VM। **প্রাথমিক অবস্থা:** Nginx ইনস্টল করা; দুটি test সার্ভার চলছে (`python3 -m http.server 3001` ও `3002` , আলাদা ডিরেক্টরিতে)।

1. upstream block লিখুন: `upstream app { server 127.0.0.1:3001; server 127.0.0.1:3002; }` এবং `proxy_pass http://app;`।
2. `sudo nginx -t && sudo systemctl reload nginx` চালান।
3. `for i in 1 2 3 4; do curl -s http://127.0.0.1/ | head -2; done` চালিয়ে দেখুন অনুরোধ দুই সার্ভারে পালা করে যাচ্ছে কি না (প্রতিটি সার্ভারের ডিরেক্টরিতে আলাদা index.html রাখলে পার্থক্য বোঝা যাবে)।
4. একটি সার্ভার বন্ধ করে আবার curl চালান — Nginx স্বয়ংক্রিয়ভাবে জীবিত সার্ভারেই পাঠায় কি না, লক্ষ করুন।

```
sudo nginx -t && sudo systemctl reload nginx
for i in 1 2 3 4; do curl -s http://127.0.0.1/; done
```

যাচাই: চারটি অনুরোধে দুই সার্ভারের জবাব পালা করে এসেছে; একটি বন্ধ থাকলেও সাইট চলেছে। **সাধারণ ভুল:** session-নির্ভর অ্যাপে round-robin-এ sticky session না থাকলে লগইন ভেঙে যেতে পারে — production-এ `ip_hash` বা shared session বিবেচনা করুন। **Cleanup:** test সার্ভার বন্ধ করুন, site ফাইল সরান, reload দিন।

Lab 12 — UFW Firewall: শুধু দরকারি দরজা খোলা

পরিস্থিতি: TechNova-র সার্ভার ইন্টারনেটে যাবে। প্রথমে firewall: SSH ও HTTP খোলা, বাকি সব বন্ধ।

উদ্দেশ্য: UFW দিয়ে allow/deny নিয়ম, সক্রিয়করণ ও যাচাই।

পূর্বজ্ঞান: অধ্যায় ১৮। **টুল:** VM (snapshot নিয়ে নিন)। **প্রাথমিক অবস্থা:** UFW নিষ্ক্রিয়।

1. `sudo ufw status` দিয়ে বর্তমান অবস্থা দেখুন।
2. `sudo ufw allow OpenSSH` চালান — **এই ধাপটি enable-এর আগে অবশ্যই করুন।**
3. `sudo ufw allow 80/tcp` ও `sudo ufw allow 443/tcp` চালান।
4. `sudo ufw enable` চালিয়ে `sudo ufw status verbose` দিয়ে নিয়মগুলো যাচাই করুন।

5. `sudo ufw deny 8080/tcp` দিয়ে একটি test port বন্ধ করে `curl http://127.0.0.1:8080/` আচরণ দেখুন (localhost-এ UFW প্রযোজ্য নাও হতে পারে — তাই বাইরের VM থেকে পরীক্ষা করাই সঠিক যাচাই)।

```
sudo ufw allow OpenSSH
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw enable
sudo ufw status verbose
```

যাচাই: status-এ SSH, 80, 443 ALLOW দেখা গেছে; default deny incoming। **সাধারণ ব্যর্থতা:** SSH নিয়ম ছাড়া enable করলে নিজেই আটকে যাবেন — তখন VM console থেকে `sudo ufw disable` করতে হয়। **Cleanup:** ল্যাব শেষে `sudo ufw disable` ও নিয়মগুলো `sudo ufw reset` দিয়ে মুছে ফেলুন (নিশ্চিত হয়ে)।

Lab 13 — WireGuard VPN ধারণা: নিরাপদ টানেলের ল্যাব

পরিস্থিতি: TechNova-র কর্মী বাড়ি থেকে অফিসের private নেটওয়ার্কে পৌঁছাতে চায়। WireGuard টানেল কীভাবে কাজ করে, দুটি VM-এর মধ্যে পরীক্ষা করুন।

উদ্দেশ্য: WireGuard interface, keypair ও peer ধারণা হাতে-কলমে বোঝা।

পূর্বজ্ঞান: অধ্যায় ১৯। **টুল:** দুটি VM (peer-a, peer-b) একই Host-only নেটওয়ার্কে। **প্রাথমিক অবস্থা:** দুটি VM পরস্পরকে ping করতে পারে।

1. দুটি VM-এ `sudo apt install -y wireguard` চালান।
2. প্রতিটিতে `wg genkey | tee privatekey | wg pubkey > publickey` দিয়ে keypair তৈরি করুন — private key কখনও শেয়ার করবেন না, শুধু public key বিনিময় করুন।
3. peer-a-তে wg0 কনফিগার করুন: `[Interface] PrivateKey = <peer-a private> Address = 10.200.0.1/24 ListenPort = 51820` এবং `[Peer] PublicKey = <peer-b public> AllowedIPs = 10.200.0.2/32 Endpoint = 192.168.56.11:51820`।
4. peer-b-তে উল্টো কনফিগারেশন করুন, দুটিতে `sudo wg-quick up wg0` চালান।
5. `ping -c 4 10.200.0.2` দিয়ে টানেলের ভেতর দিয়ে পৌঁছানো যাচাই করুন; `sudo wg show` দিয়ে handshake দেখুন।

```
wg genkey | tee privatekey | wg pubkey > publickey
sudo wg-quick up wg0
sudo wg show
ping -c 4 10.200.0.2
```

যাচাই: `wg show`-এ latest handshake সময় দেখা গেছে; টানেল IP-তে ping সফল। **সাধারণ ব্যর্থতা:** Endpoint ভুল হলে handshake হয় না — `wg show`-এ handshake অনুপস্থিত থাকলে IP/port যাচাই করুন; firewall-এ UDP 51820 খুলতে হতে পারে। **নিরাপত্তা:** private key কখনও চ্যাট, ইমেইল বা Git-এ পাঠাবেন না। **Cleanup:** `sudo wg-quick down wg0` দুটি VM-এ; key ফাইল মুছে ফেলুন।

Lab 14 — tcpdump: তারের ওপর দিয়ে কী যাচ্ছে, দেখা

পরিস্থিতি: একটি অ্যাপ মাঝে মাঝে সংযোগ হারায়। Wireshark-এর আগে tcpdump দিয়ে দ্রুত দেখুন — DNS অনুরোধ যাচ্ছে কি না, TCP handshake সম্পূর্ণ হচ্ছে কি না।

উদ্দেশ্য: tcpdump filter, capture ও পড়া।

পূর্বজ্ঞান: অধ্যায় ২২। **টুল:** VM, sudo। **প্রাথমিক অবস্থা:** ইন্টারনেট আছে।

1. `sudo apt install -y tcpdump` ইনস্টল করুন।
2. একটি টার্মিনালে `sudo tcpdump -i any -n port 53 -c 10` চালান — DNS পোর্টের ১০টি প্যাকেট ধরা হবে।
3. আরেকটি টার্মিনালে `dig example.com` চালান — প্রথম টার্মিনালে DNS query/response দেখা যাবে।
4. `sudo tcpdump -i any -n host 192.168.56.1 -c 5` দিয়ে gateway-এর ট্রাফিক দেখুন।
5. `sudo tcpdump -i any -n -c 20 -w /tmp/lab14.pcap port 80` দিয়ে capture ফাইলে লিখুন; পরে `tcpdump -r /tmp/lab14.pcap` দিয়ে পড়ুন।

```
sudo tcpdump -i any -n port 53 -c 10
sudo tcpdump -i any -n -c 20 -w /tmp/lab14.pcap port 80
tcpdump -r /tmp/lab14.pcap
```

যাচাই: DNS query ও response দুটোই ধরা পড়েছে; pcap ফাইল পড়া গেছে। **সাধারণ ভুল:** `-n` ছাড়া tcpdump প্রতিটি IP-র reverse DNS করতে গিয়ে ধীর হয়। **নিরাপত্তা:** শুধু নিজের VM-এর interface-এ capture করুন; অন্যের ট্রাফিক capture করা অনুমতি ছাড়া নিষিদ্ধ। **Cleanup:** `rm /tmp/lab14.pcap`।

Lab 15 — Wireshark: TCP Handshake চোখে দেখা

পরিস্থিতি: একটি সার্ভারে সংযোগ মাঝে মাঝে ব্যর্থ হয়। Wireshark-এ handshake দেখে বুঝতে হবে — SYN যাচ্ছে, SYN-ACK ফিরছে কি না।

উদ্দেশ্য: Wireshark-এ capture, filter ও TCP handshake বিশ্লেষণ।

পূর্বজ্ঞান: অধ্যায় ১১, ২২। **টুল:** Host কম্পিউটারে Wireshark (বা VM-এ `sudo apt install -y wireshark` — GUI না থাকলে Lab 14-এর pcap Wireshark-এ খুলুন)। **প্রাথমিক অবস্থা:** একটি pcap ফাইল বা live capture।

1. Wireshark খুলে নিজের VM-এর interface বেছে capture শুরু করুন (অথবা `File → Open` দিয়ে /tmp/lab14.pcap খুলুন)।
2. Filter বারে লিখুন `tcp.flags.syn == 1` — শুধু SYN প্যাকেট দেখা যাবে।
3. একটি SYN প্যাকেটে ক্লিক করে TCP header খুলুন: Sequence Number, Window Size, MSS দেখুন।
4. Filter `tcp.stream eq 0` দিয়ে একটি পূর্ণ সংযোগের সব প্যাকেট দেখুন — SYN, SYN-ACK, ACK ক্রমটি মেলান।
5. Filter `dns` দিয়ে DNS query/response জোড়া দেখুন — Transaction ID মিলছে কি না যাচাই করুন।

```
tcp.flags.syn == 1
tcp.stream eq 0
dns
http.request
```

যাচাই: একটি stream-এ SYN → SYN-ACK → ACK ক্রমটি দেখা গেছে; DNS query-র Transaction ID response-এর সঙ্গে মিলেছে। **সাধারণ ব্যর্থতা:** SYN গিয়ে SYN-ACK না ফিরলে সার্ভার বন্ধ বা firewall ব্লক করছে; RST প্যাকেট মানে port বন্ধ। **নিরাপত্তা:** capture ফাইলে পাসওয়ার্ড বা টোকেন থাকতে পারে — ফাইলটি শেয়ার করবেন না, ল্যাব শেষে মুছে ফেলুন। **Cleanup:** capture বন্ধ করে pcap মুছুন।

Lab 16 — NAT ও Port Forwarding ধারণা: Private থেকে Public

পরিস্থিতি: TechNova-র private নেটওয়ার্কের (192.168.56.0/24) মেশিনগুলো একটি public IP দিয়ে ইন্টারনেটে যায়। NAT কীভাবে কাজ করে, তা ল্যাবে বোঝা দরকার।

উদ্দেশ্য: NAT/PAT ধারণা, `iptables`/nftables MASQUERADE পড়া ও যাচাই।

পূর্বজ্ঞান: অধ্যায় ১২। **টুল:** VM। **প্রাথমিক অবস্থা:** VM NAT মোডে ইন্টারনেটে যুক্ত।

1. `ip route` দিয়ে default gateway ও `curl -s ifconfig.me` দিয়ে বাইরে থেকে দেখা public IP (যদি ইন্টারনেট থাকে) লিখুন — দেখুন ভেতরের private IP আর বাইরের public IP আলাদা।
2. `sudo iptables -t nat -L -n -v` বা `sudo nft list ruleset` দিয়ে NAT নিয়ম আছে কি না দেখুন (VM-এ না থাকলে VirtualBox-এর NAT-ই এই কাজ করছে — সেটিই শিক্ষা: NAT একটি gateway-এর কাজ)।
3. Port forwarding ধারণা যাচাই: VirtualBox-এ VM-এর Settings → Network → Advanced → Port Forwarding-এ Host 8080 → Guest 80 নিয়ম যোগ করে host থেকে `curl http://127.0.0.1:8080/` চালান (Nginx চললে জবাব আসবে)।
4. নিয়মটি মুছে ফেলুন।

```
curl -s ifconfig.me
sudo iptables -t nat -L -n -v
ip route
```

যাচাই: ভেতরের ও বাইরের IP আলাদা দেখা গেছে; port forwarding নিয়মে host port থেকে guest সার্ভিসে পৌঁছানো গেছে। **সাধারণ ভুল:** port forwarding-কে firewall নিয়ম ভাবা — এটি ঠিকানা অনুবাদ, নিরাপত্তা নয়; forwarded port-এ সার্ভিসটি সবার কাছে খুলে যায়। **Cleanup:** forwarding নিয়ম মুছে ফেলুন।

পরিশিষ্ট A — Guided Labs (চলমান)

Lab 17 — নেটওয়ার্ক ব্যর্থতা নির্ণয়: ধাপে ধাপে, স্তরে স্তরে

পরিস্থিতি: TechNova-র সার্ভার থেকে example.com খুলছে না। কারণ কি DNS, gateway, নাকি সার্ভার নিজে? ইচ্ছাকৃতভাবে ভাঙা অবস্থা তৈরি করে স্তরে স্তরে নির্ণয় করুন।

উদ্দেশ্য: structured troubleshooting: link → IP → gateway → DNS → application।

পূর্বজ্ঞান: অধ্যায় ২২। **টুল:** VM, snapshot। **প্রাথমিক অবস্থা:** সব ঠিক আছে।

1. সব ঠিক আছে যাচাই করুন: `ping -c 2 192.168.56.1`, `ping -c 2 8.8.8.8`, `dig example.com +short`, `curl -s -o /dev/null -w "%{http_code}\n" https://example.com`।
2. ব্যর্থতা ১ তৈরি করুন: `sudo ip route del default` — এখন কী হয় দেখুন (gateway নেই)।
3. নির্ণয় করুন: `ip route` (default নেই), `ping -c 2 8.8.8.8` ব্যর্থ, কিন্তু `ping -c 2 192.168.56.1` সফল — সমস্যা gateway-এ।
4. মেরামত করুন: `sudo ip route add default via 192.168.56.1` ও আবার যাচাই করুন।
5. ব্যর্থতা ২: `/etc/resolv.conf`-এর nameserver বদলে ফেলবেন না; বরং `dig @192.0.2.1 example.com` (TEST-NET ঠিকানা, উত্তর আসবে না) দিয়ে ভুল resolver-এর আচরণ দেখুন — timeout মানে DNS পথের সমস্যা।

```
ping -c 2 192.168.56.1
ping -c 2 8.8.8.8
dig example.com +short
sudo ip route del default
sudo ip route add default via 192.168.56.1
```

যাচাই: প্রতিটি ব্যর্থতার পর কোন ধাপে আটকেছে, তা লিখতে পেরেছেন; মেরামতের পর সব পরীক্ষা আবার সফল। **সাধারণ ভুল:** একসঙ্গে অনেক কিছু বদলানো — একটি বদলান, যাচাই করুন, তারপর পরেরটি। **Cleanup:** default route ফিরিয়ে এনেছেন কি না, `ip route` দিয়ে নিশ্চিত হোন।

Lab 18 — CDN ও Cache Header: ব্রাউজার কী সংরক্ষণ করে

পরিস্থিতি: TechNova-র সাইটের ছবি বারবার ডাউনলোড হচ্ছে। Cache-Control header দেখে বুঝতে হবে — CDN/ব্রাউজার কতক্ষণ সংরক্ষণ করবে।

উদ্দেশ্য: `curl -I` দিয়ে Cache-Control, ETag, Age header পড়া।

পূর্বজ্ঞান: অধ্যায় ২০। **টুল:** VM, ইন্টারনেট। **প্রাথমিক অবস্থা:** ইন্টারনেট আছে।

1. `curl -sI https://example.com | grep -iE "cache-control|etag|age|cf-cache|server"` চালান।
2. `curl -sI https://example.com | head -20` দিয়ে পুরো header তালিকা পড়ুন।
3. নিজের Nginx-এ একটি static ফাইলের জন্য `add_header Cache-Control "public, max-age=3600";` যোগ করে `curl -I http://127.0.0.1/file.css` দিয়ে header যাচাই করুন (Lab 10-এর Nginx থাকলে)।

```
curl -sI https://example.com | grep -iE "cache-control|etag|age|server"
curl -sI https://example.com
```

যাচাই: Cache-Control/max-age মানটি পড়তে পেরেছেন; নিজের সার্ভারের header-এ max-age=3600 দেখা গেছে। **সাধারণ ভুল:** `no-cache` মানে সংরক্ষণ নিষেধ নয় — মানে ব্যবহারের আগে revalidate করতে হবে। **Cleanup:** Nginx-এর test header সরিয়ে reload দিন।

Lab 19 — SSH Tunneling: নিরাপদ পথে ভেতরের সার্ভিসে পৌঁছানো

পরিস্থিতি: TechNova-র ডেটাবেস শুধু private নেটওয়ার্কে শোনে। SSH tunnel দিয়ে নিজের মেশিন থেকে নিরাপদে পৌঁছাতে হবে, পোর্ট সবার জন্য খুলে নয়।

উদ্দেশ্য: SSH local port forwarding (`-L`) ব্যবহার করা।

পূর্বজ্ঞান: অধ্যায় ১৯। **টুল:** দুটি VM বা একটি VM + host। **প্রাথমিক অবস্থা:** SSH চলছে; target সার্ভিস শুধু 127.0.0.1:3000-এ শোনে।

1. Target VM-এ `python3 -m http.server 3000 --bind 127.0.0.1` চালান — এটি শুধু localhost থেকে পৌঁছানো যায়।
2. নিজের মেশিন থেকে `ssh -L 8080:127.0.0.1:3000 user@192.168.56.10 -N` চালান (user ও ঠিকানা আপনার ল্যাবের)।
3. আরেকটি টার্মিনালে `curl http://127.0.0.1:8080/` চালান — tunnel দিয়ে target-এর জবাব আসবে।
4. `ss -tlnp | grep 8080` দিয়ে দেখুন 8080 শুধু আপনার মেশিনে শুনছে।

```
ssh -L 8080:127.0.0.1:3000 user@192.168.56.10 -N
curl http://127.0.0.1:8080/
ss -tlnp | grep 8080
```

যাচাই: tunnel ছাড়া 3000-এ পৌঁছানো যায় না, tunnel দিয়ে 8080-এ জবাব আসে। **সাধারণ ব্যর্থতা:** `channel 3: open failed: connect failed` মানে target সার্ভিস বন্ধ — প্রথমে target-এ সরাসরি পরীক্ষা করুন। **নিরাপত্তা:** tunnel ব্যবহার শেষে Ctrl+C দিয়ে বন্ধ করুন; `-N` মানে shell নয়, শুধু forwarding। **Cleanup:** tunnel ও test সার্ভার বন্ধ করুন।

Lab 20 — IPv6: Link-Local থেকে Global

পরিস্থিতি: TechNova-র নতুন নেটওয়ার্কে IPv6 চালু হচ্ছে। আপনার VM-এ IPv6 ঠিকানা আছে কি, link-local কীভাবে কাজ করে — দেখুন।

উদ্দেশ্য: IPv6 ঠিকানা পড়া, সংক্ষেপণ ও ping6।

পূর্বজ্ঞান: অধ্যায় ৯। **টুল:** VM। **প্রাথমিক অবস্থা:** IPv6 সক্রিয় (ডিফল্ট)।

1. `ip -6 addr show` চালান — fe80:: দিয়ে শুরু link-local ও global ঠিকানা আলাদা করুন।
2. `ping -6 -c 3 fe80::1%enp0s3` চালিয়ে দেখুন link-local-এ interface উল্লেখ (%enp0s3) কেন দরকার (interface নাম আপনার VM-এর)।
3. `ip -6 route show` দিয়ে IPv6 default route আছে কি না দেখুন।

4. `dig example.com AAAA +short` দিয়ে একটি সাইটের IPv6 ঠিকানা নিন; `ping -6 -c 2` দিয়ে পরীক্ষা করুন (ইন্টারনেটে IPv6 থাকলে)।

```
ip -6 addr show
ip -6 route show
dig example.com AAAA +short
```

যাচাই: link-local (fe80::/10) ও global ঠিকানা আলাদা করতে পেরেছেন; `::` সংক্ষেপণের উদাহরণ লিখেছেন। **সাধারণ ভুল:** link-local ঠিকানায় scope/zone (%) ছাড়া ping করা — কোন interface দিয়ে যাবে, তা OS জানে না। **Cleanup:** কিছু বদলানো হয়নি।

Lab 21 — DHCP Lease যাচাই ও Static IP দেওয়া (ল্যাব)

পরিস্থিতি: TechNova-র একটি সার্ভারকে স্থায়ী IP দিতে হবে — DHCP-তে ভরসা নয়। Netplan দিয়ে static IP কনফিগার করুন, তবে ভুল হলে ফেরার পথ রাখুন।

উদ্দেশ্য: Netplan YAML দিয়ে static address, gateway ও DNS দেওয়া।

পূর্বজ্ঞান: অধ্যায় ১৪, ২১। **টুল:** VM, snapshot (অবশ্যই নিন)। **প্রাথমিক অবস্থা:** DHCP-তে ঠিকানা পাওয়া।

1. `ls /etc/netplan/` দিয়ে বর্তমান ফাইল দেখুন; `sudo cp /etc/netplan/50-cloud-init.yaml /etc/netplan/50-cloud-init.yaml.bak` দিয়ে ব্যাকআপ নিন (ফাইলের নাম আপনার VM-এ ভিন্ন হতে পারে)।
2. ফাইলটি সম্পাদনা করে DHCP-র বদলে লিখুন: `dhcp4: false`, `addresses: [192.168.56.20/24]`, `routes: [{to: default, via: 192.168.56.1}]`, `nameservers: {addresses: [8.8.8.8, 1.1.1.1]}` — YAML-এ indentation ঠিক রাখুন (space, tab নয়)।
3. `sudo netplan try` চালান — ১২০ সেকেন্ডের মধ্যে নিশ্চিত না করলে পুরোনো অবস্থায় ফিরে যাবে; এটাই নিরাপত্তা।
4. নিশ্চিত করে `sudo netplan apply` চালিয়ে `ip addr show` ও `ping -c 2 8.8.8.8` দিয়ে যাচাই করুন।

```
sudo cp /etc/netplan/50-cloud-init.yaml /etc/netplan/50-cloud-init.yaml.bak
sudo netplan try
sudo netplan apply
ip addr show
```

যাচাই: static IP দেখা গেছে; gateway ও DNS কাজ করছে। **সাধারণ ব্যর্থতা:** YAML indentation ভুল হলে `netplan try` ভুল জানাবে — tab ব্যবহার করবেন না। **Cleanup:** ব্যাকআপ ফাইল থেকে ফিরতে `sudo cp /etc/netplan/50-cloud-init.yaml.bak /etc/netplan/50-cloud-init.yaml && sudo netplan apply`।

Lab 22 — VLAN ধারণা: এক সুইচে দুটি আলাদা নেটওয়ার্ক

পরিস্থিতি: TechNova-র অফিস ও guest নেটওয়ার্ক একই সুইচে, কিন্তু আলাদা থাকতে হবে। VLAN ট্যাগিং কীভাবে আলাদা করে, তা ল্যাবে বোঝা দরকার।

উদ্দেশ্য: Linux-এ VLAN interface তৈরি ও যাচাই (ধারণাগত ল্যাব)।

পূর্বজ্ঞান: অধ্যায় ৬, ১৮। **টুল:** VM। **প্রাথমিক অবস্থা:** একটি interface।

1. `sudo apt install -y vlan` ইনস্টল করুন (প্রয়োজন না হলেও টুলটি চেনা ভালো)।

2. `sudo ip link add link enp0s3 name enp0s3.10 type vlan id 10` চালান — VLAN 10-এর virtual interface তৈরি হলো।
3. `sudo ip addr add 192.168.10.10/24 dev enp0s3.10 && sudo ip link set enp0s3.10 up` চালান।
4. `ip -d link show enp0s3.10` দিয়ে VLAN ID ও `ip addr show enp0s3.10` দিয়ে ঠিকানা যাচাই করুন।
5. `sudo ip link del enp0s3.10` দিয়ে মুছে ফেলুন।

```
sudo ip link add link enp0s3 name enp0s3.10 type vlan id 10
sudo ip addr add 192.168.10.10/24 dev enp0s3.10
ip -d link show enp0s3.10
sudo ip link del enp0s3.10
```

যাচাই: VLAN interface-এ id 10 দেখা গেছে; মুছে ফেলার পর আর নেই। **মনে রাখবেন:** আসল VLAN আলাদা করতে সুইচে trunk/access পোর্ট কনফিগার করতে হয় — শুধু VM-এ interface বানালেই নেটওয়ার্ক আলাদা হয় না। **Cleanup:** interface মুছে ফেলা হয়েছে।

Lab 23 — Cloud Network Architecture: কাগজে VPC ডিজাইন

পরিস্থিতি: TechNova-র অ্যাপ cloud-এ যাবে। কোনো খরচ ছাড়াই, কাগজে একটি VPC ডিজাইন করুন — public/private subnet, route table, NAT gateway, security group।

উদ্দেশ্য: cloud নেটওয়ার্কের উপাদানগুলো একত্রে সাজানো (ডিজাইন ল্যাব, কোনো cloud অ্যাকাউন্ট দরকার নেই)।

পূর্বজ্ঞান: অধ্যায় ২৩, ২৪। **টুল:** কাগজ বা draw.io। **প্রাথমিক অবস্থা:** CIDR 10.0.0.0/16 বরাদ্দ।

1. 10.0.0.0/16-কে ভাগ করুন: public 10.0.1.0/24 ও 10.0.2.0/24 (দুটি availability zone), private 10.0.11.0/24 ও 10.0.12.0/24, database 10.0.21.0/24 ও 10.0.22.0/24।
2. Internet gateway public subnet-এর route table-এ (0.0.0.0/0 → IGW); NAT gateway public subnet-এ বসিয়ে private subnet-এর route table-এ (0.0.0.0/0 → NAT)।
3. Security group লিখুন: ALB-এ 80/443 সবার থেকে; app সার্ভারে শুধু ALB থেকে; database-এ শুধু app subnet থেকে 5432।
4. ডায়াগ্রামে প্রতিটি subnet, route ও security group-এর নিয়ম লেবেল করুন।

ডায়াগ্রাম — VPC 10.0.0.0/16

Internet → Internet Gateway → Public subnet (ALB, NAT GW) → Private subnet (App) → DB subnet (Database, শুধু App থেকে 5432)

Public route: 0.0.0.0/0 → IGW | Private route: 0.0.0.0/0 → NAT GW

যাচাই: প্রতিটি subnet-এর CIDR ওভারল্যাপ করে না; DB subnet-এ কোনো internet route নেই; প্রতিটি tier-এর নিয়ম লেখা আছে। **সাধারণ ভুল:** database subnet-কে public করা; NAT gateway-কে private subnet-এ বসানো (সেটি কাজ করবে না, NAT-কে public-এ থাকতে হয়)। **Cleanup:** এটি ডিজাইন ল্যাব — কোনো resource তৈরি হয়নি, খরচ শূন্য।

Lab 24 — Multi-Tier Network Health Check: পুরো বইয়ের সমন্বয়

পরিস্থিতি: TechNova-র তিন-স্তরের (web/app/db) ল্যাব নেটওয়ার্কের সাপ্তাহিক স্বাস্থ্য পরীক্ষা। একটি স্ক্রিপ্টে সব যাচাই একত্র করুন।

উদ্দেশ্য: interface, route, DNS, port, HTTP ও TLS যাচাইয়ের একক চেকলিস্ট চালানো।

পূর্বজ্ঞান: অধ্যায় ২১–২৪, Lab 01–23। **টুল:** VM, Nginx (Lab 10)। **প্রাথমিক অবস্থা:** Nginx চলছে।

1. একটি চেকলিস্ট ফাইল তৈরি করুন: `ip -brief addr`, `ip route`, `dig example.com +short`, `ss -tlnp`, `curl -s -o /dev/null -w "%{http_code}\n" http://127.0.0.1/`, `sudo ufw status`।
2. প্রতিটি কমান্ড চালিয়ে ফলাফল লিখুন — কোনটি সফল, কোনটি ব্যর্থ, কেন।
3. ইচ্ছাকৃতভাবে একটি ব্যর্থতা তৈরি করুন (Nginx বন্ধ করুন: `sudo systemctl stop nginx`), চেকলিস্ট আবার চালিয়ে কোন ধাপে ধরা পড়ল, তা লিখুন।
4. `sudo systemctl start nginx` দিয়ে ফিরিয়ে এনে শেষ যাচাই করুন।

```
ip -brief addr; ip route
dig example.com +short
ss -tlnp
curl -s -o /dev/null -w "%{http_code}\n" http://127.0.0.1/
sudo ufw status
```

যাচাই: Nginx বন্ধ থাকলে curl ধাপে ব্যর্থতা ধরা পড়েছে; চালু থাকলে সব ধাপ সবুজ। **সাধারণ ভুল:** একটি ধাপের ব্যর্থতাকে পুরো নেটওয়ার্কের ব্যর্থতা ভাবা — ধাপটি লিখুন, তারপর কারণ খুঁজুন। **Cleanup:** Nginx চালু অবস্থায় রেখে যান; চেকলিস্ট ফাইলটি সংরক্ষণ করুন — এটি আপনার প্রথম runbook।

পরিশিষ্ট B — Independent Assignments (স্বাধীন অ্যাসাইনমেন্ট)

নিচের ১৫টি অ্যাসাইনমেন্টের সমাধান এই বইয়ে দেওয়া নেই — সমাধান-নির্দেশিকা আলাদা ভলিউমে আছে। প্রতিটি অ্যাসাইনমেন্ট করার আগে সংশ্লিষ্ট অধ্যায় ও ল্যাব শেষ করুন। উত্তর লেখার সময় শুধু ফলাফল নয়, পদ্ধতি ও যুক্তিও লিখুন।

Beginner — অ্যাসাইনমেন্ট ১–৫

1. **নিজের নেটওয়ার্কের মানচিত্র:** আপনার VM-এর `ip addr`, `ip route` ও `ip neigh` আউটপুট থেকে একটি ডায়াগ্রাম আঁকুন — interface, IP, gateway, DNS resolver ও প্রতিবেশী MAC-সহ। প্রতিটি উপাদানের কাজ এক লাইনে লিখুন।
2. **Binary অনুশীলন:** 192.168.10.25, 255.255.255.0 ও 172.16.0.1-কে binary-তে রূপান্তর করুন; 11000000.10101000.00001010.00011001-কে decimal-এ ফেরত নিন। প্রতিটি ধাপ দেখান।
3. **OSI-তে একটি অনুরোধ:** ব্রাউজারে example.com খোলার যাত্রা OSI-র সাতটি স্তরে লিখুন — প্রতিটি স্তরে কী যোগ হয়, কোন ডিভাইস কাজ করে।
4. **Port শনাক্তকরণ:** আপনার VM-এ `ss -tlnp` চালিয়ে প্রতিটি listening port-এর সার্ভিস, process ও ঝুঁকি লিখুন। কোন port-টি বাইরে থেকে পৌঁছানো উচিত নয়, এবং কেন?
5. **DNS তদন্ত:** আপনার পছন্দের একটি ডোমেইনের A, AAAA, MX ও NS রেকর্ড `dig` দিয়ে বের করুন। TTL মানগুলো লিখে ব্যাখ্যা করুন — TTL বদলালে propagation-এ কী প্রভাব পড়ে?

Subnetting ও গণনা — অ্যাসাইনমেন্ট ৬–৮

1. **Subnet ভাগ:** 10.10.0.0/22 নেটওয়ার্ককে চারটি সমান subnet-এ ভাগ করুন। প্রতিটির network, broadcast, usable range ও host সংখ্যা লিখুন। তারপর একটি /23 subnet-এর জন্য একই হিসাব করুন।
2. **VLSM পরিকল্পনা:** একটি অফিসে চারটি বিভাগের প্রয়োজন — 100, 50, 25 ও 10টি host। 192.168.40.0/24 থেকে VLSM দিয়ে প্রতিটি বিভাগের subnet বরাদ্দ করুন, যাতে কোনো ঠিকানা অপচয় না হয়।
3. **CIDR যাচাই:** 172.16.8.0/21-এর network, broadcast ও usable host সংখ্যা বের করুন। 172.16.15.255 ঠিকানাটি এই নেটওয়ার্কের ভেতরে কি না, যুক্তি দিয়ে দেখান।

Protocol ও HTTP — অ্যাসাইনমেন্ট ৯–১১

1. **TCP বনাম UDP সিদ্ধান্ত:** ভিডিও কল, ফাইল ডাউনলোড, DNS lookup ও অনলাইন গেম — প্রতিটির জন্য TCP না UDP, যুক্তিসহ লিখুন। একটি ক্ষেত্রে ভুল প্রোটোকল বেছে নিলে কী ক্ষতি হবে?
2. **HTTP পরিদর্শন:** `curl -v` দিয়ে একটি সাইটের GET অনুরোধের request header ও response header লিখুন। প্রতিটি header-এর কাজ ব্যাখ্যা করুন; 301 ও 302-এর পার্থক্য লিখুন।
3. **TLS যাচাই:** `openssl s_client` দিয়ে দুটি সাইটের certificate-এর issuer, মেয়াদ ও TLS সংস্করণ তুলনা করুন। একটি certificate মেয়াদ শেষ হলে ব্যবহারকারী কী দেখবে, তা লিখুন।

ডিজাইন ও Troubleshooting — অ্যাসাইনমেন্ট ১২–১৫

1. **Firewall নীতি ডিজাইন:** একটি web সার্ভারের জন্য UFW নীতি লিখুন — SSH (শুধু অফিস IP থেকে), HTTP/HTTPS (সবার থেকে), বাকি সব বন্ধ। প্রতিটি নিয়মের যুক্তি ও ভুলের ঝুঁকি লিখুন।
2. **নেটওয়ার্ক ডায়াগ্রাম:** একটি ছোট স্টার্টআপের নেটওয়ার্ক ডিজাইন করুন — router, switch, firewall, web/app/db সার্ভার, VLAN। IP পরিকল্পনাসহ ডায়াগ্রাম আঁকুন ও প্রতিটি সংযোগে প্রোটোকল লিখুন।
3. **Troubleshooting কেস:** "সার্ভার থেকে ping যায়, কিন্তু curl ব্যর্থ" — সম্ভাব্য পাঁচটি কারণ লিখুন এবং প্রতিটি যাচাইয়ের কমান্ড দিন। কোন কারণটি সবচেয়ে সম্ভাব্য, যুক্তি দিন।
4. **Production স্থাপত্য:** TechNova-র অ্যাপের জন্য একটি পূর্ণ network স্থাপত্য প্রস্তাব লিখুন — public/private segmentation, DNS, TLS, reverse proxy, load balancing, firewall, DB isolation, monitoring। প্রতিটি সিদ্ধান্তের trade-off (খরচ, জটিলতা, নিরাপত্তা) এক অনুচ্ছেদে লিখুন। সঙ্গে Lab 17-এর ব্যর্থতা পরীক্ষার ভিত্তিতে একটি এক পৃষ্ঠার incident রিপোর্ট যোগ করুন — লক্ষণ, নির্ণয়ের ধাপ, মূল কারণ, সমাধান, ভবিষ্যতে প্রতিরোধ।

পরিশিষ্ট C — Major Projects (তিনটি বড় প্রজেক্ট)

Project 01 — Network Investigation Toolkit

ব্যবসায়িক পরিস্থিতি: TechNova-র সাপোর্ট দল প্রতিদিন একই প্রশ্ন পায় — "সার্ভার কি নেটওয়ার্কে আছে?" প্রতিবার হাতে হাতে কমান্ড চালানো ধীর ও ভুলপ্রবণ। আপনার কাজ: একটি Linux diagnostic toolkit তৈরি করা, যা এক কমান্ডে interface, IP, route, DNS, listening port ও connectivity যাচাই করে একটি প্রতিবেদন দেয়।

প্রয়োজনীয়তা: Bash স্ক্রিপ্ট; `ip`, `ss`, `dig`, `ping`, `curl` ব্যবহার; প্রতিটি যাচাইয়ের ফলাফল PASS/FAIL; আউটপুট ফাইলে সংরক্ষণ; কোনো credential বা সংবেদনশীল তথ্য আউটপুটে নয়।

আর্কিটেকচার — Toolkit

toolkit.sh → ip addr/route → ss -tlnp → dig + ping + curl → প্রতিবেদন (report.txt)

প্রতিটি ধাপ স্বাধীন; একটি ধাপ ব্যর্থ হলেও বাকিগুলো চলবে।

মাইলফলক: (১) যাচাইয়ের তালিকা লেখা; (২) স্ক্রিপ্ট লেখা ও VM-এ চালানো; (৩) ইচ্ছাকৃত ব্যর্থতা (Nginx বন্ধ, DNS ভুল) দিয়ে পরীক্ষা; (৪) সীমাবদ্ধতা নথিভুক্ত করা।

ডেলিভারেবল: toolkit.sh, নমুনা প্রতিবেদন, README (ব্যবহার ও সীমাবদ্ধতা)। **গ্রহণযোগ্যতা:** সব যাচাই চলে; ব্যর্থতা সঠিকভাবে FAIL দেখায়; স্ক্রিপ্ট শুধু পড়ার কাজ করে, কোনো সার্ভিস বদলায় না।

মানদণ্ড	ভালো	উত্তম
Coverage	৫টি যাচাই আছে	৮+ যাচাই, প্রতিটির ব্যাখ্যা আছে
নির্ভরযোগ্যতা	স্বাভাবিক অবস্থায় চলে	ব্যর্থ অবস্থাতেও থামে না, স্পষ্ট বার্তা দেয়
নথি	ব্যবহার লেখা আছে	সীমাবদ্ধতা ও ভুল-ধারণা লেখা আছে

উন্নত চ্যালেঞ্জ: JSON আউটপুট যোগ করুন; একটি ধাপে latency মাপুন; cron-এ সাপ্তাহিক চালানোর পরিকল্পনা লিখুন।

Project 02 — Secure Web Application Network

ব্যবসায়িক পরিস্থিতি: TechNova-র web অ্যাপ সরাসরি port 3000-এ চলছে — কোনো TLS নেই, firewall নেই, ডোমেইন নেই। আপনার কাজ: Nginx-এর পেছনে অ্যাপটি সুরক্ষিতভাবে প্রকাশ করা।

প্রয়োজনীয়তা: DNS রেকর্ড পরিকল্পনা (A/AAAA), TLS (self-signed নয় — ল্যাবে Let's Encrypt ধারণা বা locally trusted certificate-এর পরিকল্পনা; আসল ডোমেইন না থাকলে self-signed শুধু ল্যাবে, সীমাবদ্ধতাসহ), Nginx reverse proxy, UFW firewall, upstream health যাচাই, verification রিপোর্ট।

আর্কিটেকচার — Secure Web App

Internet → DNS → Firewall (80/443) → Nginx (TLS termination, reverse proxy) → App 127.0.0.1:3000

SSH শুধু অফিস নেটওয়ার্ক থেকে; DB থাকলে শুধু App থেকে।

মাইলফলক: (১) DNS ও TLS পরিকল্পনা; (২) Nginx reverse proxy (Lab 10); (৩) TLS চালু ও `openssl s_client` দিয়ে যাচাই; (৪) UFW নিয়ম ও বাইরে থেকে যাচাই; (৫) verification রিপোর্ট।

ডেলিভারেবল: নেটওয়ার্ক ডায়াগ্রাম, Nginx কনফিগারেশন, UFW নিয়মের তালিকা, TLS যাচাইয়ের আউটপুট, verification রিপোর্ট। **গ্রহণযোগ্যতা:** HTTP থেকে HTTPS-এ redirect; TLS মেয়াদ ও chain যাচাই সফল; firewall-এ শুধু প্রয়োজনীয় port খোলা; upstream বন্ধ হলে 502-এর বদলে অর্থবোধক error পৃষ্ঠা।

মানদণ্ড	ভালো	উত্তম
TLS	HTTPS চলে	chain, মেয়াদ, redirect যাচাই নথিভুক্ত
Firewall	80/443 খোলা	SSH সীমিত, বাকি সব বন্ধ, যাচাই আছে
যাচাই	curl দিয়ে পরীক্ষা	ব্যর্থতার পরীক্ষা (upstream বন্ধ, ভুল পথ) নথিভুক্ত

উন্নত চ্যালেঞ্জ: rate limiting যোগ করুন; দুটি upstream দিয়ে load balancing যোগ করুন; security header (HSTS) পরিকল্পনা লিখুন।

Project 03 — TechNova Production Network Capstone

ব্যবসায়িক পরিস্থিতি: TechNova একটি fictional সফটওয়্যার কোম্পানি। তাদের web অ্যাপে তিনটি tier — web, application, database। বর্তমানে সবকিছু এক নেটওয়ার্কে, কোনো segmentation নেই। আপনি জুনিয়র নেটওয়ার্ক ইঞ্জিনিয়ার হিসেবে production নেটওয়ার্কের পূর্ণ নকশা ও ল্যাব প্রদর্শন করবেন।

প্রয়োজনীয়তা: Public/private segmentation, IP ও subnet পরিকল্পনা, routing ও gateway, DNS, TLS, reverse proxy, load balancing, firewall নীতি, database isolation, troubleshooting পরিকল্পনা, availability, monitoring ও নথিপত্র।

আর্কিটেকচার — TechNova Production Network

Internet → Firewall → Public subnet (Load Balancer / Nginx) → Private subnet (App tier) → DB subnet (Database, শুধু App থেকে 5432)

DNS: public zone + private zone | Monitoring: প্রতিটি tier-এর health check | Backup: DB subnet থেকে আলাদা নেটওয়ার্কে

মাইলফলক: (১) প্রয়োজন সংগ্রহ ও IP পরিকল্পনা (VLSM-সহ); (২) ডায়াগ্রাম ও firewall নীতি; (৩) ল্যাবে Nginx reverse proxy + load balancing (Lab 10–11); (৪) DNS ও TLS পরিকল্পনা; (৫) ব্যর্থতার পরীক্ষা — একটি app সার্ভার বন্ধ, একটি subnet-এর route মুছে ফেলা, DNS ভুল — প্রতিটির নির্ণয় নথিভুক্ত; (৬) চূড়ান্ত প্রতিবেদন।

ডেলিভারেবল: স্থাপত্য নথি (ডায়াগ্রামসহ), IP/subnet পরিকল্পনার টেবিল, firewall নীতি, DNS রেকর্ড তালিকা, TLS পরিকল্পনা, ল্যাব যাচাইয়ের প্রতিবেদন, incident response চেকলিস্ট, monitoring পরিকল্পনা।

গ্রহণযোগ্যতা: DB subnet ইন্টারনেট থেকে পৌঁছানো যায় না; প্রতিটি tier-এর নিয়ম লেখা ও যাচাই করা; একটি app সার্ভার বন্ধ হলেও সাইট চলে; প্রতিটি ব্যর্থতার নির্ণয়ের ধাপ নথিভুক্ত।

মানদণ্ড	ভালো	উত্তম
Segmentation	তিনটি tier আলাদা subnet-এ	প্রতিটি tier-এর firewall নিয়ম যাচাইসহ

Availability	একটি সার্ভার বন্ধ হলেও চলে	failover পরীক্ষার প্রমাণ আছে
নথি	ডায়াগ্রাম ও IP তালিকা আছে	runbook, incident চেকলিস্ট ও monitoring আছে

উন্নত চ্যালেঞ্জ: দ্বিতীয় availability zone-এর পরিকল্পনা যোগ করুন; VPN দিয়ে admin access-এর নকশা করুন; খরচের হিসাব (cloud-এ) লিখুন — কোন উপাদানে কত খরচ, তা অনুমানসহ।

পরিশিষ্ট D — Final Examination (চূড়ান্ত পরীক্ষা)

মোট ১০০ নম্বর। সময়: ৩ ঘণ্টা। ল্যাবের কাজগুলো নিজের VM-এ করবেন; তত্ত্বের প্রশ্নগুলো লিখে উত্তর দেবেন। পাস নম্বর ৬০।

Practical — ৫টি কাজ (প্রতিটি ৮ নম্বর, মোট ৪০)

1. আপনার VM-এর interface, IP, gateway ও DNS-এর একটি প্রতিবেদন তৈরি করুন (`ip`, `dig`)। প্রতিটি মানের অর্থ এক লাইনে লিখুন।
2. 192.168.80.0/24 নেটওয়ার্ককে ৮টি সমান subnet-এ ভাগ করুন; তৃতীয় subnet-এর network, broadcast ও usable range লিখুন এবং `ipcalc` দিয়ে যাচাই করুন।
3. Nginx reverse proxy বসিয়ে port 3000-এর test অ্যাপকে port 80-এ প্রকাশ করুন; `curl` দিয়ে যাচাই করুন।
4. UFW-তে SSH, 80 ও 443 ছাড়া সব বন্ধ করুন; `ufw status verbose` দিয়ে প্রমাণ দিন।
5. tcpdump দিয়ে DNS query ধরে দেখান — কোন নামের জন্য query গেছে, কোন উত্তর এসেছে।

Short-Answer — ১০টি প্রশ্ন (প্রতিটি ৩ নম্বর, মোট ৩০)

1. OSI মডেলের স্তরগুলো কেন দরকার? একটি স্তরের ভুল কীভাবে অন্য স্তরে দেখা দেয়, উদাহরণ দিন।
2. TCP-র three-way handshake কেন তিন ধাপের? SYN-ACK না ফিরলে কী হয়?
3. Private IP ইন্টারনেটে সরাসরি ব্যবহার করা যায় না কেন? NAT কীভাবে সমস্যাটি সমাধান করে?
4. DNS-এ recursive ও iterative resolution-এর পার্থক্য কী?
5. HTTP 301, 404 ও 503-এর অর্থ ও একটি করে বাস্তব কারণ লিখুন।
6. TLS handshake-এ certificate-এর ভূমিকা কী? Certificate মেয়াদ শেষ হলে কী হয়?
7. Forward proxy ও reverse proxy-এর পার্থক্য কী? প্রতিটির একটি ব্যবহার লিখুন।
8. Stateful ও stateless firewall-এর পার্থক্য কী?
9. CDN কেন ওয়েবসাইটকে দ্রুত করে? Origin server-এর ভূমিকা কী থাকে?
10. IPv6-এ NAT-এর প্রয়োজন কম কেন? Link-local ঠিকানার কাজ কী?

Troubleshooting Case Studies — ৪টি কেস (প্রতিটি ৫ নম্বর, মোট ২০)

1. **"ping যায়, curl ব্যর্থ":** সম্ভাব্য কারণগুলো স্তরে স্তরে লিখুন; প্রতিটি যাচাইয়ের কমান্ড দিন।
2. **"DNS বদলের ২৪ ঘণ্টা পরেও পুরোনো সাইট দেখাচ্ছে":** TTL ও cache-এর ভূমিকা ব্যাখ্যা করুন; যাচাইয়ের ধাপ লিখুন।
3. **"Nginx 502 Bad Gateway দিচ্ছে":** upstream, port ও log দিয়ে নির্ণয়ের ধাপ লিখুন।
4. **"একটি subnet থেকে DB-তে পৌঁছানো যাচ্ছে, আরেকটি থেকে নয়":** routing ও firewall — দুই দিকের যাচাই লিখুন।

Architecture Design — ১টি প্রশ্ন (১০ নম্বর)

একটি ই-কমার্স সাইটের জন্য তিন-স্তরের নেটওয়ার্ক ডিজাইন করুন — public/private subnet, DNS, TLS, reverse proxy, load balancing, firewall, DB isolation। ডায়াগ্রাম আঁকুন, IP পরিকল্পনা দিন, প্রতিটি সিদ্ধান্তের যুক্তি লিখুন।

Assessment Rubric — নিজেকে মাপুন

স্তর	লক্ষণ	পরবর্তী ধাপ
Memorisation	শব্দ মনে আছে, packet-এর যাত্রা বলতে পারেন না	প্রতিটি প্রোটোকলের যাত্রা নিজের ভাষায় লিখুন
Application	গাইড দেখে ল্যাব করতে পারেন	গাইড বন্ধ করে আবার করুন
Understanding	ব্যর্থতার কারণ খুঁজে বের করতে পারেন	ইচ্ছে করে ভাঙা নেটওয়ার্ক ঠিক করুন
Judgement	trade-off বুঝে নকশা করতে পারেন	TechNova capstone সম্পূর্ণ করুন

পরিশিষ্ট E — Glossary (শব্দকোষ)

ইংরেজি শব্দ, বাংলা অর্থ ও সংক্ষিপ্ত সংজ্ঞা; শেষ কলামে সংশ্লিষ্ট অধ্যায়।

Term	বাংলা অর্থ / সংজ্ঞা	অধ্যায়
Address Resolution Protocol (ARP)	IP ঠিকানা থেকে MAC ঠিকানা বের করার প্রোটোকল	৬, ১৪
Anycast	একই ঠিকানায় একাধিক সার্ভার; নিকটতমটি উত্তর দেয়	২০
Application Layer	OSI/TCP-IP-র সর্বোচ্চ স্তর; অ্যাপের প্রোটোকল থাকে	৩, ৪
Asymmetric Encryption	public ও private key-এর জোড়ায় এনক্রিপশন	১৬
Autonomous System (AS)	এক প্রশাসনের অধীনে নেটওয়ার্কের সমষ্টি; BGP-র একক	১২
Availability Zone (AZ)	Cloud-এ আলাদা ডেটা সেন্টার এলাকা; failover-এর ভিত্তি	২৩
Bandwidth	পথের সর্বোচ্চ ধারণক্ষমতা; bits per second-এ মাপা হয়	১
Border Gateway Protocol (BGP)	Autonomous System-গুলোর মধ্যে routing-এর প্রোটোকল	১২
Broadcast	এক নেটওয়ার্কের সবাইকে পাঠানো বার্তা	৬
Broadcast Domain	যে এলাকায় broadcast পৌঁছায়	৬
Cache	দ্রুত পুনঃব্যবহারের জন্য সংরক্ষিত কপি	১৩, ২০
Cache-Control	HTTP header; কতক্ষণ cache রাখা যাবে, তা জানায়	১৫, ২০
Certificate Authority (CA)	সার্টিফিকেট দেওয়া ও স্বাক্ষর করা বিশ্বস্ত সংস্থা	১৬
Certificate Signing Request (CSR)	সার্টিফিকেট পাওয়ার আবেদন; public key ও পরিচয় থাকে	১৬
CIDR (Classless Inter-Domain Routing)	IP বরাদ্দের পদ্ধতি; /24-এর মতো prefix দিয়ে প্রকাশ	৮
Client	সেবা চাওয়া কম্পিউটার বা প্রোগ্রাম	১
Cloudflare	CDN, DNS ও নিরাপত্তা সেবা দেওয়া edge নেটওয়ার্ক	২০
Collision Domain	যে এলাকায় দুটি সংকেত ধাক্কা খেতে পারে	৬
Congestion Control	নেটওয়ার্ক ভিড় কমাতে TCP-র গতি সমন্বয়	১১
Content Delivery Network (CDN)	বিশ্বজুড়ে edge সার্ভারে কনটেন্ট ছড়িয়ে দেওয়া নেটওয়ার্ক	২০
Cookie	ব্রাউজারে সংরক্ষিত ছোট তথ্য; session চেনার কাজে লাগে	১৫
Data Link Layer	OSI স্তর ২; frame, MAC ও একই ল্যানে পৌঁছে দেওয়া	৩
Datagram	UDP-র স্বাধীন বার্তা একক; পৌঁছানোর নিশ্চয়তা নেই	১০
Decapsulation	প্যাকেট থেকে স্তরে স্তরে header খোলা	৩, ৪

Default Gateway	নিজের নেটওয়ার্কের বাইরে যাওয়ার প্রথম router	১২
Demilitarized Zone (DMZ)	ইন্টারনেট ও ভেতরের নেটওয়ার্কের মাঝের সুরক্ষিত এলাকা	১৮
DHCP (Dynamic Host Configuration Protocol)	IP, gateway, DNS স্বয়ংক্রিয়ভাবে বরাদ্দের প্রোটোকল	১৪
DHCP Lease	DHCP-পাওয়া ঠিকানার নির্ধারিত ব্যবহারের সময়	১৪
dig	DNS lookup-এর Linux কমান্ড	১৩, ২১
Digital Signature	private key-এর স্বাক্ষর; সত্যতা ও অখণ্ডতা যাচাই করে	১৬
DNS (Domain Name System)	ডোমেইন নাম থেকে IP ঠিকানা বের করার ব্যবস্থা	১৩
DNS over HTTPS (DoH)	HTTPS-এর ভেতর দিয়ে এনক্রিপ্ট করা DNS query	১৩
DNS over TLS (DoT)	TLS-এর ভেতর দিয়ে এনক্রিপ্ট করা DNS query	১৩
DNS Record	DNS-এর তথ্য একক; A, MX, CNAME ইত্যাদি	১৩
Domain	ইন্টারনেটে নামের ঠিকানা; যেমন example.com	১৩
DORA	DHCP-র চার ধাপ: Discover, Offer, Request, Acknowledge	১৪
Dual-Stack	IPv4 ও IPv6 একসঙ্গে চালানো	৯
Dynamic Routing	প্রোটোকল দিয়ে স্বয়ংক্রিয়ভাবে route শেখা	১২
Edge Location	CDN-এর ব্যবহারকারীর কাছের সার্ভার কেন্দ্র	২০
Encapsulation	ডেটার চারপাশে স্তরে স্তরে header যোগ করা	৩, ৪
Encryption	তথ্য গোপন করতে অপর পাঠ্যরূপে বদলানো	১৬
Endpoint	সংযোগের এক প্রান্ত; VPN-এ peer-এর ঠিকানা	১৯
Ethernet	ল্যানের প্রধান প্রযুক্তি; frame ও MAC ব্যবহার করে	৬
Ethernet Frame	Ethernet-এ ডেটার একক; header, payload, FCS থাকে	৬
Firewall	নিয়ম অনুযায়ী ট্রাফিক আটকানো বা ছাড়া সুরক্ষা ব্যবস্থা	১৮
Flow Control	প্রাপক সামলাতে পারে, এমন গতিতে পাঠানো	১১
Forward Proxy	ক্লায়েন্টের হয়ে ইন্টারনেটে যাওয়া proxy	১৭
Full-Duplex	একই সঙ্গে দুই দিকে যোগাযোগ	৬
Gateway	দুটি নেটওয়ার্কের মাঝে যাওয়ার দরজা	১২
Global Unicast Address	IPv6-এ বিশ্বজুড়ে অনন্য ঠিকানা	৯
Health Check	সার্ভার জীবিত কি না, নিয়মিত পরীক্ষা	১৭

Hop	প্যাকেটের পথে একটি router পার হওয়া	১৪
Host	নেটওয়ার্কে যুক্ত যেকোনো ডিভাইস	১
HTTP (Hypertext Transfer Protocol)	ওয়েবের request/response প্রোটোকল	১৫
HTTP/2, HTTP/3	HTTP-র নতুন সংস্করণ; দ্রুত ও কার্যকর	১৫
HTTPS	TLS-এ সুরক্ষিত HTTP	১৫, ১৬
ICMP (Internet Control Message Protocol)	ভুল ও অবস্থা জানানোর প্রোটোকল; ping-এ ব্যবহৃত	১৪
Interface	নেটওয়ার্কে যুক্ত হওয়ার পয়েন্ট; যেমন enp0s3	৫, ২১
Internet Gateway (IGW)	Cloud VPC-কে ইন্টারনেটের সঙ্গে যুক্ত করা gateway	২৩
Internet Layer	TCP/IP-র স্তর; IP, routing থাকে	৪
IP Address	নেটওয়ার্কে ডিভাইসের সংখ্যাগত ঠিকানা	৭
IP Forwarding	Kernel-এর প্যাকেট এক interface থেকে আরেকটিতে পাঠানো	১২
iproute2	`ip` কমান্ডের টুলসেট	২১
IPv4	৩২-bit ইন্টারনেট ঠিকানা পদ্ধতি	৭
IPv6	১২৮-bit ইন্টারনেট ঠিকানা পদ্ধতি	৯
Keep-Alive	এক সংযোগে একাধিক HTTP অনুরোধ পাঠানো	১৫
LAN (Local Area Network)	ছোট এলাকার নেটওয়ার্ক; অফিস, বাড়ি, ক্যাম্পাস	১
Latency	বার্তা যেতে লাগা সময়; দেরি	১
Layer 2 / Layer 3	OSI স্তর ২ (switching) / স্তর ৩ (routing)	৩, ৫
Link-Local Address	একই লিংকে ব্যবহারের IPv6 ঠিকানা (fe80::/10)	৯
Load Balancer	ট্রাফিক একাধিক সার্ভারে ভাগ করা ব্যবস্থা	১৭
Loopback Address	নিজের মেশিনের ঠিকানা; 127.0.0.1, ::1	৭
MAC Address	নেটওয়ার্ক কার্ডের হার্ডওয়্যার ঠিকানা	৬
MAC Address Table	Switch-এর MAC → port তালিকা	৬
MAN (Metropolitan Area Network)	শহরজুড়ে নেটওয়ার্ক	১
Maximum Transmission Unit (MTU)	এক প্যাকেটে পাঠানো সর্বোচ্চ আকার	৬
Mesh Topology	প্রতিটি নোড অন্য অনেক নোডের সঙ্গে যুক্ত	২
Multicast	একটি গ্রুপের সদস্যদের পাঠানো বার্তা	৬
NAT (Network Address Translation)	Private ঠিকানাকে public ঠিকানায় বদলানো	১২

NAT Gateway	Cloud-এ private subnet-কে ইন্টারনেটে যাওয়ার পথ দেওয়া সেবা	২৩
Netplan	Ubuntu-র নেটওয়ার্ক কনফিগারেশন টুল	২১
Network Address	Subnet-এর প্রথম ঠিকানা; নেটওয়ার্ক চেনায়	৮
Network Interface Card (NIC)	নেটওয়ার্কে যুক্ত হওয়ার হার্ডওয়্যার কার্ড	৫
Network Mask / Subnet Mask	ঠিকানার network ও host অংশ আলাদা করা মাস্ক	৭, ৮
Network Segment / Segmentation	নেটওয়ার্ককে আলাদা আলাদা অংশে ভাগ করা	২, ১৮
Next Hop	প্যাকেটের পথে পরের router	১২
nftables	Linux-এর আধুনিক firewall কাঠামো	১৮
Nginx	Web server, reverse proxy ও load balancer সফটওয়্যার	১৭
nslookup	DNS lookup-এর পুরোনো কমান্ড	২১
NTP (Network Time Protocol)	কম্পিউটারের ঘড়ি মেলানোর প্রোটোকল	১৪
Open Systems Interconnection (OSI)	নেটওয়ার্ক যোগাযোগের সাত স্তরের মডেল	৩
Origin Server	CDN-এর পেছনে আসল কনটেন্ট রাখা সার্ভার	২০
Packet	নেটওয়ার্কে পাঠানো ডেটার একক	৪
Packet Loss	প্যাকেট গন্তব্যে না পৌঁছানো	২২
PAT (Port Address Translation)	এক public IP-তে অনেক private ঠিকানা; port দিয়ে আলাদা করা	১২
Payload	প্যাকেটে header বাদে আসল ডেটা	৪
Peer-to-Peer	কেন্দ্রীয় সার্ভার ছাড়া সমানদের মধ্যে নেটওয়ার্ক	২
Ping	ICMP echo দিয়ে পৌঁছানো যাচাই	১৪
Port	এক মেশিনে সার্ভিস চেনার সংখ্যা (0–65535)	১০
Port Forwarding	বাইরের port-এর ট্রাফিক ভেতরের সার্ভিসে পাঠানো	১২
Private IP Address	শুধু private নেটওয়ার্কে ব্যবহৃত ঠিকানা (RFC 1918)	৭
Protocol	যোগাযোগের নিয়মের সমষ্টি	৩
Protocol Data Unit (PDU)	প্রতিটি স্তরের ডেটা একক; frame, packet, segment	৪
Proxy	অন্যের হয়ে অনুরোধ পাঠানো মধ্যস্থতাকারী	১৭
Public IP Address	ইন্টারনেটে অনন্য, সবার কাছে পৌঁছানো ঠিকানা	৭
Rate Limiting	নির্ধারিত হারের বেশি অনুরোধ আটকানো	১৭

Recursive Resolver	ব্যবহারকারীর হয়ে পূর্ণ DNS যাত্রা করা resolver	১৩
Redundancy	ব্যর্থতায় বিকল্প পথ বা সার্ভার রাখা	২৪
Request / Response	ক্লায়েন্টের চাওয়া ও সার্ভারের জবাব	১৫
Retransmission	হারানো প্যাকেট আবার পাঠানো	১১
Reverse Proxy	সার্ভারের সামনে বসে অনুরোধ বিতরণ করা proxy	১৭
Ring Topology	নোডগুলো বৃত্তাকারে যুক্ত	২
Root Server	DNS hierarchy-র সর্বোচ্চ স্তরের সার্ভার	১৩
Round Robin	সার্ভারগুলোতে পালা করে অনুরোধ পাঠানো	১৭
Route Table	কোন নেটওয়ার্কে কোন পথে যেতে হবে, তার তালিকা	১২
Router	নেটওয়ার্কের মধ্যে প্যাকেট পাঠানো ডিভাইস	৫
Routing	প্যাকেটের সেরা পথ বাছাই	১২
Secure Shell (SSH)	এনক্রিপ্ট করা দূরবর্তী লগইন ও ফাইল স্থানান্তর	১৪
Segment	TCP-র ডেটা একক	৪, ১০
Server	সেবা দেওয়া কম্পিউটার বা প্রোগ্রাম	১
Session Layer	OSI স্তর ৫; সংযোগের session পরিচালনা	৩
Socket	IP + port-এর জোড়া; সংযোগের প্রান্ত	১০
Split Tunneling	শুধু কিছু ট্রাফিক VPN দিয়ে, বাকি সরাসরি পাঠানো	১৯
ss	Socket ও port দেখার Linux কমান্ড	২১
Star Topology	সব নোড কেন্দ্রীয় ডিভাইসের সঙ্গে যুক্ত	২
Stateful Firewall	সংযোগের অবস্থা মনে রেখে সিদ্ধান্ত নেওয়া firewall	১৮
Stateless Firewall	প্রতিটি প্যাকেট আলাদাভাবে দেখা firewall	১৮
Static Routing	হাতে লেখা route	১২
Status Code	HTTP প্রতিক্রিয়ার সংখ্যাগত অবস্থা; যেমন 200, 404	১৫
Subnet	বড় নেটওয়ার্কের ছোট ভাগ	৮
Subnetting	নেটওয়ার্ককে subnet-এ ভাগ করার কাজ	৮
Switch	MAC ঠিকানা দিয়ে একই ল্যানে frame পাঠানো ডিভাইস	৫, ৬
Symmetric Encryption	একই key দিয়ে এনক্রিপ্ট ও ডিক্রিপ্ট করা	১৬
SYN / SYN-ACK / ACK	TCP handshake-র তিনটি বার্তা	১১

TCP (Transmission Control Protocol)	নির্ভরযোগ্য, ক্রমবদ্ধ, সংযোগভিত্তিক transport প্রোটোকল	১০, ১১
TCP/IP Model	ইন্টারনেটের চার স্তরের মডেল	৪
tcpdump	কমান্ড লাইনে প্যাকেট ধরা ও দেখার টুল	২২
Throughput	আসলে কত ডেটা পৌঁছেছে; bandwidth-এর চেয়ে কম হতে পারে	১
Time to Live (TTL)	DNS-এ cache-র মেয়াদ; IP-তে hop-এর সীমা	১৩, ১৪
Top-Level Domain (TLD)	ডোমেইনের শেষ অংশ; যেমন .com, .bd	১৩
Topology	নেটওয়ার্কের গঠন; নোড কীভাবে যুক্ত	২
Traceroute / Tracepath	প্যাকেটের পথে hop-গুলো দেখার টুল	১৪
Transport Layer	OSI স্তর ৪ / TCP-IP transport; TCP, UDP থাকে	৩, ৪, ১০
Tunnel / Tunneling	এক প্রোটোকলের ভেতর দিয়ে আরেকটি পাঠানো	১৯
UDP (User Datagram Protocol)	দ্রুত, সংযোগহীন, অনিশ্চিত transport প্রোটোকল	১০
UFW (Uncomplicated Firewall)	Ubuntu-র সহজ firewall টুল	১৮
Unicast	এক প্রাপকের কাছে পাঠানো বার্তা	৬
Upstream	Reverse proxy-র পেছনের সার্ভার	১৭
Virtual Private Cloud (VPC)	Cloud-এ বিচ্ছিন্ন virtual নেটওয়ার্ক	২৩
Virtual Private Network (VPN)	Public নেটওয়ার্কের ওপর নিরাপদ private টানেল	১৯
VLAN (Virtual LAN)	এক সুইচে যৌক্তিকভাবে আলাদা নেটওয়ার্ক	৬, ১৮
VLSM (Variable Length Subnet Mask)	প্রয়োজন অনুযায়ী ভিন্ন আকারের subnet বানানো	৮
WAN (Wide Area Network)	বড় এলাকাজুড়ে নেটওয়ার্ক; ইন্টারনেট	১
WebSocket	এক সংযোগে দুই দিকে বার্তা পাঠানোর প্রোটোকল	১৭
Well-Known Port	0–1023; পরিচিত সার্ভিসের port; যেমন 80, 443	১০
Window Size	TCP-তে একবারে কত ডেটা পাঠানো যাবে	১১
WireGuard	আধুনিক, হালকা VPN প্রোটোকল	১৯
Wireless Access Point (AP)	তারহীন ডিভাইসকে নেটওয়ার্কে যুক্ত করা ডিভাইস	৫
Wireshark	প্যাকেট বিশ্লেষণের জনপ্রিয় টুল	২২
X.509 Certificate	পরিচয় ও public key বাঁধা ডিজিটাল সার্টিফিকেটের মান	১৬

পরিশিষ্ট F — References (তথ্যসূত্র)

নিচের সূত্রগুলো শুধু অফিসিয়াল ও প্রাথমিক উৎস থেকে নেওয়া। কোনো সূত্র উদ্ভাবন করা হয়নি। URL-গুলো প্রকাশের সময় (৮ নভেম্বর ২০২৬) চালু থাকা অফিসিয়াল পাতা।

> **CONCEPT:** এই বইয়ে তিন ধরনের তথ্য আলাদা করা হয়েছে — (১) **পরীক্ষিত তথ্য:** Ubuntu 24.04 LTS-এ প্রযোজ্য কমান্ড ও সংস্করণ; (২) **সাধারণ নীতি:** OSI, TCP/IP, subnetting-এর মতো সংস্করণ-নিরপেক্ষ ধারণা; (৩) **সংস্করণ-নির্ভর আচরণ:** যেমন Nginx বা Wireshark-এর নতুন বৈশিষ্ট্য — এগুলো বদলাতে পারে, তাই সবসময় অফিসিয়াল ডকুমেন্টেশন যাচাই করুন।

IETF RFC — প্রোটোকলের প্রাথমিক সংজ্ঞা

- RFC 791 — Internet Protocol (IPv4): https://www.rfc-editor.org/rfc/rfc791
- RFC 793 — Transmission Control Protocol (TCP): https://www.rfc-editor.org/rfc/rfc793
- RFC 768 — User Datagram Protocol (UDP): https://www.rfc-editor.org/rfc/rfc768
- RFC 826 — Address Resolution Protocol (ARP): https://www.rfc-editor.org/rfc/rfc826
- RFC 792 — Internet Control Message Protocol (ICMP): https://www.rfc-editor.org/rfc/rfc792
- RFC 1034 — Domain Names: Concepts and Facilities: https://www.rfc-editor.org/rfc/rfc1034
- RFC 1035 — Domain Names: Implementation and Specification: https://www.rfc-editor.org/rfc/rfc1035
- RFC 1918 — Address Allocation for Private Internets: https://www.rfc-editor.org/rfc/rfc1918
- RFC 2131 — Dynamic Host Configuration Protocol (DHCP): https://www.rfc-editor.org/rfc/rfc2131
- RFC 8200 — Internet Protocol, Version 6 (IPv6) Specification: https://www.rfc-editor.org/rfc/rfc8200
- RFC 8446 — The Transport Layer Security (TLS) Protocol Version 1.3: https://www.rfc-editor.org/rfc/rfc8446
- RFC 9110 — HTTP Semantics: https://www.rfc-editor.org/rfc/rfc9110

Ubuntu ও Linux

- Ubuntu Server Documentation: https://documentation.ubuntu.com/server/
- Ubuntu Networking Guide: https://documentation.ubuntu.com/server/reference/networking/
- Ubuntu Manpages: https://manpages.ubuntu.com/
- Linux man pages (man7.org): https://man7.org/linux/man-pages/
- Linux Kernel Documentation — Networking: https://www.kernel.org/doc/html/latest/networking/
- Netplan Documentation: https://netplan.readthedocs.io/
- systemd-networkd Documentation: https://www.freedesktop.org/software/systemd/man/systemd.network.html
- UFW Documentation (Ubuntu): https://documentation.ubuntu.com/server/reference/firewalls/ufw/

Nginx, Wireshark ও tcpdump

- Nginx Documentation: https://nginx.org/en/docs/
- Nginx Reverse Proxy Guide: https://docs.nginx.com/nginx/admin-guide/web-server/reverse-proxy/
- Nginx Load Balancing: https://docs.nginx.com/nginx/admin-guide/load-balancer/http-load-balancer/
- Wireshark Documentation: https://www.wireshark.org/docs/
- Wireshark User's Guide: https://www.wireshark.org/docs/wsug_html_chunked/
- tcpdump Documentation: https://www.tcpdump.org/manpages/tcpdump.1.html
- WireGuard Documentation: https://www.wireguard.com/
- OpenSSL Documentation: https://www.openssl.org/docs/

Cloud Networking (প্রদানকারীভেদে পার্থক্য আছে — প্রতিটি লিংক সংশ্লিষ্ট প্রদানকারীর অফিসিয়াল ডকুমেন্টেশন)

- AWS VPC Documentation: https://docs.aws.amazon.com/vpc/
- Microsoft Azure Virtual Network Documentation: https://learn.microsoft.com/en-us/azure/virtual-network/
- Google Cloud VPC Documentation: https://cloud.google.com/vpc/docs
- IBM Cloud VPC Documentation: https://cloud.ibm.com/docs/vpc
- Cloudflare Learning Center — CDN and DNS: https://developers.cloudflare.com/learning-paths/

প্রকাশনা তথ্য

First Edition, 8 November 2026. Primary environment: Ubuntu Server 24.04 LTS (Noble Numbat, April 2024, supported until April 2029). Tools referenced: Nginx (Ubuntu 24.04 repository version), Wireshark, tcpdump, WireGuard, OpenSSL — exact patch versions vary with Ubuntu security updates; `apt show <package>` দিয়ে নিজের সিস্টেমের সংস্করণ যাচাই করুন। এই বইয়ের টার্মিনাল আউটপুটগুলো illustrative হিসেবে চিহ্নিত — সেগুলো লাইভ সার্ভারে চালিয়ে নেওয়া প্রমাণ নয়।